

TRABAJO FIN DE MÁSTER

Pricing Intelligence en APIs de correo: catálogo de datasheets y comparación sistemática de la capacidad

Realizado por

Daniel Vela Camacho

Para la obtención del título de

Máster en Ingeniería del Software: Cloud, Datos y Gestión TI

Dirigido por

Antonio Ruiz Cortés

Manuel Resinas Arias de Reyna

Tercera convocatoria, curso 2025/26

Agradecimientos

En primer lugar, quiero expresar mi más sincero agradecimiento a mis tutores, Antonio Ruiz-Cortés y Manuel Resinas. Gracias por la confianza que depositasteis en mí al abrirme las puertas de este proyecto, y por una guía que me ha hecho crecer como ingeniero y me ha enseñado a mirar cada problema con más rigor. Hago extensivo este agradecimiento a Ramón Gavira y a Daniel Ruiz, cuyo trabajo es el punto de partida de este. Sin las herramientas que construyeron, este trabajo no habría sido posible.

También quiero dar las gracias a Daniel Romero. Ya me ayudó a sacar adelante mi Trabajo Fin de Grado, y quién me iba a decir entonces que hoy trabajaríamos juntos y compartiríamos casa. Has vivido de cerca el día a día de este trabajo, con sus noches largas y sus dudas, y siempre has tenido una palabra para que siguiera adelante. Gracias por seguir ahí, dentro y fuera del trabajo.

En un lugar aparte, quiero mencionar a mi pareja, Leticia. Gracias por tu apoyo y por haber estado conmigo en las etapas más difíciles de este camino, sobre todo en aquellas en las que yo mismo dudaba de poder terminarlo. Cuando el peso se hacía más grande, tú lo hacías más ligero. No podría haber cerrado mejor este trabajo que contigo a mi lado.

Finalmente, dedico el agradecimiento más profundo a mi familia. A mis padres, por seguir siendo la base sobre la que se sostiene todo lo que hago, y por enseñarme que las cosas se terminan aunque cuesten. A mi hermano, porque a pesar de la distancia te siento cerca en cada paso, y cada logro mío sigue siendo también tuyo. Este trabajo, como el anterior, es en gran parte vuestro, y saber que estáis orgullosos de mí sigue siendo el mayor motor que tengo.

Resumen

Un servicio de correo transaccional suele ofrecer dos interfaces: la API REST y el relé SMTP, que recibe el correo por el protocolo estándar de envío. Los términos de consumo del servicio son el precio y la información de cómo, cuándo, cuánto y dónde se consume. Cada interfaz puede tener límites propios: en el plan Starter de MailerSend, el relé SMTP acepta como máximo cinco conexiones simultáneas. Para saber qué proveedor admite el envío que necesita una aplicación, hay que reunir a mano los términos de cada interfaz, y esos términos no se comparan de forma sistemática.

El *Pricing Intelligence* es la línea de investigación que convierte la información de los planes de precios en conocimiento para elegir plan o proveedor. En esa línea, dos Trabajos Fin de Máster del grupo de investigación ISA aportan el *API Capacity Model* (ACM), que describe cuánto deja consumir cada plan y en qué periodo. Definen también la Datasheet, un documento legible por máquina con el precio y los límites de una API, escrito en su esquema YAML, ACM2YAML. Sobre ella, su motor PRIME4API calcula el plan más barato para un volumen o el tiempo mínimo para enviarlo.

El objetivo de este Trabajo Fin de Máster es llevar ese instrumental a un dominio real y a escala. Para ello se construye un catálogo de Datasheets de la API REST de ocho servicios de correo transaccional. Cada dato del catálogo lleva su fuente y su fecha. El catálogo se ha propuesto como contribución a SPHERE, la plataforma del grupo que publica planes de precios.

El catálogo lo construye un proceso agéntico diseñado y supervisado por el autor. Unos agentes de inteligencia artificial redactan la Datasheet de cada API con el precio y los límites que extraen de las fuentes. Otros agentes revisan cada Datasheet, y lo que queda abierto lo resuelve el autor.

El envío suele tener dos límites: una cuota (el volumen máximo en un periodo, como un mes) y un *rate* (las peticiones que admite la API por segundo o por minuto). En seis de las ocho APIs, mientras la cuenta esté aprobada, el *rate* deja enviar la cuota entera antes de que acabe su periodo, así que el envío se dimensiona con la cuota. El estado de la cuenta (en revisión, en pruebas o por su reputación) cambia esos límites en seis APIs, no todas las anteriores, y ninguna publica el umbral que lo provoca.

El estudio de las ocho APIs muestra lo que ACM2YAML no puede expresar en sus campos, como un plan que se vende en varios volúmenes, cada uno con su precio, o los topes del relé SMTP. Para cubrir parte de ello se proponen extensiones del esquema, cada una con el caso real que la motiva. Este trabajo establece un método de comparación que responde qué APIs admiten un patrón de envío y cuánto cuesta en cada una, y lo aplica a un caso de ejemplo: una aplicación que envía 100.000 correos al mes.

Palabras clave: Pricing Intelligence, APIs de correo electrónico, *datasheets* de APIs, capacidad de APIs, *rate limits*, cuotas de uso, comparación sistemática de APIs, ACM2YAML, propiedades no funcionales, selección de servicios

Abstract

A transactional email service usually has two interfaces: the REST API and the SMTP relay, which accepts email through the standard protocol for sending email. The service's consumption terms are the price and the information on how, when, how much, and where it is consumed. Each interface can have limits of its own: on MailerSend's Starter plan, the SMTP relay accepts up to five concurrent connections. Finding out which provider can handle what an application needs to send means gathering the terms of each interface by hand, and those terms are not compared systematically.

Pricing Intelligence is the line of research that turns the information in pricing plans into knowledge for choosing a plan or a provider. Within that line, two Master's Theses from the ISA research group provide the API Capacity Model (ACM), which describes how much consumption each plan allows and over what period. They also define the Datasheet, a document that machines can read, with an API's price and limits, written in their YAML schema, ACM2YAML. From a Datasheet, their PRIME4API engine computes the cheapest plan for a volume or the minimum time to send it.

The goal of this Master's Thesis is to take that toolset to a real domain, at scale. The work builds a catalogue of Datasheets for the REST API of eight transactional email services. Every value in the catalogue carries its source and date. The catalogue has been proposed as a contribution to SPHERE, the group's platform for publishing pricing plans.

The catalogue is built by an agentic process designed and supervised by the author. AI agents write each API's Datasheet with the price and limits they extract from the sources. Other agents review each Datasheet, and the author resolves what remains open.

Sending usually has two limits: a quota (the maximum volume over a period, such as a month) and a rate (the requests the API accepts per second or per minute). In six of the eight APIs, as long as the account is approved, the rate allows the whole quota to be sent before its period ends, so sending is planned around the quota. The account status (under review, in testing or by reputation) changes those limits in six APIs, only partly the same ones, and none publishes the threshold that triggers it.

The study of the eight APIs shows what ACM2YAML cannot express in its fields, such as a plan sold in several volumes, each with its own price, or the caps of the SMTP relay. Schema extensions are proposed to cover part of this, each with the real case behind it. This work establishes a comparison method that answers which APIs support a sending pattern and what it costs in each one, and applies it to an example: an application that sends 100,000 emails a month.

Keywords: Pricing Intelligence, email APIs, API datasheets, API capacity, rate limits, usage quotas, systematic API comparison, ACM2YAML, non-functional properties, service selection

Índice general

1. Introducción **1**
- 1.1. Contexto . . . 1
  - 1.1.1. Canales, interfaces y términos de consumo . . . 1
  - 1.1.2. Pricing Intelligence y su ecosistema . . . 4
  - 1.1.3. Un ejemplo: 100.000 correos al mes . . . 6
- 1.2. Motivación . . . 7
- 1.3. Objetivos y preguntas de investigación . . . 8
- 1.4. Contribuciones . . . 10
- 1.5. Estructura de la memoria . . . 10

2. Metodología y planificación **13**
- 2.1. Metodología . . . 13
- 2.2. Planificación . . . 15
- 2.3. Presupuesto . . . 16
- 2.4. Desviaciones . . . 18

3. Estado del arte **22**
- 3.1. Modelado y análisis de pricings de SaaS . . . 22
- 3.2. Modelado y análisis de la capacidad de las APIs . . . 25
- 3.3. El ecosistema SPHERE . . . 28
  - 3.3.1. La plataforma SPHERE . . . 32
- 3.4. Las APIs de correo . . . 33
- 3.5. Huecos que aborda el trabajo . . . 35

4. El catálogo de Datasheets de APIs de correo **36**
- 4.1. Selección del dominio . . . 36
  - 4.1.1. Fuentes de inclusión . . . 37
  - 4.1.2. Criterios de inclusión y regla de reemplazo . . . 38
  - 4.1.3. Ventana de toma de datos . . . 39
- 4.2. El ciclo de construcción por API . . . 40
- 4.3. Trazabilidad y escalera de validación . . . 47
  - 4.3.1. Fuente, semáforo y procedencia . . . 47
  - 4.3.2. La escalera de validación . . . 49
  - 4.3.3. Dos casos de fuente a contradicción . . . 51
- 4.4. El catálogo resultante . . . 52
- 4.5. Conformidad y consumo por PRIME4API . . . 54
- 4.6. Del catálogo a SPHERE . . . 55
  - 4.6.1. La colección y su maqueta . . . 57
  - 4.6.2. Lo que Pricing2YAML no modela . . . 58
  - 4.6.3. Términos de consumo y versionado . . . 59
- 4.7. Respuesta a la pregunta de investigación . . . 60

5. Evolución del esquema ACM2YAML **62**

5.1. Cómo una carencia llega a propuesta . . . 62
5.2. Inventario . . . 64
5.3. Propuestas de expresividad . . . 68
5.3.1. P-01. La cuota declara volúmenes contratados . . . 68
5.3.2. P-02 y P-03. El `workload` deja de ser una terna escalar . . . 70
5.3.3. P-06. El estado de la suscripción y la reputación se declaran una vez . . . 72
5.3.4. P-07. El sobrecoste declara su tope . . . 74
5.3.5. P-08. Un cupo que cuenta destinatarios distintos y no se repone 75
5.3.6. P-09. La cuota declara qué hace con lo que la supera . . . 76
5.4. Defectos de contrato . . . 77
5.4.1. La forma de `workload`: mapa o lista . . . 77
5.4.2. P-04. Distinguir «sin límite» de «límite no publicado» . . . 78
5.4.3. P-05. Declarar un plan cuyo precio existe y no es público . . . 80
5.5. Lo que queda fuera . . . 80
5.6. Respuesta a la RQ2 . . . 84

6. Límites de uso y *rates* 86
6.1. Lectura geométrica . . . 86
6.2. Lo que publica cada proveedor . . . 89
6.3. Lo que enseña el catálogo . . . 94
6.4. Lo que responde PRIME4API . . . 99
6.4.1. Correspondencia con el cálculo a mano . . . 99
6.4.2. «No publicado» frente a «sin límite» desde el motor . . . 101
6.4.3. Los dos regímenes del sobrecoste . . . 101
6.4.4. La cuota que no cambia de unidad . . . 103
6.4.5. Capacidad con un presupuesto menor que el precio del plan . 104
6.5. Respuesta a la RQ3 . . . 104

7. Comparación sistemática 107
7.1. Catálogo de preguntas de decisión . . . 107
7.2. Procedimiento y cobertura . . . 110
7.3. Las preguntas que piden HARVEY4API . . . 114
7.4. El caso de los 100.000 correos . . . 117
7.5. Comparación funcional . . . 120
7.5.1. *Webhooks* en el plan gratuito . . . 120
7.5.2. IP dedicada al menor precio . . . 121
7.5.3. Retención de la actividad . . . 122
7.5.4. Lo que la matriz añade . . . 122
7.6. Respuesta a la RQ4 . . . 122

8. Conclusiones 124
8.1. Respuestas a las preguntas de investigación . . . 124
8.2. Objetivos y contribuciones . . . 124
8.3. Amenazas a la validez . . . 128
8.3.1. Validez de constructo . . . 128
8.3.2. Validez interna . . . 129
8.3.3. Validez externa . . . 130

8.3.4. Validez de conclusión 131
8.4. Lecciones del método agéntico 132
8.5. Trabajo futuro 132

9. Bibliografía **135**

A. Uso de inteligencia artificial **141**
A.1. Alcance del uso 142
A.2. La consulta de la selección 143
A.3. Cómo se aplicó 144

Índice de figuras

1.1. Canales e interfaces del servicio de MailerSend 2
1.2. Conceptos fundamentales de *Pricing Intelligence* 4

2.1. Cronograma previsto y al cierre 16

3.1. El pricing de Postmark en su página de precios y en SPHERE 23
3.2. Curva de capacidad de un *BoundedRate* 28
3.3. Correo transaccional frente a correo de *marketing* 34

4.1. Ciclo de construcción de cada Datasheet 40
4.2. Los cinco artefactos de Postmark 43
4.3. Estructura comercial de Postmark frente a su proyección en el bloque `plans` de ACM2YAML v0.3 46
4.4. La colección de los ocho pricings en SPHERE 56
4.5. Maqueta de la colección: ficha de MailerSend 58

6.1. Curva de capacidad $C(t)$ de Resend Pro, SendGrid Essentials 50K, MailerSend Hobby y Amazon SES 88
6.2. Curva de capacidad de MailerSend Hobby en peticiones 103

A.1. Arquitectura de agentes del trabajo 142

Índice de tablas

2.1. Actividades del proceso DSRM 14
2.2. Paquetes de trabajo de la línea base 15
2.3. Presupuesto, previsto y al cierre 17
2.4. Hitos previstos y reales 18
2.5. Horas previstas y al cierre por paquete 19
2.6. Entregables de la línea base y su estado 19
2.7. Desviaciones de alcance 20

3.1. Operaciones del catálogo de Fresno-Aranda et al. 27
3.2. Las diez preguntas de análisis de Gavira Sánchez 30
3.3. Piezas del canal API que usa este trabajo: lo que ya existía, con su autoría, y lo que aporta este trabajo. 32

4.1. Las ocho APIs del catálogo de Datasheets 36
4.2. Fuentes de inclusión del catálogo 37
4.3. Elementos no soportados documentados en el piloto 44
4.4. Fuente principal de las entradas del registro de procedencia 49
4.5. Las ocho Datasheets del catálogo 52
4.6. Resultado de ingerir las ocho Datasheets del catálogo en PRIME4API con la herramienta de verificación 55
4.7. Las 20 limitaciones de Pricing2YAML en las ocho APIs, agrupadas . 59
4.8. Cobertura de la condición de la RQ1 sobre las Datasheets 61

5.1. Inventario de elementos no soportados sobre las ocho APIs del catálogo 66
5.2. Propuestas del capítulo y su origen 67
5.3. Promovidos sin número de propuesta y con una forma esbozada . . . 82

6.1. Relación entre *rate* y cuota en Postmark, Resend, SendGrid y MailerSend 91
6.2. Relación entre *rate* y cuota en Brevo, Amazon SES, Mailgun y Mailchimp 93
6.3. Tiempo mínimo para 100.000 correos con corte estricto y con presupuesto que no limita 102

7.1. Catálogo de preguntas de decisión, categorías A y B 109
7.2. Catálogo de preguntas de decisión, categorías C y D 110
7.3. Catálogo de preguntas de decisión, categorías E a G 111
7.4. Cobertura del catálogo de preguntas sobre las ocho APIs 113
7.5. Las cuatro preguntas que piden HARVEY4API, contestadas por Claude Opus 5.5 con las ocho Datasheets en el prompt y sin ellas. Cada celda resume tres repeticiones con la misma conclusión y las mismas cifras. 116
7.6. Respuesta del catálogo al caso de los 100.000 correos 118

8.1. Respuestas a las preguntas de investigación 125
8.2. Estado de cada objetivo específico y de la contribución que le corresponde 126

A.1. Ficha de la consulta, según el fichero de la selección. 144

Índice de extractos de código

1.1. Fragmento de la Datasheet de SendGrid del catálogo: la cuota mensual del plan Essentials 50K con su sobrecoste, el *rate* de envío, que la Datasheet declara en el plan, y el plan, con su precio en USD al mes y el tope de correos por petición. Las líneas `# ...` marcan lo que se omite. 7
4.1. Serialización del semáforo de confianza en la Datasheet del piloto (fragmento). 48
5.1. P-01: volúmenes contratados dentro de un plan, con precio y sobrecoste propios (caso Resend, plan Scale, tres de los seis volúmenes). 70
5.2. P-02 y P-03: composición de dimensiones anidadas y cota expresada en otra métrica (caso Resend para P-02 y caso Postmark para P-03). 71
5.3. P-06: estados de la suscripción y de la reputación declarados una vez, con su disparador y con el `workload` que la guía no documenta (caso MailerSend). 74
5.4. P-09: una cuota que retiene lo que la supera hasta un tope fijo (caso Brevo, plan gratuito, correo transaccional). 77

1. Introducción

1.1. Contexto

Este trabajo se ha realizado en el contexto de dos líneas de investigación del grupo ISA (Ingeniería del Software Aplicada) de la Universidad de Sevilla. La primera es el *Pricing Intelligence*, una disciplina dirigida por datos que convierte la información de los planes de precios en conocimiento para decidir [1]. La segunda es el análisis de la capacidad de las APIs (interfaces de programación de aplicaciones) [2]. De las dos nace una línea que extiende el *Pricing Intelligence* a las APIs, en la que se enmarcan los Trabajos Fin de Máster de Ramón Gavira Sánchez [3] y de Daniel Ruiz López [4]. Este trabajo se apoya en los dos y construye un catálogo de ocho Datasheets de APIs de correo, documentos legibles por máquina con el precio y los límites de cada API.

1.1.1. Canales, interfaces y términos de consumo

Un servicio SaaS (*Software as a Service*) es software que opera el proveedor y que el cliente usa a través de la red, sin instalarlo. Puede consumirse por varios canales de interacción. El canal web es para personas, y en él se publica el pricing en forma de tablas. El pricing es la estructura de planes, características (*features*) y complementos (*add-ons*) con la que el proveedor regula el acceso a su servicio [5]. Un plan es la oferta que se contrata, una *feature* es una funcionalidad del servicio y un *add-on* es un complemento que se suma al plan. El canal API sirve para que otros sistemas usen el servicio de forma programática.

En esta memoria, el canal es el modo de interacción, como el canal web o el canal API, y la interfaz es el punto concreto de acceso. El NIST (*National Institute of Standards and Technology*) de Estados Unidos define la interfaz como la frontera común entre sistemas o módulos independientes en la que tienen lugar las interacciones [6]. El canal API de un servicio de correo suele tener dos interfaces: la API REST y el relé SMTP. El relé SMTP es un servidor del proveedor que recibe el correo por SMTP, el protocolo estándar de envío de correo [7], y lo entrega a sus destinatarios.

El dominio de este trabajo es el correo transaccional. Postmark, uno de los proveedores del catálogo, lo define como el mensaje individual que el destinatario espera recibir y que suele desencadenar el propio usuario, como el restablecimiento de contraseña o el recibo de compra [8]. Lo distingue del correo de *marketing*, el mismo contenido para muchos destinatarios sin que lo desencadene ningún evento. Más detalles en la Sección 3.4. Las ocho APIs del catálogo son las de Postmark, Resend, SendGrid, MailerSend, Amazon SES, Mailgun, Brevo y Mailchimp Transactional. Algunos de estos proveedores, como Resend, venden también un producto de *marketing* en la misma cuenta, y el catálogo modela en cada API solo el producto transaccional.

El servicio de MailerSend, otro de los ocho del catálogo, tiene tres interfaces, como muestra la Figura 1.1. El panel web es donde una persona opera la cuenta: da de alta

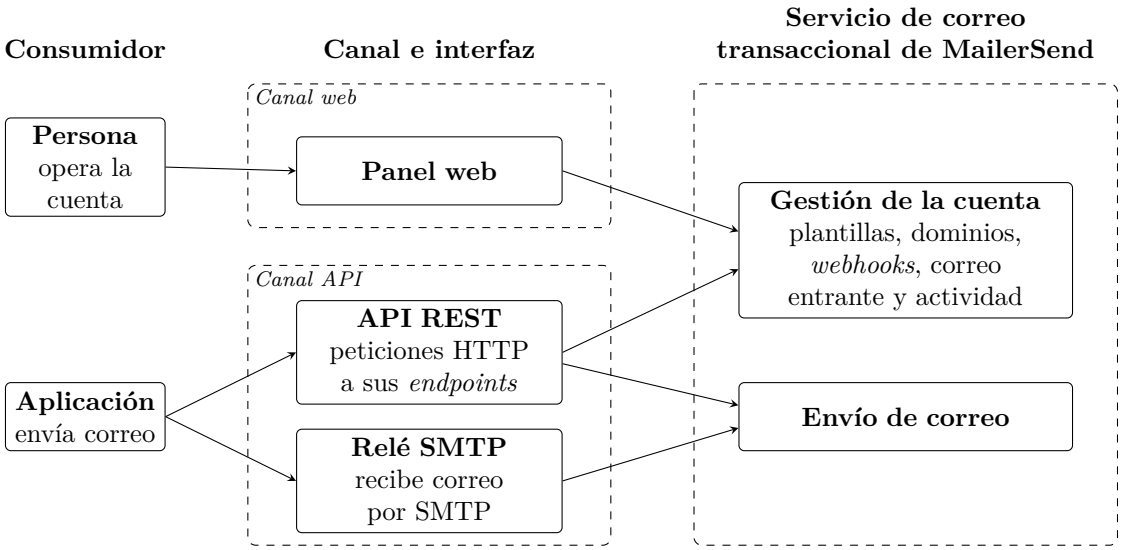


Figura 1.1: Canales e interfaces del servicio de correo transaccional de MailerSend. Una persona usa el panel web, en el canal web, y una aplicación usa la API REST o el relé SMTP, en el canal API. La API REST llega al envío y a la gestión de la cuenta, y el relé SMTP solo al envío. Elaboración propia a partir de la documentación de MailerSend [9, 10].

dominios, crea plantillas y consulta la actividad. La API REST se usa con peticiones HTTP a sus *endpoints*. Un *endpoint* es una operación de la API, identificada por un método y una ruta HTTP, como `POST /v1/email`, que envía un correo. El relé SMTP solo recibe correo para enviarlo.

Las dos interfaces del canal API se parecen en que dan acceso al mismo servicio de envío y se contratan en el mismo plan. MailerSend incluye «Email API and SMTP relay» desde su plan gratuito [11], y el panel de Postmark, visto con una cuenta propia el 3 de agosto de 2026, muestra «SMTP & REST API» en su plan Basic.

Las dos interfaces se diferencian en las funciones que dan. La API REST de MailerSend, además de enviar, tiene operaciones sobre plantillas, dominios, correo entrante, actividad y *webhooks*, los avisos que el proveedor envía al sistema del cliente cuando ocurre un evento, como una entrega o un rebote, que es un correo devuelto sin entregar [9]. El relé de MailerSend no admite la personalización avanzada ni las plantillas, que solo da la API [10]. El envío en lote, es decir, varios correos distintos en una sola petición, tampoco está siempre en las dos interfaces. En Brevo, el relé no lo admite y la API sí [12].

Los «términos de consumo» de un servicio son el precio y la información de cómo, cuándo, cuánto y dónde se consume, e incluyen el pricing. En el plan Starter de MailerSend, el precio es de 35 USD al mes, y el volumen incluido, 50.000 correos al mes, dice cuánto se puede consumir [11]. La hora a la que vuelve a cero el contador diario de peticiones, medianoche UTC, dice cuándo [13]. Que Brevo solo admita el envío en lote por la API dice cómo y dónde.

Los apuntes de Fundamentos de Ingeniería del Software para Sistemas Cloud, una asignatura de este máster, llaman términos de consumo a las condiciones del proveedor

que afectan a la velocidad y a la capacidad de consumo de su API [14]. Gavira Sánchez usa «pricing» en un sentido amplio, que incluye las limitaciones de capacidad [3]. Esta memoria reserva «pricing» para la estructura de planes, *features* y *add-ons*, y llama términos de consumo al conjunto.

Un límite de uso fija el máximo de un nivel de uso medible, como los correos enviados al mes, los dominios de la cuenta o los días que se conservan los registros. Puede renovarse cada periodo o no. La cuota es un límite de uso renovable: el volumen máximo acumulable en un periodo. Al pasar de la cuota, el proveedor puede cortar el envío o cobrar un sobrecoste por cada unidad de más. El plan Essentials 50K de SendGrid, por ejemplo, incluye 50.000 correos al mes y cobra 0,0013 USD por cada correo de más [15].

El *rate* no es un límite de uso: fija la velocidad máxima de consumo, en peticiones por unidad de tiempo. Esta memoria lo llama también caudal. Antonio Ruiz-Cortés y Miguel Ruiz-López leen la cuota y el *rate* de forma geométrica [2]. En una gráfica de las peticiones acumuladas frente al tiempo, el *rate* es una recta que sube y la cuota, una recta horizontal. Los dos operan a la vez, y la capacidad efectiva, lo que se puede consumir sin pasar ninguno de los dos, es lo que queda por debajo de ambas rectas.

Cada interfaz de un mismo servicio tiene términos de consumo propios. En el plan Starter de MailerSend, el relé SMTP acepta como máximo cinco conexiones simultáneas [16]. La API tiene un *endpoint* de envío en lote, `POST /v1/bulk-email`, que admite 500 correos distintos por llamada [17]. Cada correo lleva hasta 50 destinatarios directos, 10 en copia y 10 en copia oculta, así que una llamada llega a 35.000 destinatarios. En Starter, el *rate* de ese *endpoint* es de 15 llamadas por minuto [18].

Las conexiones simultáneas y los destinatarios por llamada no son ni cuota ni *rate*. Gavira Sánchez llama dimensión de capacidad a cada aspecto del consumo que el proveedor limita por separado, y su modelo distingue cuatro: el volumen (las cuotas), la frecuencia (los *rates*), las peticiones concurrentes y los resultados por petición, que acotan la información que devuelve una sola llamada [3]. Las conexiones simultáneas del relé son peticiones concurrentes.

Los destinatarios por llamada no son una dimensión de capacidad: dicen cuántos correos gasta una llamada, y para Gavira Sánchez eso es una regla de consumo y no una restricción de capacidad [3]. La llama CRF (*Consumption Request Factor*): la regla que dice cuánto gasta una petición de cada límite. En una API de correo, una misma llamada gasta una petición y, a la vez, tantos correos de la cuota mensual como destinatarios incluya [3].

Las dos interfaces de MailerSend comparten la cuota diaria de 100.000 peticiones del plan Starter [13], pero no con el mismo CRF. Por la API cuenta cada petición, así que una llamada de lote cuenta una. Por el relé cuenta cada mensaje [10], así que 500 correos enviados por el relé gastan 500 peticiones de la cuota.

1.1.2. Pricing Intelligence y su ecosistema

El *Pricing Intelligence* reúne herramientas, métodos y procesos para convertir los datos de los pricings en información, y la información en conocimiento para quien decide, sea una persona o un agente automático [1]. Hace con los pricings lo que la *Business Intelligence* hace con los datos de una empresa. Cavero et al. lo proponen porque varios trabajos independientes, como los de análisis automático de pricings, avanzaban por separado [1]. Lo llevan a la práctica con HARVEY, el agente conversacional del grupo, que traduce preguntas en lenguaje natural a llamadas a las herramientas de análisis de pricings del grupo y responde con sus resultados.

Gavira Sánchez explica el *Pricing Intelligence* con tres conceptos, que resume la Figura 1.2 [3]. El espacio de consultas (*Query Space*) es el conjunto de preguntas que se pueden formular sobre los pricings y que las herramientas de *Pricing Intelligence* son capaces de resolver. La base de conocimiento (*Knowledge Base*) es la información representada explícitamente, como planes, cuotas, *rates* y precios. El espacio de conocimiento (*Knowledge Space*) es el conjunto de respuestas a esas preguntas que se obtienen automáticamente a partir de esa base, así que depende de los otros dos.

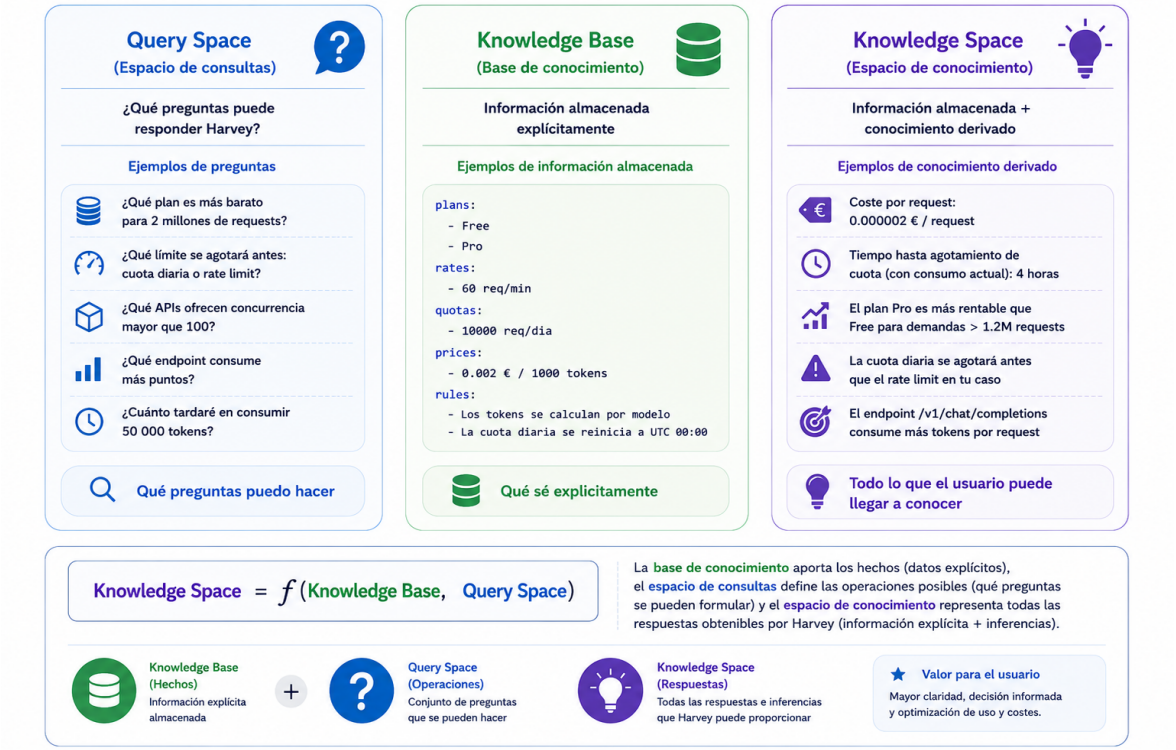


Figura 1.2: Conceptos fundamentales de *Pricing Intelligence*: espacio de consultas, base de conocimiento y espacio de conocimiento. Reproducida del Trabajo Fin de Máster de Gavira Sánchez [3], Figura 1.4.

Dos preguntas de correo lo ilustran. La primera, cuánto tiempo mínimo necesita el plan Essentials 50K de SendGrid para enviar 100.000 correos, está en el espacio de conocimiento: su respuesta sale de la cuota, el *rate* y el sobrecoste del plan, que recoge la Datasheet de SendGrid. La segunda, cuántas peticiones simultáneas admite

la API de Postmark, está en el espacio de consultas pero no en el de conocimiento. Las Datasheets, que forman la base de conocimiento de este trabajo, no tienen campo para la concurrencia.

Cavero et al. llaman tarea de *Pricing Intelligence* a la necesidad de quien decide sobre un pricing, formulada como una pregunta en lenguaje natural [1]. Un ejemplo suyo: qué plan y qué complementos se ajustan mejor al uso previsto al menor coste. Este trabajo llama pregunta de decisión a una tarea de ese tipo cuando la plantea quien va a contratar una API de correo, como qué API y qué plan cubren 100.000 correos al mes al menor coste. Las dos del párrafo anterior, la de SendGrid y la de Postmark, también lo son: las necesita quien está valorando ese plan o esa API.

El catálogo de preguntas de decisión de este trabajo tiene 32, escritas como plantillas: preguntas con huecos, como el proveedor o el volumen, que se rellenan para cada caso. Los huecos son los mismos que usan Cavero et al. [1]. Las diez preguntas de análisis de Gavira Sánchez razonan sobre los planes de una sola API [3]. Algunas preguntas de decisión hacen los mismos cálculos, como la de SendGrid, y otras comparan varias APIs o preguntan por *features*, como los *webhooks*.

Para el canal web, el grupo representa el pricing de un SaaS con el metamodelo *iPricing* de García-Fernández et al., que lo describe como planes, complementos y *features* con sus límites de uso [5]. Un pricing en iPricing se escribe en YAML, un formato de texto legible por máquina, con la serialización Pricing2YAML. iPricing es uno de los activos en los que se apoya HARVEY [3]. No representa *endpoints*, *rates* por ventana de tiempo ni el CRF, y Gavira Sánchez concluye que canales distintos necesitan modelos distintos [3].

El ecosistema del grupo para el canal API parte del *API Capacity Model* (ACM), el modelo de la capacidad contractual de una API que conceptualiza Gavira Sánchez [3] y cuya autoría comparte Ruiz López [4]. La capacidad contractual es lo que el proveedor se compromete a dejar consumir según sus términos, y no la capacidad de la infraestructura que lo sirve. El ACM la describe con las cuatro dimensiones de capacidad y el CRF presentados en la Sección 1.1.1, y la capacidad efectiva de la lectura geométrica se calcula sobre ella.

ACM2YAML es el esquema con el que el ACM se escribe en YAML: fija qué campos lleva el fichero y cómo se anidan. Cada fichero conforme a ese esquema es una Datasheet [3]. La Datasheet se organiza por plan y, dentro de cada plan, por *endpoint*. Tiene un bloque para las cuotas (`capacity`), cada una con su sobrecoste (`overage_cost`), y otro para los *rates* (`max_power`). Declara en `workload` el CRF de cada *endpoint*: las unidades que gasta una petición, entre un mínimo y un máximo, como de 1 a 1.000 correos en el envío de SendGrid. Cada valor del catálogo lleva su fuente en el registro de procedencia (*provenance*), que la guía de ACM2YAML prevé en parte. El Extracto de código 1.1 muestra un fragmento real.

Los topes propios del relé no tienen campo en la Datasheet y quedan en el registro de procedencia. Las cinco conexiones simultáneas del relé de MailerSend no lo tienen porque el esquema no recoge la concurrencia. El *rate* del relé de Mailgun, 1.500 peticiones por segundo por IP de cliente[1], no lo tiene porque el esquema cuelga de un

[1] `https://www.mailgun.com/ai-info/`, preguntas frecuentes del sitio comercial de Mailgun.

endpoint HTTP cada *rate* propio de una interfaz, y el relé no tiene ninguno. Por eso el catálogo modela la API REST de cada servicio.

PRIME4API es el motor de análisis de la capacidad de Gavira Sánchez [19]. Rehace como API REST la biblioteca Pricing4API, cuyo primer prototipo hizo Ruiz López [20], y responde leyendo la Datasheet. Calcula, por ejemplo, el tiempo mínimo para enviar un volumen, el plan más barato que lo cubre o lo que cuesta pasar de la cuota.

Gavira Sánchez acompaña el motor de diez preguntas de análisis, como cuál de dos cuotas de un plan se agota antes. Las respondió con PRIME4API sobre tres pricings de dos servicios, SendGrid en dos versiones y la API de datos bibliográficos DBLP, y esas respuestas son la referencia con la que se valida el ecosistema [3].

HARVEY4API es el agente conversacional de Ruiz López para el canal API [4]. Publica las operaciones de PRIME4API como herramientas a través del *Model Context Protocol* (MCP), el protocolo con el que un modelo de lenguaje llama a herramientas externas [21], y responde en lenguaje natural.

SPHERE es la plataforma web del grupo que reúne pricings en colecciones y da acceso a las demás herramientas [22]. Se extendió a las Datasheets a partir de los requisitos de Gavira Sánchez y Ruiz López [3], y una colección puede llevar, para cada API, su Datasheet y su pricing en Pricing2YAML. Más detalles en la Sección 3.3.1.

1.1.3. Un ejemplo: 100.000 correos al mes

Una aplicación tiene que enviar 100.000 correos al mes, con ráfagas de 1.000 peticiones por minuto, *webhooks* de eventos y envío en lote. Quien la desarrolla quiere saber qué API y qué plan lo cubren al menor coste.

Cada condición se lee en un sitio distinto. El volumen y el coste salen de la cuota, el precio y el sobrecoste del plan. La ráfaga se compara con el *rate* de envío, y el lote, con el tope de correos por petición (`workload`) del *endpoint* de envío. Los *webhooks* son una *feature* que la Datasheet no recoge. Se leen en una matriz de *features*, una tabla por API que dice qué *features* trae cada plan y que se guarda junto a su Datasheet. El Extracto de código 1.1 muestra dónde se leen en la Datasheet de SendGrid el volumen, la ráfaga y el lote.

Para el tiempo mínimo de enviar los 100.000 correos con el plan Essentials 50K de SendGrid, que incluye 50.000 y admite 10.000 peticiones por segundo, PRIME4API [19] da dos respuestas según qué ocurra al pasar de la cuota. Con un destinatario por petición y un presupuesto que cubre el sobrecoste, da 9 segundos: diez ráfagas de 10.000 peticiones, la primera en el instante cero. Si la cuota corta el envío, da 30 días y 4 segundos, porque la mitad de los correos espera a que la cuota se renueve. Con 1.000 destinatarios por petición, da menos de un segundo con sobrecoste (el motor responde 0 segundos) y 30 días con corte. El caso entero, planteado sobre las ocho APIs y con las cuatro condiciones, está en el Capítulo 7.

Extracto de código 1.1: Fragmento de la Datasheet de SendGrid del catálogo: la cuota mensual del plan Essentials 50K con su sobrecoste, el *rate* de envío, que la Datasheet declara en el plan, y el plan, con su precio en USD al mes y el tope de correos por petición. Las líneas `# ...` marcan lo que se omite.

```yaml
capacity:
  quota_email_monthly_essentials:
    value: 50000
    unit: emails
    period:
      value: 1
      unit: MONTH
    # ...
    overage_cost:
      price: 0.0013
      value: 1
max_power:
  rate_request_mailsend_essentials:
    value: 10000
    unit: requests
    period:
      value: 1
      unit: SECOND
plans:
  essentials:
    price: 19.95
    # ...
    quota: quota_email_monthly_essentials
    rate: rate_request_mailsend_essentials
    endpoints:
      /v3/mail/send:
        workload:
          - unit: emails
            min: 1
            max: 1000
            # ...
```

1.2. Motivación

El responsable técnico que elige proveedor de correo rara vez encuentra juntos en la página de precios tres de los elementos principales que caracterizan el consumo de una API: el *rate* de cada *endpoint*, los destinatarios que caben en una llamada y lo que ocurre al superar la cuota. Le falta la información que dice si el servicio admite su patrón de consumo real, es decir, cuántos correos envía su aplicación al mes, en qué ráfagas y con cuántos destinatarios por llamada, como en el caso de los 100.000 correos. Para saberlo tiene que reunir a mano los términos de consumo de cada interfaz, y esos términos no se comparan de forma sistemática.

Los términos de consumo de un mismo servicio están repartidos entre varias páginas, a veces no se publican, como la concurrencia de la API de Postmark, y a veces se contradicen. El catálogo de este trabajo registra 82 divergencias entre fuentes. Las comparativas del sector no resuelven esa dispersión: las publican empresas de correo

como Mailtrap o MailSlurp, que recomiendan su propio producto. La de Mailtrap da sin fuente los topes de destinatarios por llamada [23], y la de MailSlurp no da datos de ningún proveedor [24].

Con el ecosistema descrito en la sección anterior se puede escribir la capacidad contractual de una API en una Datasheet y analizarla, pero se ha probado poco. Las diez preguntas de análisis de Gavira Sánchez se respondieron sobre tres pricings de dos servicios, y el diseño de ACM2YAML se probó con cuatro APIs, dos de ellas de correo. La versión en uso, ACM2YAML v0.3, que Gavira Sánchez llama versión operativa parcial, recoge dos de las cuatro dimensiones de capacidad, el volumen y la frecuencia, y deja fuera la concurrencia y los resultados por petición [3].

La distancia entre las cuatro dimensiones del ACM y las dos que escribe ACM2YAML v0.3, probadas sobre pocos casos, motiva este trabajo y explica también su dificultad. Las herramientas de análisis, el motor PRIME4API y el agente HARVEY4API, ya existen. Lo que falta tiene tres partes. La primera es de reconstrucción: obtener la capacidad contractual de cada API de sus términos de consumo, repartidos entre páginas, y contrastar cada dato con otra fuente. La segunda es de modelado. Cada servicio que no encaja en el esquema obliga a decidir si es un caso particular que puede simplificarse o una carencia del modelo que debe corregirse, y esa decisión exige un juicio sobre el dominio.

La tercera parte es de coordinación. Este trabajo se construye sobre artefactos que desarrollan en paralelo los trabajos de Gavira Sánchez y Ruiz López, como ACM2YAML, PRIME4API y HARVEY4API. Cualquier cambio propuesto al esquema debe verificarse contra las herramientas que consumen la Datasheet, empezando por PRIME4API.

1.3. Objetivos y preguntas de investigación

Del contexto y la motivación expuestos se deriva el objetivo global de este Trabajo Fin de Máster.

Objetivo del Trabajo Fin de Máster

Construir un espacio de conocimiento sobre las APIs de correo electrónico mediante Datasheets conformes al modelo ACM, y establecer un método de comparación sistemática que responda a necesidades funcionales y no funcionales, evolucionando el modelo con las carencias que revele el estudio del dominio.

De este objetivo salen cuatro preguntas de investigación.

RQ1. ¿Qué APIs de correo deben incluirse en un análisis representativo del dominio, y bajo qué criterios reproducibles?

RQ2. ¿Es ACM2YAML suficientemente expresivo para representar la capacidad contractual de las APIs de correo? ¿Qué extensiones requiere?

RQ3. ¿Cómo se relacionan los límites de uso y los *rates* en las APIs de correo, y qué implicaciones tiene esa relación para el consumidor?

RQ4. ¿Cómo pueden compararse las APIs de correo de forma sistemática para responder a preguntas de decisión que involucran simultáneamente requisitos funcionales y no funcionales?

En la RQ1, representativo significa que el conjunto cubre cuatro formas de cobro (planes por volumen mensual, pago por uso, producto vendido como añadido de otro y planes con límite diario además del mensual) y que sus APIs difieren en los rasgos de sus límites, como un *rate* publicado por *endpoint* o un caudal negociado no publicado. No significa que sea una muestra estadística del mercado. La selección se completó con una consulta a un modelo de lenguaje, registrada en el Anexo A. Lo reproducible son los criterios y ese registro: repetir la consulta no garantiza las mismas APIs.

De las cuatro preguntas se derivan los cinco primeros objetivos específicos. El sexto no deriva de ninguna pregunta. Lleva el resultado a SPHERE, porque ahí el catálogo deja de ser un conjunto de ficheros de este trabajo y pasa a formar parte de la línea de investigación.

OBJ-01 Selección justificada del dominio. Definir y aplicar un criterio de inclusión reproducible que determine el conjunto de APIs analizadas, documentando el proceso de forma que un tercero pueda replicarlo.

OBJ-02 Construcción del catálogo de Datasheets. Recoger de forma sistemática los términos de consumo que publica cada proveedor, con su versión y su fecha. A partir de ellos, documentar la capacidad contractual de cada API conforme a ACM2YAML, con trazabilidad explícita de las fuentes de cada dato.

OBJ-03 Evolución del esquema. Identificar las carencias de expresividad que revele el estudio del dominio y proponer las extensiones correspondientes, justificando cada una con el caso real que la motiva.

OBJ-04 Análisis de la capacidad. Caracterizar cómo se relacionan los límites de uso y los *rates* en el dominio, con la lectura geométrica de la capacidad y las operaciones de análisis de PRIME4API.

OBJ-05 Método de comparación. Establecer un catálogo de preguntas de decisión propias del dominio y un procedimiento para responderlas sobre el conjunto de Datasheets.

OBJ-06 Contribución a SPHERE. Llevar el pricing de cada API del catálogo a Pricing2YAML, el formato que consume SPHERE, documentar lo que la plataforma no admite y proponer cómo mostrar sus términos de consumo.

El componente de investigación está sobre todo en OBJ-03 y OBJ-04: lo que el estudio de un dominio real revela sobre las carencias de expresividad de un modelo, y lo que se aprende del dominio una vez formalizado. El catálogo es el medio para llegar a ello.

1.4. Contribuciones

Estos objetivos llevan a cuatro contribuciones.

C-01 Catálogo de Datasheets de APIs de correo. Un conjunto curado y versionado de Datasheets del dominio, construido sobre los términos de consumo que publica cada proveedor. Lleva trazabilidad de fuentes, está preparado como contribución a SPHERE y lo pueden reutilizar las demás herramientas del ecosistema.

C-02 Evolución de ACM2YAML. Una propuesta de extensión del esquema justificada caso a caso, en la que cada elemento nuevo responde a un escenario real que la versión previa no era capaz de representar.

C-03 Caracterización de la relación entre límites y *rates*. Un análisis de cómo se relacionan la cuota y el *rate* en el dominio del correo y de sus consecuencias para el consumidor. El ACM, que conceptualiza Gavira Sánchez en su Trabajo Fin de Máster, define la relación [3], pero no la caracteriza sobre el conjunto de APIs de un dominio.

C-04 Método de comparación sistemática. Un catálogo de preguntas de decisión funcionales y no funcionales, junto con el procedimiento para resolverlas: las no funcionales sobre las Datasheets y las funcionales sobre una matriz de *features* por plan que queda fuera del YAML.

Las cuatro contribuciones se apoyan en un método de ingeniería del software agéntica con supervisión humana (*human in the loop*). Para cada tarea diseñamos *skills* (conjuntos de instrucciones empaquetadas que un modelo de lenguaje sigue), agentes (modelos de lenguaje que ejecutan una tarea con herramientas) y flujos de trabajo que recogen, modelan y verifican los datos de cada API. Después revisamos lo que el proceso deja abierto y lo aceptamos o lo corregimos. Las *skills* del ciclo de construcción de cada Datasheet, como la que recoge en bruto los términos de consumo de cada proveedor, son de este trabajo.

En el ámbito académico, el catálogo sirve a quienes desarrollan ACM2YAML, PRIME4API y SPHERE: les da ocho APIs reales sobre las que probar las herramientas, y cada carencia que aparece en ellas vuelve al esquema como extensión. En el ámbito industrial, sirve a los equipos de desarrollo que eligen un proveedor de correo: pueden comparar límites de uso y *rates* con una misma estructura, en lugar de reunirlos a mano de las páginas de cada proveedor. Queda sin probar si las extensiones propuestas y el método de comparación sirven en otros dominios de APIs.

1.5. Estructura de la memoria

La memoria tiene ocho capítulos y un anexo, y se organiza del siguiente modo. Del Capítulo 4 al 7, cada capítulo responde a una pregunta de investigación, en el orden de las RQ, y cierra con su respuesta.

En primer lugar, el **Capítulo 1** presenta las interfaces de un servicio de correo y sus términos de consumo, el *Pricing Intelligence* y su ecosistema, el caso de los 100.000 correos al mes y la motivación del trabajo. Formula el objetivo, las cuatro preguntas de investigación y los seis objetivos específicos, y enumera las cuatro contribuciones.

A continuación, el **Capítulo 2** describe la metodología, basada en el proceso DSRM (*Design Science Research Methodology*) de Peffers et al., un proceso de investigación en ciencia del diseño que ordena el trabajo en seis actividades [25]. Expone también el método agéntico con su supervisión humana y la planificación del proyecto: paquetes de trabajo, cronograma, presupuesto y desviaciones respecto a lo previsto.

El **Capítulo 3** revisa el modelado y análisis de pricings de SaaS y de la capacidad de las APIs, y presenta el ecosistema SPHERE. Separa lo que ya ofrecían el ACM, las Datasheets, PRIME4API y SPHERE de lo que aporta este trabajo. Termina con el dominio de las APIs de correo y los cuatro huecos que aborda el trabajo, uno por contribución.

El **Capítulo 4** construye el catálogo de ocho Datasheets y responde a la RQ1 (OBJ-01). El catálogo responde al OBJ-02 y es el medio para las RQ2 a RQ4. Su salida hacia SPHERE responde al OBJ-06, que no deriva de ninguna RQ. El capítulo fija el criterio de selección de las APIs y el ciclo de construcción de cada Datasheet, con la escalera de validación por la que pasan los datos que ninguna fuente da de forma literal o en los que dos fuentes chocan. Sus cuatro peldaños son el contraste con otra fuente, la observación del panel con una cuenta propia, el sondeo, que comprueba un límite con peticiones controladas contra la API real, y la medición formal.

El mismo capítulo comprueba si las ocho Datasheets superan el validador de formato de este trabajo, que revisa los tipos de cada campo y las referencias entre bloques, y si PRIME4API puede consumirlas.

El **Capítulo 5** responde a la RQ2 (OBJ-03). Inventaría los 35 hechos del dominio que ACM2YAML v0.3 no puede expresar o que PRIME4API no consume, y cuenta en cuántas de las ocho APIs aparece cada uno. De ese inventario salen ocho propuestas de extensión del esquema, y la novena, del fallo de PRIME4API ante el precio `custom`, sin cifra publicada, del plan Enterprise de Resend. Cada una lleva el caso real que la motiva, como los seis volúmenes contratados que Postmark vende dentro de un mismo plan.

El **Capítulo 6** responde a la RQ3 (OBJ-04). Combina el *rate*, la cuota, el periodo de la cuota y los destinatarios por petición en una curva de capacidad: el máximo de peticiones que se pueden hacer desde el inicio hasta cada instante sin pasar la cuota ni el *rate*. Calcula, plan a plan, cuántas veces cabe la cuota en lo que el *rate* admite durante ese periodo. Ese cálculo a mano se contrasta con las respuestas de PRIME4API.

El **Capítulo 7** responde a la RQ4 (OBJ-05). Presenta un catálogo de 32 plantillas de pregunta de decisión y el procedimiento que las responde sobre las Datasheets, y mide su cobertura, la proporción de preguntas que se pueden responder. El procedimiento se aplica al caso de los 100.000 correos al mes de la Sección 1.1.3, y el capítulo termina con la comparación funcional de los planes.

Por último, el **Capítulo 8** reúne en una tabla la respuesta a cada pregunta de

investigación y el estado de cada objetivo y de cada contribución. Después agrupa las amenazas a la validez en cuatro ejes, extrae las lecciones del método agéntico y plantea el trabajo futuro.

El Anexo A declara el uso de inteligencia artificial en el trabajo y recoge la consulta a un modelo de lenguaje que completó la selección de las APIs del catálogo. Las Datasheets, sus fuentes y sus evidencias no se reproducen enteras en la memoria. Se entregan junto a ella en el repositorio de datos del trabajo[2].

[2] https://github.com/danvelcam/tfm-pricing-apis-correo.

2. Metodología y planificación

Este capítulo explica cómo se ha hecho el trabajo. Primero presenta la metodología (Sección 2.1). Después recoge la planificación, el presupuesto y las desviaciones respecto al plan (Secciones 2.2 a 2.4).

2.1. Metodología

El trabajo adopta el enfoque de investigación en ciencia del diseño (*Design Science Research*, DSR) formulado por Hevner et al. [26]. Se elige porque el objetivo de la Sección 1.3, un catálogo de Datasheets y un método de comparación, pide construir artefactos que resuelvan un problema relevante y evaluar con rigor su utilidad, y no describir un fenómeno. Para aplicar el enfoque se sigue el proceso DSRM (*Design Science Research Methodology*) de Peffers et al. [25], que ordena la investigación en seis actividades y admite iterar entre ellas.

El trabajo tiene cuatro contribuciones (C-01 a C-04). Tres son artefactos de alguno de los cuatro tipos que distinguen Hevner et al.: constructos, modelos, métodos e instanciaciones. El catálogo de Datasheets es una instanciación (C-01). Las extensiones al esquema son un modelo (C-02), construido sobre el ACM y su serialización ACM2YAML, de Gavira Sánchez [3]. El método de comparación sistemática es un método (C-04). El análisis de límites y *rates* (C-03) no es un artefacto: es lo que se aprende al aplicar las operaciones de PRIME4API al catálogo.

La Tabla 2.1 recorre las seis actividades del proceso DSRM. Para cada una dice qué se hace en este trabajo, en qué capítulo se recoge y a qué objetivo responde.

Cada Datasheet se construye con un ciclo de cinco fases: recolección de fuentes, borrador, validación, revisión y consolidación. La validación es automática, con el validador de formato, y la revisión examina el borrador campo a campo con las fuentes a la vista (Sección 4.2). El ciclo se probó y se ajustó con Postmark, la API piloto, antes de aplicarlo al resto de APIs. Las decisiones de modelado que se resuelven, como qué se considera cuota, se anotan como reglas en el libro de criterios, un fichero de reglas común a todas las Datasheets, y no se vuelven a plantear. Las decisiones de alcance se anotan en un registro, y de él sale la comparación entre lo previsto y lo ocurrido de la Sección 2.4.

El trabajo lo ejecutan agentes de inteligencia artificial basados en Claude, el asistente de Anthropic (Anexo A), y lo supervisa el autor. Los agentes recogen las fuentes, construyen cada Datasheet y la verifican, en flujos paralelos. El autor diseña el procedimiento, fija los criterios de modelado (como qué fuente prevalece cuando dos se contradicen), resuelve lo que queda abierto y valida el resultado. La verificación la hace un agente distinto del que construye la Datasheet, y reabre cada URL citada. La Datasheet de Postmark, la del piloto, se construyó antes de que se separaran la

construcción y la verificación, y se cerró sin informe de verificador. Las otras siete lo tienen. Postmark sí pasó por el validador de formato y por PRIME4API (Tabla 4.6).

Actividad	**En este trabajo**	**Dónde**	**Objetivo**
A1. Problema y motivación	Capacidad contractual dispersa en fuentes heterogéneas, e instrumental validado sobre pocos servicios	Caps. 1 y 3	—
A2. Objetivos de la solución	Preguntas RQ1 a RQ4 y objetivos específicos, contra los que se evalúa cada actividad posterior	Sec. 1.3	OBJ-01 a OBJ-06
A3. Diseño y desarrollo	Criterio de selección, ciclo de construcción por API, trazabilidad de cada valor a su fuente y escalera de validación. Sobre el catálogo terminado, las extensiones del esquema y el catálogo de preguntas de decisión	Caps. 4 y 5, Sec. 7.1	OBJ-01, OBJ-02 y OBJ-03
A4. Demostración	Las ocho Datasheets se ejecutan con PRIME4API y se preparan como contribución a SPHERE. Sobre ellas se analiza la capacidad y se aplica el método de comparación	Tabla 4.6, Sec. 4.6, Caps. 6 y 7	OBJ-04, OBJ-05 y OBJ-06
A5. Evaluación	Validador de formato, verificación por uso sobre PRIME4API y reapertura de cada URL citada. Criterio de promoción, que decide qué elementos no soportados (lo que el esquema no expresa o PRIME4API no consume) entran en el modelo, y clase de cada pregunta, que dice con qué se responde o qué falta para responderla	Caps. 4 a 7	Todos
A6. Comunicación	Esta memoria y el repositorio de datos abierto	Cap. 8	—

Tabla 2.1: Actividades del proceso DSRM [25], con lo que cada una hace en este trabajo, el capítulo que la recoge y el objetivo al que responde.

El límite del enfoque está en la evaluación. El ecosistema no tiene un validador oficial de Datasheets. El validador de formato es de este trabajo: contrasta cada Datasheet con la guía de ACM2YAML v0.3 y comprueba los tipos de cada campo y las referencias entre bloques, como que exista la cuota que cita un plan. La ejecución en PRIME4API comprueba que el motor consume la Datasheet y que sus respuestas coinciden con el cálculo a mano en las preguntas que lo permiten. La utilidad para quien elige un proveedor, que es lo que DSR pide evaluar, no se ha medido, porque no hay un estudio con integradores, los desarrolladores que conectan su aplicación a la API de un proveedor.

Paquete	Entregable	Objetivo	Rol	Horas
PT0. Gestión y seguimiento	Registro de decisiones, seguimiento del plan y repositorio versionado	—	Gestor de proyecto	20
PT1. Estudio previo	Estado del arte sobre modelado de pricings y de capacidad de APIs	—	Analista de sistemas	30
PT2. Selección y piloto	Criterio de inclusión, conjunto de APIs y Datasheet piloto	OBJ-01	Analista de sistemas	25
PT3. Construcción del catálogo	Una Datasheet por API, con fuentes, observación de panel y sondeos. Inventario de elementos no soportados por el esquema y propuestas de extensión	OBJ-02 y OBJ-03	Desarrollador	90
PT4. Límites y *rates*	Datasheets ejecutadas en PRIME4API y caracterización de la relación entre límites y *rates*	OBJ-04	Analista de sistemas	30
PT5. Comparación sistemática	Catálogo de preguntas de decisión y procedimiento para responderlas	OBJ-05	Analista de sistemas	25
PT6. Memoria y defensa	Memoria depositada y defensa	—	Gestor de proyecto	80
Total				**300**

Tabla 2.2: Paquetes de trabajo de la línea base, con su entregable, el objetivo específico de la Sección 1.3 al que responde, su rol y las horas previstas.

2.2. Planificación

El Trabajo Fin de Máster tiene 12 créditos ECTS [27]. A 25 horas por crédito, el mínimo del Real Decreto 1125/2003 [28], el esfuerzo previsto es de 300 horas. El proyecto empieza el 13 de febrero de 2026. La línea base reparte las horas entre siete paquetes de trabajo (Tabla 2.2). PT0 y PT6 son transversales, y PT1 a PT5 siguen el orden del método. Cada paquete tiene un entregable, el objetivo específico al que responde y un rol, que fija su tarifa en el presupuesto.

La línea base distribuye el trabajo entre febrero y septiembre (Figura 2.1): selección y piloto hasta el 15 de abril, catálogo hasta el 15 de julio, límites y comparación hasta el 31 de julio y memoria hasta el 30 de septiembre. La línea base se reconstruyó el 29 de septiembre de 2026: el reparto de horas y las ventanas son una asignación nominal del total, y el primer plan escrito con fechas es del 2 de agosto. Por eso, las desviaciones de calendario frente a ella miden la distancia a ese reparto y no el incumplimiento de un compromiso previo.

El 1 de septiembre se replanifica el trabajo restante con un calendario semanal. Ese calendario fija el cierre del catálogo el 28 de septiembre, la versión final de la memoria el 12 de octubre y el depósito el 15 de octubre. El 30 de septiembre, el calendario oficial de la tercera convocatoria sitúa el depósito el 2 de noviembre y la defensa entre el 9 y el 13 de noviembre de 2026.

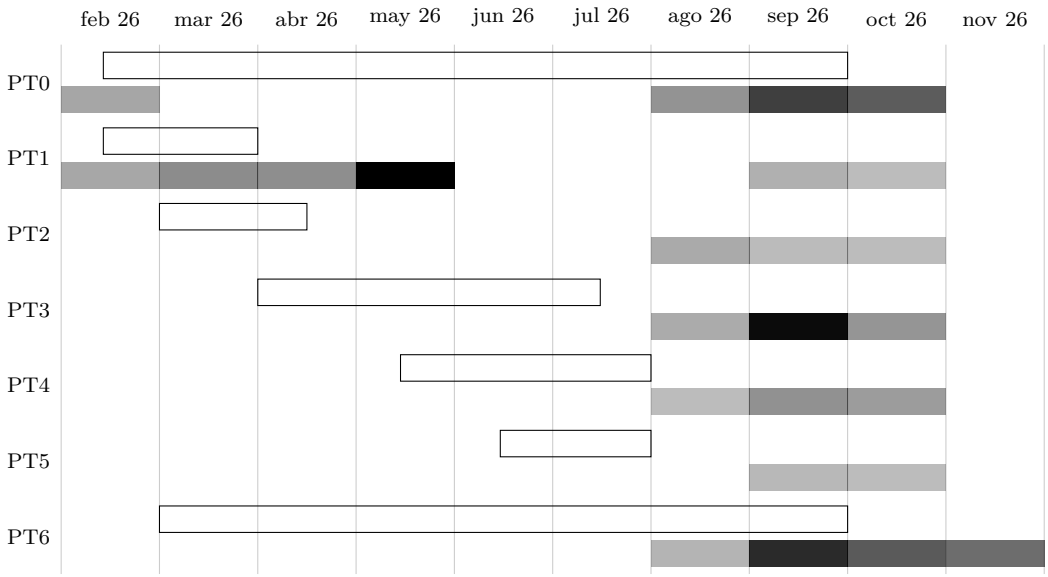


Figura 2.1: Cronograma del proyecto. Por cada paquete, la barra sin relleno es su ventana en la línea base, y las celdas rellenas de debajo son las horas al cierre de cada mes, más intensas cuantas más horas. Las de octubre y noviembre incluyen horas pendientes.

2.3. Presupuesto

El presupuesto tiene tres partidas: personal, amortización del equipo y costes fijos, que son licencias y suministros. Sobre el subtotal previsto se añade una contingencia del 10 %. El cálculo se apoya en las siguientes hipótesis.

Horas. Las previstas son las de la línea base. Las ejecutadas son las dedicadas por el autor hasta el 29 de octubre de 2026, incluida la revisión de lo que producen los agentes, y suman 199. De ellas, 109 se reconstruyen con la hora de cada mensaje del autor en los registros de sesión: una pausa de más de 45 minutos abre una sesión nueva, y cada sesión suma 15 minutos al final. Las otras 90 son estimaciones para los periodos sin registro. Las pendientes son las 15 que quedan para preparar la defensa. Las horas al cierre son las ejecutadas más las pendientes. No son horas reales, porque las pendientes son una previsión.

Tarifas. Cada rol se valora con la parte de costes directos, el 77,46 %, del precio medio de referencia que fija la Instrucción 1/2023 de la Agencia Digital de Andalucía [29]. Ese precio es el de una empresa contratista, y la parte de costes directos excluye sus gastos generales y su beneficio industrial. La cotización a la Seguridad Social ya está incluida.

Amortización. El equipo es un portátil Huawei MateBook D 16 (2024). Se toma como coste de adquisición el precio de partida de la gama, 799,00 € [30], con una vida útil de 60 meses. Se amortiza linealmente durante los 9 meses que van del inicio a la defensa.

Licencias. Se imputan dos: un mes de suscripción a Claude Max 5x, por su importe

facturado (90,00 €), y GitHub Pro, el plan individual de pago de GitHub, donde se aloja el repositorio de datos. GitHub Pro se valora a su precio de lista, aunque el autor no lo paga por ser estudiante: 4 USD al mes, el precio que GitHub fijó en abril de 2020 según su documentación [31]. Al tipo de cambio de referencia del Banco Central Europeo [32] del 30 de octubre de 2026 (1 € = 1,1172 USD), son 3,58 € al mes. GitHub Pro cuenta los mismos 8 meses que los suministros: el repositorio de datos se creó en octubre, pero el trabajo debió alojarse en GitHub desde el principio. Los proveedores del catálogo no suman coste, porque se usaron sus planes gratuitos o de prueba, abiertos a cualquiera.

Suministros. Internet (20,00 € al mes) y electricidad (25,00 € al mes) cuentan en los 8 meses con horas ejecutadas o pendientes, de febrero a mayo y de agosto a noviembre. PRIME4API y SPHERE se ejecutan en local, en el mismo equipo.

Costes que no dependen del número de horas. La amortización y los costes fijos entran igual en la columna prevista y en la de cierre. No se reparten entre lo ejecutado y lo pendiente.

La amortización resulta de

$$C_{\text{amortización}} = \frac{799{,}00}{60} \times 9 = 119{,}85\ € \tag{2.1}$$

Partida	**Horas prev./cierre**	**Previsto**	**Ejecutado**	**Pendiente**	**Al cierre**	**Desviación**
Gestor de proyecto (43,88)	100 / 102	4.388,00	3.817,56	658,20	4.475,76	+87,76
Analista de sistemas (35,09)	110 / 74	3.859,90	2.596,66	0,00	2.596,66	−1.263,24
Desarrollador (27,73)	90 / 38	2.495,70	1.053,74	0,00	1.053,74	−1.441,96
Personal	*300 / 214*	*10.743,60*	*7.467,96*	*658,20*	*8.126,16*	*−2.617,44*
Amortización del equipo		119,85			119,85	0,00
Claude Max 5x (1 mes)		90,00			90,00	0,00
GitHub Pro (8 meses)		28,64			28,64	0,00
Internet (8 meses)		160,00			160,00	0,00
Electricidad (8 meses)		200,00			200,00	0,00
Subtotal		11.342,09			8.724,65	−2.617,44
Contingencia (10 %)		1.134,21			—	
Total		**12.476,30**			**8.724,65**	**−3.751,65**

Tabla 2.3: Presupuesto previsto y al cierre, en euros sin IVA. Entre paréntesis, la tarifa de cada rol en euros por hora. El ejecutado valora las horas dedicadas hasta el 29 de octubre de 2026, y el pendiente, las previstas hasta la defensa. La columna al cierre es la suma de ambos y no incluye contingencia.

El subtotal al cierre se desvía −2.617,44 € del previsto (−23,1 %). Como los costes que no dependen del número de horas son iguales en las dos columnas, toda la diferencia es de personal (−24,4 %). Por roles, la desviación es de +2 horas en el gestor de proyecto, −36 en el analista y −52 en el desarrollador. El total al cierre no consume la contingencia y se desvía −3.751,65 € del total previsto (−30,1 %). De los 8.126,16 € de personal al cierre, 658,20 € corresponden a horas pendientes, todas del rol de gestor de proyecto.

2.4. Desviaciones

La ejecución se compara con la planificación según cuatro criterios por separado: calendario, horas, entregables y alcance. El resultado de un criterio no indica el de otro. Un paquete puede cerrar su entregable con menos horas de las previstas y fuera de su ventana.

Calendario. La Tabla 2.4 compara las fechas de los hitos con la línea base y con la replanificación del 1 de septiembre. Respecto a la línea base, el catálogo se completa 68 días más tarde y la memoria, 29 días más tarde. Respecto a la replanificación, el catálogo se adelanta 7 días, y la memoria y el depósito se retrasan 17 y 18 días. De las 199 horas ejecutadas, las de febrero a mayo son las de la reunión de arranque (4 horas) y las del estudio previo: 20 de lectura del estado del arte hasta abril y 30 de puesta al día en las líneas de investigación del grupo en mayo. Junio y julio no tienen horas, y el resto se concentra entre agosto y octubre (Figura 2.1).

Hito	Línea base	Replanif. 1 sep	Real o previsto	Días frente a línea base	Días frente a replanif.
Catálogo completo	15 jul	28 sep	21 sep	+68	−7
Memoria terminada	30 sep	12 oct	29 oct[a]	+29	+17
Depósito	—	15 oct	2 nov[a]	—	+18
Defensa	—	—	9–13 nov[a]	—	—

Tabla 2.4: Fechas de los hitos en 2026. La línea base no fija fecha de depósito ni de defensa. [a] Fecha prevista.

Horas. Las horas al cierre suman 214 frente a las 300 previstas, −86 horas (−28,7 %, Tabla 2.5). Suben la gestión (PT0) y el estudio previo (PT1), y bajan los paquetes PT2 a PT6. Las horas se asignan a cada paquete según el contenido de cada sesión de trabajo. Las actividades de PT4 y PT5 se ejecutaron dentro del ciclo de construcción de cada Datasheet y de la redacción de la memoria, y sus sesiones se asignan sobre todo a PT3 y a PT6. Por eso la desviación de PT3, PT4 y PT5 por separado no mide el esfuerzo de cada uno. Juntos pasan de 145 a 54 horas (−91, −62,8 %).

Las horas registran el tiempo del autor. El tiempo en que los agentes de inteligencia artificial trabajan sin intervención suya no se registra, así que el total no mide el esfuerzo completo del proyecto. Tampoco hay una medición del mismo trabajo sin agentes ni con el catálogo de diez a doce APIs de la línea base. Con estos datos, la reducción de horas no puede atribuirse a una causa.

Entregables. La Tabla 2.6 da el estado de cada entregable de la línea base y dónde está su evidencia. Cinco se completan, el del catálogo con ocho APIs en lugar de diez a doce. El de la comparación queda parcial y el de la memoria sigue en curso. Se añade un entregable que la línea base no preveía, la contribución a SPHERE.

Paquete	Previstas	Ejecutadas	Pendientes	Al cierre	Desv.	Desv. (%)
PT0. Gestión	20	47	0	47	+27	+135,0
PT1. Estudio previo	30	53	0	53	+23	+76,7
PT2. Selección	25	5	0	5	−20	−80,0
PT3 a PT5. Catálogo y análisis	145	54	0	54	−91	−62,8
PT6. Memoria	80	40	15	55	−25	−31,3
Total	300	199	15	214	−86	−28,7

Tabla 2.5: Horas previstas y al cierre por paquete de trabajo. PT3, PT4 y PT5 van en una sola fila porque sus actividades (construir el catálogo, ejecutarlo en PRIME4API y comparar las APIs) se ejecutaron dentro del mismo ciclo. Las ejecutadas llegan hasta el 29 de octubre de 2026, y las pendientes son previsión hasta la defensa.

Paquete	Entregable	Objetivo	Evidencia	Estado
PT0	Registro de decisiones y repositorio versionado	—	Repositorio de datos	Completado
PT1	Estado del arte	—	Capítulo 3	Completado
PT2	Criterio de inclusión, conjunto de APIs y Datasheet piloto de Postmark	OBJ-01	Secciones 4.1 y 4.2	Completado
PT3	Datasheets del catálogo, inventario de elementos no soportados y propuestas de extensión del esquema	OBJ-02 y OBJ-03	Sección 4.4, Capítulo 5 y repositorio de datos	Completado con ocho APIs de las diez a doce previstas
PT4	Datasheets ejecutadas en PRIME4API y relación entre límites y *rates*	OBJ-04	Sección 4.5 y Capítulo 6	Completado dentro del ciclo de PT3
PT5	Catálogo de preguntas de decisión y procedimiento de comparación	OBJ-05	Secciones 7.1 y 7.2	Parcial: las preguntas que exigen HARVEY4API no se ejecutan
—	Colección de las ocho APIs con su pricing en Pricing2YAML, una maqueta (prototipo web de cómo se vería la colección en SPHERE) y una propuesta de sección, en la ficha de cada API de la colección, para los términos de consumo que su pricing no recoge	OBJ-06	Sección 4.6	Añadido el 15 de octubre y cargado en SPHERE el 27 de octubre, en una colección privada, sin publicar
PT6	Memoria y defensa	—	Este documento	En curso: depósito el 2 de noviembre y defensa del 9 al 13

Tabla 2.6: Entregables de la línea base y su estado. La fila sin paquete es un entregable añadido después de la línea base.

Alcance. Las decisiones de modelado se registran como reglas en el libro de criterios y no cuentan como desviaciones. La Tabla 2.7 recoge las decisiones que cambian el alcance, con su fecha y el motivo que consta en el registro de decisiones.

Fecha	Previsto	Ejecutado	Motivo registrado
3 ago	Una versión nueva del esquema (v0.8 o v1.x)	Datasheets conformes a ACM2YAML v0.3. Los elementos no soportados se registran, y de ellos salen las propuestas de extensión (Capítulo 5)	Mantener las Datasheets consumibles por PRIME4API y SPHERE sin bifurcar el esquema
5 ago	Revisar con cuenta propia el panel del proveedor solo en algunas APIs	Se revisa el panel con cuenta propia en las ocho APIs, y ninguna Datasheet se cierra sin esa revisión (Sección 4.2)	Si unas APIs se revisan más que otras, las diferencias entre sus Datasheets reflejan el esfuerzo dedicado a cada una y no lo que publica cada proveedor
1 sep	Diez a doce APIs	Ocho APIs, con un mínimo de seis, cerradas sin sustituciones (Sección 4.1)	Se fija junto con el calendario de la replanificación
1 sep	Tres fuentes de selección: un trabajo previo, las APIs ya modeladas y una consulta a un modelo de lenguaje	Se ejecutan la segunda y la tercera, y la tercera sustituye a la primera (Sección 4.1)	La memoria del trabajo previo llegó el 14 de septiembre, con la selección ya hecha
6 sep	Las Datasheets, en anexos de la memoria	En el repositorio de datos. La memoria recoge los hallazgos	Los ficheros de datos se publican en un solo medio, el repositorio
7 sep	Ejecución sobre el despliegue público del ecosistema	PRIME4API en local, con las preguntas que ejecuta su motor (Sección 7.1). Las preguntas que exigen HARVEY4API quedan sin ejecutar	Sin acceso al despliegue público
15 oct	Sin colección en SPHERE. Pricing2YAML, fuera del alcance	Los ocho pricings en Pricing2YAML y una maqueta de la colección con el histórico de cada pricing (Sección 4.6)	Ampliación: la colección lleva el catálogo a la plataforma de la línea de investigación
23 oct	Cinco objetivos específicos. La recogida de los términos de consumo, implícita en el catálogo	OBJ-02 nombra la recogida sistemática de los términos de consumo, y OBJ-06 recoge la contribución a SPHERE (Sección 1.3)	Reformulación de objetivos para que reflejen el alcance ejecutado

Tabla 2.7: Desviaciones de alcance, por fecha de decisión en 2026. Cada fila recoge lo previsto, lo ejecutado y el motivo que consta en el registro de decisiones.

Dos decisiones reducen el alcance: el catálogo de ocho APIs y las preguntas que exigen HARVEY4API, que quedan sin ejecutar. Dos lo amplían: la contribución a SPHERE y la recogida sistemática de los términos de consumo como parte de OBJ-02. Las otras cuatro cambian cómo se ejecuta o se presenta lo previsto, no cuánto se hace.

Lo que queda fuera pasa a trabajo futuro. La primera tarea es ejecutar las preguntas que exigen HARVEY4API. La segunda está en la maqueta: la ficha de cada API tiene dos pestañas, la del pricing y la del resto de los términos de consumo, y cada una guarda sus versiones por separado, así que falta una línea temporal única para toda la ficha.

Las dos últimas tareas tratan de *skills*, instrucciones que sigue un agente basado en un modelo de lenguaje para leer los términos de consumo. La tercera es una *skill* que interprete la diferencia entre dos versiones de esos términos, pricing incluido, como un cambio del dominio, por ejemplo un descuento anual. La cuarta es publicar las *skills* en un mercado en línea (*marketplace*) y conectarlas con SPHERE.

3. Estado del arte

3.1. Modelado y análisis de pricings de SaaS

El pricing de un servicio SaaS es la estructura de planes, características (*features*) y complementos (*add-ons*) con la que el proveedor regula el acceso a su servicio. Postmark, uno de los ocho proveedores de correo transaccional que estudia este trabajo, publica cuatro planes en su página de precios (Figura 3.1a): Free, por 0 USD, y Basic, Pro y Platform, que con 10.000 correos al mes cuestan 15, 16,50 y 18 USD [33]. Pro añade sobre Basic, entre otras, una característica: el procesado del correo entrante. La IP dedicada, una dirección IP de envío reservada a un solo cliente, es un complemento, desde 50 USD al mes por IP.

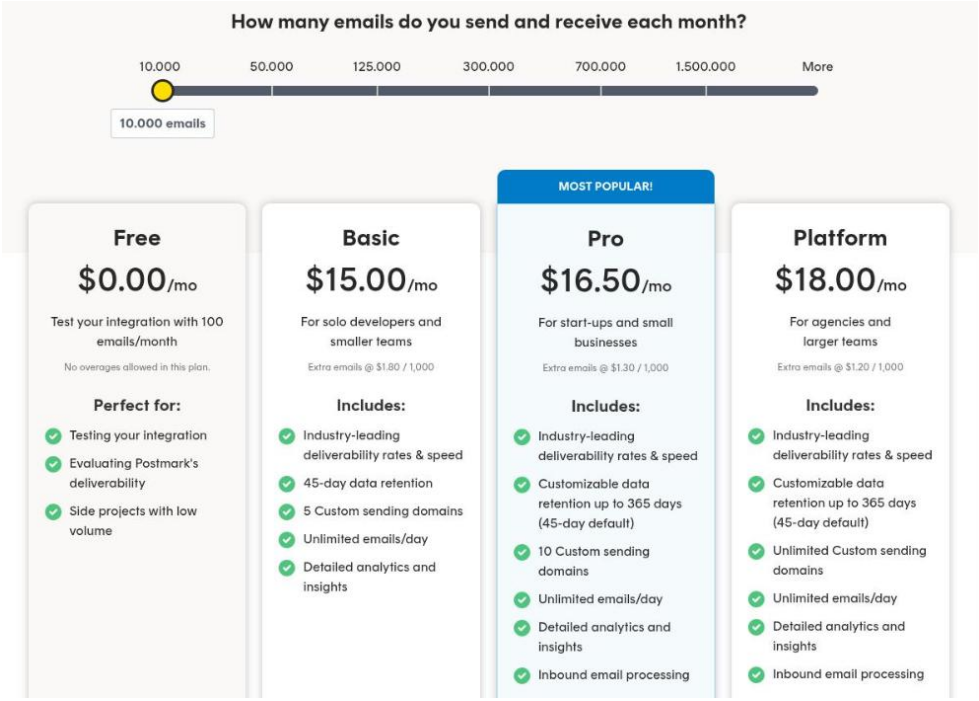
Junto al pricing, la página de Postmark publica los límites de uso de cada plan, que fijan cuánto correo entra en él. Free incluye 100 correos al mes y no admite pasar de ellos. En Basic, Pro y Platform, el volumen más bajo que se contrata es de 10.000 correos, y la página deja elegir volúmenes mayores (Figura 3.1a). Con 10.000 correos, cada millar de más se cobra a 1,80, 1,30 y 1,20 USD (el sobrecoste) [33].

Además de fijar lo que se vende, el pricing condiciona el diseño, la operación y la evolución del propio sistema. En la operación dirigida por pricings (*pricing-driven DevOps*), un cambio de pricing puede adaptar el *frontend* y el *backend* del sistema sin redesplegarlo [34].

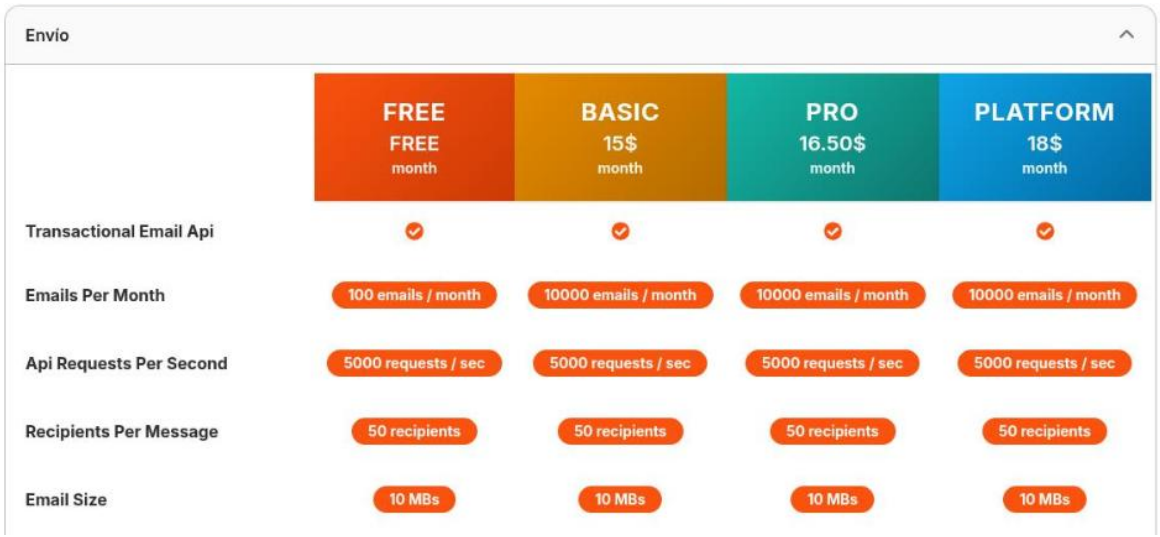
Un estudio de García-Fernández et al. sobre más de 150 pricings de una treintena de SaaS comerciales a lo largo de seis años documenta que su complejidad crece de forma sostenida y que sus espacios de configuración (el conjunto de combinaciones válidas de plan y complementos que admite un pricing) llegan a miles de combinaciones [35]. Gestionar a mano estructuras de ese tamaño no es viable, y por eso los pricings se formalizan.

La respuesta a ese crecimiento de la complejidad sigue una aproximación asentada en la investigación realizada por el grupo ISA: convertir un artefacto informal en un modelo estructurado para someterlo a análisis automatizado. El análisis automatizado empezó en el ámbito de los servicios [36] y se ha aplicado también a los procesos de negocio [37] y a los modelos de características, que describen qué combinaciones de características admite una familia de productos software [38]. Analizar un modelo de forma automatizada es responder preguntas sobre él con una herramienta, y cada tipo de pregunta es una operación de análisis. En un modelo de características, una operación de análisis cuenta cuántos productos distintos representa el modelo.

En los pricings, ese modelo estructurado es el metamodelo **iPricing**: un modelo que fija cómo se describe cualquier pricing. Se propuso con el nombre Pricing4SaaS [39] y se consolidó después con su denominación actual [5]. iPricing estructura un pricing en planes, complementos y características. Las características están sujetas a límites de uso, que son renovables si se reinician cada periodo, como las llamadas mensuales a una API, o no renovables si el recurso se agota al consumirse.



(a) Página de precios de Postmark, capturada el 1 de noviembre de 2026 [33].



(b) El pricing de Postmark escrito en Pricing2YAML por este trabajo a partir de su Datasheet, con límites que la página de precios no publica, y representado en SPHERE [22].

Figura 3.1: El pricing del servicio de correo transaccional de Postmark en su página de precios (a) y en SPHERE (b).

La serialización de iPricing, Pricing2YAML, escribe cada pricing en YAML, un formato portable que pueden leer herramientas distintas [34]. La Figura 3.1b muestra el pricing de Postmark escrito en Pricing2YAML y representado en SPHERE (*SaaS Pricing Holistic Evaluation and Regulation Environment*), la plataforma web del ecosistema que reúne pricings en colecciones [3, 22]. Postmark se consume por API, pero su página de precios tiene la forma de las del canal web, y Pricing2YAML recoge sus planes, características y complementos. Los límites propios del canal API, como el *rate* de la API REST, se escriben en él igual que la cuota mensual, sin nada que los distinga de ella.

Sobre iPricing, García-Fernández et al. presentaron en la conferencia CAiSE 2025 (*Conference on Advanced Information Systems Engineering*) siete operaciones de análisis (cardinalidad del espacio de configuración, filtrado, enumeración de suscripciones, coste, validación del pricing y de la suscripción, y suscripción óptima) [5]. Una suscripción es la configuración que contrata un consumidor: un plan con sus complementos, si los hay.

Las siete operaciones se resuelven traduciendo el pricing a un problema de satisfacción de restricciones con optimización (CSOP): unas variables con sus dominios, unas restricciones entre ellas y una función objetivo que se optimiza, como el coste. Los autores las validan sobre más de 150 pricings reales y detectan inconsistencias en 35 de ellos [5].

En torno a iPricing se ha construido un ecosistema de herramientas interoperables, que Cavero et al. presentan como la línea de *Pricing Intelligence* [1]. A-MINT automatiza la extracción: toma la URL de la página de pricing de un SaaS y devuelve un iPricing válido en Pricing2YAML. Lo hace con un pipeline de modelos de lenguaje que valida y corrige su propia salida en bucle. Según sus autores, alcanza una F1 del 70,51 % en la extracción (la media armónica de la precisión y la exhaustividad), y el 98,73 % de los modelos que genera supera la comprobación de conformidad con Pricing2YAML [40]. La evaluación se hizo sobre SPECTRA, un corpus público con los pricings de una treintena de SaaS entre 2019 y 2024 [41].

PRIME (*PRicing Intelligence and Modelling Engine*) es el motor que ejecuta las operaciones de análisis sobre el modelo [5]. HARVEY, presentado en CAiSE 2026, es el agente conversacional que traduce preguntas en lenguaje natural a llamadas a herramientas como A-MINT y PRIME, y sintetiza los resultados [1].

El canal API copia del canal web la idea de un artefacto central sobre el que trabajan todas las herramientas. En el canal web ese artefacto es el pricing formalizado en iPricing, una representación común de planes, características, complementos y límites, independiente de la implementación. En el canal API es la Datasheet. El modelo, en cambio, no se reutiliza: iPricing se concibió para el canal web y no representa *endpoints*, *rates* por ventana temporal ni reglas de consumo por petición. Gavira Sánchez concluye que canales distintos requieren modelos distintos [3], y por eso el canal API tiene su propio modelo, el ACM.

3.2. Modelado y análisis de la capacidad de las APIs

En el canal API la situación de partida es asimétrica. La superficie funcional de una API REST [42] tiene un estándar de descripción maduro, la especificación OpenAPI [43], que documenta rutas, métodos, parámetros y respuestas. Los términos de consumo no tienen un equivalente asentado: cuántas peticiones admite un *endpoint* por unidad de tiempo, qué volumen mensual permite cada plan y qué ocurre al superarlo. Son tres de los elementos principales que caracterizan el consumo de una API.

El análisis de Gámez-Díaz et al. sobre 69 ofertas comerciales de APIs RESTful muestra que las cuotas y los *rates* son las limitaciones dominantes y que la métrica más habitual es el número de peticiones [44]. Los modelos de negocio son planes escalonados, con sobrecoste por exceso o sin él. Un estudio posterior, hecho con actores industriales de la gestión de APIs, analiza el papel de esas limitaciones y de los acuerdos de nivel de servicio (SLA, por sus siglas en inglés) que las contienen, y constata que no existe una representación estándar y procesable de ellas [45].

Los intentos de formalizar las cuotas y los *rates* heredan de las especificaciones de acuerdos de nivel de servicio, como WS-Agreement, pensada para servicios web [46]. SLA4OAI traslada esos acuerdos a OpenAPI como extensión de su especificación [47]. Ancla las limitaciones (cuotas y *rates*, con umbrales, periodos y costes por operación o por exceso) a las mismas rutas y métodos que el documento OpenAPI ya describe.

La sintaxis de SLA4OAI se apoya en **Pricing4APIs**, el modelo de pricings de APIs RESTful de Fresno-Aranda et al. [48]. Organiza la información en planes, limitaciones y capacidad, entendida como el máximo que el proveedor puede ofrecer, del que las limitaciones son fracciones. El modelo incorpora una operación de validación con criterios jerárquicos (del límite individual al pricing completo). Esa operación detecta inconsistencias como límites inalcanzables o planes más caros menos generosos que otros más baratos [48].

Gavira Sánchez destaca cuatro rasgos de Pricing4APIs que explican las decisiones de diseño del ACM [3]. Aquí se recogen los tres que tratan de la capacidad y sus límites. El cuarto son los criterios con los que el modelo detecta inconsistencias en un pricing. Primero, la capacidad es unidimensional. Se reduce a un valor máximo de *throughput* (la cantidad de trabajo que el servicio atiende en un tiempo dado) sobre la métrica de peticiones. Restricciones reales como la concurrencia o el tamaño de la respuesta quedan fuera de su semántica.

El segundo rasgo es que la distinción entre cuota y *rate* se delega en el *limitador*, el componente del proveedor que lleva la cuenta de las peticiones recibidas y rechaza las que exceden lo permitido. Si el limitador cuenta sobre una ventana fija, que empieza y acaba en momentos predeterminados, el límite es una cuota. Si cuenta sobre una ventana deslizante, que se abre con la primera petición, es un *rate*. Es, en palabras de Gavira Sánchez, «un detalle de implementación que la mayoría de proveedores no publica» [3]. Por eso, con solo la documentación, no siempre se puede saber si un límite publicado es cuota o *rate*.

El tercer rasgo es que el plan es el único eje de segmentación: cualquier variación en las condiciones de dos consumidores exige planes distintos [3, 48]. En la API de GitHub, por ejemplo, el límite pasa de 60 a 5.000 peticiones por hora cuando el consumidor se autentica [49], y en Pricing4APIs esa diferencia exige dos planes [3].

La gestión de la capacidad es uno de los procesos de la gestión de servicios TI [50]. En las APIs, la capacidad que interesa al consumidor es la contractual, la que el proveedor se compromete a dejar consumir, y no la de la infraestructura. Se estudia con *operaciones de análisis de la capacidad*: cómputos formales que evalúan el uso de una API bajo una carga determinada. Hay dos aproximaciones, que difieren en cómo entienden la capacidad y en cómo resuelven el análisis.

La primera aproximación, de Fresno-Aranda et al., opera sobre arquitecturas de microservicios [51] que consumen APIs externas reguladas por planes con limitaciones. Los autores llaman a esas arquitecturas LAMA (*Limitation-Aware Microservice Architecture*). La capacidad es el máximo volumen de trabajo que la arquitectura completa puede atender sin exceder ninguna limitación externa. Smart LAMA, la API REST que los autores construyen para ellas, traduce la topología (un grafo de servicios propios y APIs consumidas) a un CSOP. Resolverlo responde a operaciones como el máximo de peticiones servibles en un periodo y coste dados, el coste mínimo para un volumen dado o el tiempo mínimo para completarlo [52, 53, 54].

La línea de Fresno-Aranda et al. aporta también un catálogo de once operaciones de análisis centradas en el consumidor individual de una API [55]. El catálogo las reparte en cinco escenarios según el punto de partida, y el último es el del prosumidor, una organización que consume APIs externas y, a la vez, ofrece su propio servicio a terceros. La Tabla 3.1 enumera las once por escenario, tal como las describe Gavira Sánchez. Según él, el catálogo se definió sin implementación de referencia [3].

La segunda aproximación es geométrica. Parte de la visión de Antonio Ruiz-Cortés y Miguel Ruiz-López, que estudian la gestión de la capacidad de las APIs con un enfoque de caja negra [2]: la API se trata como un sistema del que se ve lo que entra y lo que sale, sin saber cómo aplica el proveedor sus límites por dentro. Con esa visión, el *rate* se interpreta como la velocidad máxima de consumo y la cuota como el volumen máximo acumulable en un periodo [56].

En una gráfica de peticiones acumuladas frente al tiempo, el *rate* es una recta cuya pendiente es la velocidad máxima de consumo, y la cuota, una recta horizontal a la altura del volumen máximo [56]. Los dos límites operan a la vez: la capacidad efectiva es lo que queda por debajo de las dos rectas, y el punto en que se cortan marca el instante en que se agota la cuota si se consume a la velocidad máxima.

La biblioteca Pricing4API implementa la lectura geométrica de la capacidad. Su nombre se inspira en Pricing4APIs, el modelo que Fresno-Aranda desarrolló en su tesis doctoral [54], pero es otro trabajo [3]. Su primer prototipo lo desarrolló Daniel Ruiz López, y Gavira Sánchez la extendió en su Trabajo Fin de Grado [20, 56].

El *BoundedRate* de Pricing4API agrupa un *rate* y una o varias cuotas interdependientes. Sobre él se define la **curva de capacidad** $C(t)$: el máximo número de peticiones realizables desde el instante inicial hasta t sin violar ninguna restricción. La Figura 3.2 muestra la curva de un plan con un *rate* de una petición cada dos segundos

Nº	Operación de análisis
Dada una suscripción concreta: el consumidor ya está suscrito	
1	Calcular el tiempo requerido para enviar N peticiones
2	Verificar si es viable mantener simultáneamente un *rate* y una cuota dados, identificando el motivo en caso de no serlo
3	Determinar la velocidad máxima de envío y su duración máxima sostenida
4	Estimar el tiempo de espera tras agotar su cuota
Dado un presupuesto máximo: el consumidor aún no está suscrito	
5	Identificar la combinación óptima de planes que maximice su capacidad sin exceder el límite económico establecido
Dada una capacidad deseada	
6	Encontrar la suscripción más barata que le garantice un volumen mínimo de peticiones
7	Cuantificar qué porcentaje de esa capacidad queda cubierto por su suscripción actual
Dado un conjunto de features deseadas	
8	Buscar la suscripción de menor coste que integre las funcionalidades requeridas
9	Evaluar el porcentaje de cumplimiento de esas características bajo el plan vigente
Dado un pricing deseado: el consumidor es prosumidor	
10	Buscar la suscripción a la API externa más económica que soporte el modelo de precios propio
11	Calcular la proporción de ese modelo de precios que resulta cubierta por la suscripción actual

Tabla 3.1: Las once operaciones de análisis del catálogo de Fresno-Aranda et al. [55], agrupadas por los cinco escenarios de partida. Descripción de Gavira Sánchez [3], Sección 3.3.2, con los verbos en infinitivo; llama *features* a las características.

y una cuota de 1.000 peticiones por hora. La curva sube con la pendiente del *rate* hasta que la cuota la corta con un techo horizontal [3].

La curva de capacidad tiene dos ventajas: se lee de un vistazo en una gráfica, y evaluarla en un punto cuesta menos que resolver un CSOP. Sobre ella la biblioteca responde preguntas de tiempo mínimo, capacidad acumulada, agotamiento de cuota o rentabilidad entre planes [56].

Las dos aproximaciones comparten dos características que delimitan su alcance. La primera es la entrada. Smart LAMA asume los pricings ya descritos conforme a Pricing4APIs. La biblioteca Pricing4API no contaba con un artefacto de entrada estructurado: cada plan se codificaba a mano en Python, con los parámetros copiados uno a uno de la documentación del proveedor. Ninguna de las dos responde de dónde salen, de forma trazable y a escala, las cuotas y los *rates* que analiza.

La segunda característica común es que la interacción entre límites de uso y *rates* queda definida como semántica del modelo (las restricciones del CSOP en Smart LAMA, la curva del *BoundedRate* en Pricing4API), pero no caracterizada sobre ningún dominio real. Ninguno de estos trabajos estudia qué formas adopta esa relación en las

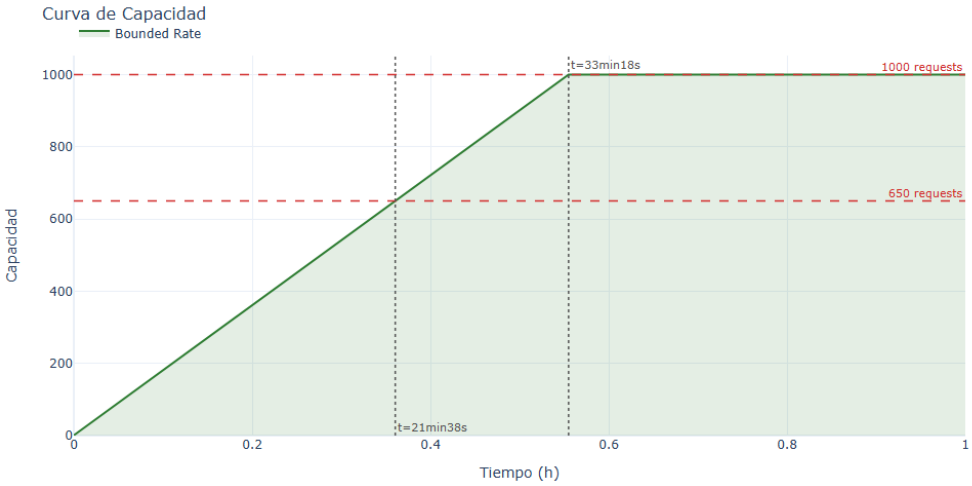


Figura 3.2: Curva de capacidad de un plan con un *rate* de una petición cada dos segundos y una cuota de 1.000 peticiones por hora. La pendiente es el *rate*, y el techo horizontal, la cuota. La línea discontinua inferior marca un objetivo de 650 peticiones, que se alcanza a los 21 minutos y 38 segundos. La cuota se agota a los 33 minutos y 18 segundos. Reproducida del Trabajo Fin de Máster de Gavira Sánchez [3], Figura 3.9.

APIs de un sector concreto ni qué consecuencias tiene para el consumidor.

3.3. El ecosistema SPHERE

Iansiti y Levien, que estudian los ecosistemas de negocio, llaman *keystone* al actor central de un ecosistema, del que dependen los demás porque les ofrece los recursos comunes sobre los que construyen su propia oferta [57]. En el ecosistema de GitHub ese papel lo tiene la plataforma de repositorios GitHub. En el ecosistema de *Pricing Intelligence* del grupo ISA lo tiene SPHERE, la plataforma web que reúne los pricings y da acceso a las herramientas del canal web [3, 22]. Gavira Sánchez la llama el *hub* del ecosistema, y su extensión al canal API reúne también las Datasheets [3]. Por eso esta memoria habla del ecosistema SPHERE.

En el ecosistema SPHERE funcionan artefactos y herramientas de los dos canales, y cada pieza del canal web tiene su par en el canal API [3]. Los artefactos son el modelo (iPricing y el ACM), su serialización (Pricing2YAML y ACM2YAML) y el documento que describe la oferta de un proveedor concreto (el pricing y la Datasheet). Las herramientas son la extracción automática (A-MINT y A-MINT4API), el motor de análisis (PRIME y PRIME4API) y el agente conversacional (HARVEY y HARVEY4API). En el canal API, SPHERE da acceso a HARVEY4API. El resto de la sección describe las piezas del canal API. De ellas, este trabajo aplica y extiende el ACM, ACM2YAML, la Datasheet, PRIME4API y HARVEY4API.

El modelo, su serialización, el motor y el agente del canal API proceden de dos Trabajos Fin de Máster coordinados del mismo grupo. El de Gavira Sánchez aporta el modelo de capacidad, su serialización y el motor de análisis, y se defendió pocos meses

antes que este [3]. El de Daniel Ruiz López, desarrollado en paralelo, se centra en el agente conversacional HARVEY4API y comparte la autoría del modelo [4]. Los dos son el punto de partida operativo de este trabajo.

El **ACM** (*API Capacity Model*) reconceptualiza la capacidad para superar los tres rasgos señalados antes: la capacidad unidimensional, la distinción entre cuota y *rate* delegada en el limitador y el plan como único eje de segmentación. Define la capacidad como un conjunto de *dimensiones* independientes: *Volume* o volumen (las cuotas), *Frequency* o frecuencia (los *rates*), *Concurrent Requests* o peticiones concurrentes y *Results Per Request* o resultados por petición. La dimensión fija qué aspecto del consumo se limita, y la métrica, en qué unidad se cuenta. Una misma métrica, como las peticiones, puede aparecer en varias dimensiones [3].

A las cuatro dimensiones se añade el CRF (*Consumption Request Factor*), la regla que declara cuántas unidades de cada métrica gasta una petición. En una API de correo, una misma llamada consume una petición y, a la vez, tantos correos de la cuota mensual como destinatarios incluya [3]. El ACM incorpora también la lectura geométrica, la curva de capacidad sobre el *BoundedRate* (Figura 3.2).

En lugar de delegar en el limitador, el ACM clasifica los límites comparándolos entre sí. De los que una API publica, el de menor unidad temporal es el *rate*, que gobierna la velocidad, y el de mayor unidad es la cuota, que gobierna el volumen. La API de GitHub cuenta parte del consumo en puntos: una petición GET gasta uno y una POST, cinco [49]. Con este criterio, su *rate* es el límite de 900 puntos por minuto y su cuota, el de 5.000 peticiones por hora, sin conocer el mecanismo interno del proveedor [3].

La serialización del modelo es **ACM2YAML**, y cada API se describe en ella con una **Datasheet**: un documento orientado a máquinas que reúne en una sola representación la capacidad de la API. Esa capacidad está dispersa entre la documentación de desarrollador, los artículos de soporte, las especificaciones OpenAPI y las políticas de uso aceptable [3].

La Datasheet combina los conceptos de capacidad con los elementos de pricing que los contextualizan. Los primeros son las cuotas, en el bloque `capacity`, cada una con su sobrecoste (`overage_cost`), los *rates*, en `max_power`, y el CRF, en el `workload` de cada *endpoint*. Los segundos forman la jerarquía Plan → Endpoint → Alias. Un alias es un modo de consumo distinto de una misma operación, cuyas restricciones cambian con una condición que no es el plan, como el estado de reputación de la cuenta. La jerarquía asocia cada restricción al nivel al que aplica sin crear planes artificiales, y así resuelve el tercer rasgo, el plan como único eje de segmentación [3].

La versión operativa, ACM2YAML v0.3, captura dos de las cuatro dimensiones del modelo (*Volume* y *Frequency*, además del CRF) y deja fuera *Concurrent Requests* y *Results Per Request*. Gavira Sánchez la califica de versión operativa parcial, priorizada para poner en marcha el ecosistema antes que para agotar la expresividad del modelo [3].

Sobre la Datasheet operan las herramientas del canal API. **PRIME4API** refactoriza la biblioteca Pricing4API como API REST y convierte el análisis en un proceso dirigido por la Datasheet: cada pregunta se reduce a operaciones sobre el *BoundedRate*

y la curva de capacidad [3]. Junto al motor, Gavira Sánchez define diez preguntas de análisis en tres categorías: la capacidad de una suscripción, la suscripción óptima y el equilibrio entre cuotas heterogéneas. La Tabla 3.2 las recoge. Gavira Sánchez las responde con PRIME4API sobre tres pricings de dos servicios (SendGrid, en dos versiones, y la API de datos bibliográficos DBLP) y usa los resultados como respuestas de referencia (*ground truth*) para validar las herramientas que consumen el motor [3].

N.º	**Pregunta**
Análisis de la capacidad de una suscripción	
1	Dado un plan o un pricing, ¿cuánto tiempo mínimo se necesita para consumir un volumen objetivo V?
2	Dado un plan o un pricing, ¿a qué velocidad máxima instantánea se puede consumir y en qué instante se agota la cuota?
3	Dado un plan o un pricing y una ventana de duración W, ¿cuánta capacidad se acumula durante ese intervalo?
4	Dado un plan o un pricing, ¿cuánto tiempo hay que esperar tras agotar la cuota para poder volver a consumir?
5	Dado un plan o un pricing y una demanda definida por un *rate* R y una cuota Q, ¿puede sostenerse ese consumo durante un período P?
Selección de la suscripción óptima	
6	Dada una capacidad deseada V, ¿cuál es la suscripción más barata que la cubre, con o sin *overage cost*?
7	Dada una capacidad deseada V y un presupuesto máximo B, ¿cuál es la suscripción más barata que cubre V sin exceder B?
Equilibrio entre cuotas heterogéneas	
8	Dado un plan o un pricing con dos cuotas y un CRF, ¿cuál de las cuotas se agota primero?
9	Dado un plan o un pricing con dos cuotas, ¿qué valor de CRF hace que ambas se agoten simultáneamente?
10	Dado un plan o un pricing, un CRF y un volumen objetivo que excede una de las cuotas, ¿cuánto cuesta completarlo incurriendo en *overage*?

Tabla 3.2: Las diez preguntas de análisis de Gavira Sánchez, por categorías. Formulación literal de su Trabajo Fin de Máster [3], Sección 5.4.1. *Overage* es el consumo por encima de la cuota, y *overage cost*, el sobrecoste que se paga por él.

La tercera categoría, el equilibrio entre cuotas heterogéneas, trata las cuotas de un mismo plan que cuentan métricas distintas, como las peticiones y los correos. Es la más próxima a este trabajo, porque en el correo una petición gasta tantos correos de la cuota como destinatarios lleve. Sus preguntas son las tres últimas de la Tabla 3.2. Esa categoría no tiene correspondencia en el catálogo de operaciones de Fresno-Aranda, porque su modelo carecía del CRF [3].

HARVEY4API es el agente conversacional del canal API: publica las operaciones de PRIME4API como herramientas mediante el *Model Context Protocol* (MCP), el protocolo con el que un modelo de lenguaje llama a herramientas externas [4, 21]. A-MINT4API extrae Datasheets de forma automática [3].

Según Gavira Sánchez, la cadena completa (extracción, modelo, motor, agente y *hub*) funciona sobre ACM2YAML v0.3, y él mismo declara lo que le falta [3]. Las

respuestas de referencia de sus diez preguntas de análisis se calculan sobre tres pricings, y las Datasheets de GitHub, SendGrid, DBLP y MailerSend sirvieron de banco de pruebas durante el diseño. Las dos dimensiones que v0.3 no captura carecen de serialización. Los elementos de pricing de la Datasheet se eligieron por lo que necesitaban las herramientas, sin un modelo formal de la estructura comercial de una API. Integrar capacidad y estructura comercial queda como trabajo futuro, igual que completar la serialización y someterla a validación formal.

La segunda pregunta de investigación de este trabajo, la RQ2, responde a una de esas líneas de trabajo futuro: completar la serialización ACM2YAML, que Gavira Sánchez deja pendiente de forma explícita [3]. El enunciado de la RQ2 es «¿Es ACM2YAML suficientemente expresivo para representar la capacidad contractual de las APIs de correo? ¿Qué extensiones requiere?» (Sección 1.3).

Las otras tres preguntas de investigación, formuladas en la misma sección, salen de lo que el análisis de la capacidad deja abierto. La RQ1 pregunta qué APIs de correo deben incluirse en un análisis representativo del dominio y bajo qué criterios reproducibles, porque el modelo no se ha validado sobre un dominio entero. La RQ3 pregunta cómo se relacionan los límites de uso y los *rates* en las APIs de correo y qué implicaciones tiene esa relación para el consumidor, porque esa relación no se ha caracterizado.

La RQ4 pregunta cómo pueden compararse las APIs de correo de forma sistemática para responder a preguntas de decisión que involucran simultáneamente requisitos funcionales y no funcionales, porque la comparación existente entre APIs parte de una demanda dada.

La Tabla 3.3 separa, pieza a pieza, lo que ya existía de lo que aporta cada contribución de este trabajo. En la fila de HARVEY4API, las preguntas de decisión del método de comparación (C-04) que piden el agente cuentan como respondibles según la especificación que publica Ruiz López, porque el agente no se ha ejecutado en este trabajo.

Pieza	Lo que ya existía	Lo que aporta este trabajo
ACM y ACM2YAML	Modelo de capacidad en cuatro dimensiones más el CRF, y su serialización v0.3, que captura dos de ellas. Gavira Sánchez, con Ruiz López como coautor del modelo [3, 4].	Extensión del esquema justificada caso a caso (C-02).
Datasheet	Documento que reúne la capacidad de una API. Gavira Sánchez [3].	Catálogo de Datasheets de APIs de correo (C-01).
PRIME4API	Motor de análisis de la capacidad dirigido por la Datasheet, con diez preguntas de análisis. Gavira Sánchez [3].	Caracterización de la relación entre límites y *rates* en el correo (C-03).
HARVEY4API	Agente conversacional que publica las operaciones del motor como herramientas MCP. Ruiz López [4].	Método de comparación sistemática (C-04). Las preguntas que exigen el agente se clasifican por especificación.
SPHERE	*Hub* del ecosistema [22]. Su extensión al canal API, con colecciones de Datasheets, la programó un técnico de investigación con los requisitos de Gavira Sánchez y Ruiz López [3].	Catálogo preparado como contribución a SPHERE (C-01).

Tabla 3.3: Piezas del canal API que usa este trabajo: lo que ya existía, con su autoría, y lo que aporta este trabajo.

3.3.1. La plataforma SPHERE

SPHERE es la plataforma web del grupo ISA desde la que se usan las herramientas de *Pricing Intelligence* sin instalar ni configurar nada en local. Es a la vez el repositorio compartido de los pricings que modelan sus usuarios y el punto de acceso público a las demás herramientas [3, 22].

Para los pricings del canal web, Gavira Sánchez describe tres funciones de SPHERE [3]. Las colecciones públicas agrupan los pricings. Según esa descripción, el editor de Pricing2YAML comprueba las mismas reglas de consistencia sobre las que razona PRIME y muestra el pricing a medida que se edita. HARVEY se consulta desde la propia plataforma, sin configurar el agente aparte. SPHERE tiene otras dos funciones: dentro de una colección, cada pricing aparece en una tarjeta con el tamaño de su espacio de configuración, y la plataforma conserva las versiones de los pricings que suben sus usuarios [22].

La extensión de SPHERE al canal API la programó un técnico de investigación a partir de los requisitos que Gavira Sánchez y Ruiz López definieron y supervisaron [3]. Su código está en una copia aparte (*fork*) del repositorio original de la plataforma [58]. Añade una sección de Datasheets organizada en colecciones y una página para cada

Datasheet. En esa página, cada plan es una tarjeta con su precio y su *rate*, y debajo van los *endpoints* con sus límites. La plataforma guarda cada cambio de la Datasheet como una versión fechada. Desde SPHERE se llega a HARVEY4API de dos formas: por el menú de herramientas o desde la página de cada Datasheet.

El editor de Datasheets de la extensión muestra el YAML a un lado y, al otro, la Datasheet representada, que se actualiza al momento. Gavira Sánchez escribe que cada cambio «se valida contra la sintaxis del modelo» [3]. En la versión del código del 10 de junio de 2026, esa comprobación consiste en que el texto se pueda leer como YAML [58]. Si no se puede leer, el editor muestra el error. Si se lee, lo representa sin comprobar los campos ni los tipos de ACM2YAML. No se ha revisado qué comprueba la versión de la plataforma que está en línea. Este mismo editor se usó para validar visualmente las Datasheets que genera A-MINT4API: se cargaban y se comprobaba que se representaban bien [3].

Este trabajo prepara para SPHERE una colección con las ocho APIs del catálogo, cada una con su Datasheet y su pricing en Pricing2YAML. Los ocho pricings están cargados desde el 27 de octubre de 2026, en una colección privada hasta el depósito de esta memoria, y la Figura 3.1b muestra el de Postmark. Este trabajo propone también una sección de términos de consumo para la ficha de cada API, su página dentro de la colección.

3.4. Las APIs de correo

Un correo transaccional se define como un mensaje individual, único para cada destinatario, que este espera recibir y que suele desencadenar el propio usuario. La definición es de Postmark, uno de los ocho proveedores que estudia este trabajo [8]. Postmark pone como ejemplos el restablecimiento de contraseña, el correo de bienvenida, la factura, el recibo de compra y el aviso de un pago fallido, y añade que este correo no necesita enlace para darse de baja.

El correo de *marketing*, que Postmark llama también masivo o promocional, no es transaccional: lleva el mismo contenido, como el anuncio de un producto o una promoción, a muchos destinatarios, sin que lo desencadene ningún evento [8]. La Figura 3.3 ilustra la diferencia.

En el canal API, el servicio de envío de cada proveedor suele tener dos interfaces, la API REST y el relé SMTP (el servidor del proveedor que recibe el correo por el protocolo estándar de envío). Las dos dan acceso al mismo servicio, pero no todos los términos de consumo valen para las dos: en SendGrid, el tope de 1.000 destinatarios por petición es un límite de la API REST [59].

El correo transaccional es el dominio elegido para confrontar el modelo y las herramientas del canal API del ecosistema SPHERE. Varios proveedores venden el mismo servicio y lo limitan con combinaciones distintas de *rates* por *endpoint*, cuotas por plan y CRF: una petición puede gastar de la cuota un correo por cada destinatario o uno por cada correo de un lote (varios correos en una sola petición). Esa variedad

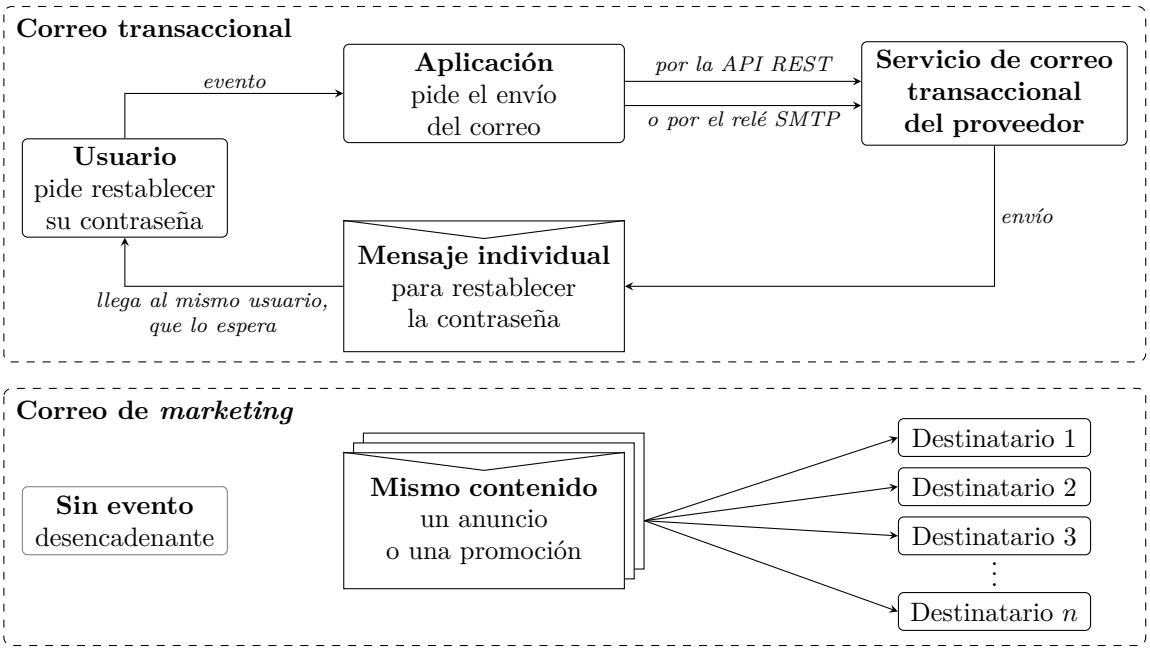


Figura 3.3: Correo transaccional frente a correo de *marketing*. El transaccional lo suele desencadenar una acción del usuario y va a un único destinatario, que lo espera. La aplicación pide su envío al proveedor por la API REST o por el relé SMTP. El de *marketing* lleva el mismo contenido a muchos destinatarios, sin un evento que lo desencadene. Elaboración propia a partir de la definición de Postmark [8].

hace del correo transaccional un banco de pruebas exigente para el ACM, la Datasheet y PRIME4API.

Sobre cómo se comparan estas APIs se han revisado dos guías públicas del sector, las de Mailtrap y MailSlurp. Las publican empresas que venden servicios de correo y recomiendan su propio producto [23, 24]. Mailtrap ordena seis proveedores. Describe cómo los probó (cuenta gratuita, IP compartida y la misma plantilla), mide la entregabilidad (la proporción de correos que llegan a la bandeja de entrada) y enlaza la página de precios de cada uno, pero da sin fuente los *rates* y los destinatarios por llamada. MailSlurp propone un procedimiento de evaluación de seis pasos, sin datos de ningún proveedor. Ninguna reúne los límites de uso y los *rates* de cada proveedor en un esquema común, con la fuente de cada dato.

En los trabajos del grupo sobre capacidad, las APIs de correo aparecen como casos de ilustración o validación. SendGrid, en dos versiones, está entre los pricings sobre los que Gavira Sánchez calcula las respuestas de referencia del canal API, y MailerSend sirvió como banco de pruebas en el diseño de ACM2YAML [3]. En los trabajos revisados no se ha encontrado un catálogo estructurado y trazable de las cuotas, los *rates* y los topes por petición de varias APIs de correo, ni un procedimiento que combine requisitos funcionales y no funcionales para compararlas.

El antecedente más próximo es el Trabajo Fin de Máster de Noguerol Tirado, del mismo programa [60]. Parte de una demanda histórica, los comentarios del New York Times tratados como traza de notificaciones, y busca la combinación de plan, complementos y estrategia de envío de menor coste en SendGrid y Amazon SES. Simula

tres estrategias de agrupación, comprueba cada una contra la cuota, el *rate* y los topes por petición, y compara el coste y el retraso de notificación de los dos proveedores.

Noguerol Tirado toma los límites de dos proveedores para servir una demanda concreta [60]. Este trabajo recoge los términos de consumo de ocho APIs sin una demanda previa, con la fuente de cada dato y en un formato que lee una herramienta: la Datasheet de ACM2YAML, sobre la que opera PRIME4API.

3.5. Huecos que aborda el trabajo

La revisión deja cuatro huecos, uno por contribución.

Primero: el instrumental existe, pero no se ha validado a escala de dominio. Las diez preguntas de análisis de Gavira Sánchez se responden con PRIME4API sobre tres pricings, y el diseño de ACM2YAML se probó con cuatro APIs. Ninguno de los trabajos revisados confronta todavía el modelo con un conjunto representativo de las APIs de un sector, elegido con un criterio declarado e incluidas las que no encajan con naturalidad. Es el hueco del catálogo (C-01).

Segundo: las carencias de ACM2YAML v0.3 están declaradas, pero no inventariadas. Se sabe qué dimensiones faltan, pero no qué extensiones exige un dominio real ni con qué prioridad. Es el hueco de la evolución del esquema (C-02).

Tercero: la relación entre límites de uso y *rates* está definida, pero no caracterizada. PRIME4API la ejercita sobre casos concretos, y Noguerol Tirado la comprueba en dos proveedores frente a una demanda dada. Falta ver qué patrones adopta en las APIs de un dominio y qué le cuestan al consumidor (C-03).

Cuarto: la comparación entre APIs existente parte de una demanda dada. El catálogo de operaciones de Fresno-Aranda y las diez preguntas de análisis de Gavira Sánchez razonan sobre un pricing, y Noguerol Tirado toma los límites de dos proveedores para servir una demanda concreta y compara los dos por coste y retraso. Elegir una API para unas necesidades dadas exige preguntas funcionales y no funcionales y un procedimiento para responderlas sobre Datasheets comparables (C-04).

4. El catálogo de Datasheets de APIs de correo

Este capítulo presenta el catálogo de ocho Datasheets de APIs de correo, que es la contribución C-01 (Sección 1.4), y responde a la RQ1: qué APIs de correo deben incluirse en un análisis representativo del dominio y bajo qué criterios reproducibles. El modelo ACM, su esquema ACM2YAML y la Datasheet son de Gavira Sánchez y Ruiz López [3, 4], y este trabajo los aplica a un dominio nuevo. Lo propio es el método que construye el catálogo y el catálogo que resulta.

El capítulo se organiza del siguiente modo. En primer lugar, la Sección 4.1 fija el dominio y las ocho APIs. A continuación, la Sección 4.2 describe el ciclo de construcción de cada Datasheet, y la Sección 4.3 las reglas que hacen revisable cada dato. Después, la Sección 4.4 presenta el catálogo resultante, y la Sección 4.5 mide si PRIME4API lo puede consumir. La Sección 4.6 describe su salida hacia SPHERE. Por último, la Sección 4.7 responde a la RQ1.

4.1. Selección del dominio

El catálogo de Datasheets de este trabajo reúne ocho APIs de correo transaccional. La Sección 3.4 define el correo transaccional y lo distingue del correo de *marketing*. Esas ocho APIs no son todas las que existen: la consulta a un modelo de lenguaje que eligió cuatro de ellas propuso además otras cuatro de reserva (Sección 4.1.2). La Tabla 4.1 presenta las ocho con el servicio que se modela, su proveedor y la vía por la que entraron en el catálogo.

API y servicio que se modela	**Proveedor**	**Vía de entrada**
Postmark	ActiveCampaign	API piloto del método
Resend, su producto transaccional	Resend	Modelada antes en el ecosistema
Twilio SendGrid Email API	Twilio	Modelada antes en el ecosistema
MailerSend, API y relé SMTP	MailerSend	Modelada antes en el ecosistema
Brevo, el correo transaccional de su plataforma de *marketing*	Brevo	Consulta a un modelo de lenguaje
Amazon SES, versión 2 de la API	Amazon Web Services	Consulta a un modelo de lenguaje
Mailgun	Sinch	Consulta a un modelo de lenguaje
Mailchimp Transactional, antes Mandrill, complemento de Mailchimp	Mailchimp	Consulta a un modelo de lenguaje

Tabla 4.1: Las ocho APIs del catálogo de Datasheets, en el orden en que se construyeron. La vía de entrada es la fila de la Tabla 4.2 por la que entró cada una.

En el canal API, un proveedor de correo suele ofrecer dos interfaces: la API REST y el relé SMTP, el servidor del proveedor que recibe el correo por el protocolo SMTP (Sección 1.1.1). Con esa terminología el objeto de estudio es el servicio de envío de correo y sus interfaces programáticas. Las Datasheets modelan la API REST, porque ACM2YAML v0.3 asigna cada límite al plan entero o a un *endpoint* HTTP, y el relé no es un *endpoint*. Lo que el relé comparte con la API, como la cuota diaria de peticiones de MailerSend, entra en la Datasheet, y los topes propios del relé se anotan como elementos que el esquema no soporta.

4.1.1. Fuentes de inclusión

El protocolo de selección parte de tres fuentes de inclusión, y la Tabla 4.2 recoge lo que aportó cada una. Las tres fuentes y la API piloto dan las ocho APIs de la Tabla 4.1.

Fuente	**Qué es**	**Qué aportó al catálogo**
Primera	Las APIs analizadas en el TFM de Noguerol Tirado [60], que compara SendGrid y Amazon SES para una demanda dada	Ninguna
Segunda	Las APIs ya modeladas en el ecosistema	SendGrid y MailerSend, modeladas en el trabajo de Gavira Sánchez [3] y en el repositorio de la línea, y Resend, modelada en la versión 0.1 de ACM2YAML en ese repositorio [61]
Tercera	Una consulta a un modelo de lenguaje con cinco criterios fijados antes de hacerla	Amazon SES, Mailgun, Brevo y Mailchimp Transactional, más cuatro APIs de reserva (Mailjet, Zoho ZeptoMail, SparkPost y SMTP2GO)
Piloto del método	No es fuente del protocolo. Es la API con la que el método se probó y se ajustó	Postmark

Tabla 4.2: Fuentes de inclusión del catálogo. Las tres primeras filas son las fuentes del protocolo. La última recoge la API piloto, con la que el método se probó y se ajustó antes de aplicarlo a las demás, y que ya formaba parte del catálogo cuando se hizo la consulta. Resend figura en la segunda fuente porque el repositorio de la línea la tenía modelada en la versión 0.1 de ACM2YAML, aunque este trabajo empezó a construirla antes de la consulta.

La primera fuente no aportó ninguna API, porque la memoria de Noguerol Tirado se obtuvo cuando la selección ya estaba cerrada, y la tercera fuente ocupó su lugar. Sus dos APIs están aun así en el catálogo: SendGrid por la segunda fuente y Amazon SES por la tercera. Las tres APIs de la segunda fuente se construyeron en v0.3 con datos recogidos entre el 2 de agosto y el 21 de septiembre de 2026. El modelado previo sirvió de punto de partida en Resend y de contraste en MailerSend, y SendGrid se construyó sin consultarlo.

4.1.2. Criterios de inclusión y regla de reemplazo

La consulta pidió cuatro APIs de correo transaccional para completar un catálogo de ocho, con cinco criterios ordenados por peso. Los criterios 1, 3, 4 y 5 buscan formas de cobro distintas, uso real, algún rasgo de límites que las demás APIs no tengan y proveedores de varios tamaños y regiones. El segundo añade una condición propia del método: que la Datasheet pueda construirse con trazabilidad, es decir, con la URL, la fecha de acceso y la cita literal que respaldan cada valor. Los criterios, en el orden en que los da el prompt (el texto de la consulta), son estos:

1. Diversidad del modelo de precios. El catálogo debe cubrir cuatro formas de cobro: planes por volumen mensual, pago por uso puro, producto vendido como añadido de otro y planes con límite diario además del mensual.
2. Documentación pública de precios y de límites (como los *rates*, los tamaños máximos o los destinatarios por petición), consultable sin darse de alta.
3. Adopción real en producción, para que la comparación tenga interés práctico.
4. Un rasgo diferenciador para analizar la relación entre límites de uso y *rates*, como un *rate* publicado por *endpoint*, un caudal negociado no publicado o un límite diario.
5. Diversidad geográfica y de tamaño de proveedor, con al menos una API europea.

La respuesta justifica tres de las cuatro APIs por la forma de cobro que añaden. A Amazon SES le atribuye el pago por uso puro, a Brevo el límite diario además del mensual y a Mailchimp Transactional el producto vendido como añadido. Mailgun entra por su rasgo de límites, que la respuesta describe como «el caso limpio de rate documentado», y Brevo cubre además la API europea del quinto criterio. Tres de estas atribuciones no se sostuvieron al construir las Datasheets (Sección 4.7). La única exclusión aplicada fue la del propio prompt, que impedía proponer las cuatro APIs ya presentes.

La consulta devolvió también cuatro APIs de reserva, por este orden: Mailjet, Zoho ZeptoMail, SparkPost y SMTP2GO. La respuesta íntegra está en el repositorio de datos del trabajo[1]. Si una API del catálogo no puede documentarse con trazabilidad, entra la primera de la reserva. La sustitución queda registrada con fecha y motivo. La regla no llegó a aplicarse, porque las ocho APIs se documentaron con fuente, fecha y cita para cada valor, y el validador de formato de este trabajo, que contrasta cada Datasheet con la guía de ACM2YAML v0.3, no devolvió ningún error en ninguna.

Un último criterio, que no está entre los cinco del prompt, acota qué se modela de cada proveedor. El catálogo de Datasheets modela solo el canal API. El canal web, que el ecosistema representa con *iPricing* [5], no entra en las Datasheets, y el pricing de cada API se escribe aparte en Pricing2YAML, la serialización de *iPricing*, para llevarlo a SPHERE (Sección 4.6).

Varios proveedores venden además más de un producto bajo la misma cuenta, y

[1] `03_datos/SELECCION_APIS_2026-09-01.md`, en https://github.com/danvelcam/tfm-pricing-apis-correo.

Resend, por ejemplo, ofrece un producto transaccional facturado por correos enviados y otro de *marketing* facturado por contactos. El catálogo modela en cada API el producto transaccional. Es el comparable entre proveedores y el que consume la API de envío. El otro producto queda en el registro de fuentes y se anota como elemento que el esquema no soporta.

Cada API del catálogo tiene un pricing publicado que cubre el canal API, y de él salen los planes y los precios de su Datasheet. En Brevo es el de su plataforma de *marketing*, que incluye el correo transaccional. El caso dudoso era Amazon SES, que se cobra por uso dentro de AWS (Amazon Web Services, la nube de Amazon), pero su página de precios publica tres planes y una tarifa de 0,10 USD por cada mil correos [62], así que también lo tiene.

Un mismo servicio puede llegar al cliente por más de una vía: SendGrid se contrata con Twilio, que lo comercializa, y también a través de RapidAPI, un mercado de APIs, y Gavira Sánchez modela el pricing de esa segunda vía [3]. El catálogo toma siempre el proveedor directo y no compara las dos vías.

4.1.3. Ventana de toma de datos

La toma de datos principal del catálogo, con sus sondeos (peticiones controladas contra la API real para comprobar un límite que la documentación no fija o no deja claro), fue del 2 de agosto al 21 de septiembre de 2026. Fuera de esa ventana se hicieron, sin cambiar ningún valor de las Datasheets, su ingesta en PRIME4API, unos sondeos complementarios de Resend y Amazon SES, el 18 de octubre de 2026, y la captura en bruto de las páginas de cada proveedor, el 23 de octubre (Sección 4.2).

Después de la ventana, dos revisiones sí cambiaron cifras de las Datasheets o su semáforo de confianza, el color que marca el respaldo de cada valor (Sección 4.3.1). El 18 de octubre se aplicó a SendGrid, Mailgun, Brevo y MailerSend la regla de que, si otra fuente da otro valor, se modela el de la página de precios: el sobrecoste del plan Essentials de SendGrid pasó de 0,00133 a 0,0013 USD por correo, y la cuota diaria del plan gratuito de Mailgun, de ámbar a verde.

El 31 de octubre se volvió a abrir cada fuente citada en las Datasheets de Brevo, Amazon SES, Mailgun y Mailchimp Transactional. Esa verificación no cambió ninguna cifra y dejó en verde dos valores que estaban en ámbar, uno de ellos el *rate* del resto de operaciones de Amazon SES (Sección 4.3.3). Abrió también una discrepancia sobre la unidad del caudal de envío del entorno de pruebas de Amazon SES, en el que empieza toda cuenta nueva: unas fuentes lo cuentan en mensajes y otras en destinatarios, y el valor bajó de verde a ámbar. El mismo día, una nueva lectura de dos páginas de MailerSend añadió a su Datasheet una nota: en septiembre una de ellas se leyó sin ejecutar JavaScript y dio los precios anuales de 2024.

Los precios, planes y límites de esta memoria son los de la toma principal, y una comparación entre dos APIs puede cruzar fechas distintas dentro de ella. El esquema v0.3 lleva una sola fecha por documento, en el campo `date`, y la fecha de acceso de cada valor se conserva en el registro de fuentes.

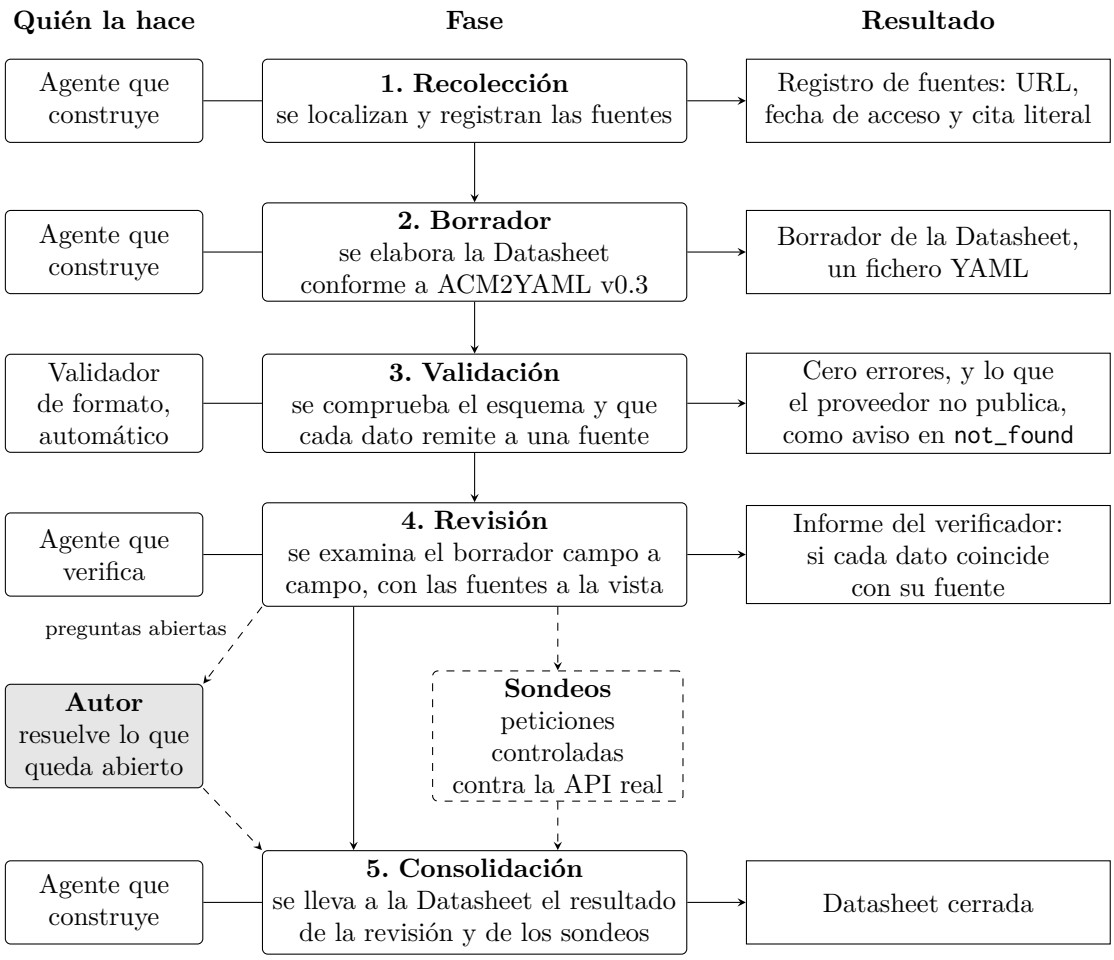


Figura 4.1: Ciclo de construcción de cada Datasheet: sus cinco fases, quién hace cada una y qué resultado deja. Las flechas discontinuas son desvíos que solo se recorren cuando la revisión los motiva: los sondeos y las preguntas abiertas, que resuelve el autor. Dos de estos resultados, el registro de fuentes y la Datasheet cerrada, son también artefactos de cada API, y la Figura 4.2 muestra los cinco. Postmark, la API piloto, se cerró sin informe del verificador. Elaboración propia.

4.2. El ciclo de construcción por API

Cada API del catálogo pasa por un ciclo de cinco fases: recolección, borrador, validación, revisión y consolidación. La Figura 4.1 lo representa con quién hace cada fase y qué resultado deja. El ciclo y sus reglas son de este trabajo.

El ciclo separa a quien construye de quien verifica, dos papeles que desempeñan agentes de inteligencia artificial distintos. El agente que verifica recibe solo la Datasheet, sin el resto del trabajo de quien la construye, vuelve a abrir cada URL citada y deja un informe que dice, campo a campo, si el dato coincide con su fuente. Esa separación entró en el ciclo el 1 de septiembre de 2026, y Postmark, la API piloto, se cerró sin informe del verificador. Las otras siete lo tienen.

En la *recolección* se localizan y registran las fuentes, que son las páginas donde el proveedor publica sus términos de consumo, es decir, el precio y la información

de cómo, cuándo, cuánto y dónde se consume el servicio. Son la página de precios, la documentación de desarrollador, las especificaciones, los artículos de soporte y las políticas de uso. Desde el piloto, la recolección incluye también el panel con cuenta propia, la aplicación web en la que el cliente gestiona su cuenta en el servicio.

En el *borrador* se elabora la Datasheet conforme a ACM2YAML v0.3 [3], siguiendo la guía del esquema que publica API-TaskForce, la organización de GitHub que reúne también el repositorio de PRIME4API y el de la extensión de SPHERE al canal API [61].

La *validación* es automática: el validador de formato de este trabajo contrasta la Datasheet con la guía de v0.3 y comprueba dos cosas. La primera es que el documento cumple el esquema (campos obligatorios, enumeraciones, unidades declaradas y referencias entre planes, cuotas y *rates*), y la segunda, que cada dato remite a una fuente registrada. La validación solo se da por superada con cero errores, y lo que el proveedor no publica queda como aviso justificado en la lista `not_found`, que registra lo que se buscó y no se encontró.

La *revisión* examina el borrador campo a campo con las fuentes a la vista. Resuelve las ambigüedades que la documentación no cierra, como la ventana de la cuota diaria de Resend, que no dice desde cuándo se cuentan sus 24 horas, y registra como pregunta abierta la que ninguna regla del libro de criterios (Sección 2.1) resuelve todavía. El acta de revisión la escribe el agente que construye, con esas preguntas abiertas, y el agente que verifica deja aparte su informe. La *consolidación* lleva a la Datasheet el resultado de la revisión y de los sondeos empíricos que esta haya motivado, y cierra el ciclo.

El ciclo produce cinco artefactos por API. Cuatro son la Datasheet, el registro de fuentes con sus citas, el registro de elementos no soportados y el acta de revisión con las decisiones de representación. El quinto es una matriz de *features* por plan (*webhooks*, IP dedicada, retención, dominios, usuarios, correo entrante, SMTP y plantillas). Como v0.3 no la admite, se conserva fuera del YAML para las preguntas funcionales, como qué planes incluyen IP dedicada, una dirección IP de envío reservada a un solo cliente.

Cada artefacto nace en una fase del ciclo: el registro de fuentes en la recolección, la Datasheet en el borrador y el acta en la revisión. El registro de elementos no soportados crece en cualquier fase del ciclo, desde la recolección hasta la consolidación, y la matriz se rellena con las fuentes de la recolección.

En el registro de elementos no soportados, cada rasgo del proveedor que v0.3 no puede escribir queda con su cita y un identificador F-nn. La F viene de «fricción», el nombre que reciben esos registros en el repositorio de datos (fichero `fricciones.md`), y el número sigue el orden en que apareció cada elemento. La Tabla 4.3 recoge los ocho del piloto, y la Tabla 5.1, los treinta y cinco de las ocho APIs.

La memoria no reproduce enteros los cinco artefactos. La Figura 4.2 muestra un fragmento de cada uno en Postmark, y los de las ocho APIs están completos en el repositorio de datos del trabajo[2], donde cualquier dato se puede auditar con lo que está publicado en él.

[2] `03_datos/apis/`, en https://github.com/danvelcam/tfm-pricing-apis-correo.

Cada fragmento de la figura se copia tal cual de su fichero, con las marcas de formato de YAML o de Markdown. Las marcas entre corchetes, como [F1] o [F11], remiten al registro de fuentes, y los identificadores con guion, como F-02, al de elementos no soportados. El fragmento (4) es la novena pregunta del acta. En los fragmentos (3) y (4), la escalera es la lista de volúmenes contratados, cada uno un escalón, *overage* es el sobrecoste y *provenance*, el registro de procedencia. Una raya en la matriz indica que ninguna fuente registrada da el dato.

Postmark [33] es la API piloto: con ella el método se probó y se ajustó antes de aplicarlo a las otras siete. El ecosistema no la tenía modelada, así que el ciclo se recorrió desde la recolección, sin un modelado previo del que partir. La Figura 4.2 muestra cómo quedó en sus cinco artefactos, y los párrafos siguientes, qué salió de cada fase.

La recolección de Postmark, el 2 y el 3 de agosto de 2026, registró doce fuentes en tres pasadas: once páginas públicas del proveedor y el panel con cuenta propia. La página de precios da cuatro planes, uno gratuito y tres de pago que empiezan en 10.000 correos al mes. La documentación describe el error que la API devuelve a quien pasa del *rate*, pero no publica el valor del *rate* (F-01). Cada una de las doce fuentes entró en el registro de fuentes con su URL, su fecha de acceso y sus citas literales.

El panel produjo tres de los ocho elementos no soportados del piloto, y ninguno se podía reconstruir con las páginas públicas. Son las cuotas por volumen contratado (F-02), los *add-ons* que cambian la capacidad (F-07) y el plan real como *tier* por volumen (F-08). El *tier* es el nivel de funcionalidades que se contrata: en Postmark, Basic, Pro o Platform. De esta fase salió el primer cambio del método: desde el 5 de agosto, el panel es obligatorio en todas las APIs, y una Datasheet sin panel no se da por cerrada.

En el borrador, escribir la Datasheet contra ACM2YAML v0.3 obligó a las primeras decisiones de representación. Postmark se contrata eligiendo un *tier* funcional y un volumen mensual, y en v0.3 cada plan lleva una cuota con un solo valor. Había dos salidas: enumerar como planes unas dieciocho combinaciones de *tier* y volumen, o modelar un plan por *tier* con la cuota de su volumen mínimo. Se eligió la segunda, que recoge el fragmento (4) de la Figura 4.2, y la Figura 4.3 muestra la cuota por volumen contratado (F-02) que queda fuera del bloque `plans`.

La pregunta que abrió esta fase fijó el 3 de agosto el marco de todo el catálogo: la Datasheet se escribe conforme a v0.3, sin campos propios, y lo que el esquema no puede escribir va al registro de elementos no soportados. Un elemento pasa a candidato de extensión si reaparece en otra API, que es el criterio de recurrencia (Sección 5.1). El piloto dejó ocho, de F-01 a F-08, que la Tabla 4.3 recoge con su estado al cerrarlo. Ninguno impide la conformidad de la Datasheet, y cada uno se registra con la API y la cita que lo originan.

La validación no dio ningún error en la Datasheet de Postmark, y los avisos del validador de formato eran los dos huecos deliberados del borrador. El primero es el bloque de *rates*, vacío porque Postmark no publica el de su API REST. El segundo es el máximo de mensajes de `/email/bulk`, una de las rutas con las que Postmark envía varios mensajes en una sola petición, que el proveedor acota por el tamaño total de la petición, 50 MB, y no por un número de mensajes (F-06). El hueco del *rate* quedó además en `not_found`, con la búsqueda hecha (Sección 4.3.1).

1 Datasheet — 03_datos/apis/postmark/datasheet.yaml

```
capacity:
  [...]
  quota_email_included_basic:
    value: 10000                                    # ●
    [...]
    overage_cost:
      price: 0.0018                                 # ●
      value: 1
[...]
plans:
  [...]
  basic:
    price: 15.00                                    # ●
    [...]
    quota: quota_email_included_basic
    rate: rate_request_rest
```

2 Registro de fuentes — 03_datos/apis/postmark/fuentes.md

```
## [F1] Página de pricing

- **URL:** https://postmarkapp.com/pricing
- **Fecha de acceso:** 2026-08-03 (re-verificada; primera visita 2026-08-02)
- **Citas literales:**
  [...]
  - Basic — `"$15.00/mo"`, `"Starting at 10,000 emails/month"`,
    `"Extra emails @ $1.80 / 1,000"`
```

3 Registro de elementos no soportados — 03_datos/apis/postmark/fricciones.md

```
## F-02 · Cuotas por escalón, no por valor único — y con overage por escalón

**Qué pasa.** Los tres planes de pago dicen *"Starting at 10,000 emails/month"* [F1].
El panel [F11] revela la estructura completa: una escalera de volumen (10k/$15 ·
50k/$55 · 125k/$115 · 300k/$245 · 700k/$455 · 1,5M/$775) en la que **el propio
overage depende del escalón** ($1.80/1.000 hasta 125k, $1.70/1.000 desde 300k), y
[...]
**Qué hace v0.3.** Una cuota tiene un `value` escalar y un `overage_cost` único. Ni
la escalera ni el overage-por-escalón caben.
[...]
**Propuesta.** Permitir que una cuota declare escalones (`tiers`) con precio y
overage propios por escalón, más un escalón `custom` para el tramo negociado.
```

4 Acta de revisión — 03_datos/apis/postmark/REVISION.md

```
### Q9 · Proyección del producto tier × volumen en `plans` (F-08) — ✓ DECIDIDA [...]
El panel [F11] revela que Basic/Pro/Platform son **tiers funcionales** y el volumen
va aparte, en escalera. La datasheet mantiene la proyección "3 planes = 3 tiers, con
la cuota del escalón mínimo" y documenta la escalera en provenance/F-02. Alternativa
descartada: enumerar las ~18 combinaciones tier×escalón como planes. [...]
```

5 Matriz de *features* por plan — 03_datos/apis/postmark/features.md

```
## Features por plan

| Feature | Free | Basic | Pro | Platform | Fuente · escalón |
|---|---|---|---|---|---|
[...]
| Usuarios | — | `"Up to 4 users"` | `"Up to 6 users"` | `"Unlimited users"` | [F11] panel |
```

Figura 4.2: Fragmento literal de cada uno de los cinco artefactos de Postmark, todos con datos del plan Basic: su cuota y su sobrecoste en la Datasheet (1), la cita de la página de precios que los respalda (2), el elemento no soportado F-02 (3), la decisión de modelar cada *tier* con la cuota de su volumen mínimo (4) y la fila de usuarios de la matriz de *features* (5). [...] marca un corte. Los círculos de color sustituyen a los emojis del semáforo de confianza (Sección 4.3.1), y la marca de verificación, al emoji de decisión cerrada. Elaboración propia.

Id	Elemento no soportado por ACM2YAML v0.3	Estado
F-01	Un *rate* que existe contractualmente pero sin valor publicado: el bloque vacío se confunde con «sin límite»	pendiente
F-02	Cuotas por volumen contratado, con sobrecoste dependiente a la vez del volumen y del *tier*, y tramo superior negociado	pendiente
F-03	Factor de consumo multiplicativo (mensajes × destinatarios) que una terna única no puede expresar	pendiente
F-04	Restricciones declaradas que el proveedor admite no aplicar o que la respuesta a la petición no refleja	discusión
F-05	La dimensión *Results Per Request* aparece en el dominio y v0.3 no la serializa	pendiente
F-06	Un *workload*, lo que consume una petición, acotado por otra métrica (bytes del cuerpo de la petición, ya codificado) y no por un entero	pendiente
F-07	*Add-ons* que modifican dimensiones de capacidad, condicionados a volumen y a *tier*	pendiente
F-08	El «plan» real es *tier* × volumen, con superficie de API dependiente del *tier*	pendiente

Tabla 4.3: Elementos no soportados documentados en el piloto. «Pendiente» marca un dato que v0.3 no sabe escribir, como el consumo de mensajes por destinatarios de F-03, y que pasa a proponerse como extensión si aparece en al menos otra API o si su efecto se reproduce al ingerir la Datasheet en PRIME4API. «Discusión» marca una pregunta de fondo: si un límite que el proveedor declara y admite no aplicar forma parte de la capacidad.

En la revisión de Postmark, hecha sin agente que verifica, el acta recogió diez preguntas que ni la documentación ni las reglas de modelado de entonces cerraban. Una era cuántos destinatarios admite un mensaje, y la resolvió el primer sondeo del catálogo, el 3 de agosto, con el token de pruebas de Postmark, una clave de acceso que valida las peticiones sin enviar correo ni consumir cuota [63].

La documentación fija un máximo de 50 destinatarios en cada campo To, Cc y Bcc, pero una petición con 51 repartidos entre To y Cc recibió un 422, el código HTTP con el que la API rechaza una petición que no cumple sus reglas. El tope de 50 es por mensaje, como dice el propio error: «Maximum of 50 recipients allowed per email message». Ese choque fue la primera entrada del registro de discrepancias, donde se enfrentan, con sus citas, las fuentes que se contradicen entre sí o con lo observado.

El tope de 50 destinatarios es un término de consumo, porque Postmark lo publica y dice cuánto cabe en una petición, pero no es una dimensión de capacidad. Como los destinatarios por llamada de la Sección 1.1.1, acota cuántos correos de la cuota gasta una petición, y en el modelo ACM eso es una regla de consumo, el CRF, que la Datasheet escribe en el *workload* del *endpoint* de envío. La respuesta de la API casa con esa lectura: la petición con 51 destinatarios recibe un 422, el de una petición mal

formada, y no el 429 que la API devuelve a quien se ha pasado del *rate*. De la revisión salió el segundo cambio del método: el sondeo contra la API real es desde entonces un peldaño ordinario de la escalera de validación de cada Datasheet (Sección 4.3.2).

La consolidación llevó a la Datasheet el tope de 50 destinatarios como valor confirmado, y dejó en el registro de procedencia los seis volúmenes contratados que muestra el panel, porque cada plan se modeló con el volumen mínimo de su *tier*. La matriz de *features*, el quinto artefacto, se construyó el 7 de septiembre, cuando el ciclo la incorporó, con las fuentes que ya tenía.

Ya con otras APIs del catálogo en construcción, el 6 y el 7 de septiembre, tres sondeos más confirmaron el tope en el envío con plantilla y el lote de 500 mensajes que publica la documentación de `/email/batch`, y midieron un contador de 5.000 peticiones por segundo en una cabecera, uno de los campos de metadatos que acompañan a cada respuesta de la API. Ese valor entró como *rate* de la Datasheet, pero sin darlo por definitivo, porque ningún sondeo llegó al 429, y de este caso sale la segunda regla de lectura de los sondeos (Sección 4.3.2).

Después de cerrar el catálogo, la recolección ganó una *skill* propia, un conjunto de instrucciones empaquetadas que un modelo de lenguaje sigue para una tarea. Esa *skill* recoge en bruto los términos de consumo de cada proveedor y está en el repositorio de datos del trabajo[3]. No intervino en las ocho Datasheets, que ya estaban cerradas: guarda el texto de cada página con su fecha, para que una captura posterior diga qué páginas cambiaron. Su estructura es la de las *skills* de Ruiz López [64]: un único fichero de reglas, una salida en YAML con su esquema y un programa que comprueba esa salida.

El fichero `SKILL.md` reúne las reglas de la *skill*: el árbol de decisión que dice, en un orden fijo, si cada enlace entra en la captura o se descarta, y el presupuesto de peticiones de cada ejecución. La carpeta `references/` guarda el esquema del fichero de salida, un glosario, los dominios oficiales de cada proveedor, los patrones de texto que reconocen cada tema y unos ejemplos con un proveedor ficticio, que enseñan cómo se aplica el árbol de decisión. La carpeta `scripts/` guarda el programa que recoge las páginas, el que comprueba la salida y la definición del contenedor en el que corren, un entorno aislado que lee el repositorio sin poder modificarlo y solo escribe en la carpeta de la ejecución.

La *skill* trabaja en dos fases. La primera, descubrir, parte de las fuentes registradas y decide qué enlaces entran, respetando el `robots.txt`, el fichero en el que cada sitio dice qué páginas pueden recorrer los programas automáticos, y sin guardar ningún texto. La segunda, extraer, guarda cada página tal como llega, con su URL, su fecha de captura y su huella sha256. La huella es un resumen de los bytes de la página: si cambia uno solo, cambia la huella, y por eso sirve para comprobar después que la página sigue igual.

En las ocho APIs hubo 14.362 páginas candidatas, las del sitio de cada proveedor enlazadas desde alguna de las 204 fuentes registradas o listadas en el índice de páginas que publica el propio sitio. De ellas entraron en la captura 1.197: las fuentes registradas

[3] `skills/recogida-terminos-consumo/`, en https://github.com/danvelcam/tfm-pricing-apis-correo.

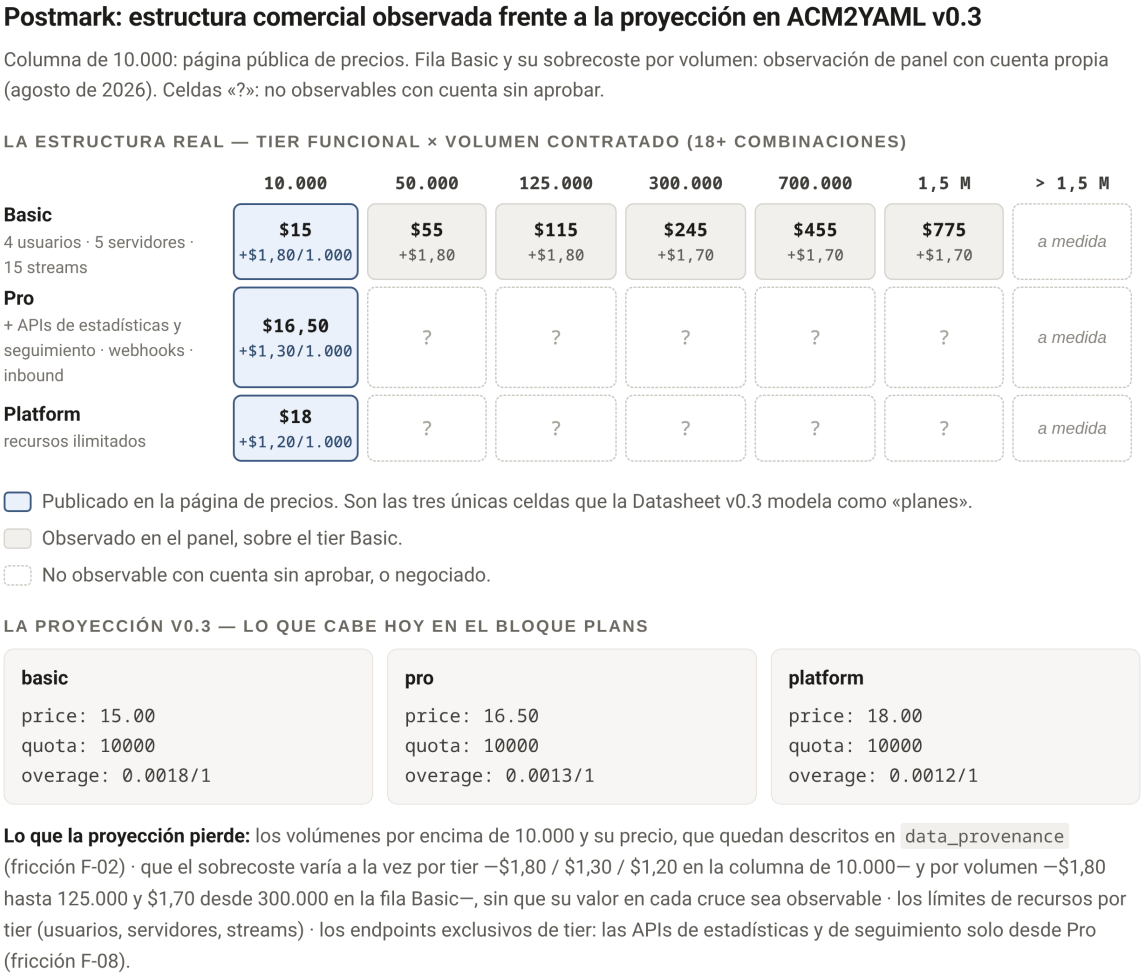


Figura 4.3: Estructura comercial de Postmark (*tier* funcional × volumen contratado) frente a la proyección representable en el bloque `plans` de ACM2YAML v0.3. La columna de 10.000 procede de la página de precios [33]. La fila Basic y su sobrecoste por volumen proceden de la observación de panel con cuenta propia. Las celdas sin valor no son observables con una cuenta sin aprobar. Elaboración propia.

y las que tratan alguno de los ocho temas de la *skill* (precios, límites y cuotas, uso aceptable, SMTP, *webhooks*, errores, reputación de la cuenta y envío). Los *webhooks* son los avisos que el proveedor envía a una URL del cliente cuando ocurre algo con un correo, y la reputación es la valoración que el proveedor hace de los envíos de la cuenta. Se capturaron 1.174, y las otras 23 quedan anotadas con su motivo.

Cada proveedor deja un único fichero `recogida.yaml` con el enlace, el título, la fecha y las huellas de cada página. La propia *skill* comprueba después el formato de ese fichero, la cobertura de cada tema y que el texto en bruto no quede en la carpeta que se publica. De esas páginas solo se publican las citas literales del registro de fuentes, fragmentos cortos que respaldan cada valor. El texto entero no se publica, porque las condiciones de uso de los sitios de cinco de los ocho proveedores prohíben copiarlo.

4.3. Trazabilidad y escalera de validación

4.3.1. Fuente, semáforo y procedencia

La regla central del método de construcción del catálogo de Datasheets es que ningún dato entra en una Datasheet sin fuente verificable ni se rellena por estimación. Cada valor lleva la URL de procedencia con su fecha de acceso, y la *cita literal* del texto que lo respalda. La guía de ACM2YAML ya prevé parte de este registro en los campos `sources_visited` y `data_provenance` [61]. El método añade los dos registros que ya han salido en el ciclo: la lista `not_found`, para que la ausencia quede registrada como dato, y el registro de *discrepancias*.

Los términos de consumo de un mismo servicio están repartidos entre varias páginas del proveedor: la de precios, la documentación de cada interfaz, la ayuda o el blog. La página de precios fija la verdad del dato, porque es lo que el cliente compra. Las demás fuentes (documentación, ayuda, blog, panel o sondeo) detectan incoherencias del pricing, pero no lo corrigen. Si una de ellas da otro valor, se modela el de la página y el otro se registra como incoherencia, con las dos citas. Si la propia página se contradice, esta regla no decide: el campo se marca como interpretado mientras la discrepancia siga abierta.

Sobre esa base, este trabajo introduce como aportación metodológica propia un *semáforo de confianza*: cada campo de la Datasheet se etiqueta con el respaldo que tiene su valor. Verde, si el valor está respaldado: es literal en lo que publica el proveedor, o lo fijó un panel con cuenta propia o un sondeo. Ámbar, si está interpretado: hay texto, pero el valor exige una lectura que la fuente no hace literal, o dos fuentes oficiales chocan. También es ámbar el valor que un sondeo de caudal leyó en una cabecera de respuesta sin llegar al rechazo por exceso de caudal (Sección 4.3.2). Rojo, si el proveedor no lo publica, y eso ya es un hallazgo.

La etiqueta se serializa como comentario YAML con el color y nada más, así que no altera la conformidad del documento, y qué fuente respalda cada dato lo dice el registro de procedencia (*provenance*).

El semáforo tiene una función operativa, que es dirigir la revisión hacia lo que no está en verde, y otra analítica, que nace de agregar. Que el bloque de *rates* de Postmark estuviera en rojo en el piloto era un hueco de un proveedor, pero que lo estuviera en todas las APIs del catálogo sería un rasgo del dominio. Agregar los colores por API y por dimensión mide qué parte de su capacidad contractual publican los proveedores de correo y qué dimensiones no publican, y la Sección 6.3 lo hace con la ventana de las cuotas.

Para que ese recuento sea posible, el hueco se registra en `not_found` con la búsqueda documentada cuando la dimensión existe pero su valor no está publicado. Así el bloque vacío queda acompañado de un registro que dice que el límite existe y no se publica. La guía de v0.3 no define ese registro, y el validador de formato lo admite porque no da error por una clave que la guía no recoge (Sección 4.5).

El Extracto de código 4.1 muestra el semáforo y ese registro del hueco sobre la

Datasheet del piloto, antes de que adoptara su *rate*. Su bloque de cuota es el del primer fragmento de la Figura 4.2. Una cuota interpretada (ámbar) convive con un sobrecoste literal (verde), y el bloque de *rates* queda vacío, con el hueco en rojo. La cuota es una interpretación porque la página de precios dice que el plan empieza en 10.000 correos al mes, no que los incluya. Al cierre del catálogo, ese bloque declara en ámbar el valor que devolvió una cabecera. En la memoria, los emojis del fichero se representan con los círculos 🟢, 🟡 y 🔴.

Extracto de código 4.1: Serialización del semáforo de confianza en la Datasheet del piloto (fragmento).

```
capacity:
  quota_email_included_basic:
    value: 10000                                                    # 🟡
    unit: emails
    period:
      value: 1
      unit: MONTH
    window_type: fixed_billing_cycle                                # 🟡
    overage_cost:
      price: 0.0018                                                 # 🟢
      value: 1

# 🔴 rate no publicado: hueco deliberado (ver metadata.not_found)
max_power: {}
```

Cómo se obtuvo el valor lo dice un segundo eje, la procedencia, con cinco valores. *Publicado*: el valor es literal de una página del proveedor, como las 10 peticiones por segundo de Resend, comunes a sus planes Free, Pro y Scale. *Interpretado*: hay texto y el modelador decide qué significa, como en la cuota diaria de Resend, que pide esperar 24 horas sin decir desde cuándo. Su ventana queda en ámbar. *Medido*: lo fijó un sondeo o un panel con cuenta propia, como el tope de destinatarios de Resend: la API responde 422 con 51 en `to` y 200 con 25 en `to` más 25 en `cc`.

Los otros dos valores no se leen en ninguna fuente ni se observan: se derivan. *Calculado*: es una función reproducible de valores publicados o medidos, como el coste que PRIME4API deriva del precio del plan y del sobrecoste, y de él se guardan la fórmula y las entradas, no el resultado. *Inferido por agente*: es una conclusión que ninguna página afirma y ninguna fórmula devuelve, producida por un modelo de lenguaje o por una *skill* que cita sus fuentes y su versión. En las ocho Datasheets no hay ningún valor así. Que un agente escriba un valor no lo hace inferido: el publicado, el interpretado y el medido parten de un texto o de una medida que la cita reproduce, y el inferido es una conclusión que no dice ningún texto.

Dos reglas enlazan el color con la procedencia. Un valor inferido por agente no pasa de ámbar hasta que lo respalde un valor publicado o medido, y un valor calculado toma el peor color de sus entradas. Las páginas del proveedor son el origen de cada dato, todavía sin contrastar, y la Datasheet es una vista construida a partir de ellas. La regla de la página de precios decide qué valor entra en la Datasheet, y el registro de procedencia dice si un valor publicado se confirmó después con un contraste, un panel o un sondeo.

Los términos de consumo que reúne una Datasheet salen de fuentes distintas del

Fuente principal	Entradas	%
Sondeo contra la API	63	14,4
Panel con cuenta propia	42	9,6
Página de precios	109	24,9
Documentación y demás páginas del proveedor	216	49,4
Nota aclaratoria, sin dato	7	1,6
Total	437	100

Tabla 4.4: Fuente principal de las 437 entradas del registro de procedencia de las ocho Datasheets. Cuando una entrada cita varias fuentes, cuenta la que aparece antes en la tabla. Porcentajes redondeados a un decimal.

mismo servicio: sus páginas, el panel y los sondeos. La Tabla 4.4 reparte por su fuente principal las 437 entradas del registro de procedencia de las ocho Datasheets. La página de precios es la fuente principal de una entrada de cada cuatro, y la documentación y las demás páginas del proveedor, de casi la mitad. Cuando una entrada cita varias fuentes, cuenta la que va antes en la tabla, y ese orden solo sirve para contar: en un choque con la página de precios, entra el valor de la página.

4.3.2. La escalera de validación

Los campos en ámbar o en rojo pasan por una escalera de validación de cuatro peldaños. Es una escalera porque los peldaños están ordenados de menor a mayor coste, de leer otra página del proveedor a medir el servicio bajo carga, y un campo solo sube al siguiente cuando el anterior no lo resuelve. El panel es la excepción: desde el piloto se observa en todas las APIs, aunque el primer peldaño ya haya resuelto el campo.

El primer peldaño es el *contraste documental*: buscar una segunda fuente oficial que confirme o contradiga la primera y resolver el choque con la regla de la página de precios del apartado anterior. Si la página no interviene, el choque se registra como discrepancia.

El segundo es la *observación de panel* con cuenta propia. Se da de alta una cuenta en el servicio y se confronta lo publicado con lo que muestra el panel, con capturas fechadas como evidencia. El panel no sobrescribe el precio publicado: si muestra otra cifra para el mismo dato, la diferencia queda como incoherencia.

El tercero es el *sondeo empírico ligero* contra la API real. Prefiere un sondeo inocuo, que no entrega ningún correo: el que usa un token de pruebas, como el de Postmark (Sección 4.2), el que solo hace lecturas o el que provoca que la API rechace la petición antes de entregarla. Si ninguno sirve, admite un envío real a destinatarios propios, con un número máximo de envíos declarado en el script.

Lo que el sondeo puede ver depende de cómo aplica el proveedor cada límite. El sondeo parte de la lista de códigos de error que publica el proveedor, porque cada límite que se aplica rechazando la petición tiene un error observable. No todos los límites se aplican así. Superada la cuota diaria de su plan Free, Brevo retiene en cola hasta 1.000 correos más [65]. El lote de MailerSend, hasta 500 mensajes en una petición, se acepta

con un `202`, el código HTTP de una petición aceptada para procesarla después, y el error solo aparece al consultar su estado.

El cuarto es la *medición empírica formal* con `datasheet-usage-testing`, el banco de pruebas de API-TaskForce para medir límites de uso [66]. Cubre lo que los anteriores no alcanzan, como los umbrales no publicados o el comportamiento bajo carga. Esa medición queda fuera del ciclo que construye el catálogo. Los campos candidatos para ejecutarla sobre el catálogo terminado son los adoptados desde una cabecera, como los *rates* de Postmark y Mailgun.

Dos reglas gobiernan la lectura de los sondeos, igual en las ocho APIs. La primera dice qué valida un sondeo sin envío real. Un sondeo con token de pruebas o en modo *sandbox* (un modo en el que la API procesa la petición sin entregar el correo) valida los topes que la API comprueba al recibir la petición, como los destinatarios o el tamaño del mensaje, pero no la cuota consumida ni el caudal. Cuando una petición con un token real se rechaza porque no cumple las reglas de la API, el contador de caudal sí descuenta, y en MailerSend también la cuota diaria de peticiones.

La segunda dice cuándo un valor observado entra en la Datasheet: cuando el sondeo se diseñó para medirlo y es inocuo. Una cabecera aparecida en un sondeo con otro propósito se registra y no se adopta. Adoptarla exige un sondeo de caudal: si confirma el contador, el valor entra en ámbar, y solo pasa a verde cuando el sondeo llega al rechazo por exceso de caudal. Si el sondeo mide menos que la página de precios, se mantiene el valor publicado. La medida queda como incoherencia del pricing.

Al cruzar fuentes se distinguen tres fenómenos, siempre con el mismo nombre. Hay *contradicción* si dos fuentes dan valores o condiciones incompatibles para el mismo atributo. En Brevo, la documentación de límites de su API admite 1.000 peticiones por segundo al *endpoint* de envío en su categoría General, y su guía de envío por lotes da para la misma ruta 6.000 por hora, seiscientas veces menos que los 3.600.000 por hora que suman 1.000 por segundo [67]. Cuando una de las dos fuentes es la página de precios y la otra no, la contradicción se registra como incoherencia, y en los demás casos, como discrepancia.

Hay *ausencia* si una fuente no publica el límite, como el contrato legible por máquina de la versión 2 de Amazon SES, que no publica ninguno. Ese contrato es la descripción formal de las operaciones de la API, pensada para que la lean los programas.

Hay *diferencia de interfaz* si dos fuentes dan valores o reglas distintos porque describen interfaces distintas del mismo servicio. La cuota diaria de peticiones de MailerSend es común a la API REST y al relé SMTP, pero la página general de su API dice que cuenta cada petición, y la de su relé, que cuenta cada mensaje. No se contradicen: por la API, un envío en lote (`POST /v1/bulk-email`) de hasta 500 mensajes distintos es una sola petición, y por el relé cada mensaje descuenta uno.

Una discrepancia puede venir también del modo de leer las fuentes, y no del proveedor. Los agentes de inteligencia artificial de la recolección leen parte de las páginas sin ejecutar su JavaScript, y reciben el HTML tal como lo envía el servidor, antes de que el navegador lo actualice. En MailerSend, ese HTML conserva los precios de 2024 con facturación anual: 5,60 USD al mes por el plan Hobby, frente a los 6,30 que ve en el navegador cualquier cliente. Lo detectó el verificador, y la Datasheet recoge el precio

que muestra el navegador.

4.3.3. Dos casos de fuente a contradicción

Dos casos muestran la cadena completa, desde la fuente hasta la evidencia que resuelve la contradicción o la deja abierta.

El primero es de Brevo y se cierra midiendo. La página de caudal de Brevo da una fila propia de 2 peticiones por segundo, 7.200 por hora [67], a `GET /v3/smtp/emails`, una ruta de su API REST aunque el nombre diga smtp, pero tres filas más abajo pone todas las rutas bajo `/v3/smtp/` a 300 por hora. Una nota al pie exime a dos rutas, y esta no es ninguna de ellas, así que, si el prefijo alcanzara a la ruta, la Datasheet sobrestimaría su caudal 24 veces. Como la contradicción está dentro de la página, el método no la da por resuelta: modela el valor de la fila propia, en ámbar, y deja abierta la discrepancia.

Una lectura sin envío devolvió la cabecera `x-sib-ratelimit-limit: 2`, que coincide con la fila propia, pero la cabecera sola no basta para cambiar el color. El sondeo de caudal sí basta: tres peticiones en el mismo segundo devuelven 400, 400 y 429, el código HTTP con el que la API rechaza una petición por pasar del *rate*. Las dos primeras, rechazadas porque les falta un parámetro, también descuentan del contador. Justo después, una lectura de otra ruta bajo `/v3/smtp/` devuelve 200, así que el contador agotado es el de la fila y no el del prefijo.[4] El rechazo cae en la tercera petición de un segundo y no en la 301 de una hora, así que rige la fila propia, y el campo pasa a verde con 2 peticiones por segundo.

El segundo es de Amazon SES y la medición lo deja abierto. La página de límites de SES declara que todas las acciones salvo las de envío se limitan a una petición por segundo: «All actions (except for SendEmail, SendRawEmail, and SendTemplatedEmail) are throttled at one request per second» [68]. La Datasheet lo modeló en ámbar como un *rate* de 1 petición por segundo para el resto de operaciones, porque la cita nombraba operaciones de la versión 1 de la API y la Datasheet modela la versión 2. La página cambió entre el 23 y el 31 de octubre de 2026, y hoy exime las dos operaciones de envío de la versión 2, SendEmail y SendBulkEmail. El sondeo hizo dos pasadas contra `GET /v2/email/account`, una lectura sujeta a ese límite.

En la pasada secuencial, doce peticiones a 5,7 por segundo respondieron 200, sin rechazo ni cabecera de caudal. En la concurrente, treinta peticiones a 140,8 por segundo dieron 21 respuestas 200 frente a 9 rechazos 429 con el error `TooManyRequestsException`. El servicio aplica, por tanto, un límite, pero no el publicado: acepta más de cinco veces el valor declarado, y el umbral real queda entre 5,7 y 140,8 peticiones por segundo sin que dos pasadas lo fijen. Con la página nueva, que ya cubre esa operación, el valor publicado pasa a verde, pero la medición sigue sin casar con él. El caso suma a F-04, las restricciones declaradas que la respuesta no refleja[5].

[4] Evidencia en `03_datos/apis/brevo/evidencias/probe_brevo_a_429_rps_2026-09-18_195606.json`. La discrepancia está registrada en `03_datos/apis/brevo/fuentes.md`.

[5] Evidencia en `03_datos/apis/ses/evidencias/probe_ses_caudal_lectura_2026-09-18_100218.json`, con las dos pasadas. La discrepancia está registrada en `03_datos/`

API	Planes	Cuota de entrada	Caudal de envío	Cuenta del sondeo
Postmark	Free, Basic, Pro, Platform	100 al mes (Free); 10.000 al mes (Basic)	no publicado; 5.000 pet./s en cabecera •	Free en modo de prueba, sin aprobar
Resend	Free, Pro, Scale, Enterprise	100 al día y 3.000 al mes (Free); 50.000 al mes (Pro)	10 pet./s por equipo	Free sin dominio verificado
SendGrid	Free Trial, Essentials, Pro, Premier	100 al día durante 60 días (Trial); 50.000 al mes (Essentials 50K)	10.000 pet./s	prueba gratuita, envíos en modo *sandbox*
MailerSend	Free, Hobby, Starter, Professional	500 al mes y 100 peticiones al día (Free); 5.000 al mes (Hobby)	120 pet./min; 10 pet./min medidas sin aprobar	cuenta en estado Sandbox, nunca aprobada
Brevo	Free, Starter, Standard, Professional, Enterprise	300 al día (Free); 5.000 al mes (Starter)	1.000 pet./s en la categoría General •	Free, un envío real
Amazon SES	pago por uso, Essentials, Pro, Enterprise	sin cuota incluida; 200 por 24 h en pruebas	1 pet./s en pruebas; producción sin cifra	entorno de pruebas, sin acceso de producción
Mailgun	Free, Basic, Foundation, Scale	100 al día (Free); 10.000 al mes (Basic)	80.000 pet. por ventana de unos 10 s, en cabecera •	Free con dominio de pruebas, 13 envíos reales
Mailchimp Transactional	bloques de 25.000 correos, sin planes	500 envíos sin renovación (prueba)	25 correos/h en prueba; pago sin cifra	cuenta de prueba sin dominio verificado

Tabla 4.5: Las ocho Datasheets del catálogo, en el orden en que se construyeron. Cuota de entrada: la del plan gratuito o de prueba y la del primer plan de pago al primer volumen. Las ocho llegaron al tercer peldaño de la escalera de validación.

4.4. El catálogo resultante

La Tabla 4.5 resume las ocho Datasheets en el orden en que se construyeron. Las ocho llegaron al tercer peldaño de la escalera, el sondeo, y ninguna al cuarto. Ninguna cuenta contrató un plan de pago, así que lo medido describe una cuenta gratuita o de prueba, y los valores de los planes de pago salen de lo que publica el proveedor.

Cada párrafo recoge los rasgos de una API que v0.3 no puede representar, con su identificador F-nn. De F-01 a F-08 son los del piloto de la Tabla 4.3, y los siguientes se numeraron a medida que aparecían en las demás APIs. Un identificador se repite cuando el mismo rasgo vuelve a aparecer en otra API.

Postmark [33], la API piloto, se describe fase a fase en la Sección 4.2. No publica el *rate* de su API REST (F-01), así que el caudal de la tabla sale de una cabecera de respuesta y está en ámbar. La cuenta está en modo de prueba y pendiente de aprobación, y ningún mensaje con token real llegó a salir.

Resend [69] cuenta la cuota por destinatario, así que el consumo es mensajes por destinatarios (F-03), y el sondeo fija el tope de 50 sobre la suma de `to` y `cc` (26 en `to` más 25 en `cc` dan 422). Pro y Scale se venden por volúmenes contratados, y una cuota de v0.3 admite un solo volumen (F-02). Enterprise no publica precio, volumen ni *rate*, y, mientras figura en el fichero, PRIME4API no devuelve ningún plan al compararlos todos, porque un precio no público invalida la evaluación. Resend es la única API cuyo *rate* coincide en documentación, panel y cabecera.

`apis/ses/fuentes.md`.

SendGrid [15] pone techo al envío mensual de Essentials y Pro: diez veces el volumen del plan, sobrecoste incluido, y v0.3 no tiene dónde declarar ese techo (F-16). Su prueba gratuita dura 60 días y después detiene el envío, y v0.3 no tiene campo de vigencia (F-14). El relé SMTP descuenta la misma cuota que la API, pero tiene topes propios (5.000 mensajes por conexión y hasta 10.000 conexiones simultáneas desde un mismo servidor), y v0.3 no tiene dónde declarar los topes de una interfaz que no es un *endpoint* (F-15). Los envíos del sondeo usaron el modo *sandbox*.

MailerSend [11] es la única API con una cuota diaria de peticiones junto a la mensual de correos. Su caudal y sus topes por petición cambian con el estado de la suscripción y la reputación, así que el plan no basta para saber cuáles rigen (F-18). Mientras la cuenta está sin aprobar, en el estado que MailerSend llama Sandbox, que no es el modo *sandbox* de la Sección 4.3.2 porque sí entrega correo, solo puede enviar a dos destinatarios distintos, y ese cupo se encontró agotado sin que hubiera salido un solo correo y sin que se pudiera saber qué petición lo gastó. Ese cupo de la cuenta no se repone, y v0.3 no puede expresarlo (F-21).

Brevo [70] no publica el precio de un correo por encima del volumen de ningún plan (F-27). Lo único automático que documenta es el paso al escalón de volumen siguiente cuando la cuenta guarda más contactos, las direcciones de su lista de destinatarios, de los que admite el suyo. La cola de hasta 1.000 correos que Brevo retiene tras la cuota diaria del plan Free, descrita con los sondeos (Sección 4.3.2), no tiene forma en v0.3 (F-24). Publica el caudal ruta a ruta, en tres categorías, y su contrato legible por máquina no lo recoge. El caudal de todo `/v3/smtp/` se publica con un comodín de prefijo y dos rutas exentas, y v0.3 no puede escribir esa excepción (F-23).

Amazon SES [62] cobra cada correo por destinatario, a 0,10 USD por cada mil en el pago por uso y por tramos en cada plan, sin cuota incluida de la que colgar ese precio (F-29, F-30). Sus límites se aplican al par de cuenta y región, la zona geográfica de AWS en la que se usa el servicio, y el esquema no tiene dónde declarar ese ámbito (F-31). Los publica en cuatro fuentes: la página de límites, la referencia de la operación de envío, el contrato legible por máquina de la versión 2 y la guía del relé SMTP. El registro de discrepancias anota veintiuna para esta API. La cuenta no tiene acceso de producción, así que cada cifra medida es del entorno de pruebas. Como en MailerSend, el límite que rige depende del estado de la cuenta, no del plan (F-18).

Mailgun [71] tiene un selector de volumen que cambia el importe y el sobrecoste de los planes altos, y la Datasheet solo declara el tramo de entrada de cada uno (F-02). La única cifra de caudal que publica para su API REST, 1.500 peticiones por segundo, está en sus preguntas frecuentes, que dan la misma cifra para su relé SMTP (Sección 1.1.2), y está acotada a una IP de cliente, un ámbito que el esquema no sabe declarar (F-31). La Datasheet declara esas 1.500 peticiones como caudal del plan. El caudal de la Tabla 4.5, 80.000 peticiones por ventana de unos 10 segundos, es el del *endpoint* de envío y se lee en una cabecera de la respuesta. Esa misma respuesta trae un segundo contador sobre la misma ventana, de 2.000.000 de destinatarios (F-36).

Otros tres topes de Mailgun no caben bien en v0.3. El titular de una cuenta puede fijar su propio tope mensual, y el esquema no tiene sitio para un límite que pone el cliente y no el proveedor (F-37). El plan Free anuncia 100 correos al día en su página y 3.000 al mes en el panel, sin decir si los dos topes se aplican a la vez (F-38). Como v0.3

admite una sola cuota por nivel, el plan o el *endpoint*, la Datasheet cuelga la mensual del plan y la diaria del *endpoint* de envío, como si solo valiera para él. En el tamaño, Mailgun publica 25 MB por mensaje, pero la API no rechaza por tamaño un adjunto de 48 MB y solo aplica un tope de 52,4 MiB a la petición entera (F-34).

Mailchimp Transactional [72], antes Mandrill, vende bloques de 25.000 correos a seis precios según cuántos se compren, sin planes con nombre ni sobrecoste (F-32). No se vende suelto: es un complemento de los planes Standard y Premium de Mailchimp. Los 500 correos gratuitos de la cuenta de prueba no caducan ni se renuevan, y dejan vacío el periodo que v0.3 exige (F-33), un valor ausente que el motor no sabe tratar. Declara que no hay tope de destinatarios, y por encima de diez la API encola sin decir si enviará.

4.5. Conformidad y consumo por PRIME4API

PRIME4API, el motor de análisis de la capacidad de Gavira Sánchez [3, 19], ingiere una Datasheet y reduce cada pregunta a operaciones sobre sus límites, como la curva de capacidad o la recomendación de plan. En este trabajo se usa sin modificar, como motor de referencia del catálogo.

El ecosistema no tiene un validador oficial de Datasheets: lo más cercano, el editor de Datasheets de SPHERE, solo comprueba en su versión del 10 de junio de 2026 que el YAML se puede leer. Por eso la conformidad se comprueba también por uso: el motor, con la Datasheet sin manipular, tiene que responder.

Una herramienta de este trabajo ingiere cada Datasheet en el motor y la somete a diecisiete comprobaciones fijas y a una más por cada plan. Las fijas cubren la lectura del fichero, las curvas de capacidad, la curva de coste con la recomendación de plan y siete operaciones sobre los límites. La herramienta dibuja dos curvas de capacidad, y una de ellas es la acumulada, la curva $C(t)$ de la Sección 3.2: el máximo de peticiones que se pueden hacer desde el instante inicial hasta cada instante.

Las ocho Datasheets superan con cero errores el validador de formato de este trabajo (Sección 4.2), que no da error por una clave o una unidad que la guía de v0.3 no recoge. Las ocho siguen la guía salvo en dos formas que esta no define: las unidades `MBs` y `MiBs`, la segunda solo en Mailgun, para los topes de tamaño, y el registro `not_found` de lo que el proveedor no publica. Una Datasheet es plenamente consumible por PRIME4API si el motor responde sin error a todas las comprobaciones de la herramienta. Ninguna de las ocho lo es: la Tabla 4.6 recoge al menos un error en cada una.

Un fichero puede pasar el validador de formato y dejar sin responder operaciones enteras de PRIME4API. Entre el esquema y el motor que lee los ficheros hay un contrato implícito que la guía de v0.3 no escribe. El motor da por hecho que todo valor existe y que todo plan tiene precio público. Los errores de la tabla tienen tres causas. Un valor ausente que el motor convierte como nulo deja sin respuesta diez comprobaciones en Postmark, diez en Mailchimp Transactional y cinco en Resend. Un plan cuyo precio existe y no es público invalida trece en SendGrid, trece en Brevo y ocho en Resend.

Datasheet	Correctas	Con error	Causa de los errores
Postmark	11	10	Diez por valor ausente
Resend	8	13	Cinco por valor ausente y ocho por precio no público
SendGrid	8	13	Trece por precio no público
MailerSend	20	1	Uno por curva acumulada sobre un solo caudal
Brevo	9	13	Trece por precio no público
Amazon SES	20	1	Uno por curva acumulada sobre un solo caudal
Mailgun	20	1	Uno por curva acumulada con el mismo mensaje que MailerSend y Amazon SES
Mailchimp Transactional	8	10	Diez por valor ausente

Tabla 4.6: Resultado de ingerir las ocho Datasheets del catálogo en PRIME4API con la herramienta de verificación.

La tercera causa está en el motor. En MailerSend, el *workload* del lote, que dice cuánto consume cada petición, se mide en megabytes, con el tope publicado de 50 MB del cuerpo, y ninguna cuota se mide en megabytes. Es el caso de F-06: un *workload* acotado por bytes en lugar de por un entero. El motor construye para esa unidad un escenario, el conjunto de caudales y cuotas que reúne para cada unidad de consumo, con un caudal y ninguna cuota. La curva acumulada divide entonces un entero de milisegundos por un objeto de duración sin convertirlo a la misma unidad. Solo falla esa comprobación, y el resto pasa.

Amazon SES falla igual con el tope de 40 MB de su mensaje. La herramienta cuenta la curva acumulada como una sola comprobación, pero al ejecutar uno a uno los 84 escenarios de Amazon SES el fallo alcanza a 52. SendGrid lo esquiva porque su *endpoint* de envío repite el *rate* del plan, y con dos caudales el motor sí convierte la duración. En MailerSend y en Amazon SES ningún valor de la Datasheet causa el fallo, y basta con convertir la duración a milisegundos en una línea del motor para corregirlo. Mailgun termina con el mismo mensaje de error, pero su escenario no se ha aislado, así que la causa no está confirmada[6].

4.6. Del catálogo a SPHERE

Este trabajo aporta a SPHERE dos piezas. La primera es una colección con las ocho APIs del catálogo, cada una con su Datasheet y su pricing en Pricing2YAML [34]. La segunda es una propuesta de sección, en la ficha de cada API, para los términos de consumo que su pricing en Pricing2YAML no recoge, como los límites propios de cada interfaz o los que cambian con el estado de la cuenta. SPHERE, Pricing2YAML y sus herramientas son del grupo ISA, y la contribución se plantea en colaboración

[6]Salida de la herramienta en `03_datos/apis/mailgun/evidencias/verificar_prime4api_2026-09-24.txt`.

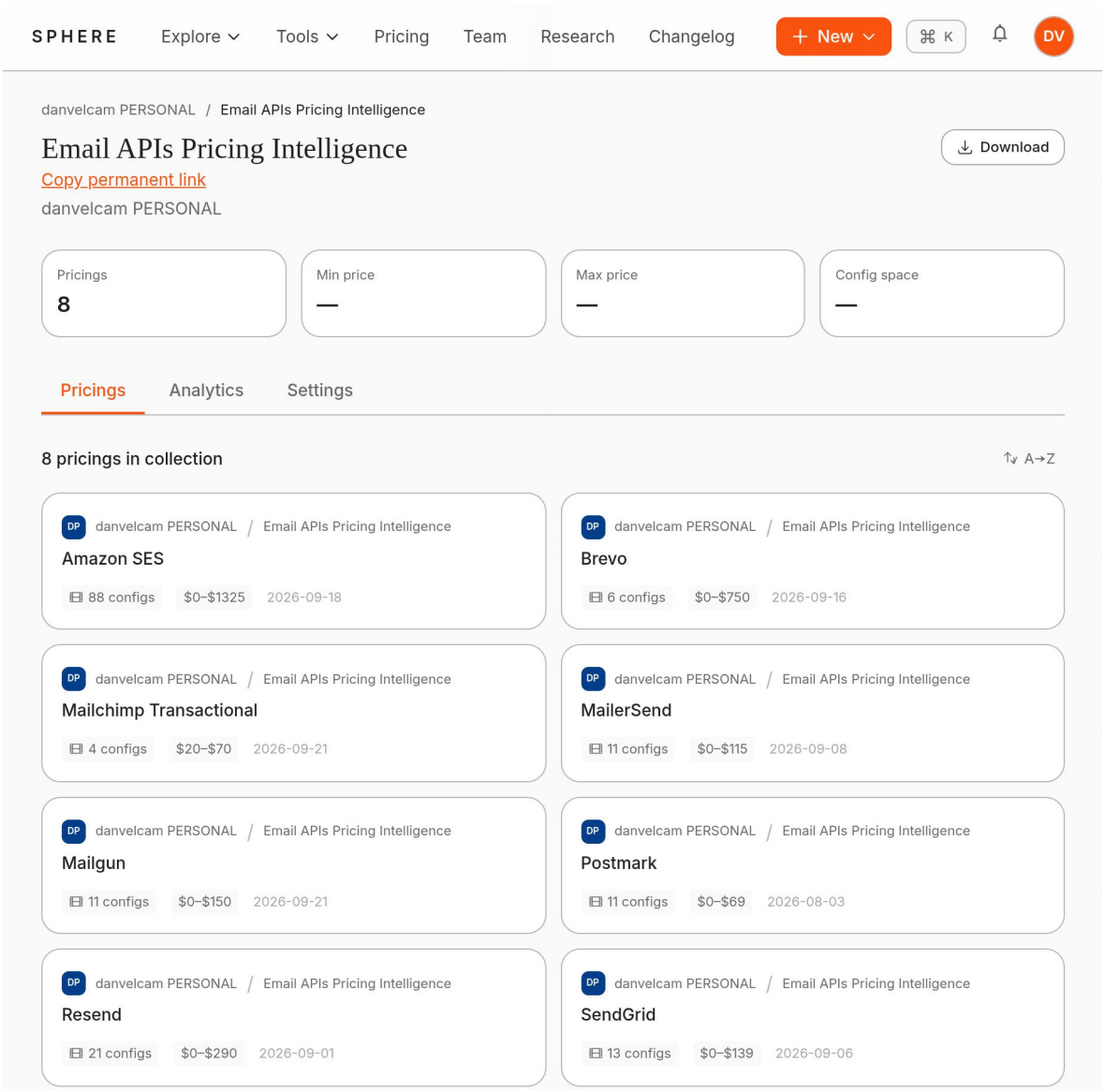


Figura 4.4: La colección «Email APIs Pricing Intelligence» en SPHERE, capturada el 28 de octubre de 2026. Cada tarjeta muestra un pricing con el tamaño de su espacio de configuración (cuántas suscripciones distintas se pueden contratar combinando un plan con sus *add-ons*) y la fecha de su Datasheet. Las cifras en USD son su precio mínimo y máximo, redondeados por SPHERE. La cabecera deja en blanco el precio y el espacio de configuración de la colección entera.

con sus desarrolladores. De este trabajo son las dos piezas y el inventario de lo que Pricing2YAML no modela.

Los ocho pricings de la primera pieza están cargados en SPHERE desde el 27 de octubre de 2026, en una colección privada hasta el depósito de este trabajo. La Figura 4.4 muestra esa colección en SPHERE.

4.6.1. La colección y su maqueta

La maqueta, un prototipo web propio que muestra cómo se vería la colección dentro de SPHERE[7], la organiza como un árbol de tres niveles: proveedor, servicio e interfaz. El árbol sigue el modelo de iCan (la plataforma del grupo que recopila los términos y condiciones de los SaaS)[8]. La maqueta se construyó por prototipos sucesivos: la primera versión agrupaba las APIs por tipo de API, y la definitiva usa ese árbol.

El árbol lo construyen dos *skills* de Ruiz López [64]. El resolvedor (`provider-service-resolver`) separa los servicios de cada proveedor y dice si cada uno lleva su pricing. Se probó en SendGrid, Amazon SES, Mailchimp Transactional y Brevo. Los tres primeros tienen varias colecciones de APIs, cada una con su pricing. Brevo es el contraejemplo: su correo transaccional va incluido en todos los planes de su plataforma de *marketing*. El analizador (`saas-interface-channel-analyzer`) enumera las interfaces de cada servicio y asigna a cada una su canal. Se aplicaron sin modificarlas a las ocho APIs, y todos los YAML de salida pasan la comprobación de estructura que trae cada una de esas *skills*.

Al elegir un servicio se ven sus interfaces y dos pestañas, cada una con sus versiones: la del pricing, con los planes, la cuota, el sobrecoste y el *rate*, y la del resto de términos de consumo, que la maqueta rotula «Otros términos» y es la sección que propone la segunda pieza.

La maqueta rellena solo lo que entra en el alcance. Solo el servicio de correo transaccional de cada proveedor lleva interfaces, Datasheet y pricing. Son 8 proveedores, 23 servicios y 60 interfaces, y los 15 servicios que el resolvedor encuentra junto al de correo transaccional se marcan fuera del alcance. Las 60 interfaces son todas las que encuentra el analizador, que cuenta como interfaz cada vía de acceso al servicio, también los *webhooks*, a los que asigna un canal de eventos. Por eso MailerSend tiene diez en la maqueta, frente a las tres de la Figura 1.1.

Cada campo de la maqueta sale de una fuente distinta. El canal de cada interfaz sale de las *skills*. Los límites y el sobrecoste de las interfaces REST y SMTP salen de la Datasheet. El tipo, el propósito y el modelo de pricing de cada interfaz se derivan del canal con una regla fija y están por validar.

La Figura 4.5 muestra la versión definitiva, con la ficha de MailerSend en la pestaña «Otros términos». Arriba está la línea temporal de las versiones de la página de límites de MailerSend que la maqueta registra, y debajo los límites del servicio, con lo que cambió respecto a la versión anterior. Las ocho APIs tienen versiones anteriores, que salen sobre todo de capturas de Wayback Machine, el archivo de versiones antiguas de páginas web de Internet Archive. Los cambios se pueden filtrar por tipo: cuotas y peticiones, *rates*, y reputación y estado de cuenta.

[7] https://danvelcam.github.io/tfm-pricing-apis-correo/sphere/coleccion.html. Es el fichero `sphere/coleccion.html` del repositorio de datos del trabajo, publicado con GitHub Pages.

[8] https://ican.score.us.es/

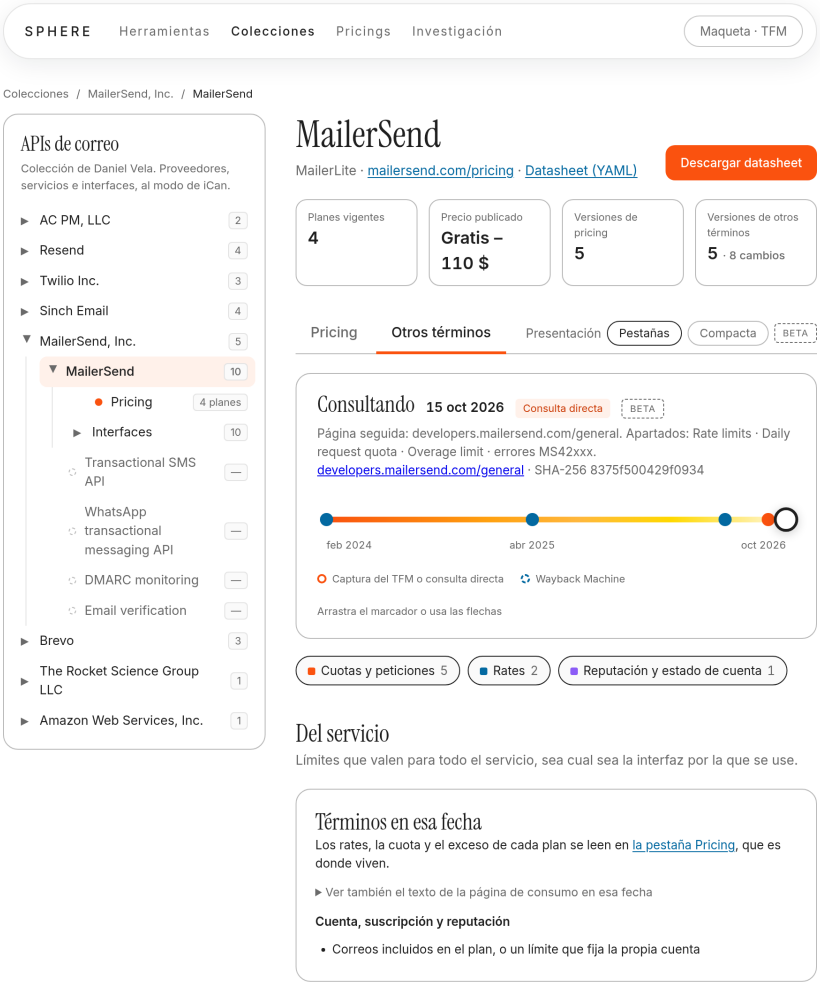


Figura 4.5: Maqueta de la colección. Ficha de MailerSend en la pestaña «Otros términos». A la izquierda, el árbol de proveedores, servicios e interfaces. A la derecha, la línea temporal de la página seguida y los límites del servicio en la versión elegida.

4.6.2. Lo que Pricing2YAML no modela

Al escribir los ocho pricings en Pricing2YAML aparecen 20 limitaciones, y cada una se documenta con el caso real que la origina, la limitación del modelo y la extensión que se propone. Los ocho ficheros pasan el analizador de pricing4ts 0.10.4 (la biblioteca con la que SPHERE lee y analiza los pricings en Pricing2YAML, en la versión que usa la plataforma) y se cargan sin errores por la interfaz de SPHERE. Las limitaciones caen en cuatro grupos, que recoge la Tabla 4.7.

Resend limita a 10 peticiones por segundo por equipo, la cuenta que comparten todas sus claves de API, y a 3.000 correos al mes en el plan Free. El primer límite protege la infraestructura y el segundo es lo que se paga, pero Pricing2YAML escribe los dos como un límite renovable con su periodo, y la analítica de SPHERE los cuenta igual. Se propone un atributo `kind` con los valores `rate` y `quota`, para que no se compare un caudal con una cuota, y el caso se repite en las ocho APIs.

MailerSend cobra 1,50 USD por cada 1.000 correos por encima de 5.000 en el plan

Hobby, y lo factura al cerrar el mes. Lo más cercano en Pricing2YAML es un *add-on* de 1.000 correos, que se compra antes de usarlo. Para el precio máximo, la analítica de SPHERE suma al plan más caro una unidad de cada *add-on*. En MailerSend sale 115,3 USD, el plan Professional más un paquete de 1.000 correos y un dominio, una cifra que no corresponde a ningún mes real. Se propone un bloque de sobrecoste en el plan que declare el precio de cada tramo y el límite al que se aplica.

Los límites de MailerSend cambian además con el estado de la cuenta: la Tabla 4.5 da 120 peticiones por minuto en el plan y 10 en una cuenta sin aprobar. Pricing2YAML, en cambio, solo admite valores que dependan del plan o de sus *add-ons*. Se propone un estado de cuenta con nombre que sobrescriba valores del plan.

Grupo	**Limitaciones**	**Lo que el modelo no admite**	**Extensión propuesta**
La forma del límite	1 a 6	El ámbito por operación, la diferencia entre caudal y cuota, el tipo de ventana, el tope por petición, y los valores ilimitado, no aplicable y a medida	Un ámbito y un tipo en cada límite, y valores reservados
Precio y consumo	7 a 12	El sobrecoste al cierre, el precio por volumen, el consumo múltiple, el precio del *add-on* según volumen y plazo, el agotamiento de la cuota y los descuentos con condición	Bloques de sobrecoste, de tramos y de descuentos, una función de consumo y una regla de agotamiento
Condiciones fuera del plan	13 a 16	El estado de la cuenta, los umbrales de uso, los dos productos bajo una cuenta y la procedencia del dato	Estados con nombre, condiciones sobre el consumo, un nivel de producto y un campo de procedencia
Clave, plazo y divisa	17 a 20	El precio por uso sin volumen incluido, la clave del contador (cuenta, región o IP), los planes con plazo y la divisa	Tramos marginales, un atributo de clave, un bloque de plazo y precios por divisa

Tabla 4.7: Las 20 limitaciones de Pricing2YAML en las ocho APIs, agrupadas. Cada una tiene su caso real, su limitación y su extensión en el repositorio de datos del trabajo.

4.6.3. Términos de consumo y versionado

La extensión de SPHERE que se propone guardaría en bruto los términos de consumo y repartiría lo que sale de ellos en dos clases: lo que se ingiere y lo que se deriva. Lo publicado, lo interpretado y lo medido entrarían en SPHERE por una ingesta, con las fechas del proveedor y un estado, pendiente de validar o validado. Ese estado llevaría a SPHERE la idea de la Sección 4.3: un valor de la página del proveedor entraría como pendiente de validar, y pasaría a validado cuando un contraste, un panel o un sondeo lo confirmara.

Lo calculado y lo inferido por agente son derivados, pero no son lo mismo (Sección 4.3.1). Lo calculado se reproduce con su fórmula y sus entradas, como el coste que da PRIME4API. Lo inferido no lo reproduce ninguna fórmula, y por eso exige validación: no pasa de ámbar hasta que lo respalde un valor publicado o medido. SPHERE versionaría los dos con sus ramas, las líneas de versiones que guarda de cada pricing, junto a su fórmula o su *skill*.

La extensión separaría además los límites según su ámbito: unos valen para una sola interfaz y otros se comparten entre varias. En el plan Starter de MailerSend, el relé SMTP admite como máximo cinco conexiones simultáneas, que es un límite de una sola interfaz. MailerSend descuenta de una misma cuota diaria las peticiones de la API y los mensajes del relé SMTP, que es un límite compartido. Esa separación en SPHERE es un diseño propuesto y no está implementado.

SPHERE ya versiona los pricings que suben sus usuarios. La maqueta versiona lo que el proveedor publicó en cada fecha junto a su URL y su huella. Son dos registros distintos: una rama de SPHERE no dice cuándo cambió el proveedor su página, y una captura fechada de esa página no dice quién cambió el pricing en la plataforma. La extensión que se propone guardaría aparte la procedencia fechada del proveedor y la enlazaría a las ramas de SPHERE, sin sustituirlas.

4.7. Respuesta a la pregunta de investigación

La RQ1 pregunta qué APIs de correo deben incluirse en un análisis representativo del dominio, y bajo qué criterios reproducibles. La primera mitad se responde con una condición: el conjunto debe cubrir las cuatro formas de cobro del primer criterio (volumen mensual, pago por uso, producto añadido y límite diario). Sus APIs deben diferir además en los rasgos de límites que pide el cuarto, como un *rate* por *endpoint* o un caudal negociado. Las ocho APIs de la Tabla 4.5 cumplen esa condición en la ventana de toma de datos, y otro conjunto que cubriera lo mismo también la cumpliría. La Tabla 4.8 da, para cada forma de cobro y para el cuarto criterio, una API y el dato de su Datasheet que la cumple, de modo que la condición se comprueba sin repetir la consulta al modelo de lenguaje.

La segunda mitad la responde el protocolo de la Sección 4.1 (tres fuentes de inclusión, cinco criterios ordenados y una regla de reemplazo), con la consulta registrada en el Anexo A.

La construcción encontró las cuatro formas de cobro y corrigió tres afirmaciones de la respuesta. La respuesta presentaba Amazon SES como pago por uso puro, pero SES publicaba planes desde el 21 de julio de 2026, antes de la consulta. También presentaba Mailgun como «el caso limpio de rate documentado», con *rates* publicados por *endpoint*, pero Mailgun no los publica en la documentación de su API (Sección 4.4). A Brevo le atribuía la cuota diaria junto a la mensual, pero su plan Free solo tiene la diaria y sus planes de pago, la mensual, y la combinación estaba en el plan Free de Resend. Ninguna corrección activa la regla de reemplazo, que solo depende de la trazabilidad.

Condición	**API y dato de su Datasheet que la sostiene**
Volumen mensual	Postmark Basic, 10.000 correos al mes; SendGrid Essentials 50K, 50.000 correos al mes
Pago por uso	Amazon SES, sin cuota incluida y con precio por destinatario, 0,10 USD por cada mil en el pago por uso
Producto añadido	Mailchimp Transactional, bloques de 25.000 correos que solo se venden con un plan Standard o Premium de Mailchimp
Límite diario además del mensual	Resend Free, 100 correos al día y 3.000 al mes; MailerSend Free, 100 peticiones al día y 500 correos al mes
Rasgos de límites distintos (cuarto criterio)	Brevo publica el *rate* ruta a ruta, con 2 peticiones por segundo en `GET /v3/smtp/emails`; Postmark no publica el de su API REST, que se lee en una cabecera; Mailgun acota a una IP de cliente sus 1.500 peticiones por segundo

Tabla 4.8: Cobertura de la condición de la RQ1, comprobada sobre las Datasheets del catálogo. Cada fila da un ejemplo entre las APIs que la cumplen.

Las cuatro primeras APIs, Postmark, Resend, SendGrid y MailerSend, tenían planes con cuota mensual, un sobrecoste por correo que baja al subir el volumen y tope de destinatarios. Solas no cubrían la diversidad del primer criterio. Amazon SES no tiene cuota incluida, Brevo no publica el precio por encima del volumen y Mailchimp Transactional no tiene planes ni sobrecoste. Mailgun aporta el rasgo del cuarto criterio, porque es la única de las ocho que publica, en sus preguntas frecuentes, cuántas peticiones admite su API REST por IP de cliente.

La selección tiene tres límites conocidos. La cuota de mercado de cada API no se midió y no entra en la condición. Los cinco criterios solo decidieron cuatro de las ocho APIs, y la respuesta de la consulta no se repite con garantía.

5. Evolución del esquema ACM2YAML

Este capítulo presenta la contribución C-02 (Sección 1.4) y responde a la RQ2: si ACM2YAML es suficientemente expresivo para representar la capacidad contractual de las APIs de correo, y qué extensiones requiere. Al escribir las ocho Datasheets del catálogo contra ACM2YAML v0.3 aparecieron treinta y cinco hechos del dominio que el esquema no puede expresar o que PRIME4API no consume. El primero salió del piloto: Postmark vende seis volúmenes contratados dentro de un mismo plan, de 10.000 a 1.500.000 mensajes, y una cuota de v0.3 solo admite una cifra y un sobrecoste. El esquema es de Gavira Sánchez y Ruiz López [3, 4], y este trabajo lo extiende con propuestas sobre sus campos, cada una con el caso real que la motiva.

El capítulo se organiza del siguiente modo. En primer lugar, la Sección 5.1 describe cómo un elemento no soportado llega a propuesta de extensión y distingue dos clases de carencia, la de expresividad y el defecto de contrato. A continuación, la Sección 5.2 presenta el inventario de los treinta y cinco elementos, con su recurrencia en las ocho APIs y el origen de las nueve propuestas. Después, la Sección 5.3 formula las siete propuestas de expresividad, y la Sección 5.4, las dos de contrato y una rectificación de la guía. La Sección 5.5 recoge los elementos promovidos sin propuesta y la compatibilidad de las propuestas con v0.3. Por último, la Sección 5.6 responde a la RQ2.

5.1. Cómo una carencia llega a propuesta

Las ocho Datasheets del catálogo se escriben contra la guía de ACM2YAML v0.3, el documento que describe el esquema con ejemplos, sin inventar campos [61]. Las dos formas que quedan fuera de la guía, la unidad `MBs` de los topes de tamaño y el registro `not_found` de lo que el proveedor no publica, ya se describieron junto al consumo del catálogo por PRIME4API (Sección 4.5). Escribir sin campos propios tiene una ventaja: el catálogo sirve aunque no se acepte ninguna extensión, porque PRIME4API lo lee tal cual, con los fallos que recoge la Tabla 4.6.

Cuando un proveedor publica algo que el esquema no puede representar, no se añade a la Datasheet ningún campo propio. El hecho se registra como «elemento no soportado»: un hecho del dominio que ACM2YAML v0.3 no puede expresar, como los seis volúmenes contratados de Postmark, o que expresa de una forma que PRIME4API, el motor de referencia, no consume. Cada elemento lleva un número F-nn, el caso real que lo motiva, la consecuencia para el análisis y, cuando lo hay, un esbozo de resolución. En el repositorio de datos estos registros se llaman «fricciones» (fichero `fricciones.md`), y de ahí viene la F del número.

Un elemento no soportado se promueve, es decir, pasa a candidato de extensión del esquema, por una de dos vías. La primera es la recurrencia: el elemento reaparece en al menos dos APIs del catálogo. Que Postmark venda seis volúmenes contratados

para su cuota mensual podría ser una rareza suya, y verlo en un segundo proveedor indica que pertenece al dominio.

La segunda vía es la traza, que admite una sola API si se cumplen a la vez tres condiciones. La condición L1 pide que el hecho esté citado con su literal o medido en un sondeo, con la evidencia archivada en el repositorio. La condición L2 pide la consecuencia ejecutada sobre el motor: con la Datasheet sin manipular, PRIME4API devuelve un valor erróneo o deja sin responder lo que respondería si el esquema pudiera expresar el hecho. Pide además un caso de control: la misma operación del motor responde bien cuando el dato cabe en el esquema. La condición L3 pide que ningún mecanismo existente de la guía cubra la carencia y que no la explique un defecto ya conocido del motor, lo que descarta una mala lectura de la guía.

Quedar promovido no equivale a ser propuesta de extensión. Solo lleva número de propuesta, P-nn, el elemento promovido cuya forma de extensión se ha comprobado contra los campos que define la guía. La propuesta P-05 es la excepción: no sale de ningún elemento no soportado y trata el plan cuyo precio existe pero no se publica.

Un elemento que aparece en una sola API y no deja traza sobre el motor no se añade al esquema, aunque esté bien documentado. Queda en el inventario sin promover, y su fila anota la condición que no cumple. El criterio mide la generalidad dentro del catálogo, y no se ha comprobado si los once elementos no promovidos (Sección 5.2) aparecen en APIs de fuera del catálogo.

La recurrencia muestra que una carencia pertenece al dominio, pero no que la forma elegida para sortearla funcione. Esa segunda comprobación exige una herramienta que lea la Datasheet y diga si la forma elegida se procesa. Ninguno de los repositorios del ecosistema, ni el de la guía ni los de PRIME4API y SPHERE, incluye un esquema JSON, un analizador estático o una integración continua que compruebe un fichero `datasheet.yaml`. PRIME4API es un motor de cálculo y no un validador: lee los campos del YAML sin comprobarlos antes, y sus esquemas describen las peticiones de su propia API REST [19]. El repositorio de la guía aporta la norma y los ejemplos, pero no un comprobador [61].

Gavira Sánchez describe así el editor de Datasheets de SPHERE: «Cada cambio en el documento se valida contra la sintaxis del modelo» [3]. En el repositorio de SPHERE, en su versión del 10 de junio de 2026, la página de subida exige la extensión `.yaml` o `.yml` y lee el YAML solo para sacar su nombre y su versión, y el editor comprueba que el YAML se pueda leer. En ese código no se ha encontrado una validación contra los campos del modelo. Qué comprueba el editor en la versión desplegada de la plataforma [22] no se ha verificado.

Por eso la conformidad se establece por uso: una herramienta de este trabajo ingiere cada Datasheet en PRIME4API y la somete a diecisiete comprobaciones fijas y a una más por cada plan (Sección 4.5). Cada propuesta declara así dos cosas separadas: en cuántas APIs del catálogo aparece la carencia y si hay traza de ejecución que muestre su efecto sobre el motor. La primera es un argumento sobre el dominio, y la segunda, sobre el esquema.

Los elementos no soportados se reparten en dos clases, y cada una se prueba de forma distinta. Una «carencia de expresividad» aparece cuando el proveedor publica

algo que el esquema no tiene dónde poner. Postmark vende seis volúmenes contratados para su cuota mensual, cada uno con su precio y su sobrecoste. Una cuota de v0.3 solo tiene un `value` escalar y un `overage_cost` único, así que no hay campo donde escribir la lista. Esta carencia se detecta al modelar, porque el fichero no admite el dato.

Un «defecto de contrato» aparece cuando el esquema sí ofrece una forma, la Datasheet la usa bien y PRIME4API falla al leerla: se rompe el contrato implícito por el que el motor lee todo lo que la guía admite. La guía admite dejar vacío un valor cuando no hay cifra que escribir. En Postmark y en Mailchimp Transactional queda vacío el tope de destinatarios, que la fuente no pone, y en Resend el *rate* del plan Enterprise, que existe y no se publica. El motor convierte ese valor a número sin comprobar si está vacío, y la Datasheet, aun siendo conforme, deja sin responder diez comprobaciones en Postmark y en Mailchimp Transactional (Tabla 4.6). Este defecto solo se detecta al ejecutar el motor sobre el fichero, porque al modelar la Datasheet se escribe sin resistencia.

La primera consecuencia de la distinción está en la vía de promoción. La recurrencia muestra que la carencia pertenece al dominio. La traza muestra que el motor da un resultado erróneo o deja sin responder la operación, y esa prueba vale en una sola API para cualquiera de las dos clases. Un defecto de contrato se descubre ejecutando el motor, así que llega ya con esa ejecución. Una carencia de expresividad sin traza, como los volúmenes de Postmark, necesita aparecer en una segunda API.

La clase de carencia decide también dónde se arregla y en qué repositorio se comunica. Los volúmenes de Postmark solo caben si cambia el esquema y la cuota admite una lista de volúmenes. El campo vacío admite el arreglo en el esquema, en el motor o en los dos, y cada propuesta dice en cuál. El cambio de la cuota se comunica en el repositorio de la guía, y la comprobación del campo vacío, en el del motor. La separación se ha obtenido sobre un solo esquema, ACM2YAML v0.3, y un solo motor, PRIME4API, por lo que no se afirma que valga fuera de ese caso.

5.2. Inventario

El piloto con Postmark dejó ocho elementos no soportados por ACM2YAML v0.3, los de la Tabla 4.3. Las siete APIs siguientes abrieron treinta números, de F-09 a F-38, y tres se retiraron o se absorbieron en otros (F-17, F-19 y F-20), así que el inventario cierra con treinta y cinco. La Tabla 5.1 recoge cada uno con el número de APIs en que aparece, si promueve a candidato de extensión y por qué vía, y la propuesta que produce cuando la hay.

El criterio de promoción de la Sección 5.1 pide dos APIs o más o, en una sola, la traza con sus tres condiciones (L1 a L3). Con ese criterio promueven veinticuatro de los treinta y cinco elementos, veintiuno por recurrencia y tres por traza. Los otros once aparecen en una sola API y les falta L2, L3 o las dos. Si en una API el elemento se sospecha pero no se ha comprobado, esa API no suma. Al comprobarse solo puede sumar, así que cada recuento de la Tabla 5.1 es un mínimo.

La mayoría de los recurrentes aparece en cuatro APIs o más: trece de veintiuno.

F-06 y F-07 aparecen en las ocho APIs: el primero es un `workload` (el campo que declara, con un mínimo y un máximo, cuánto consume una petición) acotado por una métrica distinta de la suya, y el segundo, los *add-ons* que modifican la capacidad. F-05, la dimensión *Results Per Request* que v0.3 no serializa, aparece en siete. En el otro extremo, F-12 (la recepción consume la cuota de envío) y F-14 (el plan caduca) cumplen el umbral con dos APIs justas. Lo mismo ocurre con F-34 (dos topes de tamaño sobre la misma llamada) y F-37 (un tope de capacidad que fija el cliente).

La Tabla 5.2 dice de dónde sale cada propuesta. De las nueve, ocho salen de un elemento promovido. Cinco entran por recurrencia. Una de ellas, P-04, sale de F-01, el *rate* sin valor publicado, y tiene también la traza del campo vacío en tres Datasheets. Las otras cuatro salen de F-02 (el volumen contratado con precio propio), F-03 (el consumo multiplicativo), F-06 (el `workload` acotado por otra métrica) y F-18 (la capacidad que depende de la suscripción o de la reputación).

Tres entran por traza, y en las tres PRIME4API da un resultado que el proveedor contradice. En F-16, el tope que SendGrid impone al sobrecoste, el motor promete un volumen que SendGrid rechaza. En F-21, el cupo de destinatarios distintos de MailerSend, el motor declara alcanzable en cero segundos un objetivo que la medición no consiguió enviar. En F-24, la cuota superada de Brevo que retiene un número fijo de envíos, el motor la trata como un corte estricto: al llegar a la cifra no se envía nada más. Brevo, en cambio, publica esa cuota con una cola de reintento.

La novena, P-05 (el plan con precio que existe y no es público), no tiene fila en el inventario. Su carencia apareció al evaluar la Datasheet de Resend con PRIME4API, que deja de evaluar el documento entero cuando el precio de un plan lleva el valor `custom`. Se anotó, como la forma de `workload` de la Sección 5.4, entre las discrepancias de la guía con PRIME4API, y no entre los elementos no soportados, así que no tiene número F-nn. Los otros dieciséis promovidos no llevan número de propuesta.

Al pasar de las cuatro primeras APIs (Postmark, Resend, SendGrid y MailerSend) a las ocho, ningún elemento que promovía deja de hacerlo, y dos que no promovían pasan a hacerlo. F-18, la capacidad que depende de la suscripción o de la reputación, aparece ahora en seis APIs, porque en Postmark y en Resend se comprobó al revisar las ocho. F-22, la credencial que solo se gestiona desde la web, aparece en cuatro. Entre ellas cuenta Amazon SES, donde esa credencial se gestiona en IAM, el servicio de identidades de AWS, fuera de la API de correo, una variante que cumple el enunciado con otro mecanismo y por eso suma.

Dos reglas fijan cómo se cuenta. La primera es que una aparición parcial no suma. Una variante suma si llega por otro mecanismo al mismo efecto sobre quien integra la API, como la credencial de Amazon SES gestionada en IAM. No suma si cumple solo una parte del enunciado, como el volumen contratado de Brevo, que no lleva sobrecoste.

F-01 aparece en las ocho APIs, pero solo cuenta en las cuatro que admiten que el límite existe y no le dan cifra. Postmark describe el error `429` sin umbral [63], y Amazon SES escribe que su caudal de envío «varies based on your specific use case» [68]. Mailchimp no cifra la cuota horaria de la cuenta de pago[1]. Mailgun solo publica, en

[1]Registro de procedencia de la cuota horaria en `03_datos/apis/mailchimp/datasheet.yaml` del repositorio del trabajo, que remite a https://mailchimp.com/developer/transactional/docs/

Id	Elemento	APIs	Vía	Prop.
F-01	Un *rate* que existe y cuyo valor no se publica	4/8	recurr. y traza	P-04
F-02	Volumen contratado con precio y sobrecoste propios	5/8	recurr.	P-01
F-03	Consumo multiplicativo entre dimensiones anidadas	3/8	recurr.	P-02
F-04	Restricciones declaradas que el proveedor admite no aplicar o que la respuesta no refleja	4/8	recurr.	discus.
F-05	*Results Per Request* documentada y no serializable	7/8	recurr.	—
F-06	`workload` acotado por una métrica distinta de la suya	8/8	recurr.	P-03
F-07	*Add-ons* que modifican dimensiones de capacidad	8/8	recurr.	—
F-08	Los *endpoints* disponibles cambian con el *tier*	4/8	recurr.	—
F-09	El documento caduca entero, y sus datos caducan uno a uno	5/8	recurr.	—
F-10	Dos productos de precio con unidades de consumo distintas	3/8	recurr.	—
F-11	Límites de recursos por plan, que no son consumo	6/8	recurr.	—
F-12	La recepción consume la cuota de envío	2/8	recurr.	—
F-13	El sobrecoste es opcional y no existe en el plan gratuito	5/8	recurr.	—
F-14	El plan tiene fecha de caducidad	2/8	recurr.	—
F-15	Dos interfaces con topes propios sobre la misma cuota	5/8	recurr.	—
F-16	El sobrecoste tiene un tope que impone el proveedor	1/8	traza	P-07
F-18	La capacidad depende del estado de la suscripción o de la reputación, no del plan	6/8	recurr.	P-06
F-21	Un cupo de destinatarios distintos que se gasta al probar	1/8	traza	P-08
F-22	Una credencial que solo se gestiona desde la web	4/8	recurr.	—
F-23	Un comodín de prefijo con excepción	1/8	L3	—
F-24	Una cuota superada que retiene hasta un número fijo de envíos y deja de entregar los siguientes	1/8	traza	P-09
F-25	Un *endpoint*, dos modos de envío y una carga que depende de otro campo	1/8	L3	—
F-26	Un precio que depende de la geografía y una sola divisa en el fichero	3/8	recurr.	—
F-27	Un exceso que cambia el plan en vez de pagarse por unidad, disparado por otra dimensión	1/8	L2	—
F-28	Una lista de IP autorizadas decide si la llamada entra	3/8	recurr.	—
F-29	Un precio por unidad sin cuota incluida de la que colgarlo	1/8	L3	—
F-30	Precio por tramos marginales sobre el mismo consumo	1/8	L2 y L3	—
F-31	La unidad de gobierno es el par cuenta × región	1/8	L2	—
F-32	Precio unitario degresivo sin planes que lo sostengan	1/8	L2	—
F-33	Una franquicia que no caduca ni se renueva: la cuota sin ventana	1/8	L2	—
F-34	Dos topes de tamaño sobre la misma llamada, uno del mensaje y otro de la petición	2/8	recurr.	—
F-35	El literal de un tope trae una cifra fraccionaria y el máximo de un `workload` es entero	1/8	L2 y L3	—
F-36	Dos contadores sobre la misma ventana, y un solo caudal por nodo	1/8	L2	—
F-37	Un tope de capacidad que fija el cliente, no el proveedor	2/8	recurr.	—
F-38	Dos cuotas concurrentes de la misma unidad	1/8	L2	—

Tabla 5.1: Inventario de elementos no soportados sobre las ocho APIs del catálogo. APIs: en cuántas de las ocho aparece el elemento entero, sumando la variante que cumple el enunciado con otro mecanismo. Vía, recurr.: dos APIs o más. traza: una sola API con las tres condiciones cumplidas (Sección 5.1). En los once no promovidos, la columna Vía indica la condición de la traza que no se cumple: L2, la consecuencia ejecutada sobre PRIME4API con caso de control, o L3, que ningún mecanismo de la guía cubra la carencia ni la explique un defecto conocido del motor. Prop., discus.: promovido que se discute sin formular extensión. Los números que faltan corresponden a elementos retirados o absorbidos por otros. La matriz completa, con la marca de cada API y su fuente, está en `03_datos/RECURRENCIA_2026-09-23.md` del repositorio del trabajo.

Propuesta	Qué pide	Elemento	APIs	Vía	Clase
P-01	Volúmenes contratados en la cuota, con precio y sobrecoste propios	F-02	5/8	recurr.	expresividad
P-02	Consumo de una petición como producto de dimensiones anidadas	F-03	3/8	recurr.	expresividad
P-03	Tope de una carga expresado en otra métrica	F-06	8/8	recurr.	expresividad
P-04	Marca de valor no publicado junto al campo	F-01	4/8	recurr. y traza	contrato
P-05	Plan con precio que existe y no es público	—	—	motor	contrato
P-06	Estado de la suscripción y reputación, declarados una vez y con su disparador	F-18	6/8	recurr.	expresividad
P-07	Tope del sobrecoste	F-16	1/8	traza	expresividad
P-08	Cuota de entidades distintas, sin periodo y con ámbito de cuenta	F-21	1/8	traza	expresividad
P-09	Comportamiento de la cuota al superarse	F-24	1/8	traza	expresividad
Rectif.	Forma de lista de `workload` en la guía	—	—	—	guía

Tabla 5.2: Propuestas del capítulo y su origen. Elemento: el elemento no soportado del inventario del que sale cada propuesta. APIs: su recuento sobre las ocho del catálogo. Vía, recurr.: la carencia aparece en dos APIs o más. traza: una sola API con las tres condiciones de la Sección 5.1. motor: PRIME4API falla al leer una forma que la guía admite. Clase: expresividad, si el dominio dice algo que v0.3 no puede escribir, o contrato, si v0.3 lo admite y el motor no lo consume. En P-04 los dos apoyos no se refieren al mismo hecho: la recurrencia es la de F-01, un *rate* sin cifra, y la traza es la del campo vacío en Postmark, Resend y Mailchimp Transactional, que solo en Resend es un *rate*. P-05 no sale de ningún elemento del inventario, y el criterio de promoción no se le aplica. La última fila rectifica la guía y no extiende el esquema.

sus preguntas frecuentes, un caudal de 1.500 peticiones por segundo por IP de cliente, y deja sin cifra el de la cuenta[2].

En las otras cuatro, Resend, SendGrid, MailerSend y Brevo, una cifra cubre una parte del límite y deja otra sin cubrir. Brevo, por ejemplo, cifra los *endpoints* de su API REST y no su relé SMTP. Contar esas cuatro como plenas habría subido el recuento de F-01 de cuatro a ocho con situaciones que no son la misma.

La segunda regla afecta a F-08, los *endpoints* que cambian con el *tier*. El registro de cada API lo enuncia como el plan real que combina *tier* y volumen contratado, pero

`reputation-rejections/`.

[2] https://www.mailgun.com/ai-info/. La cita literal está en el registro de procedencia de `03_datos/apis/mailgun/datasheet.yaml`.

el volumen contratado ya lo cubre F-02, y en F-08 solo cuenta la superficie de la API. Solo cuenta que el plan cambie los *endpoints* disponibles, como el envío por lotes, que la Datasheet de MailerSend omite de su plan gratuito. Los límites de recursos por plan se cuentan aparte, como F-11. Con esa regla, F-08 aparece en Postmark, SendGrid, MailerSend y Brevo. F-08 no produce propuesta, porque v0.3 ya admite esa forma: un *endpoint* que un plan no ofrece se omite de ese plan. Promueve igualmente, porque la recurrencia cuenta en cuántas APIs aparece el patrón y no comprueba si el esquema ya lo expresa.

En treinta y una combinaciones de elemento y API la aparición no está verificada. En cuatro filas bastaría con confirmar una sola de esas combinaciones para cambiar la vía. F-31 (los límites que rigen por cuenta y región), F-35 (el tope fraccionario frente al máximo entero) y F-36 (dos contadores sobre la misma ventana) pasarían a recurrentes, y F-16 pasaría de la traza a la recurrencia. En las demás filas con apariciones sin verificar la vía no cambia, porque ya promueven por recurrencia.

F-38, dos cuotas concurrentes de la misma unidad, cuenta en Resend y pasaría también a recurrente si se confirma que Mailgun aplica las dos a la vez. Esa combinación no cuenta entre las treinta y una: está en suspenso, porque Mailgun publica cada cuota en una página distinta y ninguna de las dos dice si se aplican a la vez.

F-37 es el caso más frágil. Aparece entero en Mailgun y promueve gracias a Brevo, cuya solución Corporate deja a la cuenta maestra fijar cuántos correos puede enviar cada una de sus subcuentas, una variante que cumple el enunciado con otro mecanismo. Si esas variantes no se contaran, F-37 se quedaría solo en Mailgun.

5.3. Propuestas de expresividad

Siete de las nueve propuestas responden a una carencia de expresividad: el proveedor publica un límite y ACM2YAML v0.3 no tiene dónde escribirlo. Son la cuota con varios volúmenes contratados (P-01), el consumo por petición que multiplica dos factores (P-02) o que tiene la cota en otra métrica (P-03) y los estados de la suscripción y de la reputación (P-06). Completan las siete el tope del sobrecoste (P-07), el cupo de destinatarios distintos que no se repone (P-08) y lo que hace la cuota con lo que la supera (P-09). Las dos restantes, P-04 y P-05, son defectos de contrato, porque v0.3 admite la forma y el motor no la consume. Son un límite que existe y cuyo valor no se publica y un plan con precio que existe y no es público. Las extensiones se escriben contra la guía de v0.3 [61] y no sustituyen ningún campo que ya exista.

5.3.1. P-01. La cuota declara volúmenes contratados

El caso. Los tres planes de pago de Postmark se anuncian como *«Starting at 10,000 emails/month»* [33]. La estructura completa solo se ve desde el panel de una cuenta propia: seis volúmenes contratados, de 10.000 mensajes por 15 USD a 1.500.000 por 775 USD, con negociación directa por encima del último. En el *tier* Basic el sobrecoste baja de 1,80 a 1,70 USD por millar al pasar de 125.000 a 300.000 mensajes

(Figura 4.3, que cruza los tres *tiers* con los seis volúmenes).

Resend vende la misma estructura y la publica en su página de precios [69]. Su plan Pro tiene dos volúmenes, de 50.000 mensajes por 20 USD y de 100.000 por 35 USD. Su plan Scale recorre seis, de 100.000 mensajes por 90 USD a 2.500.000 por 1.150 USD, y el sobrecoste baja de 0,90 a 0,46 USD por millar. Resend permite demostrar la carencia con una página pública de precios, sin cuenta propia. Mailgun publica su lista de volúmenes en un selector de la página de precios, y en su plan Scale el sobrecoste baja de 1,10 a 0,40 USD por millar [71].

La carencia aparece en cinco de las ocho APIs: Postmark, Resend, Mailgun, SendGrid y MailerSend (Tabla 5.1). Brevo también vende volúmenes contratados, pero sin sobrecoste, y esa variante cumple solo una parte del enunciado, así que no suma al recuento (Sección 5.2).

Qué permite v0.3. Una cuota de v0.3 declara un `value` escalar y un solo sobrecoste en `overage_cost`. No cabe la lista de volúmenes contratados ni el sobrecoste propio de cada uno. Las Datasheets de Postmark, Resend y Mailgun modelan por eso el volumen mínimo de cada plan. La lista completa solo queda en el registro de procedencia, en prosa que ninguna herramienta lee.

Consecuencia. La Datasheet de Resend afirma que Scale son 100.000 mensajes con un sobrecoste de 0,90 USD por millar. Es cierto en el primer volumen contratado y falso en el último, donde el mismo plan son 2.500.000 mensajes a 0,46 USD por millar. Comparar ese plan con el de un proveedor de volumen fijo usando el único número serializado falsea la comparación de precios.

Propuesta. Que una cuota admita una lista de volúmenes contratados, cada uno con su precio y su sobrecoste, más una entrada explícita para el tramo negociado. El Extracto de código 5.1 muestra la forma sobre el plan Scale de Resend.

Extracto de código 5.1: P-01: volúmenes contratados dentro de un plan, con precio y sobrecoste propios (caso Resend, plan Scale, tres de los seis volúmenes).

```
quota_email_transactional_monthly_scale:
  unit: emails
  period: {value: 1, unit: MONTH}
  window_type: fixed_billing_cycle
  contracted_volumes:
    - value: 100000
      price: 90
      overage_cost: {price: 0.90, value: 1000}
    - value: 500000
      price: 350
      overage_cost: {price: 0.70, value: 1000}
    - value: 2500000
      price: 1150
      overage_cost: {price: 0.46, value: 1000}
```

Efecto sobre PRIME4API. El motor calcula la curva de coste con el precio del plan y el sobrecoste de la cuota. Con un único par de valores la curva es una recta, y el resultado solo es correcto en el volumen modelado. Con la lista, la curva cambia de pendiente en cada volumen contratado, que es lo que el proveedor cobra. La recomendación de plan que ya ejecuta el TFM de Gavira Sánchez (su Tabla 5.12, para 100.000 correos con sobrecoste) depende de esa curva [3].

5.3.2. P-02 y P-03. El `workload` deja de ser una terna escalar

Las dos propuestas salen de dos elementos no soportados que recurren por separado: F-03, el consumo de una petición multiplica dos factores, y F-06, la cota de la petición está en otra métrica. Los dos piden que el consumo de una petición deje de expresarse con una sola terna `unit`, `min`, `max`. Se presentan juntas porque reformar el campo por separado obligaría a proponer dos veces la misma forma de `workload`.

El caso de P-02. El *endpoint* `/emails/batch` de Resend acepta *«up to 100 batch emails at once»* [73]. Cada mensaje admite cincuenta destinatarios, y un sondeo con cuenta propia contra Resend mostró que el tope cuenta la suma de `to` y `cc` y no cada campo por separado. Contar cada destinatario como un correo podría ser una elección de modelado, pero Resend declara cómo cuenta los destinatarios: *«Multiple To, CC, or BCC recipients count as separate emails»* [73]. El consumo de una llamada es, por tanto, la suma de los destinatarios de sus mensajes, y su máximo, el producto de los dos topes: 100 mensajes por 50 destinatarios.

En Postmark, `/email/batch` admite quinientos mensajes por llamada y cincuenta destinatarios en total por mensaje [63]. La documentación declaraba ese tope por campo, y el sondeo del piloto (Sección 4.2), que midió el tope sobre la suma de `to` y `cc`, lo corrigió. Postmark no declara cómo cuenta ese consumo, y en su caso el producto de los dos factores se deduce de los dos topes medidos.

Amazon SES es el tercer caso: su envío en lote recibe una lista de entradas con sus propios destinatarios, y su cuota cuenta destinatarios y no mensajes [68]. SendGrid, MailerSend y Brevo ponen un tope sobre el total de destinatarios de la petición, y esa

variante acota el total sin multiplicar dos factores, así que cumple solo una parte del enunciado y no suma al recuento (Sección 5.2).

Consecuencia de P-02. La Datasheet de Resend serializa `max: 100`, que es el número de mensajes y no el de unidades consumidas. El factor de consumo por petición (el CRF, que la Datasheet serializa en `workload`) alimenta las preguntas del motor sobre agotamiento de cuota y sobrecoste. Con ese valor, el motor subestima hasta cincuenta veces el consumo de una petición, y quien consulta el motor no puede distinguir ese número, que es plausible, de uno correcto.

El caso de P-03. El *endpoint* `/email/bulk` de Postmark no tiene tope de mensajes. El proveedor admite *«as many recipients as you need, as long as your total payload (including attachments) stays under 50 MB»* [63]. La cota efectiva de la llamada es un tamaño en megabytes y no un número de correos. Resend también mide en bytes, y después de codificar, el tamaño de cada mensaje: *«max 40MB per email, after Base64 encoding of the attachments»* [73]. En los dos casos el contenido útil que cabe depende de la codificación. F-06 aparece en las ocho APIs del catálogo, con el tamaño expresado en MB, en MiB o en MB por segundo según el proveedor.

Consecuencia de P-03. En Postmark el campo `max` de `/email/bulk` queda sin cifra, porque el tope real está en megabytes, y en Resend el tope de 40 MB solo consta en el registro de procedencia. Las otras seis Datasheets declaran el tope de tamaño como un segundo `workload` del *endpoint* con unidad `MBs`, una forma que la guía no recoge (Sección 4.5).

Propuesta. Que `workload` admita dimensiones anidadas que se multiplican, y que `max` pueda expresarse en una métrica distinta de la unidad declarada. El Extracto de código 5.2 muestra la primera forma sobre Resend y la segunda sobre Postmark.

Extracto de código 5.2: P-02 y P-03: composición de dimensiones anidadas y cota expresada en otra métrica (caso Resend para P-02 y caso Postmark para P-03).

```
# P-02. El consumo es el producto de los dos factores.
workload:
  - unit: messages
    min: 1
    max: 100
    composes:
      unit: emails
      max_per_unit: 50
# P-03. El tope real esta en otra metrica: el tamano de la peticion.
workload:
  - unit: emails
    min: 1
    max_by: {unit: MBs, value: 50, scope: payload}
```

Efecto sobre PRIME4API. Las dos formas alimentan el mismo cálculo. El factor de consumo por petición deja de ser un entero y pasa a depender de la petición, de modo que el motor tiene que evaluar el consumo máximo y no solo el declarado. El cambio es mayor que el de P-01, porque afecta también a las operaciones que recorren la Datasheet, como la que devuelve los rangos del factor de consumo de cada *endpoint*, además de a las que calculan capacidad y coste.

5.3.3. P-06. El estado de la suscripción y la reputación se declaran una vez

El caso. MailerSend publica cuatro estados de reputación con nombre (*Healthy*, *Under review*, *Rebuilding* y *Sending paused*) y da cifra para dos de ellos [13]. Cuando la cuenta entra bajo revisión, `/v1/email` pasa de 120 a 10 peticiones por minuto y el envío en lote, `/v1/bulk-email`, de 10 a 1 petición por minuto, sin cambiar de plan. Un sondeo sobre una cuenta recién creada, que nunca se aprobó ni contrató plan, midió esos mismos 10 y 1 peticiones por minuto, una sola dirección en `to` por envío y cinco mensajes por lote. Esa cuenta no es la que describe el valor por defecto de la Datasheet, que corresponde a la cuenta aprobada.

El caso obliga a separar tres nociones. La cuenta es la identidad del cliente ante el proveedor, y puede tener varias suscripciones: Resend muestra bajo una misma cuenta una suscripción transaccional y otra de *marketing*. El «estado de la suscripción» es la fase en que el proveedor sitúa esa suscripción, como la aprobación pendiente de MailerSend, el entorno de pruebas de Amazon SES o el modo de prueba de Mailchimp Transactional. La «reputación» es la valoración que hace el proveedor de cómo consume el cliente, por ejemplo por los errores que provocan sus peticiones, y amplía o reduce la capacidad sin cambiar de suscripción.

En MailerSend, la cuenta sin aprobar es un estado de la suscripción, y la revisión que baja `/v1/email` a 10 peticiones por minuto es un efecto de la reputación. La carencia, contada sobre los estados que cambian los topes de una suscripción, recurre en seis de las ocho APIs (Tabla 5.1): además de MailerSend, en Postmark, Resend, Amazon SES, Mailchimp Transactional y Mailgun.

Qué permite v0.3. La guía documenta alias dentro de un *endpoint* (bloques con nombre que llevan los límites de cada modo de consumo, aquí de cada estado) y usa MailerSend como ejemplo [61]. Su vocabulario de ejemplo tiene dos nombres, `healthy_reputation` y `under_review_reputation`. La Datasheet de MailerSend adopta ese alias de estado que v0.3 ya ofrece y añade un tercero, `unapproved_account`, para la cuenta sin aprobar, que el proveedor documenta y el ejemplo de la guía no recoge. El estado sano conserva el valor por defecto, y el motor devuelve una fila por estado donde antes devolvía una sola. Con ese alias, v0.3 ya escribe los topes de cada estado de MailerSend, y lo que falta es lo que el alias no cubre.

Qué no cubre. Al aplicar el mecanismo a MailerSend aparecen tres carencias.

La primera es que el alias no tiene campo para el disparador del cambio de estado. Un `trigger` escalar dentro del alias se ignora sin aviso. Escrito al nivel del *endpoint*, el motor lo lee como un estado más, porque trata como estado todo diccionario cuya clave no sea una de las cuatro que conoce (`rate`, `quota`, `workload` y `cost_per_request`). Una copia de la Datasheet con el campo dentro del alias y otra con el campo al nivel del *endpoint* lo muestran. En el comodín `/*`, que fija el caudal general de las demás rutas y lleva su `rate` al nivel del *endpoint*, el motor devuelve 60 peticiones por minuto sin estado y, con el campo, las devuelve bajo un estado llamado `trigger`, que no existe.

Ninguna de las tres comprobaciones de cierre de una Datasheet detecta el estado `trigger`. El validador de formato de este trabajo, que comprueba los tipos de cada

campo y las referencias entre bloques, da 16 comprobaciones superadas y 2 avisos. El verificador de citas comprueba que cada cita literal del registro de procedencia sigue en la copia archivada de su página, cuando la hay. La verificación por uso con PRIME4API da 20 aciertos y 1 error. Las tres dan el mismo resultado que sin el campo.

La segunda carencia está en la guía. Un alias admite también un `workload` propio, y PRIME4API lo lee, pero la guía solo documenta el *rate* y la cuota. En MailerSend el tope de destinatarios por petición cambia con la aprobación de la cuenta, y ese cambio cabe en un alias aunque la guía no lo diga. Mientras la guía no lo documente, una Datasheet que la siga al pie de la letra deja fuera del alias el tope de destinatarios de la cuenta sin aprobar de MailerSend.

La tercera es que un estado de la suscripción se declara siete veces. El estado no depende del plan ni del *endpoint*, y el esquema no tiene dónde declararlo una sola vez para toda la Datasheet. En MailerSend la referencia se repite en los siete bloques de *endpoint* y nada liga las copias, de modo que cambiar el estado obliga a tocar siete sitios. Esta tercera carencia recurre, porque la Datasheet de Amazon SES repite su alias de entorno de pruebas en ocho bloques y la de Mailchimp Transactional repite el de la cuenta de prueba en tres.

El recuento del semáforo de confianza, que etiqueta cada valor según el respaldo de su fuente, cuenta cada copia como una evidencia. En MailerSend, el tope de 50 MB del lote aparece tres veces en verde, una por plan, copiado del *endpoint* en el estado de cuenta sin aprobar. Solo una nota del registro de procedencia impide leer esas tres copias como tres evidencias.

Propuesta. Se proponen tres cambios sobre el mecanismo de alias de la guía, que se conserva. El primero es un atributo de disparador con sitio propio, que diga si el cambio de estado lo impone el proveedor o lo elige el cliente y lleve su condición en prosa. El segundo es documentar el `workload` por alias, que PRIME4API ya admite. El tercero es declarar cada estado una sola vez para toda la Datasheet, con las referencias de cada *endpoint* apuntando a esa declaración. El Extracto de código 5.3 muestra los tres sobre MailerSend.

Extracto de código 5.3: P-06: estados de la suscripción y de la reputación declarados una vez, con su disparador y con el `workload` que la guía no documenta (caso MailerSend).

```
# Declaracion unica, fuera de los endpoints.
account_states:
  unapproved_account:
    trigger: {mode: provider_managed, condition: "cuenta pendiente de
       aprobacion"}
    rate: rate_request_email_unapproved
    quota: quota_email_monthly_unapproved
    workload: workload_email_unapproved   # admitido por el motor, no
       documentado
# Cada endpoint referencia el estado en lugar de repetirlo.
/v1/email:
  states: [healthy_reputation, under_review_reputation,
      unapproved_account]
```

La reputación como campo. Los estados de ejemplo de la guía llevan la reputación en el nombre, pero ningún campo dice cómo se mueve ni cuánto cambia la capacidad. Fuera del mecanismo de alias, se propone un campo de reputación que declare cuánto amplía o reduce la capacidad y qué lo dispara. Con él, la Datasheet registraría si el proveedor publica cómo funciona la reputación de la cuenta de forma completa, con la capacidad de cada estado, y determinista, con el umbral que hace cambiar de estado. Ninguna de las ocho APIs la publica así. Seis dicen que la reputación mueve la capacidad sin dar a la vez la cifra de cada estado y el umbral, y Postmark y Brevo solo la nombran para su infraestructura de envío.[3]

Efecto sobre PRIME4API. El motor ya devuelve una fila por estado, así que leer la capacidad por estado no le exige ningún cambio. Con el disparador declarado, una consulta puede separar los estados que elige el cliente de los que impone el proveedor, que es la diferencia entre una capacidad contratable y una contingencia. Con la declaración única, el recuento de evidencia deja de contar copias. Hace falta además una corrección del motor y no del esquema: avisar de la clave que no reconoce y no convertirla en estado.

5.3.4. P-07. El sobrecoste declara su tope

El caso. Desde el 26 de marzo de 2026, SendGrid limita el envío mensual de los planes Essentials y Pro a diez veces el volumen contratado, sobrecoste incluido. Una cuenta con el plan Essentials de 50.000 correos puede enviar 500.000 al mes, y por encima recibe el error *«Maximum Credits Exceeded»*.[4] El literal cuenta *requests*, que el mismo artículo define como correos procesados más descartados (*«Requests = Processed + Dropped»*), así que la unidad no es la petición HTTP. El dato aparece en un artículo de resolución de errores, y la página de precios y el artículo de sobrecostes no lo mencionan.

[3] Las fuentes de cada API están en `03_datos/apis/<api>/fuentes.md`, en `fricciones.md` y en `05_salida/_cap5/hechos_<api>.md` del repositorio del trabajo.

[4] Artículo de soporte de SendGrid, https://support.sendgrid.com/hc/en-us/articles/35466138799899-Understanding-the-Maximum-Credits-Exceeded-error, consultado el 6 de septiembre de 2026.

Qué permite v0.3. `overage_cost` es un precio unitario sin límite de cantidad: la guía lo define como un objeto con precio y valor, sin ningún subcampo que acote la cantidad [61]. La Datasheet declara los 50.000 correos incluidos y el precio de cada correo extra, y eso se lee como capacidad ilimitada a ese precio. El tope solo cabe en el registro de procedencia, en prosa, donde ninguna herramienta lo evalúa.

Lo que responde PRIME4API. Pedida al motor la recomendación de presupuesto para 600.000 correos sobre ese plan, responde 734,95 USD y declara el objetivo alcanzable en 59 segundos. El punto de control por debajo del tope, 400.000 correos, devuelve 474,95 USD con la misma forma, y el motor cruza los 500.000 sin que nada cambie.[5] En el caso de control, con un presupuesto de 100 USD en la consulta, el mismo motor declara el objetivo inalcanzable. El motor declara inalcanzable un objetivo cuando el límite le llega por la consulta o por la Datasheet, así que el error de los 600.000 correos procede de que la Datasheet no puede declarar el tope.

Propuesta. Que `overage_cost` admita un máximo opcional, expresado como factor sobre el volumen contratado o en unidades absolutas. Con él, la Datasheet de SendGrid declara diez veces el plan y el motor deja de prometer un volumen que el proveedor rechaza.

5.3.5. P-08. Un cupo que cuenta destinatarios distintos y no se repone

El caso. La cuenta de MailerSend sin aprobar puede escribir a dos direcciones distintas, y solo a esas dos [13]. El cupo es de la cuenta y no de la petición. Solo lo levanta la aprobación, y la documentación no le da ventana. El listado de destinatarios de la API (`GET /v1/recipients`) devuelve cero, así que el cupo no se puede leer por ese *endpoint*. Los sondeos lo encontraron alcanzado sin haber enviado nunca un correo, y de ahí se deduce que lo gastaron intentos que no enviaron nada. Qué petición lo gastó no se determinó.

Qué permite v0.3. Una cuota tiene valor, unidad y periodo, y aquí el valor cabe, dos direcciones, pero fallan la unidad y el periodo. La unidad son direcciones distintas, que no equivalen a correos ni a peticiones. No hay periodo, porque el cupo no se renueva. Y, como los sondeos encontraron el cupo gastado sin un solo correo enviado, lo consume el intento y no el envío, dos cosas que el esquema no distingue.

La guía da seis ejemplos de unidad (peticiones, correos, SMS, registros, gigabytes y *tokens*), a los que sus formatos estándar suman horas y minutos [61]. Ninguna de esas unidades cuenta entidades distintas. Una unidad nueva, como la `MBs` del catálogo, no bastaría, porque la cuota seguiría sin decir su ámbito, su falta de periodo ni qué la consume. La Datasheet deja el cupo en el registro de procedencia, en prosa.

Lo que responde PRIME4API. El motor lee la Datasheet de MailerSend y atribuye a la cuenta sin aprobar una capacidad acumulada de 110 correos en un mes, por encima incluso de los 100 correos al mes que la Datasheet le declara, cuando la

[5] Ejecución en `05_salida/_cap5/prime4api_ejecucion_d042/tope_sobrecoste_2026-10-18/` del repositorio del trabajo.

capacidad medida es cero. Un objetivo de tres correos lo declara alcanzable en cero segundos y sin coste.[6] La cuenta del sondeo no envió ni un correo y la cuota mensual de correos quedó intacta. Solo bajó la diaria de peticiones, de 100 a 97. Con la misma Datasheet, el motor declara inalcanzable un objetivo de 100.000 correos por el límite de caudal, así que sabe negarse cuando el límite está escrito. El objetivo de tres correos sería alcanzable en una cuenta recién abierta, que aún conserva sus dos direcciones, y deja de serlo en cuanto el cupo se ha gastado.

Propuesta. Que una cuota pueda declarar cuatro cosas que v0.3 no declara. Son que su unidad son entidades distintas, que su ámbito es la cuenta, que no tiene periodo y qué la consume, si el intento o el resultado. En el catálogo el cupo es solo de MailerSend: las otras siete APIs se comprobaron y no lo tienen (Tabla 5.1). Por eso la propuesta se sostiene en una sola API, con la respuesta errónea que PRIME4API da sobre la cuenta sin aprobar.

Frontera con P-06. El cupo aparece en la cuenta sin aprobar, igual que los topes reducidos de P-06, pero pide otra cosa. En P-06 el esquema sabe escribir el número de cada estado y no sabe decir cuál rige ni qué dispara el cambio. En P-08 el límite no cabe ni con estado ni sin él, porque su unidad no existe en v0.3. Se separan para que ninguna carencia cuente dos veces y cada propuesta pida una sola forma.

5.3.6. P-09. La cuota declara qué hace con lo que la supera

El caso. La cuota del plan gratuito de Brevo es de 300 correos al día. Al agotarla, el correo transaccional no se corta en seco: *«once you reach your daily limit, up to 1,000 additional emails are held in a retry queue. Emails beyond this queue are not delivered»* [65]. La cuota tiene así dos umbrales, el de la cifra publicada y el de la cola, y lo que ocurre entre los dos solo vale para el correo transaccional. El literal no dice si el cliente recibe algún error por los correos que la cola ya no admite, y la cola no se ha observado.

Qué permite v0.3. De los campos de la cuota, solo `overage_cost` habla de su borde [61]. Sin él, la cuota se lee como un corte estricto: al llegar a la cifra no se envía nada más. Con precio cero, se lee como un exceso que no se cobra ni se acaba. Ninguna de las dos lecturas es la de Brevo. La Datasheet declara los 300 correos sin sobrecoste, que en v0.3 se escribe igual que un corte estricto.

Lo que responde PRIME4API. Con la Datasheet sin manipular, PRIME4API devuelve para esa cuota 300 correos al día sin sobrecoste y sin rastro de la cola.[7] Es la misma forma que devuelve para la prueba gratuita de SendGrid, que sí es un corte estricto, mientras que Brevo retiene hasta 1.000 correos más en la cola de reintento. El caso de control es el plan Essentials de SendGrid: su Datasheet declara sobrecoste, y el motor lo devuelve con su precio. El motor distingue un corte estricto de un exceso que se cobra, pero no de una cola, porque la Datasheet no tiene cómo decirlo.

[6] Ejecución en `05_salida/_cap5/prime4api_ejecucion_d042/cupo_destinatarios/` del repositorio del trabajo.

[7] Ejecución en `03_datos/apis/brevo/evidencias/l2_prime4api_2026-09-18.json` del repositorio del trabajo.

Propuesta. Que la cuota admita un bloque `overflow` con las unidades que retiene por encima de su valor y lo que hace con ellas. Si el bloque falta, la cuota es un corte estricto. Si hay sobrecoste, sigue en `overage_cost`, y el bloque nuevo solo describe la franja que no se cobra. El Extracto de código 5.4 muestra la forma sobre el caso de Brevo.

Extracto de código 5.4: P-09: una cuota que retiene lo que la supera hasta un tope fijo (caso Brevo, plan gratuito, correo transaccional).

```
quota_email_daily_free:
  value: 300
  unit: emails
  period: {value: 1, unit: DAY}
  overflow:              # P-09: lo que la cuota hace al superarse
    capacity: 1000       # envios retenidos en la cola de reintento
    behavior: queued     # en cola, y el exceso no se entrega
```

Frontera con P-07. El tope de SendGrid de P-07 acota lo que se cobra por encima de la cuota, y la cola de Brevo describe lo que se retiene sin cobrarlo. Las dos propuestas tocan el mismo borde de la cuota, y reunirlas en una sola forma es una decisión que corresponde a la guía.

5.4. Defectos de contrato

La Tabla 4.6 (Sección 4.5) recoge lo que ocurre al ingerir las ocho Datasheets en PRIME4API, y sus errores tienen tres causas. Dos son defectos de contrato, es decir, formas que la guía de ACM2YAML admite y que el motor no procesa. Son un campo declarado y vacío en Postmark, Resend y Mailchimp Transactional, y un precio que existe y no es público en Resend, SendGrid y Brevo.

La tercera causa es el fallo del propio motor al calcular la curva acumulada de MailerSend y Amazon SES, que mezcla unidades de tiempo (Sección 4.5), y no produce propuesta de esquema porque ningún valor de esas Datasheets lo causa. Mailgun termina con el mismo mensaje, pero su causa no está confirmada. La forma de `workload` es un caso distinto, porque ninguna Datasheet del catálogo usa la variante con la que falla el motor. La guía de ACM2YAML y una Datasheet de ejemplo de su repositorio escriben el campo de dos formas distintas, y se resuelve corrigiendo la guía.

5.4.1. La forma de `workload`: mapa o lista

La guía del esquema documenta `workload` como un mapa [61]. Una Datasheet de ejemplo del mismo repositorio lo serializa como una lista, y el TFM de Gavira Sánchez usa también la lista en sus extractos y en el anexo de SendGrid [3]. PRIME4API tolera las dos formas en las operaciones de cálculo y solo acepta la lista en las de navegación, que recorren la estructura del documento. Una Datasheet escrita según la guía produce entonces números correctos y falla cuando la recorren el agente conversacional HARVEY4API o la plataforma SPHERE.

La forma correcta es la lista, porque la exige el modelo de consumo. Un mapa solo puede declarar una métrica por *endpoint*, y Gavira Sánchez sostiene que un *endpoint* que consume varias métricas a la vez tiene que declararlas por separado [3]. Las ocho Datasheets del catálogo usan la lista. La propuesta es una rectificación: corregir la guía para que documente la lista. De forma opcional, el motor puede convertir el mapa en lista al leerlo, para que los ficheros escritos con la redacción actual de la guía sigan siendo navegables.

5.4.2. P-04. Distinguir «sin límite» de «límite no publicado»

El caso. Resend no publica el *rate* de su plan Enterprise, y su Datasheet deja presente y vacío el campo que lo declara. Ese campo vacío dice que el límite existe y que su valor no se conoce. En Postmark y en Mailchimp Transactional la misma forma dice lo contrario: el proveedor no pone tope en la unidad del campo. Si solo lee el campo vacío, PRIME4API no puede distinguir el caso de Resend del de Postmark, y P-04 propone cerrar esa ambigüedad.

En Postmark y en Mailchimp Transactional la cota que se publica está en otra métrica, los bytes de la llamada, que es la carencia que cubre P-03 (Apartado 5.3.2). Postmark no pone tope de destinatarios en `/email/bulk` y lo acota por el tamaño de la petición, 50 MB. Mailchimp niega por escrito el tope de destinatarios en su API, «there's no recipient limitation», y limita el JSON de cada llamada a menos de 10 MB[8]. En el sondeo de Mailchimp, una llamada con 5.000 destinatarios devolvió HTTP 200 sin rechazo.

P-04 se promueve por dos vías, y cada una se refiere a un hecho distinto. La primera es la recurrencia de F-01, un *rate* que existe y no tiene cifra publicada, que cuenta en Postmark, Amazon SES, Mailchimp Transactional y Mailgun (Tabla 5.1). En Mailchimp ese *rate* es la cuota horaria de la cuenta de pago, que el proveedor no cifra, y no el tope de destinatarios. La segunda es la traza: el campo vacío hace fallar al motor en tres Datasheets, aunque solo en Resend es un *rate*. En Postmark y en Mailchimp es un tope de destinatarios. Lo que P-04 generaliza es lo que muestra la traza, un campo vacío que el motor no distingue de la ausencia de límite.

La traza. PRIME4API convierte a número el valor de esos campos sin comprobar si está vacío, y falla con un error de conversión de tipo. En Postmark caen por esta causa diez comprobaciones: la consulta por rangos del factor de consumo por petición, tres curvas de capacidad (una es la acumulada), la curva de coste, la recomendación de plan y cuatro operaciones de cálculo. Esas cuatro dan el tiempo mínimo para alcanzar una capacidad, la capacidad acumulada en un instante, el instante en que se agota la cuota y la espera tras agotarla. En Resend el mismo error aparece en cinco comprobaciones, y otras ocho fallan por el precio que existe y no es público, el valor `custom` del plan Enterprise.

[8] https://mailchimp.com/developer/transactional/docs/outbound-email/, consultada el 21 de septiembre de 2026.

Como el fichero de Resend contiene los dos defectos, se generaron dos variantes, cada una con un solo defecto corregido. Con el campo vacío relleno, la herramienta de verificación de este trabajo da trece errores, todos por el precio no público. Con el precio no público sustituido por números, da doce, todos por el campo vacío[9]. Cada comprobación informa solo del primer error que encuentra. Por eso, con los dos defectos a la vez, el campo vacío aparece en cinco y el precio no público en ocho, aunque cualquiera de los dos basta por sí solo para hacer fallar doce o trece.

En una tanda aparte de diecisiete llamadas a la API del motor con la Datasheet de Mailchimp Transactional, catorce responden HTTP 500 con el mismo error de conversión, y solo contestan las tres consultas que no calculan tiempo. Si el campo vacío del tope se rellena en el documento ya cargado por el motor, sin tocar el fichero, el error pasa a ser una división por cero. Esa segunda caída la causa otro campo vacío, el periodo de la franquicia de la cuenta de prueba, 500 correos gratuitos que no caducan ni se renuevan (F-33). Con los dos campos rellenos, la misma llamada devuelve HTTP 200. La herramienta de verificación de este trabajo, con sus propias comprobaciones, da a Mailchimp diez errores con el mismo mensaje que a Postmark.

Por qué el esquema no ofrece salida. ACM2YAML v0.3 deja dos alternativas, y las dos tienen coste. Omitir el campo hace que la Datasheet afirme implícitamente que no hay límite. En Resend, donde el límite existe, la omisión produciría una respuesta optimista y falsa. En Postmark y en Mailchimp coincidiría con lo que dice el proveedor, pero PRIME4API no distinguiría esa omisión de la que haría Resend con un límite que existe. Declarar el campo vacío, la otra alternativa, deja sin responder operaciones enteras del motor: diez comprobaciones en Postmark, cinco en Resend y catorce de diecisiete llamadas en Mailchimp.

La Datasheet de Amazon SES elige la omisión para el caudal de una cuenta con acceso de producción: no lo declara y anota la ausencia en su registro `not_found`. Ese registro, que usan las ocho Datasheets, dice qué falta en el documento, pero no en qué campo, y una marca junto al propio campo dejaría esa anotación donde el motor la puede leer.

Propuesta. Se propone una marca explícita de valor no publicado junto al campo. El esquema declara la marca y el motor la respeta: omite esa dimensión y sigue evaluando el resto en lugar de abandonar el documento. Con la marca, el límite sin cifra de Resend y la ausencia de tope de Postmark y Mailchimp dejan de escribirse igual. Resend la lleva en su caudal de Enterprise. Postmark y Mailchimp pueden omitir el campo, y la omisión dice lo mismo que su fuente: no hay tope de destinatarios. La cota en bytes ya la declara la Datasheet de Mailchimp como un `workload` aparte, y la de Postmark necesita P-03.

La misma distinción alcanza al periodo de una cuota, el segundo campo vacío de Mailchimp. Ahí, igual que en su tope de destinatarios, no falta una cifra, porque la franquicia de 500 correos de la cuenta de prueba no caduca ni se renueva (Sección 4.4).

[9]Las tres versiones del fichero (la original y las dos variantes) y su salida están en `03_datos/apis/resend/evidencias/control_motor_causas_2026-09-23/`.

La marca queda para los valores que existen y no se publican, y la cuota sin periodo es otro elemento, F-33.

5.4.3. P-05. Declarar un plan cuyo precio existe y no es público

El caso. Resend no publica el precio ni la cuota de su plan Enterprise, y la Datasheet escribe en los dos campos el valor `custom`. Es lo que publica el proveedor, y es la forma que la guía documenta para un valor negociable [61].

La traza. PRIME4API intenta convertir la cadena `custom` a número y falla. El fallo no se queda en el plan Enterprise. La evaluación recorre todos los planes del documento, así que ese único valor impide evaluar también los planes Free, Pro y Scale, que tienen precio publicado. Con ese error fallan las tres consultas de *rates*, cuotas y límites, las cuatro operaciones de cálculo y la consulta del precio del plan Enterprise, que son ocho de los trece errores de la Datasheet de Resend. Con el campo vacío relleno, los trece pasan a esta causa (Apartado 5.4.2). SendGrid y Brevo reproducen el fallo con trece errores cada una, todos con el mismo mensaje: la cadena `custom` no se puede convertir a número.

Por qué importa. En siete de las ocho APIs del catálogo hay al menos un precio que el proveedor no publica, y la excepción es Amazon SES. En Postmark, Resend, SendGrid, MailerSend, Brevo y Mailgun es un plan a medida, y en Mailchimp Transactional es el precio por encima de cuatro millones de correos al mes. Resend, SendGrid y Brevo escriben ese plan con `custom`, como admite la guía, y el motor falla en las tres. Postmark, MailerSend y Mailgun lo dejan fuera de sus planes.

Propuesta. Se proponen dos cambios separables. En el esquema, una forma explícita de declarar un plan que tiene precio aunque no lo publique, en sustitución de la cadena `custom` que el motor no sabe leer. En el motor, aislar el fallo por plan, de modo que un plan no evaluable devuelva su error y los demás se sigan calculando. El segundo cambio no depende del primero, y por sí solo recupera tres de los cuatro planes de la Datasheet de Resend.

Las dos propuestas de esta sección se han medido solo contra PRIME4API. Otro motor podría tolerar el campo vacío o la cadena `custom` sin que el esquema cambiara.

5.5. Lo que queda fuera

De los veinticuatro elementos no soportados que superan el criterio de promoción, porque aparecen en dos APIs o más o porque una ejecución de PRIME4API muestra su efecto sobre la Datasheet, dieciséis no llevan número de propuesta. Su patrón se repite en el catálogo, pero les falta una forma concreta contrastada con el texto de la

guía de ACM2YAML. Es un límite asumido: la guía recibe su recuento, no una forma que pueda adoptar. Llevar número depende de tener esa forma, no de la recurrencia: F-11, los límites de recursos por plan, recurre en seis APIs y no lo lleva, y F-21, el cupo de destinatarios distintos de MailerSend, aparece en una sola API y sí lo lleva.

F-07, los *add-ons* que modifican dimensiones de capacidad, recurre en las ocho APIs. El caso común es la IP dedicada, una dirección IP de envío reservada a un solo cliente, que se paga aparte o exige un plan mínimo. Aun así no se propone extensión, por una razón de alcance: el esquema deja los *add-ons* al canal web, y el catálogo modela solo la API REST, una de las interfaces del canal API (Sección 4.1.2). Un bloque de *add-ons* en la Datasheet repetiría lo que el pricing del canal web ya modela, y enlazar la capacidad con el pricing es una línea futura que Gavira Sánchez deja en el trabajo que introduce el modelo [3]. El catálogo aporta a esa línea ocho casos.

F-04 reúne los límites que el proveedor declara y que la respuesta a la petición no refleja. Plantea si un límite que la respuesta no refleja forma parte de la capacidad contractual. Es una pregunta sobre la definición del ACM que no se resuelve con un campo nuevo, y por eso no se convierte en propuesta. En MailerSend, el *endpoint* de lote acepta con un `202` un envío con un remitente de dominio sin verificar, que la validación diferida rechaza después. Los dos lotes medidos terminan con cinco errores de validación y ningún correo enviado[10]. Quien mida su consumo por la respuesta cuenta cinco correos que nunca salieron.

F-04 aparece también en los sondeos de Mailchimp Transactional y de Amazon SES, que respondieron con `200` por encima del límite que declara su proveedor. Mailchimp Transactional declara que el JSON de cada llamada debe ocupar menos de 10 MB [74], y respondió con `200` a una llamada de unos 40 MB[11]. Amazon SES declara una petición por segundo en sus acciones de gestión [68], y respondió con `200` a doce peticiones seguidas a 5,7 por segundo (Sección 4.3.3). En los dos casos la Datasheet conserva la cifra publicada, y la medida no tiene campo donde escribirse. El registro de elementos no soportados esboza una salida: distinguir en el esquema el límite publicado del aplicado, cada uno con su fuente.

F-05, los resultados por petición que fija la documentación, es una de las dos dimensiones del ACM que ACM2YAML v0.3 deja fuera de su versión operativa. Gavira Sánchez ya recoge completar esa serialización como línea futura [3], así que proponerla repetiría esa línea. F-08, la superficie de la API que cambia con el *tier*, ya cabe en v0.3: un *endpoint* que un plan no tiene se omite de ese plan (Sección 5.2).

Los doce restantes están en la Tabla 5.3. El registro de cada API esboza la forma de once, y la de F-12 la enuncia este trabajo, pero ninguna se ha contrastado con la guía. F-22, la credencial que solo se gestiona desde la web, está en la frontera de lo que el esquema describe. Su forma registra qué canal ofrece cada operación, y el esquema describe precios, cuotas y límites.

Dos de los doce tienen argumentos que no dependen de cuántas APIs los presentan. En F-09, la caducidad de los datos, el repositorio de la guía conserva una

[10] Evidencia en `03_datos/apis/mailersend/evidencias/probe_sandbox_limites_2026-09-09.json` y en la repetición del mismo sondeo, en la misma carpeta.

[11] Evidencia en `03_datos/apis/mailchimp/evidencias/sondeo_a_preflight_2026-09-21.md`.

Dónde	Id	Carencia	APIs	Forma esbozada
Documento	F-09	El documento caduca entero, y sus datos caducan uno a uno	5/8	Fecha de verificación por afirmación en el registro de procedencia
Documento	F-10	Dos productos de precio con unidades de consumo distintas	3/8	Varios catálogos por servicio, o un campo que acote qué producto se modela
Plan	F-11	Límites de recursos por plan, que no son consumo	6/8	Bloque de recursos en el plan, con nombre, valor y unidad y sin periodo
Plan	F-13	El sobrecoste es opcional y no existe en el plan gratuito	5/8	Modo del sobrecoste declarado en el plan
Plan	F-14	El plan tiene fecha de caducidad	2/8	Vigencia del plan y plan al que cae al expirar
Precio	F-26	Un precio que depende de la geografía y una sola divisa en el fichero	3/8	Precio como lista de ofertas, cada una con su divisa
Cuota	F-12	La recepción consume la cuota de envío	2/8	Declarar que un evento entrante reenviado descuenta de la cuota de envío, con un factor que depende de la ruta
`workload`	F-34	Dos topes de tamaño sobre la misma llamada, uno del mensaje y otro de la petición	2/8	`workload` que declara el objeto medido además de la unidad
Interfaz	F-15	Dos interfaces con topes propios sobre la misma cuota	5/8	Interfaces declaradas, cada una con sus topes, o `workload` declarado por interfaz
Acceso	F-28	Una lista de IP autorizadas decide si la llamada entra	3/8	Condiciones de admisión declarables en el plan o en la credencial
Bloque nuevo	F-37	Un tope de capacidad que fija el cliente, no el proveedor	2/8	Bloque de límites que configura el cliente, fuera de la capacidad contractual
Operación	F-22	Una credencial que solo se gestiona desde la web	4/8	Qué canal ofrece cada operación, registrado por operación

Tabla 5.3: Promovidos sin número de propuesta y con una forma esbozada en el registro de elementos no soportados de cada API, salvo F-12, cuya forma enuncia este trabajo. Dónde: la parte del esquema que tocaría la forma. APIs: recuento sobre las ocho del catálogo. Ninguna forma se ha trabajado ni contrastado con la guía.

Datasheet de MailerSend cuyos precios y sobrecostes difieren de los publicados cinco meses después [61]. Ese mismo ejemplo define caudales de 15 y 30 peticiones por minuto que ningún plan usa, un error de modelado distinto de la caducidad. En F-10, Resend vende un producto transaccional que se factura por correos y otro de *marketing* que se limita por contactos, y la Datasheet modela el primero sin poder declarar que solo cubre ese.

Los once elementos sin promover aparecen en una sola API, así que solo podían entrar por la traza, y a cada uno le falta al menos una de sus tres condiciones (Tabla 5.1). En F-23, el prefijo `/v3/smtp/` de la API REST de Brevo (rutas HTTP, no el relé SMTP) con dos rutas exentas, el esquema ya lo expresa si se enumeran las rutas

del prefijo que no tienen fila propia. En F-25, un *endpoint* de Brevo con dos modos de envío, la guía expresa los dos modos con alias, aunque no documenta un `workload` por alias. En F-29, un precio por unidad sin cuota de la que colgarlo, falta comprobar si la guía lo cubre.

A F-32 (precio degresivo sin planes), F-33 (franquicia sin ventana), F-36 (dos contadores sobre la misma ventana) y F-38 (dos cuotas concurrentes de la misma unidad) les falta una ejecución archivada de PRIME4API que muestre su efecto con la Datasheet sin manipular. En F-33, la división por cero del Apartado 5.4.2 solo aparece tras rellenar a mano otro campo vacío. A F-30 (precio por tramos marginales) y F-35 (tope fraccionario en un máximo entero) les faltan esa ejecución y el contraste con la guía.

F-31, los límites que rigen por cuenta y región, y F-27, el exceso de Brevo que cambia el plan en lugar de pagarse por unidad, tienen ejecución, pero ninguno de los valores que devuelve se puede dar por falso. La de F-27 devuelve un coste plano, y con él no se distingue si Brevo deja de enviar al llegar a la cuota o pasa la cuenta al plan superior.

F-36 y F-38 piden que un mismo nodo de la Datasheet, sea un plan, un *endpoint* o un alias, admita varios *rates* o varias cuotas a la vez. Es lo que pide el *issue* «Soportar listas de rates y cuotas en datasheet», que Gavira Sánchez abrió en el repositorio del motor [19], y el motor ya lo implementa aunque la guía no lo documente.

Todas las propuestas salvo tres son retrocompatibles: añaden campos opcionales que un documento escrito contra v0.3 puede no declarar, y no cambian cómo se lee ninguno de los que ya existen. Las dos que convierten el factor de consumo por petición en una expresión, P-02 y P-03, obligan a tocar las operaciones que lo leen. P-05, el plan con precio no público, sustituye la cadena `custom` que usan las Datasheets de Resend, SendGrid y Brevo. La corrección de la forma de `workload` rectifica la guía para que documente la lista que ya usan las ocho Datasheets (Sección 5.4), y queda fuera de ese recuento. Su coste recae sobre los documentos que siguen la redacción actual y escriben un mapa.

La guía no tiene mecanismo para lo que piden las cuatro últimas propuestas: el estado de la suscripción declarado una vez y con su disparador (P-06), el tope del sobrecoste (P-07), el cupo de destinatarios distintos (P-08) y la cola tras la cuota (P-09). Aun así, declara que el esquema es preliminar y que habrá que extender campos [61]. En P-06, la parte que documenta el `workload` por alias no cuesta nada, porque PRIME4API ya lo lee.

Las cuatro añaden claves que v0.3 no define, y un motor que rechazara las claves desconocidas invalidaría los documentos que las declaren. Ninguna Datasheet del catálogo las declara. Por eso la corrección del motor que pide P-06 avisa de la clave desconocida y no la rechaza: así sigue evaluando el resto del documento y no invalida estas cuatro extensiones antes de que la guía las defina.

Antes de comunicar una carencia se comprueba si ya está reportada. En los repositorios de la guía, de PRIME4API, de SPHERE y de las pruebas de uso de Datasheets solo hay un *issue* pertinente al esquema, el de listas de *rates* y cuotas, que cubre F-36 y F-38. Fuera de ese *issue*, ninguna de las carencias de P-01 a P-05 está reportada, y

las de P-06 a P-09 no se han contrastado con los *issues* abiertos.

5.6. Respuesta a la RQ2

La RQ2 pregunta si ACM2YAML es suficientemente expresivo para representar la capacidad contractual de las APIs de correo, y qué extensiones requiere. Con las ocho APIs del catálogo, la respuesta a la primera mitad es que no. Las ocho Datasheets se escriben sin campos propios, pero dejan en prosa del registro de procedencia, que ninguna herramienta lee, datos como los volúmenes contratados de Postmark, Resend y Mailgun o el tope del sobrecoste de SendGrid. En tres casos, el tope de SendGrid, el cupo de destinatarios de MailerSend y la cola de Brevo, PRIME4API devuelve un resultado que el proveedor contradice.

La respuesta a la segunda mitad son las nueve propuestas de la Tabla 5.2: siete amplían lo que el esquema expresa y dos corrigen formas que la guía admite y PRIME4API no lee. Ocho de ellas salen de un inventario de treinta y cinco elementos no soportados, y la novena, P-05, del fallo del motor ante un plan cuyo precio no es público. A ellas se suma una rectificación de la guía de ACM2YAML: la guía documenta `workload` como un mapa, y una Datasheet de ejemplo de su propio repositorio lo escribe como lista (Apartado 5.4.1).

Gavira Sánchez declara que ACM2YAML v0.3 captura dos de las cuatro dimensiones de capacidad del metamodelo, el volumen (las cuotas) y la frecuencia (los *rates*), y deja fuera las peticiones concurrentes y los resultados por petición [3]. Los resultados por petición (*Results Per Request*) están documentados en siete APIs del catálogo, y Gavira Sánchez ya recoge su serialización como línea futura [3]. Las peticiones concurrentes aparecen en el catálogo en los topes del relé SMTP que reúne F-15: SendGrid admite hasta 10.000 conexiones simultáneas desde un mismo servidor, y MailerSend, cinco en sus planes Free, Hobby y Starter. Las dos cifras quedan en el registro de procedencia.

Tres propuestas tratan lo que v0.3 sí recoge pero declara con un único valor escalar: la cifra de la cuota y el consumo de cada petición. Postmark vende seis volúmenes contratados dentro de un mismo plan, donde el esquema admite una sola cifra de cuota (P-01, Apartado 5.3.1).

Las otras dos tratan el consumo de cada petición. Una llamada al lote de Resend admite hasta cien mensajes con hasta cincuenta destinatarios cada uno y consume hasta el producto de las dos cifras, 5.000 correos. El esquema, en cambio, describe el consumo de una petición con una sola terna de unidad, mínimo y máximo (P-02). El tope de `/email/bulk` de Postmark es un tamaño, 50 MB por petición, y el esquema solo admite acotar una petición en su propia unidad, que es el correo (P-03). Los dos casos se detallan en el Apartado 5.3.2.

La capacidad depende además del estado de la suscripción y de la reputación de la cuenta. Cuando MailerSend pone una cuenta bajo revisión, su envío baja de 120 a 10 peticiones por minuto sin cambiar de plan, y ese eje aparece en seis de las ocho APIs (P-06, Apartado 5.3.3). La guía permite declarar dentro de cada *endpoint* un alias por

estado, con la cifra de cada uno, pero obliga a repetirlo en todos los *endpoints*. Tampoco tiene campo para el disparador que cambia el estado, como la aprobación de la cuenta en MailerSend o su revisión por reputación.

Otras dos propuestas tratan lo que ocurre al superar la cuota. SendGrid limita el envío mensual a diez veces el volumen contratado, sobrecoste incluido, y el esquema no tiene campo para ese tope (P-07, Apartado 5.3.4). Brevo retiene hasta 1.000 correos por encima de la cuota diaria y deja de entregar los siguientes (P-09, Apartado 5.3.6). F-13, el sobrecoste opcional que no existe en el plan gratuito, se refiere también al exceso sobre la cuota en cinco APIs, y su forma queda esbozada en la Tabla 5.3.

Al margen del exceso sobre la cuota, la cuenta de MailerSend sin aprobar solo puede enviar a dos direcciones distintas, con un cupo que no se repone (P-08, Apartado 5.3.5). P-07, P-08 y P-09 aparecen en una sola API cada una. Se promueven porque PRIME4API devuelve sobre la Datasheet sin manipular un resultado que el proveedor contradice, y la misma operación responde bien en un caso de control en que el límite le llega escrito, en la Datasheet o en la consulta.

Las dos propuestas restantes son de contrato: el esquema admite el valor que describe al proveedor y PRIME4API falla al leerlo. En el plan Enterprise de Resend, un límite vacío no dice si no se publica o si no existe, y el motor falla al convertirlo a número (P-04, Apartado 5.4.2). El mismo plan lleva el precio `custom`, que la guía admite y que impide al motor evaluar el documento entero (P-05, Apartado 5.4.3).

Además de las nueve propuestas, la Tabla 5.3 recoge doce elementos que aparecen en dos APIs o más y cuya forma está esbozada sin contrastarla con la guía. Otros cuatro elementos promovidos quedan fuera de las propuestas (Sección 5.5): los resultados por petición, los *add-ons*, los límites que la respuesta no refleja y la superficie de la API que cambia con el plan. Los *add-ons*, como la IP dedicada, aparecen en las ocho APIs, pero el esquema los deja al canal web y el catálogo solo modela la API REST, una de las interfaces del canal API.

Tampoco hay propuesta para los límites que el proveedor declara y que la respuesta HTTP no refleja. MailerSend acepta con un `202` un lote que después rechaza, y Mailchimp Transactional responde con `200` a una llamada de unos 40 MB cuando declara un tope de 10 MB. Estos casos plantean qué cuenta como capacidad contractual, y esa cuestión no la resuelve ningún campo nuevo. La superficie de la API que cambia con el plan ya cabe en v0.3, porque un *endpoint* que un plan no ofrece se omite de ese plan.

Las nueve propuestas y los dieciséis elementos promovidos sin propuesta valen para un dominio y un motor: la recurrencia se cuenta solo en las ocho APIs de correo transaccional, y las Datasheets solo se han evaluado con PRIME4API.

6. Límites de uso y *rates*

Este capítulo responde a la RQ3: cómo se relacionan los límites de uso y los *rates* en las APIs de correo, y qué implicaciones tiene esa relación para el consumidor. Un límite de uso es el máximo de un nivel de uso medible, como los correos enviados al mes o los días que se conservan los registros, y puede renovarse o no. La cuota es un límite de uso renovable: el volumen máximo que se puede enviar en un periodo, llamado ventana, y que se repone cuando la ventana se cierra. El *rate*, o caudal, es la velocidad máxima de envío, en peticiones o en correos por unidad de tiempo.

En seis de las ocho APIs (Postmark, Resend, SendGrid, MailerSend, Brevo y Mailgun), el *rate* deja pasar en la ventana de la cuota entre 103,7 y 129.600.000 veces la cuota de los planes que recogen sus Datasheets, de MailerSend Starter al plan Free de Postmark. Con los volúmenes contratados que el proveedor publica y la Datasheet no recoge (Apartado 5.3.1), el mínimo baja a 2,59 veces en MailerSend Professional 2M. En ellas la cuota se agota antes de que el *rate* frene el envío.

Las cifras de esa relación no salen de la misma fuente en todas las APIs. Postmark no publica su *rate* y lo devuelve en una cabecera de la respuesta, uno de los campos que la API añade a cada respuesta HTTP. Amazon SES no publica cifras para la cuenta de producción, la que Amazon autoriza a enviar a cualquier destinatario al sacarla del entorno de pruebas.

El capítulo se organiza del siguiente modo. En primer lugar, la Sección 6.1 define las cuatro magnitudes del cálculo (el *rate*, la cuota, su ventana y los destinatarios por petición) y la curva de capacidad que las combina. A continuación, la Sección 6.2 recoge lo que publica cada proveedor y calcula, plan a plan, cuántas veces admite el *rate* la cuota. Después, la Sección 6.3 extrae seis afirmaciones con su recuento sobre las ocho APIs, y la Sección 6.4 contrasta el cálculo a mano con las respuestas de PRIME4API. Por último, la Sección 6.5 responde a la RQ3.

6.1. Lectura geométrica

En el plan Pro de Resend el *rate* es de 10 peticiones por segundo, el valor por defecto que el proveedor publica por equipo, compartido por todas sus claves de API [73]. A ese ritmo, un mes de 30 días admite 25.920.000 peticiones, y la cuota del plan es de 50.000 correos al mes [69]. El *rate* deja pasar, por tanto, 518 veces la cuota en su ventana de 30 días, contando un destinatario por petición. En SendGrid Essentials 50K [15] el mismo cálculo da 518.400 veces, con un *rate* de 10.000 peticiones por segundo. El proveedor publica ese *rate* una sola vez, en una pregunta frecuente que no lo liga a ningún plan. En la cuenta de prueba de Mailchimp Transactional el cálculo no se puede hacer. Su franquicia de 500 correos no tiene ventana declarada, y lo único que pone tiempo es un caudal de 25 correos a la hora [74].

El cálculo usa cuatro magnitudes de la Datasheet. El *rate* R se expresa en peti-

ciones por segundo y sale del bloque `max_power`. La cuota Q se expresa en correos, y su ventana W, el periodo tras el que la cuota se repone (un día o un mes), en segundos. Las dos están en el bloque `capacity`. El factor k es el número de destinatarios que lleva cada petición, de uno al máximo que admite el proveedor, que la Datasheet declara en el bloque `workload`.

El factor k corresponde al «CRF» (*Consumption Request Factor*) de Gavira Sánchez [3], la regla que dice cuántas unidades de una métrica descuenta cada petición. Contar k en destinatarios supone que cada destinatario descuenta una unidad de cuota. Esa equivalencia está sostenida en cuatro de las ocho APIs: Resend y SendGrid la publican, y en Brevo y Amazon SES la midió un sondeo, una tanda de peticiones reales contra la API para medir un límite (Apartado 4.3.2). En Postmark, MailerSend, Mailgun y Mailchimp no consta que cada destinatario descuente una unidad de cuota.

Para un envío uniforme a un ritmo medio c, en correos por segundo, los dos límites fijan dos umbrales de ritmo y un tiempo:

$$c_{\text{cuota}} = \frac{Q}{W}, \qquad c_{rate} = R \cdot k, \qquad t_{\text{mín}} = \frac{Q}{R \cdot k}. \tag{6.1}$$

El primer umbral es el ritmo medio que agota la cuota justo al cerrar su ventana. El segundo es el ritmo máximo de correos que deja pasar el *rate*. El tiempo $t_{\text{mín}}$ es el que tarda una ráfaga sostenida a ese ritmo máximo en agotar la cuota.

De los dos límites, la restricción activa es el de umbral menor: el que se alcanza primero y frena el envío, mientras el otro no llega a actuar. En Resend Pro la cuota pide 0,019 correos por segundo de media y el *rate* deja pasar 10, así que la restricción activa es la cuota. El cociente $R \cdot W/Q$ mide cuántas veces deja pasar el *rate* la cuota dentro de su ventana: 518 en Resend Pro y 518.400 en SendGrid Essentials 50K. Se calcula con $k = 1$ para comparar las APIs en peticiones, y k solo entra en el umbral del *rate* y en $t_{\text{mín}}$.

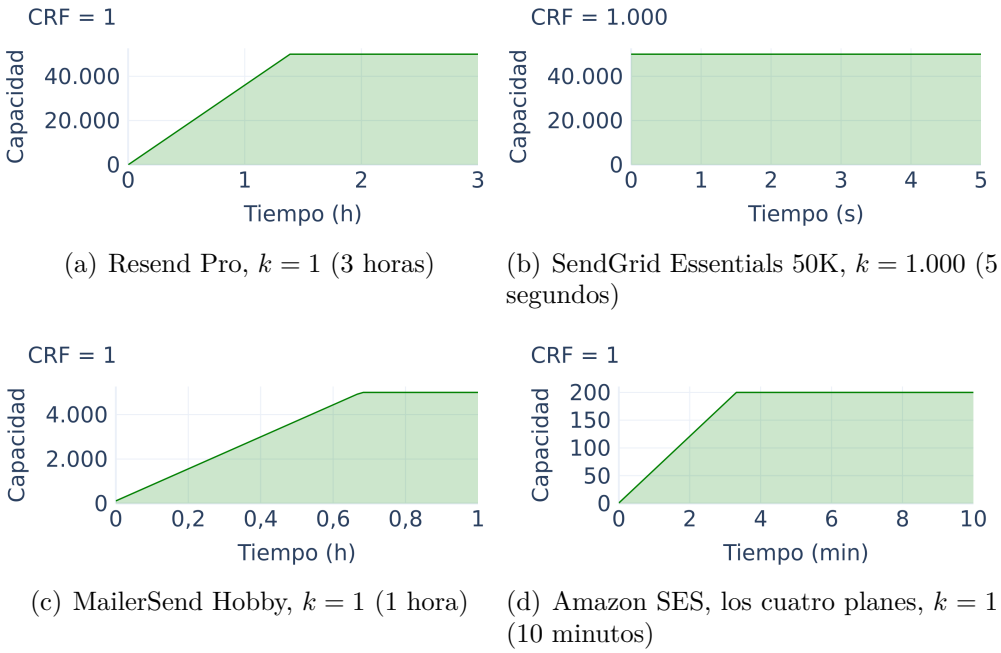
Las cuatro magnitudes son las que PRIME4API lee de la Datasheet para construir los límites de cada plan [3]. El mes se toma de 30 días (2.592.000 segundos) y el día de 86.400 segundos. Con meses de 31 días los cocientes cambian un 3,3 %.

Con ocho APIs, esta notación no cubre tres casos: los dos contadores de Mailgun, el caudal de Mailchimp en correos por hora y su franquicia sin ventana. En Mailgun, la cabecera de la respuesta de cada envío aceptado trae dos contadores sobre el mismo *endpoint*: uno de 80.000 peticiones y otro de 2.000.000 de destinatarios. Los dos comparten una ventana que Mailgun no publica y que dos mediciones situaron en unos diez segundos (Sección 6.2). Hay entonces dos términos de *rate*, R en peticiones y R_d en destinatarios, y cuál de los dos corta antes depende de k. La Tabla 6.2 calcula el cociente de Mailgun con el contador de peticiones, y el de destinatarios solo entra en su $t_{\text{mín}}$ con el k máximo.

En Mailchimp el caudal se declara en correos por hora y no en peticiones, así que k no multiplica nada: el *rate* solo frena el envío cuando el ritmo medio supera los 25 correos por hora. La franquicia de 500 correos de su cuenta de prueba tampoco tiene ventana. Sin W no hay cociente que formar, porque los 500 correos de la franquicia se gastan una sola vez y no se reponen. En Mailchimp no se calcula el cociente y se anota el tipo de límite.

La misma relación se lee como una curva. Ruiz-Cortés y Ruiz-López interpretan el *rate* como la velocidad máxima de consumo y la cuota como el volumen máximo acumulable en un periodo, y la capacidad efectiva sale de aplicar los dos límites a la vez [2]. El ACM adopta esa interpretación como su semántica. Sobre ella, la curva de capacidad $C(t)$ de Gavira Sánchez [3] es el máximo de correos acumulables desde el inicio de la ventana hasta el instante t sin violar ningún límite.

Con un *rate* y una cuota, la curva tiene dos tramos. Mientras manda el *rate*, la curva es una recta de pendiente $R \cdot k$. Cuando manda la cuota, se vuelve horizontal a la altura de Q. El codo entre ambos cae en $t_{\text{mín}}$. Cuanto antes llega el codo dentro de la ventana, más holgado es el *rate*. Esa holgura se mide con $W/t_{\text{mín}} = R \cdot W \cdot k/Q$, que es el cociente $R \cdot W/Q$ multiplicado por k. En Resend Pro con $k = 1$ el codo llega a los 5.000 segundos, unos 83 minutos, y el tramo horizontal ocupa el resto de los 30 días. Con los 50 destinatarios por petición que admite Resend, el codo se adelanta a los 100 segundos. La curva no recoge los dos contadores de Mailgun ni la franquicia sin ventana de Mailchimp.



(a) Resend Pro, $k = 1$ (3 horas)

(b) SendGrid Essentials 50K, $k = 1.000$ (5 segundos)

(c) MailerSend Hobby, $k = 1$ (1 hora)

(d) Amazon SES, los cuatro planes, $k = 1$ (10 minutos)

Figura 6.1: Curva de capacidad $C(t)$ de cuatro planes, dibujada por PRIME4API [19] en local. Entre paréntesis, el intervalo que muestra cada gráfica. Tras el codo, la curva sigue horizontal hasta que la cuota se repone: a los 30 días en Resend, SendGrid y MailerSend, y a las 24 horas en Amazon SES. El área sombreada es la capacidad acumulable.

La Figura 6.1 muestra la curva de cuatro planes. La dibuja el propio PRIME4API [19], ejecutado en local con la misma imagen de contenedor que despliega SPHERE. El motor lee las Datasheets y su renderizador traza la curva. Cada gráfica muestra el intervalo que hace visible el codo, porque en una ventana de 30 días el tramo inclinado no se distinguiría de una vertical. El motor da por enviadas en el instante $t = 0$ las peticiones que el *rate* admite en su primer periodo, un segundo en Resend Pro y un

minuto en MailerSend Hobby. Esa es la primera ráfaga, y por eso el motor da 4.999 segundos en Resend Pro donde el cálculo a mano da 5.000.

En SendGrid Essentials 50K se toma $k = 1.000$, el máximo de destinatarios que el proveedor admite por petición. Con él la cuota cabe en 50 peticiones, el tramo inclinado dura cinco milésimas de segundo y la curva alcanza la cuota casi desde el origen. En MailerSend Hobby con $k = 1$ el tramo inclinado sube 120 correos por minuto y el codo llega a los 41 minutos, con una cuota de 5.000 correos al mes. La gráfica de MailerSend Hobby se pide en correos, la unidad que fija el parámetro `capacity_unit` del motor, y por eso no muestra la cuota diaria de peticiones del plan (Apartado 6.4.4). Con esa cuota diaria de 1.000 peticiones, los 5.000 correos con $k = 1$ tardan 4 días y 8 minutos, y no 41 minutos.

En Amazon SES el precio del plan no cambia la curva. Sus cuatro planes, los tres de la página de precios y la tarifa por uso sin plan, de 0 a 500 USD al mes, devuelven la misma curva, porque la Datasheet les asigna la misma cuota y el mismo caudal. Son los del entorno de pruebas (*sandbox*), el único para el que Amazon publica cifras. Toda cuenta nueva de SES empieza en ese entorno, solo puede enviar a direcciones y dominios verificados, y pasa a producción cuando el titular lo pide y Amazon lo aprueba. En ese entorno el *rate* es de una petición por segundo. Con un destinatario por petición, la cuota de 200 correos al día se agota en 200 segundos (199 en el motor), y la cuenta no envía nada más hasta que la cuota se repone, a las 24 horas.

El entorno de pruebas es un estado de la cuenta: la condición en que el proveedor tiene la cuenta o su suscripción, con independencia del plan contratado. El estado de la cuenta reúne el estado de la suscripción, como la cuenta en pruebas o pendiente de aprobación, y la reputación, que separa la cuenta sana de la cuenta en revisión. En el catálogo, MailerSend baja el *rate* con la cuenta en revisión, y la cuenta de prueba de Mailchimp Transactional tiene su propio caudal de 25 correos por hora. Una cuenta en estado sano es la que está aprobada y sin restricción por reputación. El Apartado 5.3.3 separa el estado de la suscripción de la reputación con el caso de MailerSend.

6.2. Lo que publica cada proveedor

Las Tablas 6.1 y 6.2 dan, plan a plan, la cuota Q y el *rate* R que declara cada Datasheet, con el semáforo de confianza del Apartado 4.3.1. Un dato va en verde si es literal del proveedor o lo fijó un panel con cuenta propia o un sondeo, en ámbar si exige una lectura que la fuente no hace literal o si dos fuentes oficiales chocan, y en rojo si el proveedor no lo publica. Con esos pares las tablas calculan el cociente $R \cdot W/Q$ y el tiempo $t_{\text{mín}}$ que tarda el *rate* en agotar la cuota, definidos en la Sección 6.1. Cada cifra tiene su respaldo en el registro de procedencia de su Datasheet, que guarda para cada valor la URL, la fecha de acceso y la cita literal de la fuente.

Cada proveedor reparte sus términos de consumo, el precio y la información de cómo, cuándo, cuánto y dónde se consume, entre varias páginas: la de precios, la de límites de la API, la guía del relé SMTP o unas preguntas frecuentes. Los párrafos siguientes recogen los casos en que la fuente de la cifra cambia el cálculo, como el *rate*

de Postmark, que solo da la cabecera de la respuesta, o el de MailerSend, que baja con la cuenta en revisión.

Una cifra que solo se lee en una cabecera y otra que no se puede observar. En Postmark el *rate* de la API REST no está publicado, y la documentación describe el error 429 «Rate Limit Exceeded» sin dar el umbral [63]. La cifra sale de la propia API: los cuatro *endpoints* de envío probados devuelven `ratelimit-limit: 5000` y `x-ratelimit-limit-second: 5000`, tanto en la respuesta aceptada (200) como en la rechazada por datos no válidos (422). Que el límite sea por segundo es una lectura del nombre de la cabecera, no un literal del proveedor. Un sondeo de 200 peticiones no llegó al 429, y la Datasheet adopta 5.000 peticiones por segundo en ámbar para los cuatro planes, aunque solo se observó en la cuenta gratuita.

SendGrid está en el caso opuesto, con una cifra publicada que no se puede observar. Da 10.000 peticiones por segundo para `/v3/mail/send` una sola vez, en las preguntas frecuentes del *endpoint* y sin plan ni ámbito[1]. La Datasheet la pone en verde porque es un literal del proveedor. No hay con qué contrastarla: el envío en modo *sandbox* no devuelve cabeceras `x-ratelimit-*`. Ese modo es una opción de la petición que la valida sin entregar el correo, y no tiene que ver con el entorno de pruebas de Amazon SES.

Un *rate* que cambia con el estado de la cuenta. MailerSend ata su *rate* por minuto a la categoría del *endpoint* y al estado de la cuenta [13]. Con la reputación sana, `/v1/email` admite 120 peticiones por minuto en los cuatro planes, es decir, $R = 2$, y `/v1/bulk-email` entre 10 y 30 según el plan. Con la cuenta bajo revisión, `/v1/email` baja a 10 peticiones por minuto y `/v1/bulk-email` a 1. El sondeo del Apartado 5.3.3 midió esos 10 en una cuenta sin aprobar, que es un estado de la suscripción distinto de la revisión por reputación para la que el proveedor publica la cifra. Las 120 peticiones por minuto están publicadas, pero no observadas.

Cada plan de MailerSend tiene además una cuota diaria de peticiones, común a las dos interfaces de envío, la API REST y el relé SMTP. Va de 100 peticiones en Free a 500.000 en Professional. Esa cuota no entra en la Ecuación (6.1) porque cuenta peticiones y no correos, y la Tabla 6.1 la anota aparte.

[1] https://www.twilio.com/docs/sendgrid/for-developers/sending-email/v3-mail-send-faq. La cita literal está en el *provenance* de `03_datos/apis/sendgrid/datasheet.yaml`.

API · plan	$R \cdot W$	Q	$R \cdot W/Q$	$t_{\text{mín}}$ (k=1 / k máx)	**Datos en ámbar o rojo**
Postmark · Free	$1{,}296 \cdot 10^{10}$	100	129.600.000	0,02 s / 0,0004 s	*rate* medido, no publicado ●
Postmark · Basic	$1{,}296 \cdot 10^{10}$	10.000	1.296.000	2 s / 0,04 s	*rate* ●, cuota y ventana ●
Resend · Free (día)	864.000	100	8.640	10 s / 0,2 s	ventana diaria ●
Resend · Free (mes)	25.920.000	3.000	8.640	300 s / 6 s	ventana mensual ●
Resend · Pro	25.920.000	50.000	518,4	83,3 min / 100 s	*rate* por defecto
Resend · Scale	25.920.000	100.000	259,2	2,78 h / 200 s	*rate* por defecto
Resend · Scale 2,5M	25.920.000	2.500.000	10,37	69,4 h / 83,3 min	volumen contratado no recogido
Resend · Enterprise	n.c.	Custom	n.c.	n.c.	*rate* ●
SendGrid · Trial (día)	864.000.000	100	8.640.000	0,01 s / 1 petición	*rate* no observable
SendGrid · Essentials 50K	$2{,}592 \cdot 10^{10}$	50.000	518.400	5 s / 0,005 s	*rate* no observable
SendGrid · Pro 100K	$2{,}592 \cdot 10^{10}$	100.000	259.200	10 s / 0,01 s	*rate* no observable
SendGrid · Pro 2,5M	$2{,}592 \cdot 10^{10}$	2.500.000	10.368	250 s / 0,25 s	volumen contratado no recogido
SendGrid · Premier	$2{,}592 \cdot 10^{10}$	Custom	n.c.	n.c.	cuota *custom*, ventana ●
MailerSend · Free	5.184.000	500	10.368	250 s / 4 s	ventana ●, k ●, *rate* sin observar. Diaria de 100 pet.: 4 d 50 s (k=1)
MailerSend · Hobby	5.184.000	5.000	1.036,8	41,7 min / 36 s	k ●. Diaria de 1.000 pet.: 4 d 8 min (k=1)
MailerSend · Starter	5.184.000	50.000	103,7	6,94 h / 6 min	k ●, lote de 15/min ●
MailerSend · Starter 1M	5.184.000	1.000.000	5,18	5,8 d / 2 h	volumen contratado no recogido. Diaria de 100.000 pet.: 10 d (k=1)
MailerSend · Professional 2M	5.184.000	2.000.000	2,59	11,6 d / 4 h	volumen contratado no recogido
MailerSend · Starter 1M, rev.	432.000	1.000.000	0,43	$>$ 30 d / 24 h	*rate* de 10/min medido en la cuenta sin aprobar, volumen contratado no recogido

Tabla 6.1: Relación entre *rate* y cuota en Postmark, Resend, SendGrid y MailerSend. $R \cdot W$ va en peticiones por ventana y Q en correos, con un mes de 30 días. $t_{\text{mín}}$ se da con $k = 1$ y con el k máximo de cada API: 50 en Postmark y Resend, 1.000 en SendGrid y 70 en MailerSend, este en ámbar. Cuando la cuota se agota en unas pocas peticiones con el k máximo, $t_{\text{mín}}$ se da en número de peticiones en lugar de en segundos. La fila «rev.» usa el *rate* de una cuenta de MailerSend bajo revisión. Un volumen contratado «no recogido» lo publica el proveedor y la Datasheet no lo recoge (Apartado 5.3.1). Postmark Pro y Platform tienen la misma cuota y el mismo *rate* que Basic, y se omiten. n.c.: no calculable.

Categorías de *rate* sin plan asignado. Brevo da tres cifras para el mismo `POST /v3/smtp/email`, un *endpoint* de la API REST pese a su nombre: 1.000 peticiones por segundo en la categoría General, 2.000 en Advanced y 6.000 en Extended [67]. La página no cierra qué plan cae en cada una, porque una viñeta reserva Advanced a Professional y Enterprise, y el distintivo «Available in» que encabeza la tabla de Advanced enumera los cinco planes, Free incluido. Los tres valores van en ámbar, y la Tabla 6.2 calcula Professional con la cota inferior, la de General, y Enterprise con la superior, la de Extended.

La guía de envío por lotes da otra cifra para el mismo *endpoint*, «Execute the *endpoint* up to 6000 times per hour», 600 veces menos que los 3.600.000 por hora de la categoría General. La contradicción sigue abierta, y la tabla calcula con la página de categorías.

Cifras que solo valen para el entorno de pruebas. En Amazon SES las

cifras publicadas son las del entorno de pruebas, que es un estado de la cuenta y no un plan: un correo por segundo, 200 correos cada 24 horas y 50 destinatarios por mensaje [68]. Para la cuenta de producción, la página de límites escribe que «this number varies based on your specific use case», y la referencia de la API reserva en `Max24HourSend` el valor -1 para una cuota ilimitada, sin decir qué cuenta lo recibe. Por eso los cuatro planes que la Datasheet toma de la página de precios [62] reciben la cuota y el *rate* del entorno de pruebas.

Hay dos límites de SES que la Datasheet no pudo modelar con ACM2YAML v0.3. El primero es el tiempo que SES pide esperar tras un rechazo por limitación, que no tiene campo en el esquema. La página de errores de la cuota de envío da «up to 10 minutes», y la de problemas del relé SMTP, para las peticiones SMTP rechazadas, una serie de esperas de 5, 10 y 30 segundos seguida de 20 minutos. La primera página cubre también los rechazos SMTP, así que las dos indicaciones chocan.

El segundo es la petición por segundo que la página de límites aplicaba, cuando se consultó, a todas las operaciones salvo SendEmail, SendRawEmail y SendTemplatedEmail, tres nombres de la versión 1 de la API. La Datasheet modela la versión 2, y el esquema no podía acotar esa cifra a sus operaciones porque la página no decía cuáles cubría: el envío por lotes no estaba ni entre las exentas ni entre las limitadas. Un sondeo contra `GET /v2/email/account` aceptó más de cinco veces ese valor sin fijar el umbral (Apartado 4.3.3). Desde finales de octubre de 2026 la página exime SendEmail y SendBulkEmail, y con ese texto la cifra ya se puede acotar.

Una cifra publicada y otra medida, con ámbitos distintos. Mailgun publica una sola cifra de *rate* para su API REST, «1,500 API requests per second per client IP», en las preguntas frecuentes de su sitio comercial y no en la documentación de desarrollador[2]. Leída al pie de la letra es un tope por IP de cliente, un ámbito que el esquema no sabe declarar, y la Datasheet la pone en ámbar como caudal del plan.

Lo medido es otro contador, el de la cabecera del envío aceptado: `x-ratelimit-limit: 80000`, 80.000 peticiones por ventana. La ventana la abre la primera petición, y dos mediciones de su duración dieron 9,85 y 9,87 segundos. La misma respuesta trae `x-recipient-limit: 2000000`, un contador que limita los destinatarios y no las peticiones. Es el segundo *rate*, R_d, que la Sección 6.1 admite junto al de peticiones, y no está en la Datasheet porque el esquema admite un solo *rate* por nodo, sea un plan o un *endpoint*.

La Tabla 6.2 calcula con $R = 8.000$ peticiones por segundo, una cifra que no publica Mailgun ni trae la cabecera: es el cociente de esas 80.000 peticiones entre los diez segundos a los que la Datasheet redondea la ventana. La Datasheet declara las 1.500 peticiones por segundo en el plan y el contador de 80.000 en el *endpoint* de envío. PRIME4API, que suma los límites del plan a los del *endpoint*, aplica las dos cifras a la vez y calcula con la más baja, las 1.500.

La cuota del plan Free de Mailgun tiene dos cifras: la página de precios da 100 correos al día [71] y el panel de la cuenta muestra 3.000 al mes. Ninguna de las dos fuentes dice si son dos topes simultáneos, y la tabla calcula con ambos.

[2] `https://www.mailgun.com/ai-info/`. La cita literal está en el *provenance* de `03_datos/apis/mailgun/datasheet.yaml`.

Un caudal en correos por hora. Mailchimp Transactional no publica *rate* en peticiones para el envío, y ni sus páginas ni el sondeo dan cabeceras de *rate* [74]. El envío de la cuenta de prueba lo gobierna «a fixed hourly quota of 25 emails», y los correos que pasan de ese tope no se rechazan: entran en una cola que se procesa en la hora siguiente. La Datasheet interpreta ese tope como un *rate* de 25 correos por hora. La franquicia de 500 correos de esa cuenta no tiene ventana, y en la cuenta de pago la cuota es un bloque de 25.000 correos [72], con un *rate* que depende de la reputación y no tiene cifra publicada.

API · plan	$R \cdot W$	Q	$R \cdot W/Q$	$t_{\text{mín}}$ (k=1 / k **máx**)	**Datos en ámbar o rojo**
Brevo · Free (día)	86.400.000	300	288.000	0,3 s / < 1 pet.	*rate* ●, hora del reinicio diario ●
Brevo · Starter	$2{,}592 \cdot 10^9$	5.000	518.400	5 s / 3 pet.	*rate* ●, k ●, 99 destinatarios con adjunto
Brevo · Standard	$2{,}592 \cdot 10^9$	5.000	518.400	5 s / 3 pet.	ídem Starter, misma cuota y distinto precio
Brevo · Professional	$2{,}592 \cdot 10^9$	150.000	17.280	150 s / 75 pet.	categoría Advanced sin atribuir ●
Brevo · Professional 10M	$2{,}592 \cdot 10^9$	10.000.000	259,2	2 h 47 min / 5.000 pet.	volumen contratado no recogido
Brevo · Enterprise	$1{,}5552 \cdot 10^{10}$	Custom	n.c.	n.c.	categoría Extended ●
SES · pruebas	86.400	200	432	200 s / 4 s	ventana de la cuota ●, no es cuota de plan
SES · producción	n.c.	no publicada	n.c.	n.c.	cuota ● y *rate* ●
Mailgun · Free (día)	691.200.000	100	6.912.000	0,0125 s / < 1 pet.	una o dos cuotas, sin decidir ●, hora de reposición ●
Mailgun · Free (mes)	$2{,}0736 \cdot 10^{10}$	3.000	6.912.000	0,375 s / < 1 pet.	ídem, la diaria por 30 da la mensual
Mailgun · Basic	$2{,}0736 \cdot 10^{10}$	10.000	2.073.600	1,25 s / 0,05 s	ventana del *rate* fuera del esquema ●
Mailgun · Foundation	$2{,}0736 \cdot 10^{10}$	50.000	414.720	6,25 s / 0,25 s	ídem, *rate* no atribuido por plan ●
Mailgun · Scale	$2{,}0736 \cdot 10^{10}$	100.000	207.360	12,5 s / 0,5 s	ídem
Mailgun · Scale 2,5M	$2{,}0736 \cdot 10^{10}$	2.500.000	8.294,4	312,5 s / 12,5 s	volumen contratado no recogido
Mailchimp · prueba	en correos/h	500	no formable	20 h (cualquier k)	cuota sin ventana ●, k sin tope ●
Mailchimp · bloque de pago	sin cifra	25.000	no formable	n.c.	*rate* sin publicar, ventana del bloque ●

Tabla 6.2: Relación entre *rate* y cuota en Brevo, Amazon SES, Mailgun y Mailchimp, con las convenciones de la Tabla 6.1. El k máximo vale 2.000 en Brevo (en ámbar), 50 en SES y 1.000 en Mailgun. El *rate* de Mailgun es el de la cabecera del envío, 80.000 peticiones en una ventana medida de unos diez segundos. Su $R = 8.000$ es el cociente de esas 80.000 peticiones entre los diez segundos a los que la Datasheet redondea la ventana, y no una cifra publicada. Su $t_{\text{mín}}$ con k máximo lo gobierna el contador de 2.000.000 de destinatarios sobre la misma ventana. Las dos filas de SES son estados de la cuenta y valen para los cuatro planes. Con adjunto, Brevo baja el tope a 99 destinatarios por petición. Mailchimp declara su *rate* en correos por hora, y por eso no tiene $R \cdot W$. n.c.: no calculable.

La especificación formal dice menos que la prosa. La especificación formal de una API es su descripción legible por máquina: un fichero OpenAPI o Swagger que enumera cada ruta con sus parámetros y sus respuestas. La de SendGrid, en el

repositorio `twilio/sendgrid-oai`, no menciona los *rates* ni el tope de `to`, y el tope de 1.000 destinatarios por petición, que suma `to`, `cc` y `bcc`, solo aparece en la referencia en prosa [59]. La de MailerSend, publicada en SwaggerHub, tampoco declara el tope de `to` ni la respuesta 429 [13]. La de Brevo, una definición Swagger de 207 rutas, no menciona el *rate* ni una vez [67].

Amazon SES publica sus límites en cuatro documentos: la página de límites, la referencia de la operación de envío, la especificación formal de la versión 2 y la guía del relé SMTP [68]. En una tabla de doce límites por esos cuatro documentos, de la cuota diaria a la espera tras el rechazo, quedan vacías 32 de las 48 celdas, y la columna de la especificación formal lo está entera. Una Datasheet construida solo desde la especificación formal recogería menos límites que la construida desde la prosa en esas cuatro APIs, y la comparación no se hizo en las otras cuatro.

6.3. Lo que enseña el catálogo

Las Tablas 6.1 y 6.2 recogen el caudal, la cuota y la ventana de cada API, y de ellas salen seis afirmaciones sobre la relación entre límites de uso y *rates*. Cada afirmación se cuenta sobre las ocho APIs y no sobre sus planes, para que una API con cinco planes no pese cinco veces. El recuento reparte las ocho en cuatro grupos, que se escriben siempre en este orden: a favor, en contra, no comprobable y no aplica. Una API cuenta a favor cuando su Datasheet tiene el dato que la afirmación necesita y ese dato va en la dirección enunciada. Cuenta en contra cuando tiene el dato y apunta al lado opuesto, y entonces es un contraejemplo y se nombra.

Los otros dos grupos separan el dato que falta de la afirmación que no se puede plantear. Una API es no comprobable cuando la afirmación tiene sentido en ella pero ningún hecho recogido aporta el dato, y en ese caso nunca suma a favor. No aplica cuando la afirmación presupone una estructura que la API no tiene, como los planes en Mailchimp Transactional o la cuota incluida en Amazon SES.

El cociente $R \cdot W/Q$ es el de la Sección 6.1: cuántas veces cabe la cuota en lo que el *rate* admite durante su ventana. Como se calcula con un destinatario por petición ($k = 1$), es una cota inferior, porque enviar a varios destinatarios en cada petición solo puede agrandarlo.

La primera afirmación es que, con la cuenta en estado sano, la cuota es la restricción activa en toda API cuya Datasheet lleva *rate* y cuota con valor. El recuento es de seis a favor, ninguna en contra, una no comprobable y una que no aplica. El cociente $R \cdot W/Q$ va de 103,7 en MailerSend Starter a 129.600.000 en el plan Free de Postmark, es decir, de dos a ocho órdenes de magnitud. Con los volúmenes contratados que el proveedor publica y la Datasheet no recoge, el mínimo baja a 2,59 en MailerSend Professional 2M, algo más del doble.

La no comprobable es Amazon SES, porque fuera del entorno de pruebas el proveedor no publica ni el *rate* ni la cuota, que cada cuenta recibe al obtener el acceso de producción. La que no aplica es Mailchimp Transactional, que no tiene planes y cuya franquicia de prueba de 500 correos no caduca ni se renueva. Sin ventana en la

cuota, el cociente no está definido. Con 25 correos por hora, agotar esa franquicia lleva veinte horas, así que en esta API el tiempo lo pone el caudal.

En esas seis APIs el *rate* del cociente sale de fuentes distintas, recogidas en la Sección 6.2. Solo MailerSend lo publica para todos sus planes, con 120 peticiones por minuto, y Resend publica un valor por defecto por equipo. Postmark lo devuelve en la cabecera de respuesta, y la de Mailgun da 80.000 peticiones por una ventana medida de unos diez segundos, que la Tabla 6.2 divide para obtener 8.000 por segundo. SendGrid lo da en unas preguntas frecuentes sin plan, y Brevo no dice qué plan cae en cada categoría. Para dimensionar un volumen de envío, el *rate* puede ignorarse en seis de las ocho APIs mientras la cuenta esté sana, y en Amazon SES la pregunta no tiene respuesta antes de abrir la cuenta.

La segunda afirmación tiene dos mitades que se cuentan aparte: el *rate* del envío individual no varía con el plan, y lo que el plan compra es cuota. La primera mitad se limita al envío individual porque MailerSend publica para el envío por lotes, `/v1/bulk-email`, un caudal que sube con el plan: 10 peticiones por minuto en Hobby, 15 en Starter y 30 en Professional. Esa mitad queda en tres a favor, una en contra, dos no comprobables y dos que no aplican. Las tres a favor son Resend, SendGrid y MailerSend. Resend publica un valor por defecto por equipo, SendGrid publica 10.000 peticiones por segundo sin condición de plan y MailerSend publica 120 peticiones por minuto en `/v1/email` para sus cuatro planes.

El contraejemplo es Brevo, con las tres categorías de caudal para el mismo *endpoint* de envío que recoge la Sección 6.2. Pasar de la categoría General a la Extended multiplica por seis el caudal del envío individual, que así puede crecer al subir de plan. El dato va en ámbar (•) porque la página no dice qué plan cae en cada categoría. Postmark y Mailgun son no comprobables, porque su caudal de envío se observó en la cabecera de una sola cuenta gratuita y la Datasheet lo extiende a todos los planes por falta de dato. La mitad no aplica a Amazon SES, cuyo único caudal publicado es el del entorno de pruebas, un estado de la cuenta y no un plan. Tampoco aplica a Mailchimp Transactional, que no tiene planes y fija el caudal según su reputación.

La segunda mitad queda en seis a favor y dos que no aplican. Las seis a favor son Postmark, Resend, SendGrid, MailerSend, Brevo y Mailgun, cuya cuota crece con el precio del plan, en Brevo de 300 correos al día a diez millones al mes. En Brevo el plan compra además caudal de envío individual, y en MailerSend, caudal de envío por lotes. En Amazon SES el plan no compra volumen, porque ningún plan incluye cuota: los 200 correos al día del entorno de pruebas valen igual para los cuatro planes, que compran funciones y tramos de precio unitario [62]. En Mailchimp Transactional no hay planes, y lo que se compra es la propia unidad de volumen, el bloque de 25.000 correos, a un precio que baja de 20 a 10 USD por bloque [72].

La tercera afirmación es que el *rate* que publica el proveedor coincide con el que devuelve la API en sus cabeceras o con el que mide un sondeo. El recuento es de cuatro a favor, ninguna en contra, cuatro no comprobables y ninguna que no aplique. Las cuatro que coinciden publican el valor con su ámbito escrito: Resend lo da por equipo, MailerSend por estado de la cuenta, Brevo ruta a ruta y Mailchimp Transactional para la cuenta de prueba. La cabecera del envío de Brevo devolvió las 1.000 peticiones por segundo de su tabla General, y en Mailchimp Transactional la documentación, el panel

y la medición coinciden en 25 correos por hora.

En MailerSend la coincidencia es de valor y no de estado. Las 10 peticiones por minuto que el proveedor publica para la cuenta en revisión son las que el sondeo midió en una cuenta sin aprobar, que es otro estado, y las 120 de la cuenta sana siguen sin observar.

Dos de esas coincidencias tienen una segunda cifra al lado. En Brevo la coincidencia vale solo para la categoría General, porque su guía de envío por lotes da para el mismo *endpoint* 6.000 peticiones por hora, 600 veces menos. En Resend, la Datasheet de ejemplo que publica el repositorio de la guía de ACM2YAML, fechada en febrero de 2026, fija 2 peticiones por segundo [61], y la documentación consultada en agosto de 2026 publica 10 [73]. Las capturas de su página de límites en Wayback Machine muestran que cambió la oferta: 2 por segundo hasta marzo de 2026, 5 entre abril y julio, y 10 desde agosto.

Las cuatro no comprobables carecen de una cifra publicada y observada para el mismo *endpoint* y el mismo ámbito. Postmark no publica *rate* y devuelve 5.000 en la cabecera de cada respuesta. SendGrid publica 10.000 peticiones por segundo para un *endpoint* de envío que, en modo *sandbox*, no devuelve cabeceras, y en los de lectura devuelve 600 y 6 en `X-RateLimit-Limit`, valores que ninguna página documenta. Amazon SES no publica su caudal de producción.

En Mailgun, la cifra publicada y la medida tienen ámbitos distintos. Sus preguntas frecuentes dan 1.500 peticiones por segundo por IP de cliente para toda la API, y la cabecera del envío cuenta otro contador, 80.000 peticiones por una ventana de unos diez segundos (unas 8.000 por segundo si se dividen). La medición no dice cuál de los dos rige en el envío, así que no contradice la cifra publicada.

La cabecera de respuesta tampoco es una señal fiable en todo el catálogo. Amazon SES no emite ninguna cabecera de caudal, y sus nueve respuestas 429 medidas no traen `Retry-After`, la cabecera que dice cuánto esperar antes de reintentar. Mailchimp Transactional no emite cabecera de caudal en ninguna respuesta del sondeo ni la menciona en ninguna página oficial. Mailgun la emite solo en un POST aceptado, así que leer el caudal le cuesta al cliente un envío.

Donde la cabecera llega, la documentación no siempre la describe bien. Brevo afirma que llega en todas las respuestas, y en la medición llegó en una respuesta de error 400 y no en dos respuestas correctas 200. En SendGrid, `X-RateLimit-Reset` trae en tres *endpoints* un valor de una o dos cifras (6, 9 y 58), que se lee como los segundos que faltan, y en otros tres una marca de tiempo Unix. La documentación solo muestra la segunda forma [75]. Una herramienta que lea esa cabecera para calcular cuándo reintentar encuentra el mismo nombre con dos formas de valor y ningún aviso.

La alternativa a la cabecera es un *endpoint* que informa de la capacidad restante de la cuenta sin gastar cuota, y cuatro de las ocho APIs lo tienen, aunque el de Brevo da el saldo y no el límite. Amazon SES devuelve en un `GET` la cuota de 24 horas y el caudal máximo de envío de la cuenta, y su documentación lo señala como la vía para conocerlas. Mailchimp Transactional devuelve la cuota horaria junto a la reputación en `POST /users/info`, y MailerSend tiene `GET /v1/api-quota` [13]. El `GET /v3/account` de Brevo devuelve el saldo de correos y el periodo exacto del ciclo, pero ninguno de sus

campos es de límite.

Postmark, Resend, SendGrid y Mailgun no tienen ninguno, y en ellas el cliente conoce su capacidad restante después de gastarla o no llega a conocerla. Donde existe, el *endpoint* tampoco garantiza un dato al día. En MailerSend, once horas y veinticuatro minutos después de la medianoche que repone la cuota diaria, seguía sirviendo el saldo del día anterior.

La cuarta afirmación es que el proveedor publica un tope de destinatarios por petición, el valor máximo del factor k con el que se pasa del *rate*, que cuenta peticiones, a la cuota, que cuenta correos. El recuento es de siete a favor, una en contra, ninguna no comprobable y ninguna que no aplique, porque siete APIs publican ese tope y una no. Postmark, Resend y Amazon SES admiten 50 destinatarios por petición, MailerSend 70, SendGrid 1.000, Mailgun 1.000 por lote y Brevo 2.000. El contraejemplo es Mailchimp Transactional, que escribe que no hay limitación de destinatarios y que el JSON de la llamada debe pesar menos de 10 MB [74]. Ahí la cota de una llamada está en megabytes, y no hay k que pase de una unidad a la otra.

Cinco de los siete topes están verificados por sondeo: los de Postmark, Resend, SendGrid y Mailgun, y el de la cuenta sin aprobar de MailerSend. En Mailgun el destinatario 1.001 devuelve un error 400 que nombra el máximo de 1.000. En Brevo no hubo sondeo de destinatarios, y el tope de Amazon SES sale de su página de cuotas, que lo define como la suma de `To`, `Cc` y `Bcc` [68]. En Amazon SES una petición de 50 destinatarios consume la cuarta parte de la cuota diaria del entorno de pruebas, y en Brevo una de 2.000 equivale a casi siete veces la cuota diaria del plan Free.

El factor k solo conecta las dos magnitudes si cada destinatario descuenta una unidad de cuota. De las cuatro APIs en las que esa equivalencia está sostenida (Sección 6.1), Resend y SendGrid la publican, y en Brevo y Amazon SES salió de medir. En Brevo, el saldo de la cuenta bajó de 300 a 298 tras una petición con dos destinatarios. En Amazon SES, el contador marcó 5 sobre 200 tras cuatro mensajes con cinco destinatarios, porque uno llevaba la misma dirección en `to` y en `cc`. Ese recuento descarta a la vez que la unidad sea el mensaje y que el proveedor deduplique direcciones, porque cualquiera de las dos habría dado cuatro. En Postmark, MailerSend, Mailgun y Mailchimp Transactional esa equivalencia es una suposición que ningún dato confirma.

La quinta afirmación es que el proveedor no publica un caudal propio de cada plan ni cuándo se repone la cuota. En la mitad que trata del caudal del envío individual, el recuento es de siete a favor, una en contra, ninguna no comprobable y ninguna que no aplique. Un caudal común a todos los planes, como las 120 peticiones por minuto de MailerSend, cuenta a favor, porque no es propio de ningún plan. El contraejemplo es Brevo, la única que pone una cifra de ese caudal junto a una categoría de plan. Amazon SES cuenta a favor porque no publica el caudal de producción. El envío por lotes de MailerSend, con 10, 15 y 30 peticiones por minuto según el plan, queda fuera de esta mitad por el mismo motivo que en la segunda afirmación.

Cuatro de las siete Datasheets declaran de formas distintas el caudal que el proveedor no liga a un plan. La de Amazon SES deja vacío el bloque de caudal, la de Mailgun referencia un único valor desde sus cuatro planes y la de MailerSend toma las 120 peticiones por minuto como valor por defecto. La de Mailchimp Transactional

repite el caudal como estado de la cuenta en sus tres *endpoints* de envío.

La otra mitad de la quinta afirmación, la de la ventana de la cuota, queda en ocho a favor. Ninguna API tiene esa ventana en verde si cada una se clasifica por el peor semáforo de sus cuotas: cuatro están en rojo (●) y cuatro en ámbar (●). SendGrid es la que más se acerca, porque publica la medianoche UTC y el periodo de facturación, y solo la ventana de su plan Premier queda en ámbar.

Los cuatro rojos responden a tres causas. Mailgun y Brevo dan valor y periodo para las cuotas de su plan Free, pero ninguna fuente fecha su reposición. Amazon SES describe su cuota diaria como deslizante en una página, es decir, que en cada envío cuenta lo enviado en las 24 horas anteriores, y como fija en otra, en la que hay que esperar al siguiente periodo de 24 horas. El campo se deja vacío a propósito. Mailchimp Transactional escribe que su franquicia no caduca ni se renueva, mientras ACM2YAML v0.3 exige un periodo en toda cuota. En esas cuatro APIs, quien integra la API no puede calcular cuándo se repone su cuota sin observarlo en su propia cuenta.

La ventana del caudal no la declara ninguna de las ocho, y dos sondeos la acotaron. En Brevo la ruta `GET /v3/smtp/emails`, que consulta la actividad de envío, volvió a admitir peticiones antes de cumplirse un segundo desde la primera de la serie, y con una sola muestra no se distingue una ventana fija alineada con el reloj de una reposición gradual. En Mailgun la ventana dura entre 9,85 y 9,87 segundos, la abre la primera petición y no es deslizante, y ninguno de los tres tipos de ventana que admite ACM2YAML v0.3 (deslizante, fija a la medianoche UTC y fija al ciclo de facturación) la describe.

La sexta afirmación es que el estado de la cuenta condiciona la capacidad, con independencia del plan. El recuento es de seis a favor, dos en contra, ninguna no comprobable y ninguna que no aplique. Son las seis APIs en que la capacidad depende del estado de la cuenta y no del plan, recogidas como F-18 en el Apartado 5.3.3. Ese apartado describe el caso de MailerSend y la extensión que se propone para modelar el estado de la suscripción. Solo MailerSend da una cifra para cada estado: 120 peticiones por minuto en `/v1/email` con la cuenta sana y 10 con la cuenta en revisión.

Las otras cinco a favor son Amazon SES, Mailchimp Transactional, Postmark, Resend y Mailgun. Amazon SES publica los topes del entorno de pruebas y no los de producción. Mailchimp Transactional fija 25 correos por hora en la cuenta de prueba y ata la de pago a su reputación. Postmark limita la cuenta en modo de prueba a 100 correos y a dominios verificados, y con la cuenta pendiente de aprobación rechazó con el código 412 cada mensaje a otro dominio. Resend sube su valor por defecto de 10 peticiones por segundo a los remitentes de confianza, sin decir cuánto ni qué umbral los define. Mailgun eleva el caudal a medida que crece la reputación de envío, sin una sola cifra. Ninguna de las seis publica el umbral que dispara el cambio de estado.

Las dos en contra son SendGrid y Brevo. En SendGrid el único estado que cambia la capacidad es el fin de la prueba gratuita, que es la caducidad de un plan y no un estado de la cuenta. En Brevo lo que cambia el caudal es la categoría del acuerdo comercial y no el estado de la cuenta, y para la cuenta sin aprobar no hay cifra. ACM2YAML v0.3 ya cubre parte de esos estados con sus alias de estado, nombres bajo un *endpoint* que agrupan los límites de cada estado, como el `sandbox` de Amazon SES

(Apartado 5.3.3).

Fuera de las seis afirmaciones, el catálogo muestra qué pasa con el envío que supera la cuota. Cinco de las ocho APIs cobran un sobrecoste por cada correo que pasa de la cuota del plan, y en las cinco su precio baja con el volumen. Los tramos de Postmark y Resend ya se recogieron en el Apartado 5.3.1, donde un mismo plan cambia de sobrecoste con el volumen contratado. En las otras tres, MailerSend va de 1,50 a 0,50 USD por cada mil correos y Mailgun de 1,80 a 0,40 USD [71]. SendGrid fija el sobrecoste por correo, y llevado a mil correos va de 1,30 a 0,50 USD.

Las tres APIs sin sobrecoste ponen algo distinto en su lugar. Brevo niega que haya cargos adicionales y sube de plan automáticamente cuando crece el número de contactos [70]. Amazon SES no tiene cuota incluida que exceder, y sus tramos de precio bajan con el volumen sin corte ni penalización [62]. Mailchimp Transactional detiene el envío al agotar el bloque, y lo que ocupa el lugar del sobrecoste es comprar otro.

En el borde de la cuota hay tres comportamientos documentados. Amazon SES rechaza la llamada que pasa de la cuota de 24 horas de la cuenta y descarta el mensaje sin ponerlo en cola ni reintentarlo. SendGrid en su prueba gratuita, MailerSend en el plan Free y Mailchimp Transactional al agotar los bloques detienen el envío. Brevo retiene en cola hasta 1.000 correos por encima de la cuota diaria y no entrega los que pasan de esa cifra. Los Apartados 5.3.4 y 5.3.6 recogen las dos propuestas que salen de este borde: el tope de diez veces el volumen del plan que SendGrid impone al sobrecoste y la cola con la que Brevo retiene los correos tras agotar la cuota.

Ninguna de las seis afirmaciones se cumple entera en las ocho APIs, y Mailchimp Transactional y Amazon SES quedan fuera de al menos dos de ellas.

6.4. Lo que responde PRIME4API

Dos de los cálculos de la Sección 6.1, la restricción activa y el tiempo mínimo $t_{\text{mín}}$, corresponden a operaciones de PRIME4API, el motor de análisis de la capacidad de Gavira Sánchez [3], que las resuelve para cada valor del factor k. Se han ejecutado contra el motor como servicio REST en local, sobre las ocho Datasheets, con la petición y la respuesta de cada pregunta archivadas en el repositorio de datos. Sus respuestas coinciden con el cálculo a mano salvo por tres convenciones de conteo, por los volúmenes contratados que la Datasheet no declara y por el caudal que toma en Mailgun. El motor no pasa a correos la cuota diaria en peticiones de MailerSend, y sobre la Datasheet de Mailchimp Transactional no responde a la pregunta del tiempo mínimo.

6.4.1. Correspondencia con el cálculo a mano

La restricción activa a consumo uniforme corresponde a la operación que comprueba si una demanda con *rate* y cuota puede sostenerse durante un período. El tiempo mínimo $t_{\text{mín}}$ de la Ecuación (6.1) corresponde a la que calcula cuánto tarda en agotarse una cuota, y es lo que Gavira Sánchez llama umbral de agotamiento [3].

El factor de equilibrio $k^* = Q/(R \cdot W)$ es el número de destinatarios por petición con el que el *rate* y la cuota se agotan a la vez. Una de las diez preguntas de análisis de Gavira Sánchez pide el CRF con el que dos cuotas de un mismo plan o *pricing* se agotan a la vez [3]. Aquí se aplica al par *rate* y cuota, tratando el *rate* como un límite con ventana de un segundo. El motor no calcula k^* con una operación propia: devuelve el tiempo de agotamiento de cada límite para cada k, y k^* se deriva de esos tiempos.

El factor de equilibrio es el inverso del cociente $R \cdot W/Q$ de las Tablas 6.1 y 6.2, así que se lee en ellas sin otro cálculo. Vale 0,0019 en Resend Pro, $1{,}9 \cdot 10^{-6}$ en SendGrid Essentials 50K, 0,00096 en MailerSend Hobby y 0,0023 en el entorno de pruebas de Amazon SES. Como k cuenta destinatarios y no baja de uno, un k^* menor que uno significa que la cuota se agota siempre antes que el *rate*. Las dos tablas tienen una sola fila con cociente menor que uno, MailerSend Starter 1M con la cuenta bajo revisión del proveedor, y su k^* es 2,31. Ningún plan publicado con la cuenta en estado sano tiene un $k \geq 1$ que iguale los dos límites.

Mailgun sí tiene un k^* mayor que uno, pero se da entre sus dos contadores de caudal y no entre caudal y cuota. Con los 2.000.000 de destinatarios y las 80.000 peticiones por ventana que devuelve su cabecera (Sección 6.2), el contador de destinatarios corta antes a partir de 25 destinatarios por petición. En Mailchimp Transactional la operación no se puede plantear, porque el caudal ya está en correos y k no multiplica nada.

El motor reproduce el cálculo a mano en Resend y SendGrid, con tres convenciones propias. Cuenta la primera ráfaga en $t = 0$, y por eso da 4.999 segundos en Resend Pro donde a mano salen 5.000. Abre la ventana siguiente en el instante final de un intervalo, y en $t = 30$ días suma ya la primera ráfaga de esa ventana. Aplica a cada ventana mensual, y no solo a la primera, la cuota ampliada con el sobrecoste que paga el presupuesto.

Aparte de esas convenciones, el motor solo ve el primer volumen contratado de cada plan, porque la Datasheet no declara más. Con 50 USD al mes, en Resend Pro y SendGrid Essentials el motor parte del volumen de 50.000 correos y da 83.343 y 83.115 correos, cifras correctas para lo que la Datasheet declara. Los dos proveedores venden un segundo volumen contratado de 100.000 correos por 35 y 34,95 USD, que la Datasheet solo recoge en el registro de procedencia. Con ese segundo volumen el mismo presupuesto daría 116.000 y 116.722 correos, de modo que el motor se queda entre un 28 y un 29 % por debajo por un dato que la Datasheet no declara.

En Brevo y Amazon SES las respuestas del motor solo difieren del cálculo a mano por las tres convenciones. En Mailgun el motor aplica a la vez la cifra por IP que la Datasheet declara en el plan, 1.500 peticiones por segundo, y el contador de la cabecera del envío, 80.000 peticiones por ventana, y calcula con la primera, más baja. La Tabla 6.2 usa solo el contador, dividido entre diez segundos. En Mailchimp Transactional no responde a la pregunta del tiempo mínimo para enviar 100.000 correos.

El motor aplica una regla de herencia: suma los límites del plan a los del *endpoint*. Si la cuota y el *rate* de SendGrid Essentials 50K se declaran en el plan y otra vez en el *endpoint*, con $k = 1.000$ el motor da 100.000 correos a 30 días donde el proveedor publica 50.000. Por eso las Datasheets del catálogo declaran cada límite una sola vez,

en el plan si vale para todos los *endpoints* y en el *endpoint* si es propio. El validador de formato de este trabajo, el que comprueba los tipos y las referencias de cada Datasheet, falla cuando un límite se repite. La Figura 6.1 es la que devuelve el motor sobre las Datasheets con esa regla aplicada.

6.4.2. «No publicado» frente a «sin límite» desde el motor

Antes de adoptar las 5.000 peticiones por segundo de la cabecera, la Datasheet de Postmark tenía vacío el bloque de los *rates*, `max_power`, y muestra qué hace el motor en ese caso. Con el bloque vacío, el motor responde 0 segundos para agotar la cuota de 10.000 correos del plan Basic. Para 100.000 correos responde 270 días, que son nueve renovaciones de la cuota mensual, como si el *rate* fuese infinito. Con el caudal de 5.000 peticiones por segundo medido en la cabecera de respuesta, el motor responde 1 segundo para agotar la cuota (2 a mano, por la convención de la primera ráfaga). Para los 100.000 correos responde 270 días y 1 segundo.

Con un presupuesto de 10^9 USD, la capacidad a un mes pasa de $1{,}1 \cdot 10^{12}$ correos, que solo acota ese presupuesto porque ningún caudal frena el consumo, a 12.960.005.000. Esa segunda capacidad es $R \cdot W$ más la primera ráfaga de la ventana siguiente. Ante un pico de 20.000 correos en una hora, el veredicto es negativo en los cuatro planes, tanto con el bloque vacío como con el caudal declarado. Lo que cambia es la causa que el motor declara: `rate_exceeded` con el bloque vacío, aunque lo que se viola es la cuota, y `quota_exceeded` con el caudal declarado.

Tratar como infinito el caudal que el esquema deja vacío es la confusión entre «no publicado» y «sin límite» que la propuesta P-04 quiere impedir con una marca explícita de valor no publicado junto al campo (Apartado 5.4.2).

En Amazon SES el plan no cambia la respuesta del motor. Los cuatro planes devuelven 200 correos al día, una petición por segundo, 1.401 correos en siete días y 6.001 al mes, todo tomado del estado `sandbox`, el entorno de pruebas. Ante el mismo pico de 20.000 correos en una hora, el motor responde que no en los cuatro planes con la causa `rate_exceeded`. Aquí la causa es correcta, porque una petición por segundo no sostiene los 5,56 correos por segundo que pide la demanda, y la violación se produce desde el instante cero.

6.4.3. Los dos regímenes del sobrecoste

La pregunta del tiempo mínimo para enviar 100.000 correos tiene dos respuestas en el motor, según reciba o no un presupuesto. Sin el parámetro `max_budget`, el motor trata la cuota con sobrecoste como un corte estricto, que no se puede superar pagando: cuenta renovaciones y mide el tiempo en meses. En MailerSend Hobby, 100.000 correos tardan así 570 días y 41 minutos, que son diecinueve renovaciones de la cuota de 5.000 correos. Con un presupuesto que no limita, el motor expande la cuota con el sobrecoste que publica el proveedor, y el tiempo lo fija el caudal: 13 horas y 53 minutos. Esa cifra no aplica la cuota diaria de 1.000 peticiones del plan, que el motor no pasa a correos (Apartado 6.4.4): con ella, los 100.000 correos con $k = 1$ tardan 99 días y 8 minutos.

Los dos tiempos salen de datos del proveedor, y ninguno es una hipótesis de este trabajo. El segundo tampoco exige elegir un presupuesto, porque a partir de lo que cuestan los 100.000 correos la respuesta deja de cambiar. En Hobby, 100.000 correos cuestan 149,50 USD: con 149,40 USD el motor sigue respondiendo en días, y con 149,50 USD responde 13 horas y 53 minutos, igual que con mil millones.

La Tabla 6.3 pone los dos regímenes al lado en siete APIs. En Mailgun Basic el presupuesto baja el tiempo de 100.000 correos de 270 días a 1 minuto y 6 segundos. Ese tiempo sale del caudal de 1.500 peticiones por segundo por IP que declara la Datasheet, y no del contador de 80.000 peticiones por ventana de unos diez segundos. En Brevo Starter y en Amazon SES la misma llamada no mueve un segundo, porque sus cuotas no declaran sobrecoste. La distancia entre las dos columnas llega a seis órdenes de magnitud en Postmark Basic con $k = 1$, de 270 días a 19 segundos.

API y plan	k	**Corte estricto**	**Presupuesto que no limita**
Postmark Basic	1	270 d 1 s	19 s
	50	270 d	0 s
Resend Pro	1	30 d 1 h 23 min 19 s	2 h 46 min 39 s
	50	30 d 1 min 39 s	3 min 19 s
SendGrid Essentials 50K	1	30 d 4 s	9 s
	1.000	30 d	0 s
MailerSend Hobby	1	570 d 41 min	13 h 53 min
	70	570 d	11 min
Brevo Starter	1	570 d 4 s	570 d 4 s
	2.000	570 d	570 d
SES Pro (`sandbox`)	1	499 d 3 min 19 s	499 d 3 min 19 s
	50	499 d 3 s	499 d 3 s
Mailgun Basic	1	270 d 6 s	1 min 6 s
	1.000	270 d	0 s

Tabla 6.3: Tiempo mínimo para enviar 100.000 correos en un plan de pago de cada API, respondido por PRIME4API [19] en los dos regímenes. Sin `max_budget`, la cuota con sobrecoste se trata como corte estricto y el motor cuenta renovaciones. Con un presupuesto que no limita, el motor la expande y manda el caudal. Brevo y SES son los dos casos sin sobrecoste declarado, y por eso las dos columnas coinciden. Las filas de SES son las del estado `sandbox`, el único que el proveedor cifra. Mailchimp Transactional no aparece porque el motor no responde a esta pregunta sobre su Datasheet. La fila de Mailgun usa el caudal de 1.500 peticiones por segundo por IP. La fila de MailerSend Hobby no aplica la cuota diaria de 1.000 peticiones del plan (Apartado 6.4.4).

Nada en la Datasheet dice cuál de los dos regímenes describe el plan. La cuota lleva su sobrecoste, pero no declara si el proveedor lo cobra sin más, si exige aprobarlo o si lo limita un tope. El tope de sobrecoste de SendGrid, diez veces el volumen del plan (500.000 correos en Essentials 50K), tendría que entrar en la operación que calcula el importe del sobrecoste, pero la Datasheet no tiene dónde declararlo. La propuesta P-07, que añade a la cuota un campo para ese tope, lo permite (Apartado 5.3.4). El modo del sobrecoste no tiene campo: es la carencia F-13 de la Tabla 5.3, vista ahora desde el motor, y sigue entre las carencias sin forma fijada.

6.4.4. La cuota que no cambia de unidad

MailerSend declara en su plan Hobby dos cuotas en unidades distintas: 5.000 correos al mes y 1.000 peticiones al día. Su *endpoint* de envío declara un `workload` de hasta 70 destinatarios por petición, y ese factor k es el que el motor usa para pasar de una unidad a otra. Pedida la curva en correos, la respuesta trae el caudal y la cuota mensual, y no la diaria. Pedida en peticiones, trae los tres límites: el caudal y la cuota diaria tal cual, y la mensual dividida por k. De las tres conversiones que el factor permite, el motor hace dos. Multiplica por k el caudal de peticiones para darlo en correos, y divide por k la cuota mensual de correos para darla en peticiones. La cuota diaria en peticiones no la lleva a correos.

Que el motor no pase la cuota diaria a correos cambia la respuesta. Con $k = 1$, el motor responde que los 5.000 correos de Hobby se agotan en 41 minutos, mientras que con la cuota diaria de 1.000 peticiones tardarían 4 días y 8 minutos. La capacidad a un mes con presupuesto sin tope sale de 5.184.120 correos, y con la cuota diaria sería de 30.000. Con 50 USD de presupuesto el motor devuelve 33.786,67 correos, sin redondear, y con la cuota diaria también saldrían 30.000. Quien pregunta en correos, que es la unidad del dominio, recibe la cifra optimista. La Figura 6.2 es la misma llamada pedida en peticiones, y en ella la cuota diaria sí aparece: la curva sube 1.000 cada medianoche durante cinco días.

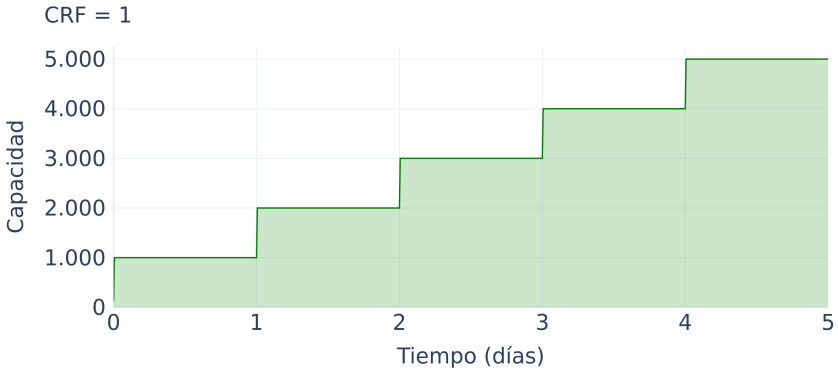


Figura 6.2: Curva de capacidad de MailerSend Hobby con $k = 1$ sobre cinco días, pedida a PRIME4API [19] en peticiones. Es la misma llamada que la gráfica (c) de la Figura 6.1, con `capacity_unit` en peticiones. La cuota diaria de 1.000 peticiones del plan sí entra, y la curva sube 1.000 cada medianoche UTC hasta los 5.000 de la cuota mensual el quinto día. Con $k = 1$ cada petición lleva un correo, así que la curva se lee también en correos. El tramo de *rate* (120 peticiones por minuto durante ocho minutos) no se distingue a esta escala.

El renderizador del motor oculta las curvas en peticiones. Si se le pide la gráfica de MailerSend Hobby sin filtrar la unidad, devuelve cuatro figuras y las cuatro son de correos. La curva en peticiones, la única que lleva la cuota diaria de Hobby, solo se dibuja con el parámetro `show_derived_requests`, cuyo valor por defecto es falso. El *endpoint* de datos, en cambio, devuelve las dos unidades sin que se le pida, así que la ausencia está solo en el dibujo.

La conversión que falta corresponde al motor. La Datasheet de MailerSend declara los tres límites con su unidad y el factor k que haría falta, que el motor usa en las otras dos conversiones. MailerSend es la única de las ocho APIs que declara cuotas en dos unidades, y por eso el problema solo aparece en ella. El defecto no cambia el plan más barato en ninguna de las tres recomendaciones de presupuesto del contraste, para 12.000, 25.000 y 40.000 correos. Queda una pregunta que el catálogo no convierte en propuesta: una Datasheet con límites en dos unidades no declara en cuál mide el plan el proveedor, y quien la consulta elige la unidad al preguntar.

6.4.5. Capacidad con un presupuesto menor que el precio del plan

La operación `/capacity-at` calcula cuántos correos caben hasta una fecha con un presupuesto dado. Con un presupuesto menor que el precio del plan, devuelve la cuota base más la primera ráfaga de la ventana siguiente. En MailerSend Professional con 50 USD responde 50.120 correos y no marca el plan como no asequible, a diferencia de `/budget/recommendation`, que recomienda planes según el presupuesto y sí lo marca. SendGrid Pro, que cuesta 89,95 USD al mes, responde 110.000 correos con el mismo presupuesto de 50 USD.

Brevo Professional cuesta 499 USD al mes, y con 50 USD el motor devuelve 151.000 correos con la categoría de caudal General y 152.000 con la Advanced. Son los 150.000 de la cuota más lo que cabe en el primer segundo de la ventana siguiente, así que la categoría de caudal no cambia el volumen del mes. En los tres casos el motor no amplía la cuota con sobrecoste, porque el presupuesto no cubre el precio del plan, pero devuelve igualmente la cuota base y no avisa de que el plan no es asequible. Quien consulta `/capacity-at` con un presupuesto menor que el precio del plan tiene que cruzar la respuesta con `/budget/recommendation` para saber si puede pagar el plan.

El contraste cubre las ocho Datasheets del catálogo. Una API con cuotas en tres unidades o con un sobrecoste que exija aprobación previa podría sacar a la luz convenciones del motor que estas ocho no activan.

6.5. Respuesta a la RQ3

La RQ3 pregunta cómo se relacionan los límites de uso y los *rates* en las APIs de correo, y qué implicaciones tiene esa relación para el consumidor.

En seis de las ocho APIs (Postmark, Resend, SendGrid, MailerSend, Brevo y Mailgun) la cuota es la restricción activa, el límite que se alcanza primero y frena el envío (Sección 6.1). En su ventana, el *rate* admite entre 103,7 veces la cuota (MailerSend Starter) y 129.600.000 veces (el plan Free de Postmark, cuyo *rate* no está publicado y sale de la cabecera que devuelve la API). La condición es que el *rate* y la cuota tengan valor en la Datasheet, publicado o medido, y que la cuenta no esté en revisión, en pruebas ni restringida por reputación. Con esa condición, quien contrate una de las seis puede dimensionar el volumen sin consultar el *rate*.

El cociente $R \cdot W/Q$ baja a 2,59 en MailerSend Professional 2M, un volumen contratado que la Datasheet no recoge, y a 0,43 en Starter 1M con la cuenta en revisión, el único caso de las dos tablas con cociente calculable en que manda el *rate*. En Amazon SES y Mailchimp Transactional no se puede decir que mande la cuota. Amazon SES no publica la cuota ni el caudal de producción, así que la relación no se puede calcular antes de sacar la cuenta del entorno de pruebas. En Mailchimp Transactional la restricción activa es el caudal: la cuenta de prueba tiene una franquicia de 500 correos y un tope de 25 correos por hora, de modo que agotarla lleva 20 horas.

Las implicaciones para quien contrata salen de tres casos en que la cuota no basta para dimensionar el envío. El primero es el estado de la cuenta, que condiciona la capacidad en seis de las ocho APIs sin que ninguna publique el umbral que dispara el cambio. Solo MailerSend da una cifra para cada estado, 120 peticiones por minuto con la cuenta sana frente a 10 en revisión.

El segundo caso es la distancia entre el *rate* publicado y el observado. Solo coinciden en cuatro APIs, las que escriben a qué se aplica el valor. Resend lo fija para cada equipo de la cuenta, MailerSend para cada estado de la cuenta, Brevo para cada *endpoint* en su categoría General y Mailchimp Transactional para la cuenta de prueba. En Mailgun, en cambio, no coinciden las 1.500 peticiones por segundo y por IP de sus preguntas frecuentes y el contador de la cabecera del envío, 80.000 peticiones por ventana de unos diez segundos.

Amazon SES y Mailchimp Transactional no emiten ninguna cabecera de caudal, y Mailgun solo la devuelve tras un envío aceptado. Para comprobar antes de enviar cuánta capacidad queda hace falta un *endpoint* que la informe, y solo la mitad del catálogo lo tiene. Amazon SES, Mailchimp Transactional y MailerSend devuelven el límite, y Brevo solo el saldo de correos.

El tercer caso es el factor k, los destinatarios por petición, que pasa el *rate* de peticiones a correos. Solo vale si cada destinatario descuenta un correo de la cuota, y eso solo consta en Resend, SendGrid, Brevo y Amazon SES. En Postmark, MailerSend, Mailgun y Mailchimp Transactional el paso de peticiones a correos es una suposición.

El contraste con PRIME4API de la Tabla 6.3, que da el tiempo mínimo para enviar 100.000 correos con y sin presupuesto, añade dos implicaciones más. La primera es que la Datasheet no dice cuál de los dos regímenes aplica el proveedor: en Postmark Basic, la respuesta es de 270 días si la cuota es un corte estricto y de 19 segundos si se paga el sobrecoste. La segunda es que el motor lee como infinito el caudal que la Datasheet deja vacío: con ese bloque vacío, agotaba en 0 segundos los 10.000 correos de Postmark Basic, cuyo *rate* el proveedor no publica.

En el borde de la cuota hay tres comportamientos, que recoge la Sección 6.3 y no el contraste con el motor. Amazon SES rechaza la llamada y descarta el mensaje, y SendGrid, MailerSend y Mailchimp Transactional detienen el envío donde no hay sobrecoste. Brevo retiene en cola hasta 1.000 correos por encima de la cuota diaria y no entrega los que pasan de esa cifra, que es lo que modela la propuesta P-09 del Apartado 5.3.6.

Esta respuesta descansa sobre una base estrecha. El par formado por el *rate* y la cuota solo tiene valor en la Datasheet de seis APIs. En cuatro de ellas el proveedor

publica el *rate* del envío, y en Postmark y Mailgun sale de una cabecera medida: en Mailgun, 80.000 peticiones por una ventana de unos diez segundos, que la Tabla 6.2 divide entre diez. La cifra que Mailgun publica, 1.500 peticiones por segundo y por IP, tiene otro ámbito. De los cuatro *rates* publicados, el de SendGrid no se liga a ningún plan y el de Resend es el valor por defecto del equipo. El de MailerSend solo vale para una cuenta aprobada, y la página de Brevo se contradice sobre qué planes incluyen cada una de sus tres categorías de caudal.

Los valores medidos tampoco describen el plan contratado: en MailerSend, Amazon SES y Mailchimp Transactional corresponden a cuentas sin aprobar o en pruebas, y en Postmark y Mailgun a una sola cuenta gratuita. La relación entre cuota y *rate* queda establecida en seis APIs, pero el margen exacto entre ambos depende de valores que los proveedores no fijan por escrito.

7. Comparación sistemática

Este capítulo responde a la RQ4: cómo pueden compararse las APIs de correo de forma sistemática para responder a preguntas de decisión que involucran simultáneamente requisitos funcionales y no funcionales. La pregunta del ejemplo de la Sección 1.1.3 es una de ellas. Una aplicación necesita enviar 100.000 correos al mes, con ráfagas de 1.000 peticiones por minuto, *webhooks* de eventos y envío en lote, y la pregunta es qué API y qué plan lo cubren al menor coste. Para responderla hay que cruzar cuatro datos en cada una de las ocho APIs. El precio con la cuota del plan, el caudal del *endpoint* de envío y el tamaño máximo del lote están en la Datasheet, pero los *webhooks* no, porque ACM2YAML v0.3 no modela funcionalidades.

El método que responde a preguntas como esta es la contribución C-04 (Sección 1.4): un catálogo de preguntas de decisión propias del dominio y un procedimiento que las responde sobre las Datasheets. Las preguntas de capacidad se calculan con PRIME4API, el motor de análisis de la capacidad de Gavira Sánchez [3], y se contrastan con un cálculo a mano. Las funcionales se leen en la matriz de *features*, la tabla de funcionalidades por plan que acompaña a cada Datasheet.

El capítulo se organiza del siguiente modo. En primer lugar, la Sección 7.1 presenta las 32 preguntas del catálogo y la clase que dice con qué se responde cada una. A continuación, la Sección 7.2 aplica el procedimiento a las ocho Datasheets y mide la cobertura, y la Sección 7.3 trata las cuatro preguntas que piden HARVEY4API. Después, la Sección 7.4 responde con el catálogo al caso de los 100.000 correos, y la Sección 7.5 compara las funcionalidades de cada plan. Por último, la Sección 7.6 responde a la RQ4.

7.1. Catálogo de preguntas de decisión

Una plantilla de pregunta es una pregunta con huecos, como la API, el plan, el volumen o el precio, que se rellenan para cada caso. El catálogo tiene 32 plantillas, y 25 de ellas (el 78 %) se pueden responder con el instrumental del canal API: la Datasheet, PRIME4API y HARVEY4API. De esas 25, 11 se responden leyendo la Datasheet, 10 con una operación de PRIME4API y 4 con HARVEY4API. Otras 2 se responden leyendo una matriz de *features* por plan, que se captura con las mismas fuentes que la Datasheet pero queda fuera del esquema. Las 5 restantes no tienen respuesta, y cada una deja escrito qué falta para tenerla.

La proporción de preguntas respondibles es la cobertura, la misma métrica con la que Cavero et al., que definen el Pricing Intelligence como disciplina, miden el alcance de sus herramientas: un 54 % sobre un banco de 56 plantillas [1].

Las dos cifras no se miden igual. Cavero et al. prueban herramientas que ya existían contra un banco que no se hizo para ellas, y aquí la clase de cada pregunta, el instrumento con el que se responde, se asigna conociendo el instrumental. La categoría

B, la de selección de suscripción, parte de las operaciones del motor, y las cuatro preguntas que el catálogo asigna a HARVEY4API cuentan por la especificación de la herramienta, que no se ha ejecutado. Sin esas cuatro, la cobertura con un instrumento leído o ejecutado es de 21 sobre 32, el 66 %.

Del trabajo de Cavero et al. se toma también la gramática de las plantillas, con siete *placeholders* (los huecos): la API (`SAAS`), el plan con sus complementos (`CONFIGURATION`), una funcionalidad (`FEATURE`), una cantidad como un volumen de correos o una ventana de tiempo (`USAGE LIMIT`), la necesidad del usuario (`ACTION`), un precio (`PRICE`) y el año de una versión del *pricing* (`YEAR`). Ninguna plantilla nombra una API, porque el proveedor solo aparece al rellenar los huecos, en el ejemplo de cada pregunta, y 13 de las 32 hablan de correo, como los envíos al mes o los límites que crecen con la reputación del remitente. Con esa gramática, cada pregunta del catálogo puede incorporarse al banco de plantillas del grupo ISA de la Universidad de Sevilla.

Cada pregunta tiene una plantilla, un ejemplo instanciado sobre una de las ocho APIs y una clase, que dice con qué se responde. Instanciar es rellenar los huecos: con SendGrid y 100.000, la plantilla de PI-B01, «suscripción más barata de `SAAS` para `USAGE LIMIT` correos al mes», da «la suscripción más barata de SendGrid para 100.000 correos al mes», una pregunta del caso de los 100.000 correos. Los ejemplos instanciados están en `03_datos/preguntas/catalogo.yaml`, en el repositorio del trabajo.

La clase D indica que basta leer la Datasheet, incluido su *provenance*, las notas en las que la Datasheet guarda lo que los campos de ACM2YAML v0.3 no admiten. Es el caso de los volúmenes contratados del plan Basic de Postmark, que se vende en seis volúmenes, cada uno con su precio.

La clase P indica que PRIME4API resuelve la pregunta con la operación que responde una de las diez preguntas de análisis de Gavira Sánchez [3], y la tabla anota cuál. El catálogo emplea siete de esas operaciones: el tiempo mínimo, la capacidad en una ventana, la suscripción más barata, la suscripción con presupuesto, el importe del sobrecoste, la demanda sostenible y las cuotas que se agotan a la vez. Su correspondencia con el cálculo a mano de la restricción activa (el límite que se alcanza primero y frena el envío), el tiempo mínimo y el factor k (los destinatarios por petición) está en la Sección 6.4.

Las otras tres clases son H, L y X. La clase H pide combinar varias Datasheets o varias operaciones, que es la tarea de HARVEY4API [4]. La clase L se responde leyendo la matriz de *features*, sin instrumento. La clase X marca una pregunta sin respuesta y dice qué falta: un campo que v0.3 no serializa, una operación que el motor no tiene o un comportamiento que ninguna dimensión de capacidad del ACM recoge. Una dimensión de capacidad es un aspecto del consumo que el proveedor limita por separado, como el volumen al mes o las peticiones por segundo. La clase depende del instrumento y no de lo que publique cada proveedor, así que el reparto es el mismo en las ocho APIs: 25 preguntas respondibles con la Datasheet, PRIME4API o HARVEY4API, 2 que se responden leyendo la matriz y 5 sin respuesta.

Las 32 plantillas se agrupan en siete categorías, de la A a la G, según la fuente de la que salen. La A recorre los bloques de capacidad de la Datasheet (cuota, caudal y sobrecoste) y la B parte de las operaciones de Gavira Sánchez sobre un plan. La C

compara APIs entre sí y la D convierte en preguntas la RQ3, la relación entre límites de uso y *rates*. Un límite de uso es el máximo de un nivel de uso medible, como la cuota de correos al mes. La E, la F y la G salen de los elementos no soportados que se registraron al construir las Datasheets, los hechos del dominio que ACM2YAML v0.3 no puede expresar. Recogen las necesidades funcionales, la vigencia de los precios y el riesgo de operar.

El catálogo se cerró en 32 preguntas sin un criterio de saturación fijado de antemano, y al pasar de tres APIs a ocho no ganó ninguna. Tampoco cubre los cinco retos abiertos que enumeran Cavero et al. [1], entre ellos la predicción de la evolución de un *pricing* y los análisis *what-if*.

Las tablas 7.1, 7.2 y 7.3 recogen las 32 preguntas con su plantilla abreviada, su clase, la operación o composición que la resuelve y lo que falta para responder las X o cómo se leen las L.

Id	Pregunta (plantilla abreviada)	Clase y operación	Qué falta (X) o cómo se lee (L)
A. Lectura de la capacidad contractual de una API (8)			
PI-A01	Correos al mes que incluye `CONFIGURATION` de `SAAS`, y qué pasa al superarlos	D	
PI-A02	*Rate* máximo con el que `SAAS` acepta peticiones de `ACTION` en `CONFIGURATION`	D	
PI-A03	Destinatarios por petición de `ACTION` en `SAAS`, y si cuentan uno a uno contra la cuota	D	
PI-A04	Si hay límite diario además del mensual en `CONFIGURATION` de `SAAS`	D	
PI-A05	Tamaño máximo de un mensaje (adjuntos incluidos) en `SAAS`	D	
PI-A06	Tiempo mínimo que necesita `CONFIGURATION` de `SAAS` para enviar `USAGE LIMIT` correos	P, tiempo mínimo	
PI-A07	Capacidad que acumula `CONFIGURATION` de `SAAS` en una ventana de `USAGE LIMIT`	P, capacidad en una ventana	
PI-A08	Peticiones simultáneas que admite `SAAS` en `CONFIGURATION`	X	Concurrencia: v0.3 no la serializa, y de las ocho APIs solo Mailgun publica algo sobre ella para la API REST: que no limita las peticiones simultáneas
B. Selección de suscripción para una necesidad (5)			
PI-B01	Suscripción más barata de `SAAS` para `USAGE LIMIT` correos al mes	P, suscripción más barata	
PI-B02	Con `PRICE` al mes, máximo de correos con `SAAS`	P, suscripción con presupuesto	
PI-B03	Con `USAGE LIMIT` al mes, si sale más barato el sobrecoste o el siguiente plan de `SAAS`	P, importe del sobrecoste	
PI-B04	Si se puede empezar gratis en `SAAS` y qué limita el plan gratuito para `ACTION`	D	
PI-B05	Plan de `SAAS` que sostiene un pico de `USAGE LIMIT` correos en `USAGE LIMIT 2`	P, demanda sostenible	

Tabla 7.1: Catálogo de preguntas de decisión, categorías A y B. Clase: D Datasheet, P PRIME4API, H HARVEY4API, L lectura de la matriz de *features*, X no respondible.

Las cinco preguntas X no se descartan: cada una deja anotado en la última columna de las tablas lo que le falta al instrumental. PI-E03, la retención de los registros de envío, pide un límite que no se consume con cada envío, y PI-D06, el comportamiento al superar el *rate*, algo que ninguna dimensión de capacidad del ACM recoge. Una pregunta X solo pasa a respondible cuando el esquema o el motor ganan lo que le falta, y por eso la cobertura del 78 % se mide sobre el instrumental actual.

Id	Pregunta (plantilla abreviada)	Clase y operación	Qué falta (X) o cómo se lee (L)
C. Comparación entre APIs (5)			
PI-C01	Para `USAGE LIMIT` correos al mes, cuál sale más barata, `SAAS 1` o `SAAS 2`	H, suscripción más barata por API, más comparación	
PI-C02	Cuál de `SAAS 1`, `SAAS 2` y `SAAS 3` da más capacidad por dólar en el primer plan de pago	H, lectura por API, más comparación	
PI-C03	Qué API de correo publica su *rate* y cuál no	D	
PI-C04	Cuál envía antes `USAGE LIMIT` correos, `SAAS 1` o `SAAS 2`	H, tiempo mínimo por API, más comparación	
PI-C05	En qué se parecen y en qué se diferencian los pricings de `SAAS 1` y `SAAS 2` para `ACTION`	X	Una operación de *diff* entre pricings que el motor no tiene
D. Relación entre límites de uso y *rates* (RQ3) (6)			
PI-D01	En `CONFIGURATION` de `SAAS`, qué se agota antes, la cuota mensual o el *rate*	P, demanda sostenible	
PI-D02	A partir de qué volumen mensual el *rate* de `SAAS` deja de ser el límite efectivo	P, demanda sostenible	
PI-D03	Si `SAAS` cobra por correo y limita por petición, cuántos destinatarios por petición igualan los dos límites	P, cuotas que se agotan a la vez	
PI-D04	Si el *rate* de `SAAS` sube al subir de plan o es el mismo en todos	D	
PI-D05	Si los límites de `SAAS` crecen con el uso (reputación) o son fijos por plan	D	
PI-D06	Qué ocurre en `SAAS` al superar el *rate*: rechazo, cola o degradación	X	El comportamiento al exceder no es una dimensión del ACM

Tabla 7.2: Catálogo de preguntas de decisión, categorías C y D. Misma leyenda que la Tabla 7.1.

Solo una de las cinco tiene una forma esbozada en el Capítulo 5: PI-F02, la vigencia de la Datasheet. Se respondería con una fecha de verificación en cada afirmación del registro de procedencia, la forma que la Tabla 5.3 esboza para F-09, la caducidad de los datos. P-09 (Apartado 5.3.6) declara qué hace una cuota al superarse, la cola de reintento de Brevo, pero no cubre el exceso sobre el *rate* que pide PI-D06. La concurrencia de PI-A08, el *diff* entre pricings de PI-C05 y la retención de PI-E03 no tienen extensión propuesta, y quedan como límite del esquema y del motor.

7.2. Procedimiento y cobertura

El catálogo se aplica a las ocho Datasheets en tres pasos. Primero, cada una de las 32 preguntas se instancia sobre cada Datasheet con los hechos que esta registra, cada dato que se recogió con su fuente al construirla, esté o no en un campo. Después, las preguntas de clase P, las que resuelve una operación de PRIME4API, se calculan a mano con la definición de la operación y se contrastan con la respuesta del motor. Por último, cada pareja de pregunta y API recibe una de cuatro salidas.

La primera salida es la respuesta con su cifra y con la fuente que la sostiene. La segunda es la respuesta parcial, cuando solo sale una parte de la pregunta, y la tercera es «no», con el dato que falta. La cuarta, «fuera de los campos», solo existe en la clase X, la de las preguntas que el instrumental no responde. Se anota cuando los hechos de

Id	Pregunta (plantilla abreviada)	Clase y operación	Qué falta (X) o cómo se lee (L)
E. Necesidades funcionales (3)			
PI-E01	Si `CONFIGURATION` de `SAAS` incluye `FEATURE`	L	Por lectura de la matriz de *features*. Con instrumento exigiría el iPricing del canal web, fuera del alcance fijado en el Capítulo 4
PI-E02	Cuál de `SAAS 1` y `SAAS 2` da `FEATURE` al menor precio para `USAGE LIMIT` correos al mes	L	Por lectura: *feature* en la matriz, capacidad en la Datasheet, composición a mano. Con instrumento faltaría el vínculo entre canales
PI-E03	Cuánto tiempo conserva `SAAS` los registros de envío en `CONFIGURATION`	X	Un límite de uso que no se consume (retención): v0.3 no lo modela
F. Evolución y vigencia (2)			
PI-F01	Si el pricing de `SAAS` ha cambiado desde `YEAR` en cuota o en *rate*	D	
PI-F02	Qué parte de la Datasheet de `SAAS` sigue vigente	X	Una Datasheet no dice qué parte sigue vigente (F-09, la caducidad de los datos): exige reverificar fuente a fuente
G. Operación y riesgo (3)			
PI-G01	Si el tráfico crece un `USAGE LIMIT` % al mes, cuándo habrá que cambiar de plan en `SAAS`	H, suscripción más barata iterada	
PI-G02	Coste de un mes de `USAGE LIMIT` correos en `SAAS` sin cambiar de plan	P, importe del sobrecoste	
PI-G03	Si se puede contratar más volumen en `SAAS` sin cambiar de plan, y a qué precio	D	

Tabla 7.3: Catálogo de preguntas de decisión, categorías E a G. Misma leyenda que la Tabla 7.1.

la API contienen el dato aunque ningún campo de la Datasheet lo recoja, por ejemplo en el *provenance* o en la matriz de *features*.

El recuento acepta «no publicado» como respuesta cuando la pregunta lo admite. Así se responde PI-A02, la velocidad máxima de envío, en Postmark. La respuesta es que el proveedor no publica el *rate*, junto con las 5.000 peticiones por segundo que la Datasheet toma de la cabecera `ratelimit-limit` leída en un sondeo, una prueba propia contra la API. Ese valor está en ámbar (●), el color que el semáforo de la Datasheet da a un valor interpretado. En las preguntas de comparación de la categoría C, una API cuenta como respondida cuando aporta su lado de la comparación.

Un mes se toma como 30 días (2.592.000 segundos). Cada respuesta vale a la fecha de su Datasheet, dentro de la ventana de toma de datos del 2 de agosto al 21 de septiembre de 2026 (Sección 4.1.3). Entre capturas sucesivas de una misma página cambió la redacción, pero ningún valor que la Datasheet modele.

Los términos de consumo de una API (el precio y la información de cómo, cuándo, cuánto y dónde se consume) están repartidos entre el panel de la cuenta y varias páginas del proveedor, como la de precios, la documentación de la API, el centro de ayuda o el blog. Cuando la página de precios y otra de esas fuentes no coinciden, la comparación usa el valor de la página de precios, que es el que modela la Datasheet: las demás fuentes señalan la incoherencia, pero no corrigen el dato (Sección 4.3.1).

El catálogo de Datasheets registra 82 divergencias entre fuentes, cada una un dato que dos fuentes dan distinto, y en 37 la página de precios es uno de los dos lados. En siete de esas 37 la página se contradice a sí misma, y la regla de la página de precios no ordena nada. En once de las 30 restantes, esa regla cambió la fuente que ganaba antes de adoptarla o cerró una divergencia que seguía abierta. Solo dos de esas once cambiaron la Datasheet: un valor, el sobrecoste de Essentials 50K en SendGrid, y un color del semáforo, el de la cuota diaria del plan gratuito de Mailgun. Las otras nueve solo cambian la procedencia anotada, sin tocar ningún campo.

Algunas preguntas presuponen un plan con su cuota, y no todos los proveedores lo tienen. PI-B02 pregunta cuántos correos compra un presupuesto mensual en un plan, y Mailchimp Transactional no tiene planes. En esos casos la pregunta se responde sobre lo que la Datasheet sí declara, como una cuenta de prueba o una tarifa sin suscripción, y la celda se rotula entre corchetes con el nombre que le da el proveedor. En Mailchimp es `[demo]`, como llama el proveedor a su cuenta de prueba. En Amazon SES es `[sandbox]`, su entorno de pruebas y el único con cuota publicada, o `[à la carte]`, su tarifa sin suscripción, que cobra solo por correo enviado. El corchete no lleva semáforo, porque no mide la fiabilidad del dato.

Ejecución contra PRIME4API. Las preguntas P se ejecutaron en 128 llamadas al motor, todas con su petición y su respuesta archivadas[1]. Fueron 45 llamadas para Postmark, Resend y SendGrid, 15 para MailerSend y 68 para las 40 instancias de Brevo, Amazon SES, Mailgun y Mailchimp. En las cuatro primeras APIs el motor contestó 58 de las 60 llamadas, y las dos que fallan piden PI-B01 con todos los planes y tropiezan con el plan Enterprise de Resend y con el de SendGrid. En las otras cuatro contestó las 17 llamadas de Mailgun y las 17 de Amazon SES, 16 de 17 en Brevo y 3 de 17 en Mailchimp. De sus 40 instancias resuelve 30, las diez de Brevo, las diez de Amazon SES y las diez de Mailgun.

Las respuestas del motor coinciden con el cálculo a mano salvo por las tres convenciones de conteo de la Sección 6.4. Una de ellas abre la ventana siguiente en el instante final del intervalo. Por eso, en PI-B02, el motor da 3.100 correos en el plan gratuito de Mailgun, cien más que su cuota mensual: suma 30 ventanas diarias de 100 correos y la que abre en $t = 30$ días, que ya es del mes siguiente.

Entre esas otras cuatro, el motor devuelve errores en Brevo y en Mailchimp. La única llamada de Brevo que no nombra los planes alcanza Enterprise, cuyo precio es `custom`, y devuelve HTTP 400, el caso de un precio que existe y no es público que recoge P-05 (Sección 5.4.3). Las otras dieciséis nombran los planes y lo dejan fuera. En Mailchimp, catorce llamadas devuelven HTTP 500 por dos campos vacíos que el esquema admite: el tope de destinatarios y el periodo de la franquicia de 500 correos de la cuenta de prueba (Sección 5.4). Las tres que contesta solo leen su cuota o su caudal, y por eso sus diez respuestas de clase P están calculadas a mano, sin confirmación del motor, y la celda lo indica en cada caso.

[1] En `05_salida/_cap5/prime4api_ejecucion*/` del repositorio del trabajo.

Salida	Postmark	Resend	SendGrid	MailerSend	Brevo	SES	Mailgun	Mailchimp	Las ocho
Respondible con los hechos	18	22	20	21	20	21	20	20	16
de ellas, con un dato del *provenance* y no de un campo del esquema	4	3	2	1	5	6	1	2	1
Parcial	4	2	4	4	4	4	5	3	2
Respondible en el catálogo, sin respuesta en los hechos	3	1	1	0	1	0	0	2	0
L, respondida leyendo la matriz de *features* de la Datasheet	2	2	2	2	2	2	2	2	2
X sin respuesta en ningún sitio	2	2	2	2	2	3	1	2	1
X con respuesta en los hechos (fuera de los campos)	3	3	3	3	3	2	4	3	2
Total	32	32	32	32	32	32	32	32	—

Tabla 7.4: Cobertura del catálogo de preguntas sobre las ocho APIs. El catálogo declara 25 respondibles con una Datasheet, PRIME4API o HARVEY4API (clases D, P y H), 2 que se responden leyendo (L) y 5 que el instrumental no responde (X), y ese reparto no cambia de una API a otra. La columna «Las ocho» cuenta las preguntas con la misma salida en las ocho APIs y no suma 32. La lista de preguntas de cada celda está en `05_salida/_cap5/analisis.md` del repositorio del trabajo.

Cobertura. De las 25 preguntas de clase D, P y H, las que el catálogo declara respondibles con una Datasheet, PRIME4API o HARVEY4API, Resend responde 22 con los hechos de su Datasheet. Amazon SES y MailerSend responden 21, SendGrid, Mailgun, Mailchimp y Brevo 20 y Postmark 18. Dieciséis preguntas se responden en las ocho APIs a la vez. La Tabla 7.4 recoge el recuento completo por API.

De las dieciséis preguntas comunes, cinco piden cuota, precio o si el *rate* está publicado (PI-A02, B03, C02, C03 y G03), y siete necesitan el valor del *rate* (PI-A06, A07, B05, C04, D01, D02 y D03). Las cuatro restantes piden el precio de un volumen (PI-B01, B02, C01 y G02). En Brevo esas cuatro se responden con los volúmenes contratados de Starter, que van de 5.000 correos por 9 USD a 100.000 por 82 USD y constan en el *provenance*, no en un campo (P-01, Apartado 5.3.1). El selector de volumen de su página de precios solo deja sin precio la última posición, «More», por encima de 100.000 correos en Starter, de 500.000 en Standard y de diez millones en Professional.

Los cuatro parciales de Brevo salen de lo que el proveedor no publica. PI-A01, la cuota y lo que ocurre al superarla, queda en parcial porque sus tres planes de pago no lo documentan y la página de precios niega que exista precio de exceso [70]. Los otros tres salen de los tres tramos de caudal de su API, General, Advanced y Extended. Professional puede acogerse a dos y Enterprise a los tres, y ninguna fuente dice cuál se les aplica.

Un caso resuelto: PI-B02. PI-B02, cuántos correos se envían como máximo con 50 USD al mes, recorre el procedimiento completo. Resend llega a 116.000 correos con el segundo volumen de Pro (100.000 por 35 USD) más 16 bloques de 1.000 a 0,90 USD. SendGrid llega a 116.722 con Essentials 100K, de 34,95 USD, más 15,05 USD de

sobrecoste a 0,0009 USD por correo. MailerSend con Starter y Mailgun con Foundation llegan los dos a 61.538: 50.000 correos por 35 USD y 15 USD a 1,30 USD el millar.

Las respuestas de Postmark y de Brevo llevan una salvedad cada una. Postmark se queda en 36.666 con Platform (18 USD de plan y 32 USD a 1,20 USD el millar), sobre una cuota marcada en ámbar (•). Brevo llega a 40.000 correos con el volumen contratado de Starter que cuesta 46 USD, un precio que consta en el *provenance* y no en un campo (P-01).

En las otras dos APIs el presupuesto no fija la respuesta. En Amazon SES los 50 USD compran 500.000 destinatarios a la tarifa sin suscripción, pero el entorno de pruebas solo deja enviar 200 correos al día, 6.000 al mes, así que la respuesta la fija la capacidad y no el precio. En Mailchimp compran 50.000 correos de cuota, pero la cuenta de pago no tiene caudal publicado, y con los 25 correos por hora de la cuenta de prueba el mes da 18.000, rotulados `[demo]`.

PRIME4API da 83.343 correos en Resend y 83.115 en SendGrid, frente a los 116.000 y 116.722 del cálculo a mano, porque de cada plan solo ve el primer volumen contratado, el de 50.000 correos (Sección 6.4). Los 100.000 correos de Pro en Resend los vende la página de precios, y ACM2YAML v0.3 no tiene campo para guardarlos (P-01, los volúmenes contratados de un plan, Sección 5.3.1).

Lectura de las columnas. Las respuestas que se apoyan en al menos un hecho en ámbar son 13 de 18 en Postmark y 12 de 20 en Mailgun, y bajan a 1 de 22 en Resend y 1 de 20 en SendGrid.

Las respuestas rotuladas entre corchetes se cuentan aparte y no se descuentan de la columna: son 17 de 21 en Amazon SES y 10 de 20 en Mailchimp.

De las cinco preguntas X, las que el instrumental no responde, seis APIs responden tres fuera de los campos. Mailgun responde cuatro, porque es la única que publica la concurrencia que pide PI-A08, y Amazon SES dos, porque ninguna de sus fuentes publica la retención que pide PI-E03.

La versión del catálogo legible por máquina (`catalogo.yaml`) se conserva en el repositorio del trabajo junto con las 128 llamadas al motor, de modo que el procedimiento puede repetirse. Su límite está en la clase H: las diez preguntas P se han ejecutado contra PRIME4API, pero las cuatro H cuentan como respondibles por la especificación de HARVEY4API, sin haberse ejecutado con esa herramienta.

7.3. Las preguntas que piden HARVEY4API

Cuatro de las 32 plantillas del catálogo de preguntas de decisión combinan varias Datasheets o varias operaciones del motor, y por eso el catálogo les asigna la clase H, la que reserva a HARVEY4API. PI-C01 compara el coste de dos APIs para un volumen mensual y PI-C02 busca cuál de tres da más correos por dólar en su primer plan de pago. PI-C04 pregunta cuál de dos APIs envía antes un volumen dado y PI-G01, en qué mes un crecimiento sostenido obliga a cambiar de plan.

HARVEY4API, el agente conversacional de Ruiz López, publica las operaciones del motor como herramientas mediante el *Model Context Protocol*, el protocolo con el que un modelo de lenguaje llama a herramientas externas [4, 21]. HARVEY4API no se ha ejecutado en este trabajo. Su despliegue en SPHERE, la plataforma web del grupo ISA que reúne pricings en colecciones, llama a un modelo de OpenAI y necesita una clave de pago de esa API. El presupuesto del trabajo (Sección 2.3) solo imputa como servicio de modelos de lenguaje la suscripción a Claude, y no incluye esa clave.

Procedimiento. Para no dejar sin respuesta esas cuatro preguntas, se hicieron a otro modelo de lenguaje, Claude Opus 5.5, de Anthropic. La diferencia con HARVEY4API está en quién calcula: HARVEY4API invoca el motor como herramienta, y aquí el modelo lee el YAML y hace él mismo las cuentas.

La prueba tiene dos condiciones. En la primera, el modelo recibe un prompt de sistema fijo, el texto de instrucciones que lee antes de cada pregunta. Contiene las ocho Datasheets y seis reglas: usar solo esos datos, citar el campo de cada cifra, enseñar el cálculo, decir qué falta, avisar del semáforo y contestar en pocas líneas. En la segunda, el modelo recibe solo la pregunta, como en un chat sin contexto.

Al modelo se le planteó una instancia acotada de cada plantilla. En PI-C01, Postmark o Resend a 50.000 correos al mes, y en PI-C02, los correos por dólar de Postmark, Resend y SendGrid. En PI-C04, Mailgun o SES para 100.000 correos, y en PI-G01, un crecimiento del 20 % mensual sobre el plan Basic de Postmark.

Cada pregunta se hizo tres veces en cada condición, sin herramientas ni acceso a la web. Las 24 respuestas están archivadas[2]. Cada respuesta se contrasta con una referencia calculada a mano sobre las Datasheets.

Respuesta de referencia. La referencia de PI-C01 a 50.000 correos al mes ordena las ocho APIs así: Amazon SES 5,00 USD, SendGrid 19,95, Resend 20, Mailgun y MailerSend 35, Mailchimp 40, Postmark 55 y Brevo 56. SES queda a la cuarta parte de SendGrid porque cobra 0,10 USD por cada 1.000 correos, sin suscripción ni volumen incluido [62].

Los precios de Postmark y de Brevo salen de volúmenes contratados que la Datasheet guarda en el *provenance*. Los 55 USD de Postmark corresponden al volumen de 50.000 correos de su plan Basic, observado en el panel [33]. Los 56 USD de Brevo son el precio del volumen de 50.000 correos de su plan Starter, leído en el deslizador de la página de precios [70]. Ninguno de los dos volúmenes está en un campo, porque una cuota de v0.3 solo admite un valor (P-01, los volúmenes contratados de un plan, Sección 5.3.1).

Si se parte de la cuota de entrada de Postmark, 10.000 correos, y se suma al precio de cada plan el sobrecoste de los 40.000 restantes, el envío costaría 66 USD en Platform, 68,50 en Pro y 87 en Basic. La Datasheet registra esa cuota de entrada en

[2] Las 24 respuestas, el prompt de sistema y el script que las obtiene están en `05_salida/_cap5/claude_ejecucion/2026-10-25/` y `05_salida/_herramientas/ejecutar_claude_contexto.py`.

Pregunta	Referencia	Con Datasheets	Sin Datasheets
PI-C01: Postmark o Resend a 50.000 correos al mes	Resend 20 USD. Postmark 55 USD en el panel, de 66 a 87 USD por sobrecoste	Resend 20 USD frente a 55 USD, con los tres cálculos por sobrecoste y el ámbar de la cuota de Postmark	Resend. Postmark entre 50 y 70 USD, sin fuente
PI-C02: correos por dólar entre Postmark, Resend y SendGrid	SendGrid 2.506, Resend 2.500, Postmark 667	Las mismas tres cifras, cada una con su campo, y el ámbar de Postmark	Las mismas tres cifras, sin fuente
PI-C04: Mailgun o SES para 100.000 correos	Mailgun 66,7 s (un destinatario) o menos de 1 s (1.000). SES 500 días en pruebas (499 si el primer envío cuenta en $t = 0$) y sin cifra en producción	Mailgun en menos de 10 s con 1.000 destinatarios, cota más holgada que la de la referencia. SES 500 días en pruebas. Producción no publicada, sin comparar	Mailgun. SES en producción con 50.000 correos al día y 14 por segundo, cifras sin fuente publicada
PI-G01: crecimiento del 20 % al mes en Postmark Basic	Parcial: cuota de 10.000 en ámbar. Falta el volumen inicial	Pide el volumen inicial, da la fórmula y avisa del ámbar	Pide el volumen inicial, da la fórmula y supone 10.000 sin fuente

Tabla 7.5: Las cuatro preguntas que piden HARVEY4API, contestadas por Claude Opus 5.5 con las ocho Datasheets en el prompt y sin ellas. Cada celda resume tres repeticiones con la misma conclusión y las mismas cifras.

ámbar (●), el color de un valor interpretado y no publicado tal cual, y los tres cálculos heredan ese color.

PI-C02 divide la cuota del primer plan de pago entre su precio. El orden es SES 10.000 correos por dólar, SendGrid 2.506, Resend 2.500, Mailchimp 1.250, MailerSend 714, Mailgun y Postmark 667 y Brevo 556. Los 667 correos por dólar de Postmark (10.000 correos entre 15 USD) heredan el ámbar de su cuota de entrada.

En PI-C04, Mailgun envía 100.000 correos en 66,7 segundos con un destinatario por petición, al caudal de 1.500 peticiones por segundo que su Datasheet lleva en ámbar. SES tarda 500 días en su entorno de pruebas, con 200 correos al día, y en producción no publica ni la cuota ni el caudal. PI-G01 solo tiene respuesta parcial, porque la pregunta no fija el volumen inicial y sin él no hay mes en que se agote la cuota. La Tabla 7.5 recoge, para cada una de las cuatro instancias, la referencia y lo que contestó el modelo en cada condición.

Resultado. Las respuestas con Datasheets y sin ellas se separan en la fuente de cada cifra. Con las Datasheets, cada respuesta cita entre 4 y 11 campos y avisa del semáforo de los que están en ámbar o en rojo. Cada una separa lo publicado, lo interpretado y lo calculado, con la procedencia que define la Sección 4.3. En PI-C01, los 20 USD de Resend son un valor publicado, la cuota de 10.000 correos de Postmark es interpretada y los 87 USD de su plan Basic se calculan con el sobrecoste. La misma respuesta da aparte los 55 USD del panel, porque no salen de un campo modelado. Sin las Datasheets ninguna cifra cita una fuente, y varias llevan «unos» o «~» delante.

Las dos condiciones se separan más en PI-C04, que pregunta si conviene Mailgun o SES para enviar 100.000 correos. Con las Datasheets, las tres respuestas dicen que SES no publica su cuota ni su caudal en producción y que esa parte no se puede comparar. Sin ellas, las tres dan para SES en producción 50.000 correos al día y 14 por segundo, dos cifras que el proveedor no publica, así que la comparación con Mailgun se apoya en valores que ninguna fuente respalda.

En las cuatro preguntas, el modelo llega con las Datasheets y sin ellas a la misma conclusión que la referencia, y las tres repeticiones de cada condición dan las mismas cifras. Son proveedores conocidos, y es probable que sus precios de entrada figuren en los datos con los que se entrenó el modelo. Por eso el acierto de las respuestas sin Datasheets se trata como una amenaza a la validez de la prueba.

Las respuestas del modelo tienen la procedencia «inferido por agente», la de los valores que deduce un modelo de lenguaje, que no pasan de ámbar mientras no los respalde un valor publicado o medido. Tampoco sustituyen a HARVEY4API, porque ningún cálculo pasa por el motor de PRIME4API. La referencia y las Datasheets son del mismo autor, y las Datasheets se construyeron con asistencia de Claude (Anexo A), la misma familia de modelos que responde, de modo que la prueba comprueba que el modelo lee bien la Datasheet, no que la Datasheet sea correcta.

7.4. El caso de los 100.000 correos

Una aplicación necesita enviar 100.000 correos al mes, con ráfagas de 1.000 peticiones por minuto (16,7 peticiones por segundo), *webhooks* de eventos y envío en lote. Es el ejemplo de la Sección 1.1.3. El catálogo debe decir qué API y qué plan lo cubren al menor coste. Las cuatro condiciones se comprueban sobre la API REST, la interfaz que modelan las Datasheets (Sección 4.1). El relé SMTP, la otra interfaz del canal API, queda fuera de la comparación.

Tres de las cuatro condiciones se leen en la Datasheet de cada API, y la cuarta fuera de ella. El volumen sale de la cuota del plan y de su sobrecoste, y el coste se calcula para un mes de 30 días sumando al precio del plan el sobrecoste de los correos que pasan de la cuota. La ráfaga se compara con el *rate* del *endpoint* de envío que recoge el campo `max_power`, el caudal máximo que admite el plan. El lote es el tope de destinatarios o mensajes por petición (el máximo del `workload`) del *endpoint* que envía varios mensajes por llamada, y el caso no fija su tamaño. Los *webhooks* no son un campo de ACM2YAML v0.3, y se leen en la matriz de *features* de cada API, fuera

API	Plan y coste al mes (USD)	*Rate* de envío	Lote	*Webhooks*	Cumple
SendGrid	Essentials 50K con sobrecoste: 19,95 + 50.000 × 0,0013 = 84,95	10.000 pet./s ●	1.000 destinatarios	2 *endpoints* incluidos	sí
Mailgun	Scale: 90 con 100.000 incluidos	1.500 pet./s ●	1.000 destinatarios	en todos los planes	sí
Postmark	Platform con sobrecoste: 18 + 90.000 × 0,0012 = 126	5.000 pet./s ●	500 mensajes	solo Pro y Platform	sí
Brevo	Professional: 499 con 150.000 incluidos	1.000 pet./s ●	2.000 destinatarios ●	en todos los planes	sí
Resend	Pro con sobrecoste: 20 + 50.000 × 0,0009 = 65	10 pet./s ●	100 mensajes	5 *endpoints* en Pro	no
MailerSend	Starter con sobrecoste: 35 + 50.000 × 0,0013 = 100	120 pet./min ●	35.000 destinatarios	50 *endpoints* en Starter	no
Amazon SES	sin cuota incluida, y el precio por destinatario no es campo	no publicado ●	sin tope publicado	no (vía SNS)	no
Mailchimp Transactional	4 bloques × 20 ● = 80, más una cuenta de Mailchimp de pago	no publicado ●	sin tope publicado	por la API, en la cuenta de pago	sin respuesta

Tabla 7.6: Plan más barato que cubre 100.000 correos al mes en cada API y comprobación de las otras tres condiciones: ráfaga de 1.000 peticiones por minuto, envío en lote y *webhooks* de eventos. Coste con los campos `price`, `value` y `overage_cost`. En Mailgun se toma `rate_request_api`, el caudal que aplica el motor. El del *endpoint* de envío, 80.000 peticiones por ventana, también cumple la ráfaga: se midió en un sondeo, con una ventana de 9,85 a 9,87 segundos que la Datasheet redondea a 10, y está en ámbar. Los *webhooks* se dan para el plan de la fila. El lote se da en destinatarios o en mensajes, según cuente cada proveedor. El *rate* de Postmark se mide en la cabecera `ratelimit-limit` de un sondeo y no está publicado. Semáforo de la Datasheet: ● valor respaldado por su fuente, ● interpretado, ● no publicado.

de la Datasheet.[3]

De cada API se toma el plan más barato que cubre los 100.000 correos y se comprueban en él las otras tres condiciones. La Tabla 7.6 recoge el resultado.

Con los campos de las Datasheets y la matriz de *features* cumplen cuatro APIs: SendGrid, Mailgun, Postmark y Brevo. Leyendo solo los campos, la más barata es

[3] `03_datos/apis/<api>/features.md` del repositorio del trabajo, fila de *webhooks* de eventos, con la fuente de cada celda.

SendGrid Essentials 50K con 50.000 correos de sobrecoste, a 84,95 USD, seguida de SendGrid Pro, a 89,95 USD con 100.000 correos incluidos, y de Mailgun Scale, a 90 USD. En Postmark el plan más barato para este volumen es Platform, a 126 USD, porque su sobrecoste por correo es el menor de los tres planes de pago (Pro saldría a 133,50 USD). Brevo exige Professional, a 499 USD, porque su Datasheet modela Starter y Standard con 5.000 correos y sin sobrecoste. El tope de sobrecoste de SendGrid, diez veces el volumen del plan, no se alcanza con 100.000 correos (P-07, el tope que v0.3 no recoge, Sección 5.3.4).

Si solo se leen los campos, Resend cubre el volumen por menos dinero que ninguna otra, 65 USD, y MailerSend lo cubre por 100 USD, pero las dos fallan en la ráfaga. Resend admite 10 peticiones por segundo por equipo, 600 por minuto, en todos sus planes. MailerSend admite 120 peticiones por minuto en `/v1/email` y 15 en `/v1/bulk-email` con el plan Starter. El requisito cuenta peticiones y no correos: leído como 1.000 correos por minuto, las dos lo cumplirían enviando en lote.

Amazon SES no cumple el caso. SES cobra por destinatario, a 0,10 USD por millar en la tarifa sin suscripción, unos 10 USD para 100.000 correos, pero ese precio está en una nota del *provenance* porque v0.3 no tiene un precio por unidad sin cuota. Su *rate* en producción no está publicado, y SES no ofrece *webhooks*, porque entrega los eventos de envío a servicios de AWS como Amazon SNS o EventBridge. El cliente puede recibirlos por HTTP con una suscripción de Amazon SNS, pero el caso solo cuenta como *webhook* el aviso que el propio servicio de correo envía a una URL del cliente, sin otro servicio intermedio.

Mailchimp Transactional queda sin respuesta, porque no publica el caudal de la cuenta de pago y sin él no se puede comprobar la ráfaga. Sus 100.000 correos son cuatro bloques de 25.000 a 20 USD, 80 USD, con el precio del bloque en ámbar. El servicio exige además una cuenta de Mailchimp de pago, que la Datasheet no cifra.

El importe de la respuesta depende de los volúmenes contratados: el proveedor vende varios por plan, en la página de precios o en el panel, y la cuota de v0.3 solo admite uno (P-01, Sección 5.3.1). Cada Datasheet modela el volumen mínimo de cada plan, y los demás volúmenes quedan en notas del *provenance*.[4] SendGrid vende Essentials con 100.000 correos a 34,95 USD, y Resend vende Pro con 100.000 correos a 35 USD. Mailgun vende Foundation con 100.000 correos a 75 USD. Postmark vende Basic con 125.000 correos a 115 USD, aunque su panel no declara *webhooks* de eventos para ese plan.

PRIME4API toma de cada plan el único volumen que guarda la cuota, el mínimo, y cobra como sobrecoste lo que pase de él. Si se usan los volúmenes del *provenance*, Mailgun baja de 90 USD con Scale a 75 con Foundation, y Brevo de 499 USD con Professional a 82 con Starter de 100.000 correos. Foundation y Starter tienen el mismo *rate*, el mismo lote y los mismos *webhooks* que Scale y Professional, así que también cumplen.

SendGrid sigue siendo la más barata, a 34,95 USD con Essentials 100K, y los 84,95 USD que dan los campos son 2,4 veces el precio que publica para ese volumen.

[4] Notas de `metadata.data_provenance` en `03_datos/apis/<api>/datasheet.yaml` del repositorio del trabajo, con la página de precios o el panel de los que se copiaron.

Con la cuota extendida que propone P-01, una lista de volúmenes con su precio y su sobrecoste, los 34,95 USD de SendGrid, los 75 de Mailgun y los 82 de Brevo saldrían de los campos.

En SendGrid Essentials 50K, el plan que gana el caso cuando solo se leen los campos, la Tabla 6.3 da el tiempo mínimo para 100.000 correos con un destinatario por petición. Sin sobrecoste son 30 días y 4 segundos, porque los otros 50.000 esperan a que la cuota se reponga, y con un presupuesto que lo cubra son 9 segundos. Las 16,7 peticiones por segundo del caso son la seiscentésima parte del *rate* que SendGrid publica, y a ese ritmo los 100.000 correos salen en unos 100 minutos, así que en este caso deciden la cuota y el presupuesto. La curva de capacidad de ese plan (Figura 6.1) se queda en su cuota de 50.000 y no suma el sobrecoste, y una curva para el caso exigiría una ejecución nueva de PRIME4API [19].

Los *rates* con los que se decide la ráfaga no tienen la misma fiabilidad. El de SendGrid es una cifra de sus preguntas frecuentes que ninguna cabecera de la respuesta de envío deja ver (Sección 6.3, sobre la procedencia de cada *rate*). Los de Mailgun, Postmark y Brevo están en ámbar. El estado de la cuenta también puede bajar el caudal sin cambiar de plan: en MailerSend pasa de 120 a 10 peticiones por minuto con la cuenta en revisión (P-06, el caudal que depende del estado de la cuenta, Sección 5.3.3).

El catálogo da veredicto para siete de las ocho APIs, aunque el precio más bajo sale de las notas del *provenance* y los *webhooks*, de la matriz de *features*, que queda fuera del esquema.

7.5. Comparación funcional

El catálogo tiene tres preguntas sobre funcionalidades: si el plan gratuito incluye *webhooks*, qué API da IP dedicada (una dirección IP de envío que usa un solo cliente) al menor precio y cuánto tiempo se conservan los registros de envío. Ninguna se responde con la Datasheet ni con PRIME4API, aunque casi todos los datos que piden están publicados en la página de precios o en el panel de cada API.

Para responderlas se lee la matriz de *features* por plan, una tabla que el ciclo de cada Datasheet rellena con las mismas fuentes (Sección 4.2). Cada celda lleva su cita y su peldaño de la escalera de validación, como el contraste documental o la observación de panel, igual que un campo de la Datasheet, y queda vacía cuando ninguna fuente da el dato. Ocho de sus filas se registran en las ocho APIs, entre ellas los *webhooks* de eventos, la IP dedicada y la retención de actividad.[5]

7.5.1. *Webhooks* en el plan gratuito

PI-E01 pregunta si el plan gratuito incluye *webhooks* de eventos, es decir, avisos HTTP que el proveedor envía cuando un correo se entrega, rebota o se abre. Como en

[5]Las ocho matrices, con el literal y la fuente de cada celda, están en `03_datos/apis/<api>/features.md` del repositorio de datos del trabajo, https://github.com/danvelcam/tfm-pricing-apis-correo.

el caso de los 100.000 correos (Sección 7.4), cuenta como *webhook* el aviso HTTP que el propio servicio de correo envía a una URL del cliente, sin otro servicio intermedio. Es de clase L, la que se responde leyendo la matriz, y las ocho APIs la contestan.

Cinco de las ocho APIs dan *webhooks* en su plan gratuito o en su cuenta de prueba. Resend Free y MailerSend Free dan un *webhook* cada uno, y la cuenta de prueba de SendGrid da un *Event Webhook*. Mailgun los incluye en sus cuatro planes, hasta tres URL por tipo de evento, sin recorte en el plan gratuito. En Postmark la respuesta es «no consta», porque el panel solo los declara en Pro y Platform. En Brevo la matriz separa dos cosas: los *webhooks* salientes están en los cinco planes, y los de entrada, que reciben en Brevo eventos de otros sistemas, solo en Professional y Enterprise.

Las dos respuestas atípicas son de las APIs que no tienen un plan gratuito con nombre. SES no tiene *webhooks* en ningún plan, porque sus eventos salen por otros servicios de AWS, como SNS o EventBridge. En Mailchimp Transactional la cuenta de prueba no los muestra en el panel y la matriz responde «no consta», aunque la documentación da la operación `webhooks/add` para la cuenta de pago.

7.5.2. IP dedicada al menor precio

PI-E02 pregunta qué API da IP dedicada al menor precio para 200.000 correos al mes. Combina un dato de la matriz, la IP dedicada, con el precio de enviar 200.000 correos, que la Datasheet da en todas las APIs salvo SES, y la suma se hace a mano. SES es la más barata, con 44,95 USD: 20,00 de envío, a 0,0001 USD por destinatario, más 24,95 de una IP estándar. SES vende la IP sin exigir un volumen mínimo de envío.

Tras SES van Mailgun, con 185 USD (plan Foundation de 100.000 correos a 75, con la IP incluida, más 100.000 de sobrecoste a 1,10 USD el millar), y Mailchimp Transactional, con 189,95 USD (ocho bloques de 25.000 correos a 20 USD más 29,95 de la IP). Resend cuesta 190 USD (plan Scale de 200.000 correos a 160 más 30 de la IP) y SendGrid 199,95 (plan Pro de 100.000 correos a 89,95, con la IP incluida, más 100.000 de sobrecoste a 0,0011). Mailchimp adelanta a Resend por cinco céntimos.

Brevo tiene la IP dedicada más barata, 251 USD al año, unos 20,92 al mes, pero solo la vende en el plan Professional. Ese plan arranca en 150.000 correos por 499 USD, y el volumen contratado que cubre 200.000 es el de 250.000, a 599 USD, así que el coste total queda en unos 620 USD al mes. Es el coste total más alto de las seis APIs que venden la IP con precio publicado, más del triple que Resend.

Dos APIs quedan fuera de la comparación. Postmark no es elegible, porque exige 300.000 correos al mes para vender la IP. MailerSend tampoco, porque su página de precios reserva la IP al plan Enterprise sin publicar su precio.

Dos de las tres cifras más bajas llevan salvedad. La de Mailchimp Transactional no incluye la cuenta Mailchimp de pago que el producto exige, y con los 25 correos por hora de la cuenta de prueba los 200.000 correos tardarían 333 días. La de SES no sale de la Datasheet, que no tiene campo para el precio por destinatario, y su entorno de pruebas solo permite 200 correos al día.

7.5.3. Retención de la actividad

PI-E03 pregunta cuánto tiempo conserva cada API los registros de envío, y es de clase X: pide un campo que ACM2YAML v0.3 no tiene. La matriz tiene valor para siete de las ocho APIs. Postmark guarda 45 días y Resend 30. SendGrid guarda entre 3 y 7 días según el plan, y MailerSend entre 1 y 30. Mailgun guarda los registros entre 1 y 30 días y Mailchimp Transactional 30 días en la cuenta de prueba. En Brevo el periodo lo fija el cliente entre 1 y 24 meses, y es el único caso de las ocho APIs en que no lo pone el proveedor. En SES ninguna fuente publica la retención, así que la pregunta no tiene respuesta en ningún sitio.

La pregunta sigue en clase X aunque la matriz tenga siete valores, porque la retención es un límite de uso, el máximo de un nivel de uso medible, y no una *feature*. A diferencia de la cuota mensual, no se gasta con cada envío, y ACM2YAML v0.3 no tiene campo para ese tipo de límite.

7.5.4. Lo que la matriz añade

Con la matriz el catálogo pasa de 25 a 27 preguntas respondibles, porque se suman las dos de clase L. Las ocho APIs ganan dos respuestas cada una y quedan entre las 20 de Postmark y las 24 de Resend. Las preguntas comunes a las ocho APIs pasan de 16 a 18, porque se añaden las dos de clase L, PI-E01 y PI-E02.

El resultado deja una asimetría entre los dos tipos de pregunta. La capacidad se responde con instrumento, ya sea leyendo campos de la Datasheet o ejecutando PRIME4API, mientras que las *features* se responden leyendo la matriz a mano. En PI-E02 la IP dedicada de cada API está en la matriz y el precio de los 200.000 correos, en la Datasheet salvo en SES, y ninguna herramienta une las dos mitades. Lo que haría a PI-E02 respondible con instrumento es un vínculo entre la Datasheet del canal API y el iPricing, el metamodelo con el que SPHERE representa los planes y las *features* que un SaaS publica en su canal web [39]. Ese vínculo cae fuera del alcance fijado en el Capítulo 4, que modela solo el canal API, y por eso la comparación funcional se queda en lectura trazada, celda a celda, sin cálculo automático.

7.6. Respuesta a la RQ4

La RQ4 pregunta cómo pueden compararse las APIs de correo de forma sistemática para responder a preguntas de decisión que involucran simultáneamente requisitos funcionales y no funcionales. La respuesta es un procedimiento que instancia cada una de las 32 plantillas del catálogo sobre cada API. Cada instancia se responde con el instrumento que declara su clase: un campo de la Datasheet, una operación de PRIME4API, una composición de HARVEY4API o la matriz de *features* por plan. Cada respuesta queda registrada con la fuente que la sostiene.

Con ese instrumental, 25 de las 32 preguntas tienen una clase que las declara respondibles con la Datasheet, PRIME4API o HARVEY4API. Si se descuentan las

cuatro de HARVEY4API, que no se ha ejecutado, quedan 21. Otras 2, fuera de esas 25, se responden leyendo la matriz de *features*. Las cinco restantes señalan lo que falta, como la concurrencia de peticiones, que ACM2YAML v0.3 no serializa, o la retención de los registros de envío, un límite que no se gasta con cada envío.

Cada API responde entre 18 de esas 25 preguntas, en Postmark, y 22, en Resend (Tabla 7.4). En SES, 17 de sus 21 respuestas no se dan sobre el plan de suscripción que presupone la pregunta: 16 se dan sobre el entorno de pruebas y una sobre la tarifa sin suscripción. En Mailchimp, 10 de 20 son de la cuenta de prueba.

La comparación es asimétrica. El coste, la cuota y el caudal se comparan calculando sobre la Datasheet con PRIME4API, y los *webhooks* o la IP dedicada se comparan leyendo a mano la matriz de *features*. El caso de los 100.000 correos al mes necesita las dos: el precio, el caudal y el lote salen de la Datasheet, y los *webhooks*, de la matriz. Cumplen SendGrid, Mailgun, Postmark y Brevo, y la más barata es SendGrid, a 34,95 USD al mes con Essentials 100K. Con los campos de v0.3 saldría a 84,95 USD, 2,4 veces el precio que SendGrid publica para ese volumen, porque la cuota no guarda el volumen de 100.000 correos (Sección 7.4).

La respuesta tiene dos límites. La base son ocho APIs, y en cinco de ellas lo observado con cuenta propia corresponde a un estado de la suscripción, la fase en que el proveedor sitúa la cuenta, y no a un plan. Son el entorno de pruebas en SES, la cuenta de prueba en Mailchimp y una cuenta gratuita o sin aprobar en MailerSend, Mailgun y Brevo. El segundo límite es que las cuatro preguntas que combinan varias Datasheets o varias operaciones cuentan como respondibles por la especificación de HARVEY4API que publica Ruiz López [4], sin que la herramienta se haya ejecutado. Las respuestas que dio Claude Opus 5.5 leyendo las ocho Datasheets llegan a la misma conclusión que la referencia calculada a mano, pero no sustituyen a la herramienta (Sección 7.3).

8. Conclusiones

Este capítulo reúne las respuestas a las cuatro preguntas de investigación, de la RQ1 a la RQ4, y evalúa los objetivos y las contribuciones con los resultados de los Capítulos 4 a 7. Las ocho Datasheets del catálogo pasan sin errores el validador de formato de este trabajo, que comprueba los tipos de cada campo y las referencias entre bloques, pero ninguna supera todas las comprobaciones que se le hacen al ingerirla en PRIME4API. Al escribirlas aparecieron 35 elementos no soportados, hechos del dominio que ACM2YAML v0.3 no puede expresar o que PRIME4API no consume, como los seis volúmenes contratados que Postmark vende en un mismo plan.

El capítulo se organiza del siguiente modo. En primer lugar, la Sección 8.1 responde a cada pregunta de investigación y la Sección 8.2 da el estado de cada objetivo y de cada contribución. A continuación, la Sección 8.3 agrupa las amenazas a la validez en cuatro ejes, cada una con su mitigación cuando la hay. Después, la Sección 8.4 recoge las lecciones del método agéntico: el riesgo que añade un agente de inteligencia artificial, los controles que lo acotan y lo que no se ha medido. Por último, la Sección 8.5 plantea el trabajo futuro y el repositorio del que parte.

8.1. Respuestas a las preguntas de investigación

La Tabla 8.1 responde a las cuatro preguntas de investigación de la Sección 1.3. Cada celda da el resultado principal, y las cifras de detalle están en el capítulo que la sostiene, el de la última columna.

8.2. Objetivos y contribuciones

La Tabla 8.2 da el estado de cada objetivo específico y de la contribución que le corresponde, con los epígrafes de la memoria que lo sostienen. OBJ-03 y OBJ-04 se justifican primero, porque el Capítulo 1 sitúa en ellos el componente de investigación.

Evolución del esquema (OBJ-03 y C-02). De los 35 elementos no soportados que reúne el inventario de las ocho APIs, 24 superan el criterio de promoción: aparecen en dos APIs o más, o una ejecución de PRIME4API muestra su efecto sobre la Datasheet (Sección 5.2). Hay nueve propuestas de extensión. Ocho responden a elementos promovidos, como P-01, los seis volúmenes contratados que Postmark vende dentro de un mismo plan. La novena, P-05, el plan con precio que existe y no es público, sale del fallo del motor ante el precio `custom` del plan Enterprise de Resend.

Los otros 16 elementos promovidos no tienen propuesta. Doce solo tienen una forma esbozada que no se ha contrastado con la guía de ACM2YAML (Tabla 5.3). Los cuatro restantes quedan fuera de las propuestas por otras razones, como los *add-ons* de IP dedicada, que el esquema deja al canal web (Sección 5.5). OBJ-03 queda cumplido

Pregunta	Respuesta	Cap.
RQ1. Qué APIs de correo incluir y con qué criterios reproducibles	El conjunto debe cubrir cuatro formas de cobro: volumen mensual, pago por uso, producto añadido y límite diario. Sus APIs deben diferir en cómo limitan el envío, por ejemplo con un *rate* por *endpoint* o con un caudal negociado con el proveedor. Las ocho del catálogo cumplen esa condición. Es reproducible el protocolo de selección (tres fuentes de inclusión, cinco criterios ordenados y una regla de reemplazo). La consulta al modelo de lenguaje no lo es, porque repetirla no garantiza las mismas APIs, y queda registrada en el Anexo A.	4
RQ2. Si ACM2YAML basta para la capacidad contractual y qué extensiones requiere	No basta: las ocho Datasheets dejan en el registro de procedencia, que ninguna herramienta lee, datos como los seis volúmenes contratados de Postmark. Las extensiones son nueve propuestas. Siete amplían lo que el esquema expresa, y dos corrigen formas que la guía de ACM2YAML admite y PRIME4API no lee: el límite vacío (P-04) y el precio `custom` del plan Enterprise de Resend (P-05).	5
RQ3. Cómo se relacionan los límites de uso y los *rates*, y qué implica para el consumidor	En seis de las ocho APIs la cuota es la restricción activa, el límite que corta antes el envío. Con los planes que recoge la Datasheet, el *rate* admite en la ventana de la cuota entre 103,7 veces la cuota (MailerSend Starter) y 129.600.000 (Postmark Free). Quien contrata una de esas seis puede dimensionar el volumen sin consultar el *rate* si su cuenta está aprobada y sin restricciones. En Amazon SES y Mailchimp Transactional no se puede decir que mande la cuota. El estado de la cuenta cambia la capacidad en seis de las ocho, esas dos incluidas, y ninguna publica el umbral que dispara el cambio.	6
RQ4. Cómo comparar las APIs con requisitos funcionales y no funcionales a la vez	Un procedimiento instancia en cada API el catálogo de 32 preguntas de decisión y responde cada una con el instrumento de su clase: la Datasheet, PRIME4API o HARVEY4API. Son respondibles por su clase 25, y 21 sin HARVEY4API, que no se ha ejecutado. El coste, la cuota y el caudal se calculan sobre la Datasheet, y los *webhooks* o la IP dedicada se leen a mano en la matriz de *features*. En el caso de los 100.000 correos al mes cumplen cuatro APIs, y la más barata es SendGrid Essentials 100K, a 34,95 USD. Los campos de ACM2YAML v0.3, con un solo volumen por plan, dan 2,4 veces ese precio.	7

Tabla 8.1: Respuesta a cada pregunta de investigación y capítulo que la sostiene.

Objetivo	RQ	Contrib.	Estado del objetivo	Estado de la contribución	Epígrafes que lo sostienen
OBJ-01. Selección justificada del dominio	RQ1	C-01	cumplido en parte	cumplida en parte	4.1, 4.7
OBJ-02. Construcción del catálogo de Datasheets	RQ2 a RQ4	C-01	cumplido	cumplida en parte	4.3, 4.5, 5.4
OBJ-03. Evolución del esquema	RQ2	C-02	cumplido en parte	cumplida	5.2, 5.5, 5.6
OBJ-04. Análisis de la capacidad	RQ3	C-03	cumplido	cumplida	6.1, 6.3, 6.5
OBJ-05. Método de comparación	RQ4	C-04	cumplido	cumplida	7.1, 7.2, 7.6
OBJ-06. Contribución a SPHERE	ninguna	C-01	cumplido	cumplida en parte	4.6

Tabla 8.2: Estado de cada objetivo específico y de la contribución que le corresponde. RQ: la pregunta de investigación a la que sirve el objetivo. C-01: catálogo de Datasheets de APIs de correo. C-02: evolución de ACM2YAML. C-03: caracterización de la relación entre límites y *rates*. C-04: método de comparación sistemática. OBJ-02 sirve de medio a RQ2, RQ3 y RQ4, porque el catálogo es la condición para responderlas. C-01 aparece tres veces porque responde a OBJ-01, OBJ-02 y OBJ-06. OBJ-06 no deriva de ninguna RQ.

en parte, porque pide la extensión de cada carencia identificada y esos doce promovidos no la tienen fijada.

C-02 se cumple: cada una de las siete propuestas que amplían lo que ACM2YAML v0.3 puede expresar responde a un caso real que el esquema no representa, como el tope con que SendGrid limita el envío mensual a diez veces el volumen contratado, sobrecoste incluido. Las otras dos quedan fuera del enunciado de C-02: P-04, la marca de valor no publicado junto al campo, y P-05. ACM2YAML v0.3 admite esos valores, y es PRIME4API el que falla al leerlos.

Análisis de la capacidad (OBJ-04 y C-03). La cuota es la restricción activa en Postmark, Resend, SendGrid, MailerSend, Brevo y Mailgun, seis de las ocho APIs (Sección 6.5). La curva de capacidad $C(t)$ que dibuja PRIME4API (Figura 6.1) da el máximo de correos que se pueden enviar desde el inicio de la ventana hasta cada instante sin pasar ningún límite. Su codo es el instante en que deja de mandar el *rate* y empieza a mandar la cuota, y con un destinatario por petición llega a los 83 minutos en Resend Pro y a los 41 en MailerSend Hobby. Dos límites del catálogo no caben en esa curva: los dos contadores de Mailgun sobre la misma ventana y la franquicia sin ventana de Mailchimp Transactional, API en la que manda el caudal.

OBJ-04 y C-03 se cumplen sobre una base estrecha. Solo en esas seis APIs la Datasheet da cifra al *rate* y a la cuota de un mismo plan, y los valores medidos en MailerSend, Amazon SES y Mailchimp Transactional salen de cuentas sin aprobar o en pruebas, no del plan contratado.

Método de comparación (OBJ-05 y C-04). El catálogo de preguntas reúne 32 preguntas de decisión, y el procedimiento que instancia cada una sobre cada API se ha aplicado a las ocho (Sección 7.2). OBJ-05 y C-04 se cumplen con tres salvedades. La primera es que las cuatro preguntas que piden HARVEY4API cuentan como respondibles por la especificación de HARVEY4API que recoge Ruiz López [4], y solo se contrastaron con las respuestas que dio el modelo de lenguaje Claude Opus 5.5 al leer las Datasheets.

La segunda salvedad es que las diez respuestas de Mailchimp Transactional que resuelve una operación de PRIME4API están calculadas a mano. El motor devuelve HTTP 500 en 14 llamadas por dos campos vacíos que el esquema admite: el tope de destinatarios y el periodo de la franquicia de la cuenta de prueba. La tercera salvedad es que una de las tres preguntas funcionales, la retención de los registros de envío, pide un campo que ACM2YAML v0.3 no tiene. Las otras dos se responden leyendo la matriz de *features* por plan.

Catálogo, selección y SPHERE (C-01 frente a OBJ-01, OBJ-02 y OBJ-06). OBJ-02, la construcción del catálogo, se cumple. Los términos de consumo de cada proveedor quedan recogidos con su URL y su fecha de acceso, y cada valor de las ocho Datasheets lleva la cita literal que lo sostiene (Sección 4.3). En Postmark no se archivaron las páginas que cita su Datasheet, y su versión se conserva solo como URL y fecha de acceso.

OBJ-01, la selección del dominio, se cumple en parte. El protocolo parte de tres fuentes de inclusión: las APIs del trabajo de Noguerol Tirado [60], las ya modeladas en el ecosistema y una consulta a un modelo de lenguaje (Sección 4.1.1). La primera no aportó ninguna API, porque se tuvo acceso a la memoria de Noguerol Tirado cuando la selección ya estaba cerrada. La consulta presentaba Amazon SES como pago por uso puro, cuando ya publicaba planes, y repetirla no garantiza las mismas APIs (Sección 4.7). Postmark, el piloto con el que se probó y se ajustó el método, no entra por ninguna de las tres fuentes.

OBJ-06, la contribución a SPHERE [22], se cumple. Los ocho pricings en Pricing2YAML pasan el analizador de pricing4ts 0.10.4, la versión que usa la plataforma, y cada una de las 20 limitaciones del formato tiene su caso real. Pricing2YAML no distingue un caudal de una cuota: en Resend Free, el caudal de 10 peticiones por segundo y la cuota de 3.000 correos al mes se escriben igual, como un límite renovable con su periodo, y la analítica de SPHERE los cuenta igual (Sección 4.6). La maqueta de la colección en SPHERE propone cómo mostrar los términos de consumo en la ficha de cada API.

C-01 se cumple en parte por tres razones. La primera es que la colección está cargada en SPHERE desde el 27 de octubre de 2026, pero es privada hasta el depósito. La segunda es que ninguna Datasheet supera todas las comprobaciones que se le hacen al ingerirla en PRIME4API (Tabla 4.6). La tercera es que HARVEY4API, la otra

herramienta del ecosistema que debería reutilizar las Datasheets, no se ha ejecutado.

En cinco de las ocho Datasheets, las comprobaciones fallan por un valor ausente o por un precio no público. En Postmark queda vacío el tope de destinatarios, que el proveedor no pone, y en Mailchimp Transactional ese mismo tope y el periodo de la franquicia de la cuenta de prueba. Fallan diez comprobaciones en cada una, y cinco en Resend, que no publica el *rate* de su plan Enterprise. Un precio que existe y no es público invalida trece comprobaciones en SendGrid, trece en Brevo y ocho en Resend. MailerSend, Amazon SES y Mailgun solo fallan la comprobación de la curva de capacidad acumulada, con el mismo mensaje de error. En MailerSend y Amazon SES el fallo es del motor, y en Mailgun no se ha aislado su causa.

8.3. Amenazas a la validez

Esta sección agrupa por eje de validez las limitaciones que declaran los Capítulos 4 a 7, cada una con su caso, su mitigación cuando la hay y lo que queda abierto.

8.3.1. Validez de constructo

La validez de constructo pregunta si lo medido es lo que se dice medir. El sondeo de MailerSend midió el estado de la cuenta y no el plan: la cuenta sin aprobar admitió 10 peticiones por minuto frente a las 120 que el proveedor publica para una cuenta sana. El sondeo de Amazon SES corrió en el entorno de pruebas (*sandbox*) y el de Mailchimp Transactional en su cuenta de prueba. Las tres Datasheets declaran el estado medido. Medir en producción exige que el proveedor apruebe la cuenta, como en la salida del *sandbox* de Amazon SES, o pagar un plan.

El *rate* de Postmark y el de Mailgun salen de la cabecera de respuesta de una cuenta gratuita, sin provocar el código `429` con que la API rechaza el exceso, y la Datasheet los extiende a todos los planes. En Postmark no lo provocaron 200 peticiones, y en Mailgun harían falta 80.000 en unos diez segundos. Lo mitiga que las dos cifras vayan en ámbar, el color del semáforo para un valor interpretado y no literal.

SendGrid publica sus 10.000 peticiones por segundo en unas preguntas frecuentes y su envío no devuelve cabeceras de *rate*, así que la cifra va en verde por ser literal, pero ningún sondeo la ha observado. Queda abierta en las tres APIs una medición bajo carga, por ejemplo con el banco de pruebas de límites de uso de API-TaskForce, el grupo que publica la guía de ACM2YAML [66].

La curva de capacidad necesita la ventana de cada límite y lo que descuenta cada envío. La ventana de la cuota, el periodo tras el que su contador vuelve a cero, está en rojo, el color del semáforo para un dato no publicado, en Mailgun, Brevo, Amazon SES y Mailchimp Transactional, y en ámbar en las otras cuatro. Ninguna de las ocho declara la ventana del *rate*, y los dos sondeos que la acotaron, en Brevo y Mailgun, no fijaron si es fija o deslizante. Que cada destinatario descuente un correo de la cuota consta en Resend, SendGrid, Brevo y Amazon SES, y en Postmark, MailerSend, Mailgun y

Mailchimp Transactional queda sin medir. Esos huecos los marca el semáforo API a API, de modo que ninguno pasa por publicado sin serlo.

La cita literal de cada valor solo se contrasta de forma mecánica si se archivó la página, y de Postmark no se archivó ninguna de las que cita su Datasheet. Lo mitigan la URL y la fecha de acceso de cada valor.

El ecosistema no tiene validador oficial de Datasheets, y la conformidad se comprueba con el validador de formato de este trabajo y con la ingestión en PRIME4API. Los defectos de contrato, valores que ACM2YAML v0.3 admite y PRIME4API no sabe leer, los confirma un motor ajeno. En cambio, las cuatro carencias que se promueven solo por recurrencia descansan en la lectura que este trabajo hace de la guía de ACM2YAML: los volúmenes contratados (P-01), el consumo de un lote de mensajes con varios destinatarios (P-02), el tope de una petición en otra unidad (P-03) y el estado de la suscripción (P-06).

P-06, el estado de la suscripción, muestra el riesgo de esa lectura. La guía documenta alias de estado, bloques que dentro de un *endpoint* dan a un estado de la cuenta sus propias cifras, con MailerSend como ejemplo, y la Datasheet de MailerSend añade uno para la cuenta sin aprobar. En un alias, la guía solo documenta el *rate* y la cuota, pero el código de referencia también lee un tope por petición. El tope de dos destinatarios por petición medido en esa cuenta cabe, por tanto, en el esquema, y lo que falta es documentarlo en la guía. P-06 sigue siendo una extensión porque el esquema no permite declarar el disparador del estado ni declarar el estado una sola vez para toda la Datasheet.

A cada pregunta se le asigna el instrumento que la responde, sea la Datasheet, PRIME4API o HARVEY4API, sabiendo ya lo que ofrece cada uno, y eso puede inflar la proporción de preguntas respondibles. Por eso el 78 % de preguntas respondibles, 25 de las 32 del catálogo de preguntas, no se compara de forma directa con el 54 % que Cavero et al. obtienen sobre 56 plantillas con herramientas que ya existían [1] (Sección 7.1). Lo mitiga que las preguntas sigan la gramática de plantillas de Cavero et al., de modo que pueden incorporarse a su banco de preguntas y medirse con sus herramientas.

El catálogo de preguntas se cerró en 32 sin un criterio de saturación fijado de antemano, aunque al pasar de tres APIs a ocho no ganó ninguna.

8.3.2. Validez interna

La validez interna pregunta si los resultados dependen de las Datasheets o del montaje con que se obtienen. El script que verifica las Datasheets contra PRIME4API clona el motor sin fijar la revisión, y un cambio del motor puede mover los errores de la Tabla 4.6 sin tocar la Datasheet. Seis de las ocho ejecuciones registran la revisión `8e6011c` del repositorio de PRIME4API y pueden repetirse contra ella, y las de Brevo y Amazon SES no la registran.

El método de sondeo tiene tres puntos débiles. En MailerSend, el *endpoint* de cuota servía el saldo del día anterior once horas después de la reposición (Sección 6.3),

así que el saldo que registra el sondeo al terminar puede ser el de la víspera. En Brevo, el sondeo lee la cifra en el error con que el proveedor rechaza un destinatario imposible de entregar, pero con la cabecera de modo *sandbox*, que solo valida el formato, el envío devolvió `201`. Lo cubren un sondeo previo que comprueba que el proveedor rechaza la petición y un tope de envíos que cuenta como real cada petición aceptada. En Mailchimp Transactional, desde once destinatarios o con un adjunto de 5 MB, el envío queda en cola, y por encima de ese umbral el sondeo no mide.

PRIME4API corrió en local con la misma imagen de contenedor que despliega SPHERE, y no en el despliegue público que cita Gavira Sánchez [3], así que sus respuestas no se han contrastado contra ese servicio. Las diez respuestas de Mailchimp Transactional que se calcularon a mano por el HTTP 500 del motor (Sección 8.2) no tienen confirmación de PRIME4API.

Los valores modelados dependen también de qué fuente se toma cuando dos discrepan. La regla que da preferencia a la página de precios vale para las 37 divergencias, de las 82 registradas, en que esa página es uno de los lados. Aun en ellas deja sin resolver las siete en que la página se contradice a sí misma, cuyo valor queda en ámbar. Las 45 restantes enfrentan fuentes distintas de la página de precios: en la mayoría, dos fuentes del mismo proveedor, como su referencia de la API y su centro de ayuda, y en las demás, una de ellas frente a un sondeo o a un tercero. Las ordenan otras reglas de modelado, como la que da preferencia al valor medido. Dieciséis de esas 45 siguen abiertas, con las dos fuentes en el registro de procedencia.

Las Datasheets y el recuento de recurrencia los construye un agente de inteligencia artificial, que podría dar por leído un dato que la fuente no contiene. Lo mitiga que el agente que construye no sea el que verifica, y queda abierto medir cuántos valores corrigió esa revisión (Sección 8.4).

8.3.3. Validez externa

La validez externa pregunta hasta dónde valen los resultados fuera del catálogo y de su ventana de toma de datos. La respuesta negativa a la RQ2, si ACM2YAML v0.3 basta para representar las APIs de correo, no depende de la muestra, porque basta un caso que v0.3 no represente, como los seis volúmenes contratados de Postmark.

Sí depende de la muestra qué propuestas se promueven al esquema. El tope de una petición medido en una unidad distinta de la suya, como los 50 MB de carga de `/email/bulk` en Postmark (P-03), aparece en las ocho APIs. En cambio, tres propuestas descansan en una sola API: el tope del sobrecoste de SendGrid (P-07), el cupo de destinatarios de la cuenta sin aprobar de MailerSend (P-08) y la cola de Brevo por encima de la cuota (P-09). Se promueven porque PRIME4API da en ellas un resultado que el proveedor contradice. Al pasar de cuatro APIs a ocho ninguna propuesta perdió su apoyo. Queda abierto repetir el inventario en otro dominio, comprobando cada propuesta con el caso que la motiva.

La marca de valor no publicado (P-04) y el plan con precio no público (P-05) se han medido contra un único motor, PRIME4API. Claude Opus 5.5, con el que se

contrastaron las preguntas de HARVEY4API, lee el YAML como texto y no cuenta como segundo motor.

En la relación entre límites de uso y *rates* (RQ3), que la cuota sea la restricción activa se explica por las unidades, y por eso cabe esperarlo fuera del catálogo: frente a una cuota mensual o diaria, un *rate* por segundo o por minuto deja pasar en la ventana de la cuota mucho más volumen. Lo que sí depende de la muestra son las excepciones, como el *rate* reducido de la cuenta sin aprobar de MailerSend (Sección 8.3.1) o la discrepancia de Mailgun. Mailgun publica 1.500 peticiones por segundo por IP de cliente, y la cabecera del envío devuelve un límite de 80.000 peticiones en una ventana medida de unos diez segundos. Que el *rate* del envío individual no cambie con el plan vale en Resend, SendGrid y MailerSend, y Brevo lo contradice.

Las cuatro formas de cobro que el catálogo cubre (volumen mensual, pago por uso, producto añadido y límite diario) salen del primer criterio de selección, cubrir formas de cobro distintas (Sección 4.1.2), y no de un censo del sector. Entre las APIs de reserva de la selección, Zoho ZeptoMail cobra con créditos prepagados, una variante que el catálogo no incluye.

El catálogo modela solo el producto transaccional, el de los mensajes individuales que suele desencadenar el propio usuario, y sus conclusiones no se trasladan al de *marketing*, que Resend factura por contactos y no por correos.

La oferta cambia con el tiempo: el repositorio de ACM2YAML conserva una Datasheet de MailerSend con precios que difieren de los publicados cinco meses después [61]. Cada valor lleva su fecha de acceso, y falta un procedimiento que diga cuándo reverificar.

8.3.4. Validez de conclusión

La validez de conclusión pregunta si los recuentos que sostienen las conclusiones están bien hechos. La Tabla 5.1 deja 31 combinaciones de elemento y API sin verificar y una en suspenso, las dos cuotas de Mailgun que ninguna página dice si se aplican a la vez. Una aparición sin verificar no se cuenta, así que verificarla solo puede añadir APIs y cada recuento es un mínimo. Bastaría confirmar una aparición más en cada uno para que pasaran a recurrentes tres elementos no promovidos: la región como unidad de gobierno, el tope fraccionario y los dos contadores sobre una misma ventana. Ninguna promoción se retiraría[1].

En Postmark, una respuesta «no publicado» cuenta como respondida en la pregunta del *rate* máximo y no en la del límite diario ni en la de si el *rate* sube con el plan. Aplicar el mismo criterio a las tres movería una o dos de sus 18 respuestas. La cobertura cuenta igual el ámbar que el verde, y el ámbar lo llevan 13 de las 18 respuestas de Postmark y 12 de las 20 de Mailgun, frente a 1 de 22 en Resend.

El mes se toma de 30 días. Con 28, el menor cociente entre *rate* y cuota de una cuenta sana, el de MailerSend Professional 2M, un volumen que la Datasheet no recoge, baja de 2,59 a 2,42. Ninguna restricción activa cambia con la longitud del mes.

[1] El detalle por fila está en `03_datos/RECURRENCIA_2026-09-23.md`, en el repositorio del trabajo.

8.4. Lecciones del método agéntico

El Anexo A declara qué tareas hicieron los agentes de inteligencia artificial y cuáles el autor.

Este trabajo separa a quien construye una Datasheet de quien la verifica. El riesgo propio de un agente es dar por publicado un dato que la fuente vigente no contiene. Le ocurrió a Claude Opus 5.5 en la prueba de la Sección 7.3, que le planteó preguntas de comparación con las Datasheets en el prompt y sin ellas. Sin ellas, atribuyó a Amazon SES en producción 50.000 correos al día y 14 por segundo. La documentación vigente no da ninguna de las dos: la primera solo sale en una entrada del blog de AWS de 2014, y la segunda no está en ninguna fuente de su Datasheet. El mismo error en un agente que construye una Datasheet pasaría al catálogo.

En este trabajo lo acotan dos controles. El primero es que otro agente recibe la Datasheet terminada y la revisa campo a campo con las fuentes a la vista, en la fase de revisión del ciclo de la Sección 4.2. El segundo es el verificador de citas, una comprobación automática que contrasta la cita literal de cada valor con la copia archivada de su página, y no se aplica a Postmark, cuyas páginas no se archivaron. No se ha contado cuántos valores de las ocho Datasheets corrigieron esos dos controles, de modo que su eficacia no está medida.

El libro de criterios es el fichero de reglas común a las ocho Datasheets. Cada duda de modelado que no cubría se decidió una sola vez y entró en él como regla nueva, de modo que no se volvió a plantear en otra API. Así, los límites de recursos, como el número de dominios o de *webhooks* de una cuenta, no se modelan como cuotas en ninguna de las ocho.

En la prueba de la Sección 7.3, con las Datasheets en el prompt, Claude Opus 5.5 leyó bien los campos que pedían las cuatro preguntas, que tocan cinco de las ocho APIs. Citó el campo de cada cifra, separó el valor publicado del interpretado y llegó a la misma conclusión que la referencia calculada a mano, la misma a la que llegó sin ellas. La prueba comprueba la lectura y no que las Datasheets sean correctas: como se construyeron con la misma familia de modelos, un valor mal leído al construirlas pasaría a la respuesta sin que la prueba lo detectase. Por eso un valor que solo ha deducido un agente se queda en ámbar, el color de lo interpretado, mientras no lo respalde un valor publicado o medido.

La reducción de horas del autor no se puede atribuir a los agentes. Las horas suman 214 frente a las 300 previstas, pero el tiempo de los agentes no se registró y no hay una medición del mismo trabajo sin ellos (Sección 2.4). Medir ese efecto exigiría registrar ese tiempo desde el inicio y construir sin agentes una muestra de Datasheets.

8.5. Trabajo futuro

Sobre ACM2YAML quedan tres líneas de trabajo. La primera es distinguir el límite publicado del aplicado, el elemento F-04, que reúne los límites que el proveedor

declara y que la respuesta a la petición no refleja (Sección 5.5). Es el caso de Mailchimp Transactional y de Amazon SES, que aceptaron llamadas por encima del tope que declaran. Queda por decidir qué cifra va a la Datasheet y cómo se declara la fuente de cada una. La segunda es fijar las doce formas esbozadas de la Tabla 5.3, de una fecha de verificación para cada afirmación del registro de procedencia (F-09) al canal que ofrece cada operación (F-22), y contrastarlas con la guía de ACM2YAML.

La tercera es llevar como *issues* las nueve propuestas de extensión de la Tabla 5.2, cada una con su caso y el número de APIs en que aparece, al repositorio que tiene que cambiar: el de la guía si cambia el esquema y el de PRIME4API si el arreglo está en el motor (Sección 5.1). De esas nueve, las cinco que van de los volúmenes contratados al plan con precio no público (P-01 a P-05) no figuran en ningún *issue* de los repositorios de la guía, de PRIME4API ni de SPHERE (Sección 5.5). Las cuatro restantes, del estado de la suscripción al comportamiento de la cuota al superarse (P-06 a P-09), aún deben contrastarse con los *issues* abiertos antes de enviarse.

En las herramientas de consulta queda ejecutar con HARVEY4API las cuatro preguntas de clase H, las que combinan varias Datasheets u operaciones de PRIME4API, y comparar sus respuestas con las de Claude Opus 5.5 y con el cálculo a mano (Sección 7.3). También queda vincular la Datasheet del canal API con el iPricing, el metamodelo con el que SPHERE representa los planes y las *features* de la página de precios [39]. Con él, la pregunta de qué API da IP dedicada al menor precio para 200.000 correos al mes, que en la Sección 7.5 se calcula a mano, podría responderse con una consulta a SPHERE.

En SPHERE, la primera línea es implementar la sección de términos de consumo que la Sección 4.6.3 deja diseñada. Lo publicado y lo medido entrarían por una ingesta que guarda la fecha de cada página, en un historial separado de las versiones de *pricing* que SPHERE ya mantiene. Ese diseño separa los límites propios de una interfaz, como las 10 conexiones SMTP concurrentes por IP que recomienda Postmark, de los compartidos entre varias. Un límite compartido es la cuota diaria de MailerSend, que cuenta las peticiones de la API REST y las del relé SMTP.

Al diseño le falta una tercera clase, la de los límites globales, que valen para el servicio entero y no para una interfaz concreta. Esa clase exige un nivel por encima de la interfaz.

La segunda línea en SPHERE es comparar las vías de contratación de un mismo servicio. En RapidAPI, un mercado de APIs, hay fichas del servicio de envío de cuatro de las ocho APIs: SendGrid, Postmark, Brevo y Mailgun[2]. Comparar el precio y los límites de SendGrid contratado en RapidAPI con los de su página de precios pide un *pricing* por vía de contratación y el árbol de proveedor, servicio e interfaz de la maqueta de la colección para SPHERE (Sección 4.6.1). Queda por aclarar si esas fichas, que publica un tercero, exigen una cuenta en el proveedor.

Las últimas líneas son *skills*, instrucciones que sigue un agente basado en un modelo de lenguaje para leer los términos de consumo, y van de la más próxima a este trabajo a la más lejana. Una agruparía las interfaces que comparten un límite, como la cuota diaria de MailerSend, y su salida se contrastaría con la página antes de

[2]Búsqueda sin cuenta en el buscador de RapidAPI, consultada el 23 de octubre de 2026.

entrar en la Datasheet. Otra listaría como candidatas las contradicciones entre páginas de un mismo proveedor, como la guía de envío por lotes de Brevo, que admite 6.000 llamadas por hora al *endpoint* de envío, 600 veces menos que los 3.600.000 por hora de su categoría General (Sección 6.2). Una tercera contaría los límites de cada versión fechada de una página, como las capturas de la página de límites de MailerSend.

La cuarta traduciría a cambios del dominio el diff sintáctico con el que SPHERE compara dos versiones de un pricing, que marca qué líneas difieren entre ellas [22]. La más lejana, porque depende de las otras, es publicarlas en un *marketplace* que registre su uso real y conectarlas con SPHERE por el *Model Context Protocol*, el mismo con el que HARVEY4API publica las operaciones de PRIME4API [21].

Cualquiera de estas líneas parte del repositorio de datos del trabajo, `https://github.com/danvelcam/tfm-pricing-apis-correo`. Reúne las ocho Datasheets con sus fuentes y evidencias junto a las herramientas del trabajo. No incluye las credenciales de las APIs ni PRIME4API, y de las copias archivadas de las páginas de los proveedores publica solo la huella sha256.

Para comprobar la instalación, la herramienta `05_salida/_herramientas/verificar_prime4api.py` se ejecuta con `python3` sobre `03_datos/apis/postmark/datasheet.yaml`. La primera vez clona PRIME4API en `~/.cache/atf/prime4api` sin fijar la revisión, así que antes hay que situar esa copia en la revisión `8e6011c` (Sección 8.3.2). El resultado debe coincidir con la fila de Postmark de la Tabla 4.6: 11 comprobaciones superadas y 10 fallidas por valor ausente.

9. Bibliografía

[1] F. J. Cavero, A. García-Fernández, J. A. Parejo, M. Resinas y A. Ruiz-Cortés, «Pricing Intelligence: Rethinking IS Engineering in Volatile SaaS Environments,» en *Advanced Information Systems Engineering (CAiSE 2026)*, serie Lecture Notes in Computer Science, vol. 16558, Springer, 2026, págs. 42-58. DOI: 10.1007/978-3-032-28110-4_3

[2] A. Ruiz-Cortés y M. Ruiz-López, «Gestión de la capacidad y el rendimiento de APIs bajo el enfoque de caja negra,» en *Actas de las XX Jornadas de Ciencia e Ingeniería de Servicios (JCIS 2025)*, J. Boubeta-Puig, ed., Sistedes, 2025. Dirección: https://hdl.handle.net/11705/JCIS/2025/47

[3] R. Gavira Sánchez, «Pricing Intelligence: Modelo y motor de análisis de la capacidad de pricing de APIs,» Máster en Ingeniería del Software: Cloud, Datos y Gestión TI, Trabajo Fin de Máster, Universidad de Sevilla, 2026.

[4] D. Ruiz López, «HARVEY4API: Modelado, extracción y análisis de pricing de APIs,» En preparación, Trabajo Fin de Máster, Universidad de Sevilla, 2026.

[5] A. García-Fernández, J. A. Parejo, P. Trinidad y A. Ruiz-Cortés, «Automated Analysis of Pricings in SaaS-Based Information Systems,» en *Advanced Information Systems Engineering (CAiSE 2025)*, serie Lecture Notes in Computer Science, vol. 15702, Springer, 2025, págs. 223-239. DOI: 10.1007/978-3-031-94571-7_13

[6] Joint Task Force, «Security and Privacy Controls for Information Systems and Organizations,» National Institute of Standards and Technology, NIST Special Publication 800-53 Rev. 5, 2020. DOI: 10.6028/NIST.SP.800-53r5

[7] J. C. Klensin, «Simple Mail Transfer Protocol,» Internet Engineering Task Force, RFC 5321, oct. de 2008. DOI: 10.17487/RFC5321

[8] Postmark, *What are Transactional emails? | Postmark Support Center*, 2026. Visitado 28 de sep. de 2026. Dirección: https://postmarkapp.com/support/article/804-what-are-transactional-emails

[9] MailerSend, *What is MailerSend?* 2026. Visitado 8 de sep. de 2026. Dirección: https://developers.mailersend.com/

[10] MailerSend, *SMTP relay*, 2026. Visitado 8 de sep. de 2026. Dirección: https://developers.mailersend.com/smtp-relay.html

[11] MailerSend, *MailerSend Pricing*, 2026. Visitado 8 de sep. de 2026. Dirección: https://www.mailersend.com/pricing

[12] Brevo, *SMTP relay integration | Brevo API Documentation*, 2026. Visitado 16 de sep. de 2026. Dirección: https://developers.brevo.com/docs/smtp-integration

[13] MailerSend, *General API resources | MailerSend Developers*, 2026. Visitado 8 de sep. de 2026. Dirección: https://developers.mailersend.com/general.html

[14] *Apuntes de Fundamentos de Ingeniería del Software para Sistemas Cloud*, Asignatura del Máster Universitario en Ingeniería del Software: Cloud, Datos y Gestión de las Tecnologías de la Información, Universidad de Sevilla, Profesores: Antonio Ruiz Cortés y Manuel Resinas Arias de Reyna. Curso 2025/26, 2025.

[15] Twilio SendGrid, *Twilio SendGrid: Email API Pricing*, 2026. Visitado 6 de sep. de 2026. Dirección: https://www.twilio.com/en-us/products/email-api/pricing

[16] MailerSend, *How to send emails via SMTP with MailerSend*, 2026. Visitado 8 de sep. de 2026. Dirección: https://www.mailersend.com/help/smtp-relay

[17] MailerSend, *Schedule bulk emails*, 2026. Visitado 8 de sep. de 2026. Dirección: https://www.mailersend.com/help/schedule-bulk-emails-with-the-bulk-endpoint

[18] MailerSend, *MailerSend Developers: Email API*, 2026. Visitado 8 de sep. de 2026. Dirección: https://developers.mailersend.com/api/v1/email.html

[19] R. Gavira Sánchez, *PRIME4API: motor de análisis de la capacidad contractual de APIs*, 2026. Visitado 10 de ago. de 2026. Dirección: https://github.com/API-TaskForce/prime4api

[20] D. Ruiz López y R. Gavira Sánchez, *Pricing4API: A Python Library for Modeling and Reasoning about API Pricing Plans*, 2024. Visitado 2 de ago. de 2026. Dirección: https://github.com/PRICING4API/Pricing4API

[21] Anthropic, *Introducing the Model Context Protocol*, 2024. Visitado 2 de ago. de 2026. Dirección: https://www.anthropic.com/news/model-context-protocol

[22] ISA Group, *SPHERE: A SaaS Platform for API Pricing Evaluation and Management*, 2026. Visitado 2 de ago. de 2026. Dirección: https://sphere.score.us.es/

[23] Mailtrap, *6 Best Transactional Email Services Compared*, 2026. Visitado 26 de oct. de 2026. Dirección: https://mailtrap.io/blog/transactional-email-services/

[24] MailSlurp, *Transactional Email Service Comparison and Evaluation Guide*, 2026. Visitado 26 de oct. de 2026. Dirección: https://www.mailslurp.com/blog/transaction-email-services-compared/

[25] K. Peffers, T. Tuunanen, M. A. Rothenberger y S. Chatterjee, «A Design Science Research Methodology for Information Systems Research,» *Journal of Management Information Systems*, vol. 24, n.º 3, págs. 45-77, 2007. DOI: 10.2753/MIS0742-1222240302

[26] A. R. Hevner, S. T. March, J. Park y S. Ram, «Design Science in Information Systems Research,» *MIS Quarterly*, vol. 28, n.º 1, págs. 75-105, 2004.

[27] Universidad de Sevilla, *Trabajo Fin de Máster (51790028). Máster Universitario en Ingeniería del Software: Cloud, Datos y Gestión de las Tecnologías de la Información*, 2026. Visitado 29 de sep. de 2026. Dirección: https://www.us.es/estudiar/que-estudiar/oferta-de-masteres/master-universitario-en-ingenieria-del-software-cloud/51790028

[28] Ministerio de Educación, Cultura y Deporte, *Real Decreto 1125/2003, de 5 de septiembre, por el que se establece el sistema europeo de créditos y el sistema de calificaciones en las titulaciones universitarias de carácter oficial y validez en todo el territorio nacional*, Boletín Oficial del Estado, núm. 224, de 18 de septiembre, 2003. Visitado 29 de sep. de 2026. Dirección: https://www.boe.es/buscar/act.php?id=BOE-A-2003-17643

[29] Agencia Digital de Andalucía, *Instrucción 1/2023, de 4 de mayo de 2023, de la Agencia Digital de Andalucía sobre perfiles, precios de referencia y desglose de costes en contratos de bienes y servicios TIC*, 2023. Visitado 29 de sep. de

2026. Dirección: https://www.juntadeandalucia.es/haciendayadministracionpublica/apl/pdc_sirec_documentacion/rest/descargar/documento/129086
[30] Xataka, *Huawei MateBook D 16 (2024), análisis*, 2024. Visitado 29 de sep. de 2026. Dirección: https://www.xataka.com/analisis/huawei-matebook-d-16-2024-analisis-caracteristicas-precio-especificaciones
[31] GitHub, *FAQ about changes to GitHub's plans*, Respuesta sobre el cambio de planes del 14 de abril de 2020, 2026. Visitado 1 de nov. de 2026. Dirección: https://docs.github.com/en/get-started/learning-about-github/faq-about-changes-to-githubs-plans
[32] European Central Bank, *ECB euro reference exchange rate: US dollar (USD)*, 2026. Visitado 1 de nov. de 2026. Dirección: https://www.ecb.europa.eu/stats/policy_and_exchange_rates/euro_reference_exchange_rates/html/eurofxref-graph-usd.en.html
[33] Postmark, *Postmark Pricing*, 2026. Visitado 2 de ago. de 2026. Dirección: https://postmarkapp.com/pricing
[34] A. García-Fernández, J. A. Parejo, F. J. Cavero y A. Ruiz-Cortés, «Trends in Industry Support for Pricing-Driven DevOps in SaaS,» *IEEE Transactions on Services Computing*, vol. 19, n.º 1, págs. 726-737, 2026. DOI: 10.1109/TSC.2025.3634801
[35] A. García-Fernández, J. A. Parejo, F. J. Cavero y A. Ruiz-Cortés, «Racing the Market: An Industry Support Analysis for Pricing-Driven DevOps in SaaS,» en *Service-Oriented Computing (ICSOC 2024)*, serie Lecture Notes in Computer Science, vol. 15405, Springer, 2025, págs. 273-288. DOI: 10.1007/978-981-96-0808-9_19
[36] A. Ruiz-Cortés, O. Martín-Díaz, A. Durán y M. Toro, «Improving the automatic procurement of web services using constraint programming,» *International Journal of Cooperative Information Systems*, vol. 14, n.º 4, págs. 439-467, 2005. DOI: 10.1142/S0218843005001225
[37] A. del-Río-Ortega, M. Resinas, C. Cabanillas y A. Ruiz-Cortés, «On the definition and design-time analysis of process performance indicators,» *Information Systems*, vol. 38, n.º 4, págs. 470-490, 2013. DOI: 10.1016/j.is.2012.11.004
[38] D. Benavides, S. Segura y A. Ruiz-Cortés, «Automated analysis of feature models 20 years later: A literature review,» *Information Systems*, vol. 35, n.º 6, págs. 615-636, 2010. DOI: 10.1016/j.is.2010.01.001
[39] A. García-Fernández, J. A. Parejo y A. Ruiz-Cortés, «Pricing4SaaS: Towards a Pricing Model to Drive the Operation of SaaS,» en *Intelligent Information Systems (CAiSE 2024 Forum)*, serie Lecture Notes in Business Information Processing, vol. 520, Springer, 2024, págs. 47-54. DOI: 10.1007/978-3-031-61000-4_6
[40] F. J. Cavero, J. A. Parejo, J. C. Alonso y A. Ruiz-Cortés, «A-MINT: An LLM Pipeline for Automated Modeling of iPricings from SaaS Pricing Pages,» en *Service-Oriented Computing (ICSOC 2025)*, serie Lecture Notes in Computer Science, vol. 16321, Springer, 2026, págs. 83-91. DOI: 10.1007/978-981-95-5015-9_6
[41] A. García-Fernández, J. A. Parejo, P. Trinidad y A. Ruiz-Cortés, *SaaS Pricing Evaluation Corpus for Temporal Representation & Analysis (SPECTRA)*, Zenodo, Versión 1.1.0, 2025. DOI: 10.5281/zenodo.17237051

[42] R. T. Fielding, «Architectural Styles and the Design of Network-based Software Architectures,» Tesis doct., University of California, Irvine, 2000.
[43] OpenAPI Initiative, *OpenAPI Specification v3.2.0*, 2025. Visitado 2 de ago. de 2026. Dirección: https://spec.openapis.org/oas/v3.2.0.html
[44] A. Gámez-Díaz, P. Fernández y A. Ruiz-Cortés, «An Analysis of RESTful APIs Offerings in the Industry,» en *Service-Oriented Computing (ICSOC 2017)*, serie Lecture Notes in Computer Science, vol. 10601, Springer, 2017, págs. 589-604. DOI: 10.1007/978-3-319-69035-3_43
[45] A. Gámez-Díaz et al., «The Role of Limitations and SLAs in the API Industry,» en *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2019)*, ACM, 2019, págs. 1006-1014. DOI: 10.1145/3338906.3340445
[46] A. Andrieux et al., «Web Services Agreement Specification (WS-Agreement),» Open Grid Forum, inf. téc. GFD-R-P.107, 2007.
[47] A. Gámez-Díaz, P. Fernández y A. Ruiz-Cortés, «Automating SLA-Driven API Development with SLA4OAI,» en *Service-Oriented Computing (ICSOC 2019)*, serie Lecture Notes in Computer Science, vol. 11895, Springer, 2019, págs. 20-35. DOI: 10.1007/978-3-030-33702-5_2
[48] R. Fresno-Aranda, P. Fernández, A. Gámez-Díaz, A. Durán y A. Ruiz-Cortés, «Pricing4APIs: A Rigorous Model for RESTful API Pricings,» *Computer Standards & Interfaces*, vol. 91, ene. de 2025, art. 103878, ISSN: 0920-5489. DOI: 10.1016/j.csi.2024.103878
[49] GitHub, *Rate limits for the REST API*, 2026. Visitado 2 de ago. de 2026. Dirección: https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api
[50] A. Grummit, *Capacity Management – A Practitioner Guide* (ITSM Library). Van Haren Publishing, 2010.
[51] M. Fowler y J. Lewis, *Microservices*, 2014. Visitado 2 de ago. de 2026. Dirección: https://martinfowler.com/articles/microservices.html
[52] R. Fresno-Aranda, P. Fernández, A. Durán y A. Ruiz-Cortés, «Semi-automated Capacity Analysis of Limitation-Aware Microservices Architectures,» en *Economics of Grids, Clouds, Systems, and Services (GECON 2022)*, serie Lecture Notes in Computer Science, vol. 13430, Springer, 2023, págs. 75-88. DOI: 10.1007/978-3-031-29315-3_7
[53] R. Fresno-Aranda, P. Fernández y A. Ruiz-Cortés, «Smart LAMA API: Automated Capacity Analysis of Limitation-Aware Microservices Architectures,» en *Actas de las XVII Jornadas de Ciencia e Ingeniería de Servicios (JCIS 2022)*, Sistedes, 2022.
[54] R. Fresno-Aranda, «Automated Capacity Analysis of Limitation-Aware Microservices Architectures,» Tesis doct., Universidad de Sevilla, 2025.
[55] R. Fresno-Aranda, P. Fernández y A. Ruiz-Cortés, «A Catalogue of Analysis Operations for API Pricing Plans,» en *Actas de las XVIII Jornadas de Ciencia e Ingeniería de Servicios (JCIS 2023)*, Sistedes, 2023.
[56] R. Gavira Sánchez, «Modelo visual y analítico de la capacidad de APIs desde la perspectiva del cliente,» Trabajo Fin de Grado, Escuela Técnica Superior de Ingeniería Informática, Universidad de Sevilla, 2025.

[57] M. Iansiti y R. Levien, *The Keystone Advantage: What the New Dynamics of Business Ecosystems Mean for Strategy, Innovation, and Sustainability*. Boston: Harvard Business School Press, 2004, ISBN: 978-1-59139-307-8.

[58] API-TaskForce, *SPHERE: SaaS Pricing Holistic Evaluation and Regulation Environment*, *Fork* con la extensión para Datasheets. Versión del 10 de junio de 2026 (commit fc1562a), 2026. Visitado 7 de sep. de 2026. Dirección: https://github.com/API-TaskForce/SPHERE

[59] Twilio SendGrid, *Twilio SendGrid Developer Documentation: Mail Send API Reference*, 2026. Visitado 6 de sep. de 2026. Dirección: https://www.twilio.com/docs/sendgrid/api-reference/mail-send

[60] J. Noguerol Tirado, «Selección visual de plan+add-ons y estrategia de envío en APIs de correo para minimizar el coste de una demanda histórica dada,» Máster en Ingeniería del Software: Cloud, Datos y Gestión TI. Convocatoria de octubre, curso 2024/25, Trabajo Fin de Máster, Universidad de Sevilla, 2025.

[61] API-TaskForce, *apis_consumption_model: API Datasheets — guía del esquema ACM2YAML y datasheets de referencia*, 2026. Visitado 3 de ago. de 2026. Dirección: https://github.com/API-TaskForce/apis_consumption_model

[62] Amazon Web Services, *Amazon Simple Email Service (SES) Pricing*, 2026. Visitado 18 de sep. de 2026. Dirección: https://aws.amazon.com/ses/pricing/

[63] Postmark, *Postmark Developer Documentation: API Overview*, 2026. Visitado 2 de ago. de 2026. Dirección: https://postmarkapp.com/developer/api/overview

[64] D. Ruiz López, `provider-service-resolver` *(v1.1) y* `saas-interface-channel-analyzer` *(v3.0): skills para caracterizar servicios SaaS*, Material sin publicar, facilitado por su autor en octubre, 2026.

[65] Brevo, *FAQs: What are the limits of the Free plan? | Brevo Help Center*, 2026. Visitado 16 de sep. de 2026. Dirección: https://help.brevo.com/hc/en-us/articles/208580669-FAQs-What-are-the-limits-of-the-Free-plan

[66] API-TaskForce, *datasheet-usage-testing: banco de pruebas para la medición empírica de límites de uso de APIs*, 2026. Visitado 3 de ago. de 2026. Dirección: https://github.com/API-TaskForce/datasheet-usage-testing

[67] Brevo, *API rate limits | Brevo Developers*, 2026. Visitado 16 de sep. de 2026. Dirección: https://developers.brevo.com/docs/api-limits

[68] Amazon Web Services, *Service quotas in Amazon SES | Amazon SES Developer Guide*, 2026. Visitado 18 de sep. de 2026. Dirección: https://docs.aws.amazon.com/ses/latest/dg/quotas.html

[69] Resend, *Resend Pricing*, 2026. Visitado 5 de ago. de 2026. Dirección: https://resend.com/pricing

[70] Brevo, *Brevo Pricing*, 2026. Visitado 16 de sep. de 2026. Dirección: https://www.brevo.com/pricing/

[71] Mailgun, *Transactional Email API Service For Developers | Mailgun Pricing*, 2026. Visitado 21 de sep. de 2026. Dirección: https://www.mailgun.com/pricing/

[72] Mailchimp, *Transactional Email Pricing | Mailchimp*, 2026. Visitado 21 de sep. de 2026. Dirección: https://mailchimp.com/pricing/transactional-email/

[73] Resend, *Resend Developer Documentation: API Reference*, 2026. Visitado 5 de ago. de 2026. Dirección: https://resend.com/docs/api-reference/introduction

[74] Mailchimp, *Fundamentals | Mailchimp Transactional API*, 2026. Visitado 21 de sep. de 2026. Dirección: https://mailchimp.com/developer/transactional/docs/fundamentals/

[75] Twilio SendGrid, *Twilio SendGrid Developer Documentation: Rate Limits*, 2026. Visitado 6 de sep. de 2026. Dirección: https://www.twilio.com/docs/sendgrid/api-reference/how-to-use-the-sendgrid-v3-api/rate-limits

A. Uso de inteligencia artificial

Este anexo declara el uso de inteligencia artificial en el trabajo, que se ha hecho con supervisión humana (*human in the loop*). El autor plantea cada tarea como un problema y diseña los agentes y flujos de trabajo que la resuelven, con herramientas para comprobar lo que producen. El alcance y las tres fuentes de inclusión de las APIs vienen del encargo del trabajo, y el autor fija las preguntas de investigación y el índice. Después revisa los datos que los agentes no pueden cerrar con una fuente y las dudas de modelado en las que los agentes no se ponen de acuerdo, y los da por buenos o los corrige. El resto de las tareas lo ejecuta Claude, el asistente de Anthropic, dentro de las reglas que fija el autor.

La Figura A.1 recoge cómo se reparte el trabajo. La sesión principal de Claude, la conversación en la que trabaja el autor, es el coordinador: reparte las tareas entre agentes y, cuando sus propuestas no coinciden, convoca el desempate. Unos agentes construyen cada Datasheet. Otro agente, distinto, la verifica: recibe solo la Datasheet terminada, vuelve a abrir cada URL citada y contrasta dato a dato. La Datasheet pasa además tres comprobaciones automáticas: el validador de formato, la verificación de que cada cita sigue en la copia archivada de su página, cuando la hay, y una ejecución sobre PRIME4API que confirma que el motor carga la Datasheet.

La Datasheet de Postmark, la del piloto, es anterior a este reparto y se cerró sin informe de verificador. Las otras siete lo tienen. Las páginas que cita no se archivaron, así que la comprobación de citas no tiene con qué contrastarlas. La de Postmark sí pasó por el validador de formato y por PRIME4API (Tabla 4.6).

El libro de criterios es el fichero de reglas de modelado que se aplica a todas las Datasheets. Una de sus reglas dice, por ejemplo, que la política de retención de datos no es una cuota. Las dudas que ese libro no resuelve las deliberan cinco agentes en paralelo, y si no coinciden, un sexto agente las examina por separado y propone el veredicto, que entra en el libro como regla nueva. Si tampoco el sexto desempata, la duda pasa al autor.

La comparación entre versiones de los términos de consumo de cada proveedor, que detecta qué cambió entre dos capturas fechadas, la hizo Claude en sesiones independientes. Es la parte menos formalizada, y por eso se dibuja con borde discontinuo. Ni las instrucciones que recibió ni los programas que extraen el texto de las versiones archivadas en la Wayback Machine, el archivo de páginas web de Internet Archive, están en el repositorio ni empaquetados como *skill*. Solo se conservan sus resultados, las tablas de versiones de cada proveedor. La captura de la versión vigente de cada página sí está versionada, porque la hace la *skill* de recogida descrita en la Sección 4.2.

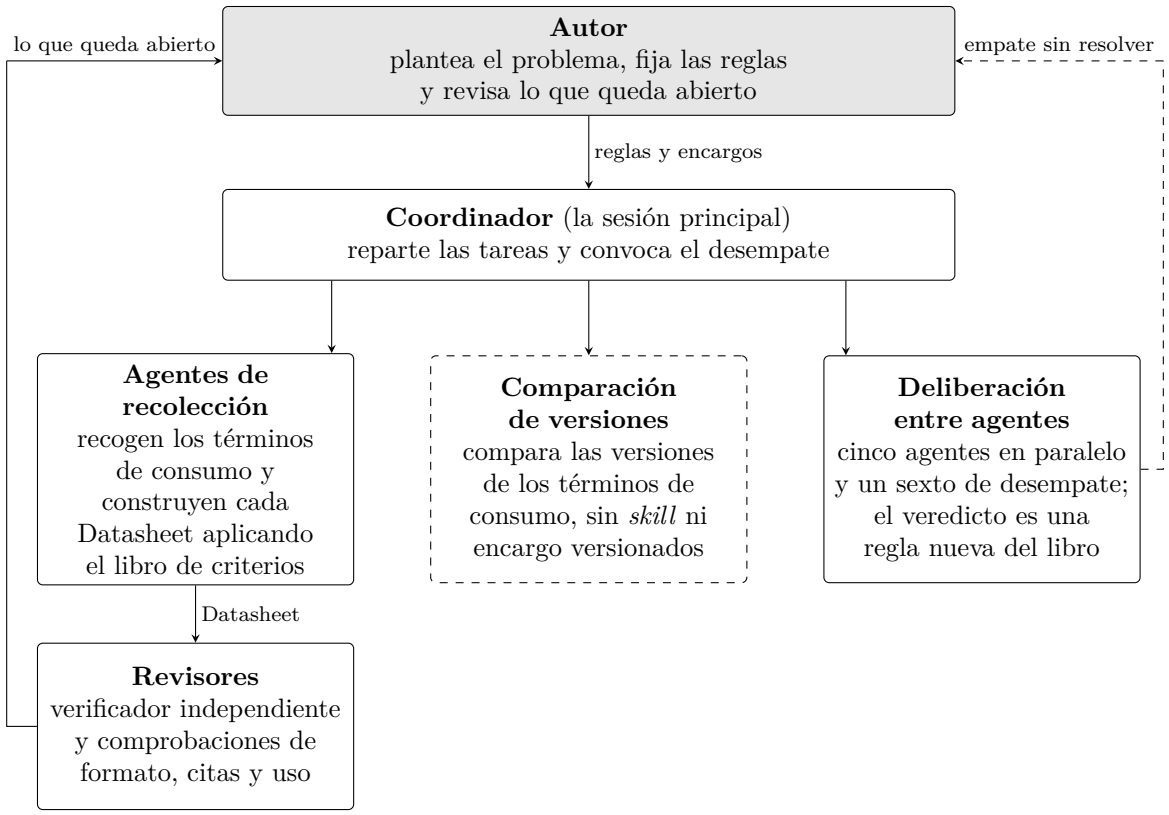


Figura A.1: Arquitectura de agentes del trabajo. Las cajas de borde continuo son procesos documentados en el repositorio, y la de borde discontinuo, uno cuyo encargo y herramientas no están versionados. La flecha discontinua es el empate que la deliberación no resuelve.

A.1. Alcance del uso

El trabajo se ha hecho con Claude, de Anthropic, en sus entornos Cowork y Claude Code, con la suscripción que imputa la Sección 2.3. Además de las dos consultas de las secciones siguientes y de la comparación entre versiones de los términos de consumo, Claude interviene en seis tareas.

Construcción y revisión de las Datasheets. Siguen el reparto de la Figura A.1, salvo en la Datasheet de Postmark, que es anterior a él.

Recogida de los términos de consumo. Una *skill* propia, con validador y pruebas automáticas, captura en bruto las páginas de términos de consumo de cada proveedor (Sección 4.2).

Sondeos empíricos. Las pruebas contra la API de un proveedor que miden un límite que su documentación no publica las ejecutan agentes, con un presupuesto declarado y con envíos limitados a tres direcciones del autor.

Análisis. Los agentes preparan el recuento de en cuántas APIs reaparece cada elemento no soportado y las ejecuciones sobre PRIME4API, y sus resultados están versionados en el repositorio de datos.

Herramientas. Los programas del repositorio, como el validador de formato o la verificación de citas, se programaron con Claude y tienen pruebas automáticas.

Redacción. Claude ayuda en la redacción: propone versiones de los párrafos con el registro del autor, tomado de su Trabajo Fin de Grado, y las pasa por una *skill* que quita las marcas de estilo habituales de los modelos de lenguaje. Otro agente, distinto del que las propone, comprueba que cada afirmación lleve su fuente y que el texto se entienda sin leer las referencias. El autor lee el texto completo, lo corrige y decide cada cambio.

Claude fue también objeto de una prueba, con las Datasheets en el prompt y sin ellas, sobre las cuatro preguntas del catálogo reservadas a HARVEY4API, el agente conversacional que consulta PRIME4API (Sección 7.3). La prueba no sustituye a HARVEY4API, que no se ejecutó en este trabajo: en ella el modelo hace él mismo las cuentas, sin llamar al motor.

A.2. La consulta de la selección

La selección de APIs del catálogo se completó con una consulta a un modelo de lenguaje (Sección 4.1). El prompt pedía cuatro APIs seleccionadas y cuatro de reserva, ordenadas, con el tipo de modelo de precios de cada una y sin ninguna cifra. La respuesta propuso Amazon SES, Brevo, Mailchimp Transactional y Mailgun, y de reserva, por este orden, Mailjet, Zoho ZeptoMail, SparkPost y SMTP2GO. La Tabla A.1 recoge las condiciones de la consulta que constan en el fichero de la selección del repositorio, y también las que no constan. El prompt literal y la respuesta íntegra están en el repositorio del trabajo[1].

[1] https://github.com/danvelcam/tfm-pricing-apis-correo, fichero `03_datos/SELECCION_APIS_2026-09-01.md`. Contiene el prompt, la respuesta con su tabla de cobertura y la lista de reserva en su orden.

Campo	Valor
Fecha	1 de septiembre de 2026
Modelo	`claude-fable-5-1`, de Anthropic
Entorno	La sesión de trabajo del proyecto, a petición del autor
Criterios	Cinco, ordenados por peso e incluidos en el prompt: diversidad del modelo de precios, documentación pública de precios y de límites, adopción en producción, un rasgo diferenciador de los límites y diversidad geográfica y de tamaño del proveedor
Catálogo previo	Postmark, Resend, SendGrid y MailerSend, que el prompt excluye
Respuesta pedida	Cuatro APIs seleccionadas y cuatro de reserva, ordenadas, y para cada una el tipo de modelo de precios, lo que aporta y el riesgo principal, sin inventar cifras
No consta en el fichero	La hora, el prompt de sistema, los parámetros de generación, el número de intentos y la transcripción de la sesión

Tabla A.1: Ficha de la consulta, según el fichero de la selección.

La consulta se hizo en la sesión de trabajo del proyecto, y la respuesta usa ese contexto: menciona dos decisiones de modelado anteriores que el prompt no recoge. Una limita la Datasheet de Resend a su API de correo transaccional. La otra pone `api_first`, y no `partial_saas`, en el campo `type` de un proveedor que tiene panel web y API. Es el campo con el que ACM2YAML clasifica el servicio. Este trabajo reserva `partial_saas` para el servicio en el que alguna de tres operaciones (enviar, crear o editar una plantilla y añadir un dominio de envío) se puede hacer en el panel y no en la API, y en los demás casos usa `api_first`.

A.3. Cómo se aplicó

La selección no cambió después de la consulta: ninguna API de la reserva sustituyó a una del catálogo (Apartado 4.1.2). La respuesta no da precios ni límites con cifra, así que ninguno de los que recoge el catálogo procede de ella. Cada uno sale de la documentación del proveedor, de su panel o de un sondeo, en la ventana de toma de datos (Apartado 4.1.3). La construcción corrigió tres afirmaciones de la respuesta, sobre Amazon SES, Mailgun y Brevo, y la Sección 4.7 las detalla.

Una segunda consulta, ajena a la selección, preguntó a Claude Opus 5.5 si doce elementos no soportados que aparecen en una sola API admiten una forma general que valga también para otros proveedores. Once son los que el inventario no promueve al modelo. El duodécimo era un supuesto tercer tope diario de MailerSend, de 100 correos en el plan Free, sin sitio en la Datasheet porque el plan y el *endpoint* ya tenían cada uno su cuota. Se retiró dos días después de la consulta: la página de precios cifra ese tope en peticiones, y en peticiones es la cuota diaria que la Datasheet ya tenía. La

respuesta a esa consulta no cambia ningún dato del catálogo ni el veredicto de ningún elemento (Sección 5.1).